

CHANGLIU LIU

ADVANCED CONTROL FOR ROBOTICS

CARNEGIE MELLON UNIVERSITY

Copyright © 2026 Changliu Liu

PUBLISHED BY CARNEGIE MELLON UNIVERSITY

TUFTE-LATEX.GITHUB.IO/TUFTE-LATEX/

Licensed under the Apache License, Version 2.0 (the “License”); you may not use this file except in compliance with the License. You may obtain a copy of the License at <http://www.apache.org/licenses/LICENSE-2.0>. Unless required by applicable law or agreed to in writing, software distributed under the License is distributed on an “AS IS” BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied. See the License for the specific language governing permissions and limitations under the License.

First printing, August 2026

Contents

Introduction 5

I Systems 7

1 *System Models* 9

2 *System Properties* 19

II Optimal Control 31

3 *Optimal Control - Finite Time* 33

4 *Optimal Control - Infinite Time* 53

5 *Model Predictive Control* 61

6 *Iterative Learning Control* 81

III State Estimation and Model Adaptation 91

7 *Probability and Least Squares* 93

8	<i>Kalman Filter, Extended Kalman Filter and Unscented Kalman Filter</i>	103
9	<i>Recursive Least Square and Stochastic Gradient Descent</i>	117
	<i>IV Indirect Adaptive Control</i>	133
10	<i>Passive Learning and Separation Principle</i>	135
11	<i>Active Learning and Dual Control</i>	147
	<i>V Value Approximation and Policy Learning</i>	155
12	<i>Temporal Difference Learning and Value Approximation</i>	157
13	<i>Policy Gradient and Actor Critic</i>	171
	<i>VI Stability and Safety</i>	187
14	<i>Lyapunov Stability</i>	189
15	<i>Passivity and Hyperstability</i>	197
16	<i>Safe Learning and Adaptation</i>	207
	<i>Bibliography</i>	223
	<i>Index</i>	225

Introduction

The goal for this class is to familiarize students with advanced control techniques. This class note is adapted from 16-899 class note.

Part I

Systems

1

System Models

Building a system model is the first step to tackle a control problem. This chapter defines states and controls in section 1.1; introduces dynamic models as a function between state and control in section 1.2; discusses the measurement model in section 1.3 and the closed-loop system in section 1.4; and finally introduce the objectives for controller design in section 1.5. Section 1.6 summarizes different controller design problems in different situations. Section 1.7 discusses the connection between these models with Markov Decision Process.

1.1 State and Control

The internal state variables x are: the smallest possible subset of system variables that can represent the entire state of the system at any given time such that the future evolution of the system only depends on the current state, not the past (Markov).¹

A state space $\mathcal{X}$ is the collection of all possible values of the state variables. State values refer to the value of the state variables. State values can be continuous or discrete. For example, we can take the grid as state in a grid world. The state values (e.g., the grid ID) are discrete. For an inverted pendulum, we can take its angular position as state, which is continuous.

A control space $\mathcal{U}$ is the set of variables that the agent can choose to affect future states. The element in the control space $u \in \mathcal{U}$ is called a control input or control signal. The control signals can be continuous or discrete. For example, in a multi-arm bandit problem, the control is discrete as the agent can only choose to one arm from multiple arms.² On the other hand, for autonomous driving, the vehicle steering angle is a continuous control variable.

A system can have either continuous or discrete state space and control space. It is not necessary that systems with continuous (or discrete) states much have continuous (or discrete) controls and vice versa. The following are examples of systems with different combina-

¹ “Markov” generally means that given the present state, the future and the past are independent.

² The name “multi-arm bandit” comes from imagining a gambler at a row of slot machines, who has to decide which machines (arm) to play at each round.

tions of states and controls.

- Continuous controls on continuous states

Example: in a vehicle model, define the state to be the planar location and the control be the steering angle and the speed. Then both the states and the controls are continuous.

- Continuous controls on discrete states

Example: in a ratchet and pawl gear system, define the state to be the number of times the pawl gear is pushed away and the control be the rotation angle of the ratchet gear. Then the state is discrete and the control is continuous.

- Discrete controls on discrete states

Example: in a grid world, define the state to the grid and the control to be movements among the grids (left, right, up, down). Then both the states and the controls are discrete.

- Discrete controls on continuous states

Example: in a water tank with binary in/out valve, define the state to the water level and the control to the on-off of the valve. Then the state is continuous, while the control is discrete.

1.2 Dynamics

Dynamic equation is a function that describes the evolution of states under control inputs. Let us first consider the deterministic case where the future states can be fully specified by the current state and the control inputs along the way. We may describe the system using continuous time or discrete time. For continuous time deterministic systems, the dynamics are described using ordinary differential equations as shown below

$$\dot{x} = f(x, u). \quad (1.1)$$

For discrete time deterministic systems, the dynamics are described using difference equations as

$$x_{k+1} = f(x_k, u_k), \quad (1.2)$$

where the subscript k denotes time step k . For simplicity, we may drop the time steps for discrete time systems as we did for the continuous time systems. In this way, we can write the difference equations as $x' = f(x, u)$ where x' is the state in the next time step.

Remark 1.1 (Discretizing a continuous time system). *A continuous time system $\dot{x} = f(x, u)$ can be turned into a discrete time system $x' = x + \delta x$ where δx is the change of the state according to the continuous time dynamics between two sampling time steps δt . The term δx can be approximated by $f(x, u)\delta t$.*

When the system is stochastic, there are additional variables to affect future states other than the current states and the control inputs along the way. We call these variables as disturbances, which are usually modeled as random variables. The dynamics of a continuous time stochastic system can be specified as

$$\dot{x} = f(x, u, w), \quad (1.3)$$

where w is a random variable that represents disturbances to the system. The dynamics of a discrete time stochastic system can be specified as

$$x_{k+1} = f(x_k, u_k, w_k), \quad (1.4)$$

where w_k is a random variable that represents disturbances to the system. Chapter 7 will characterize these random variables.

The functions f in (1.1), (1.2), (1.3), and (1.4) are called dynamic models. The dynamic model can be time varying or time invariant, depending on whether f changes with respect to time. For simplicity and when there is no ambiguity, we do not explicitly include time in the equations. In the case that we need to emphasize the time varying nature, we will use subscripts f_t for continuous time case or f_k for discrete time case.

If f is linear time invariant (LTI), then we will write (1.1), (1.2), (1.3), and (1.4) as

$$\dot{x} = Ax + Bu \quad (1.5)$$

$$x_{k+1} = Ax_k + Bu_k \quad (1.6)$$

$$\dot{x} = Ax + Bu + B^w w \quad (1.7)$$

$$x_{k+1} = Ax_k + Bu_k + B^w w_k, \quad (1.8)$$

where A, B, B^w are all model parameters.

Another important type of dynamic models is affine dynamics, where f is nonlinear but linear in the control input. For affine time-invariant (ATI) dynamics, (1.1), (1.2), (1.3), and (1.4) can be rewritten as

$$\dot{x} = f(x) + g(x)u \quad (1.9)$$

$$x_{k+1} = f(x_k) + g(x_k)u_k \quad (1.10)$$

$$\dot{x} = f(x, w) + g(x, w)u \quad (1.11)$$

$$x_{k+1} = f(x_k, w_k) + g(x_k, w_k)u_k, \quad (1.12)$$

where f is overloaded, which now represents the nonlinear inertia in the dynamic model, and g is the coefficient for control.

We will mostly focus on LTI and ATI systems in this course. Note that any system can be rearranged to be an affine system through dynamic extension. Here we use the continuous time deterministic model (1.1) as an example. By dynamic extension, we define an extended state $x^e := [x; u]$ where $[\cdot; \cdot]$ means vertical concatenation, and an extended control $u^e := \dot{u}$. Then the extended dynamics with respect to the extended state and the extended control are

$$\dot{x}^e = \begin{bmatrix} \dot{x} \\ \dot{u} \end{bmatrix} = \begin{bmatrix} f(x, u) \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u^e, \quad (1.13)$$

which is in the form of (1.9).

1.3 Measurement and Observation

In many cases, states are not directly measurable. They needed to be inferred from measurements, denoted $y \in \mathcal{Y}$, where $\mathcal{Y}$ is the observation space. A deterministic measurement function and a stochastic measurement function can be written as

$$y = h(x, u), \quad (1.14)$$

$$y = h(x, u, v), \quad (1.15)$$

where h is called the measurement function and v is the measurement noise.

1.4 Closed-Loop Systems

The dynamic models introduced in the previous section are all open-loop dynamics. The loop is closed if we design a rule to automatically choose the control input based on the system state, which is called as a feedback control law. If the feedback control law only depends on the current state, i.e., $u = \pi(x)$, it is called a state-feedback control law. If the feedback control law depends on the measurement of the current state, i.e., $u = \pi(y)$, it is called a measurement-feedback control law. When $y = x$, the measurement-feedback control law reduces to a state-feedback control law.

1.5 Objectives

A control law is always designed to optimize certain objectives. We call these control laws as optimal control laws. The optimization objective can be: stabilization, trajectory tracking, goal reaching,

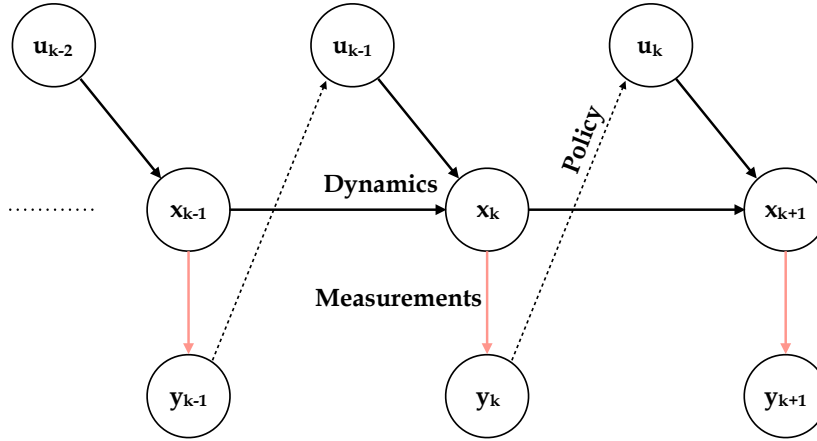


Figure 1.1: Illustration of a discrete-time measurement feedback system. The black solid arrows correspond to the propagation of dynamics (1.4). The red solid arrows are the measurement functions (1.15). The black dashed arrows represent the measurement feedback control law.

etc. The objective is either encoded as a cost function J or a reward function R . This course will use cost J .

A typical cost function consists of run-time costs and terminal costs. The run-time cost $l(x, u)$ ³ is incurred at every time instance, for example, battery consumption cost for electronics. The terminal cost $l(x)$ is only incurred if certain event happens or if at certain time, *e.g.*, arriving at a destination. Both the run-time cost and the terminal cost may be time varying functions. For simplicity and when there is no ambiguity, we do not explicitly include time in the expression. In the case that we need to emphasize the time varying nature, we will use subscripts l_t for continuous time case or l_k for discrete time case.

The agent optimizes not only the current cost, but the cumulative cost over a certain period of time. Regarding how far the agent looks into the future, we can either have a finite horizon problem and an infinite horizon problem. A finite horizon either terminates at certain time, *e.g.*, exams, or terminates at certain event, *e.g.*, goal-reaching problems. An infinite horizon problem has no termination, *e.g.*, stabilization problems. The cost that will incur in the far future may not have the same effect as the cost that incurs immediately. There will be a discount factor $\delta \in [0, 1]$ to represent how the agent weights the costs at different time steps differently.

Mathematically, the cost J for a continuous time finite horizon problem can be written as

$$J = \int_0^T \delta^t l(x(t), u(t)) dt + \delta^T l(x(T)), \quad (1.16)$$

where the horizon is set from 0 to $T \in \mathbb{R}^+$. The terminal time T can either be fixed or triggered when certain conditions apply. The

³ The run time cost can be a function on the current state, the current control, and the future state after the dynamic transition. For simplicity, we assume the run time cost is only a function on the first two. In addition, the system dynamics can be extended as a mapping from x and u to $\dot{x}$ (or x') and l considering the cost.

first case corresponds to a fixed-time problem, which the second corresponds to a fixed-target problem. The cost J for a continuous time infinite horizon problem can be written as

$$J = \int_0^{\infty} \delta^t l(x(t), u(t)) dt. \quad (1.17)$$

There is no terminal cost since the problem never terminates. Similarly, the cost J of a discrete time finite horizon problem is

$$J = \sum_{k=0}^{K-1} \delta^k l(x_k, u_k) + \delta^K l(x_K), \quad (1.18)$$

where the horizon is set from 0 to $K \in \mathbb{N}$. The cost J for a discrete time infinite horizon problem is

$$J = \sum_{k=0}^{\infty} \delta^k l(x_k, u_k). \quad (1.19)$$

This course focuses on methods to solve for the control policy π that minimizes the cost:

$$\min_{\pi} J. \quad (1.20)$$

1.6 Controller Design Problems

Using the dynamic functions, measurement functions, and objectives defined earlier, the controller design problems in different situations can be posed as follows.

Problem 1.1 (Continuous time deterministic finite horizon problem).

Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} J = \int_0^T \delta^t l(x(t), u(t)) dt + \delta^T l(x(T)), \quad (1.21a)$$

$$\text{s.t. } \dot{x} = f(x, u), \quad (1.21b)$$

$$y = h(x, u), \quad (1.21c)$$

$$u = \pi(y). \quad (1.21d)$$

Problem 1.2 (Discrete time deterministic finite horizon problem).

Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} J = \sum_{k=0}^{K-1} \delta^k l(x_k, u_k) + \delta^K l(x_K), \quad (1.22a)$$

$$\text{s.t. } x' = f(x, u), \quad (1.22b)$$

$$y = h(x, u), \quad (1.22c)$$

$$u = \pi(y). \quad (1.22d)$$

Problem 1.3 (Continuous time deterministic infinite horizon problem). *Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, and an initial state x_0 , we need to design a controller to solve the following problem:*

$$\min_{\pi} J = \int_0^{\infty} \delta^t l(x(t), u(t)) dt, \quad (1.23a)$$

$$\text{s.t. } \dot{x} = f(x, u), \quad (1.23b)$$

$$y = h(x, u), \quad (1.23c)$$

$$u = \pi(y). \quad (1.23d)$$

Problem 1.4 (Discrete time deterministic infinite horizon problem).

Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} J = \sum_{k=0}^{\infty} \delta^k l(x_k, u_k), \quad (1.24a)$$

$$\text{s.t. } x' = f(x, u), \quad (1.24b)$$

$$y = h(x, u), \quad (1.24c)$$

$$u = \pi(y). \quad (1.24d)$$

Problem 1.5 (Continuous time stochastic finite horizon problem).

Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, noise models for w and v , and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} \mathbb{E}[J] = \mathbb{E}_{w,v} \left[\int_0^T \delta^t l(x(t), u(t)) dt + \delta^T l(x(T)) \right], \quad (1.25a)$$

$$\text{s.t. } \dot{x} = f(x, u, w), \quad (1.25b)$$

$$y = h(x, u, v), \quad (1.25c)$$

$$u = \pi(y). \quad (1.25d)$$

Problem 1.6 (Discrete time stochastic finite horizon problem). *Given*

a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, noise models for w and v , and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} \mathbb{E}[J] = \mathbb{E}_{w,v} \left[\sum_{k=0}^{K-1} \delta^k l(x_k, u_k) + \delta^K l(x_K) \right], \quad (1.26a)$$

$$\text{s.t. } x' = f(x, u), \quad (1.26b)$$

$$y = h(x, u), \quad (1.26c)$$

$$u = \pi(y). \quad (1.26d)$$

Problem 1.7 (Continuous time stochastic infinite horizon problem).

Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, noise models for w and v , and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} \mathbb{E}[J] = \mathbb{E}_{w,v} \left[\int_0^{\infty} \delta^t l(x(t), u(t)) dt \right], \quad (1.27a)$$

$$\text{s.t. } \dot{x} = f(x, u, w), \quad (1.27b)$$

$$y = h(x, u, v), \quad (1.27c)$$

$$u = \pi(y). \quad (1.27d)$$

Problem 1.8 (Discrete time stochastic infinite horizon problem). Given a state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, noise models for w and v , and an initial state x_0 , we need to design a controller to solve the following problem:

$$\min_{\pi} \mathbb{E}[J] = \mathbb{E}_{w,v} \left[\sum_{k=0}^{\infty} \delta^k l(x_k, u_k) \right], \quad (1.28a)$$

$$\text{s.t. } x' = f(x, u), \quad (1.28b)$$

$$y = h(x, u), \quad (1.28c)$$

$$u = \pi(y). \quad (1.28d)$$

1.7 Markov Decision Process

The controller design problem introduced earlier can also be posed as a Markov decision process (MDP) or a partially observed Markov decision process (POMDP). An MDP includes the following items:

- A state space $\mathcal{X}$, a control space $\mathcal{U}$, an initial state x_0 ;
- A transition function $T(x, u, x') : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \mapsto \mathbb{R}^+$, which describes the probability of arriving at the particular state value at next step given the current state value and the current control action;
- A reward function $R(x, u, x') : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \mapsto \mathbb{R}$;
- A discount factor δ .

The goal in solving MDP is to obtain a state-feedback control policy that achieves the highest accumulative reward.

An MDP is equivalent to problem 1.6 (if there is a terminate state) or problem 1.8 (if there is no terminate state) with full state feedback, i.e., $y = x$. The transition function is equivalent to the discrete time stochastic dynamic function (1.4). The reward function is equivalent to the negative run-time cost function.

The POMDP models are extensions of MDP models by considering the observation space. A POMDP includes the following items:

- A state space $\mathcal{X}$, a control space $\mathcal{U}$, an observation space $\mathcal{Y}$, an initial state x_0 ;
- A transition function $T(x, u, x') : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \mapsto \mathbb{R}^+$;
- An observation function $O(y', x', u) : \mathcal{Y} \times \mathcal{X} \times \mathcal{U} \mapsto \mathbb{R}^+$, which describes the probability of obtaining certain measurement given the current state value and the last step's control action;
- A reward function $R(x, u, x') : \mathcal{X} \times \mathcal{U} \times \mathcal{X} \mapsto \mathbb{R}$;
- A discount factor δ .

The goal in solving POMDP is to obtain a measurement-feedback control policy that achieves the highest accumulative reward.

A POMDP is equivalent to problem 1.6 (if there is a terminate state) or problem 1.8 (if there is no terminate state). The observation function is equivalent to the stochastic measurement function (1.15).

1.8 Example

Let us build a system model for an autonomous driving problem on the freeway. Suppose an autonomous vehicle wants to follow its lane center with constant speed v^r and minimum energy consumption. The vehicle can be controlled through steering and longitudinal acceleration. The vehicle can measure its position through GPS and its longitudinal speed through speedometer. Assume the environment is deterministic.

- What is the state and control?

The vehicle model can be developed with different fidelity. Here we use a unicycle model. There are four state variables: planer position p_x and p_y , longitudinal velocity v , heading θ . There are two control inputs: steering $\dot{\theta}$, and $a = \dot{v}$. Hence⁴

$$x = [p_x; p_y; v; \theta], u = [a; \dot{\theta}]. \quad (1.29)$$

⁴ We follow the convention in matrix definition that “;” means vertical concatenation and “,” or space means horizontal concatenation.

- What is the dynamic model?

The dynamics model can be written as

$$\dot{x} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} u. \quad (1.30)$$

- What is the measurement model?

The vehicle can directly measure three states: p_x , p_y , and v . Hence

$$y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} x. \quad (1.31)$$

- What is the cost function?

Suppose the lane center is along $p_x = 0$. Hence the target state can be defined as $x^r = [0; p_y; v^r; 0]$. Then the run-time can be defined as

$$l(x, u) = (x - x^r)^T \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} (x - x^r) + u^T u. \quad (1.32)$$

where the first quadratic term penalizes the tracking error, and the second quadratic term penalizes the energy consumption (in terms of the magnitude of control effort). This is an infinite horizon problem, hence does not have a terminal cost.

Hence this example belongs to a continuous time deterministic infinite horizon problem [1.3](#).

2

System Properties

After obtaining the system models, it is important to characterize the system properties before deriving the control law. In particular, we need to answer the following questions: does it exist a control law to meet the control objective? is it possible to infer the system states if we observe in a long enough time horizon? The first question corresponds to controllability of the system, to be discussed in section 2.1. The second question corresponds to observability of the system, to be discussed in section 2.2. On the other hand, after we close the loop with an optimal control law, it is important to understand how well is the control objective achieved, *i.e.*, will the system be able to automatically correct any error introduced by disturbances? This question can be answered by analyzing the stability of the system, to be discussed in section 2.3. In the following discussion, we primarily focus on deterministic ATI systems (1.9) and (1.10).

2.1 Controllability

Definition 2.1 (Controllability). *A system is controllable if the control input u can drive the internal state x of a system from any initial state to any other final state in a finite time interval.*

For deterministic ATI systems (1.9) and (1.10), we denote the system trajectory as $x(t) = \Phi(x(0), u(0 : t))$, which is a function of the initial state, the control inputs, and the time.¹ Then we can determine the controllability of a system by the following feasibility test:

- The system is controllable if and only if for any x_0 and any x^* , there exists a finite t^* and a control sequence $u^*(0 : t^*)$ such that

$$x^* = \Phi(x(0), u^*(0 : t^*)). \quad (2.1)$$

In other words, if we can always solve an inverse of Φ from x^* to u^* and t^* , the system is controllable.

¹ The notation here works for both discrete time systems and continuous time systems. $u(0 : t)$ denote the set of all controls applies between time 0 and time t .

Example 1: Consider the following LTI system

$$\dot{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u. \quad (2.2)$$

The system trajectory can be written as²

$$x(t) = \begin{bmatrix} \exp(t) & 0 \\ 0 & \exp(t) \end{bmatrix} x(0) + \int_{\tau=0}^t \begin{bmatrix} 0 \\ \exp(t-\tau) \end{bmatrix} u(\tau) d\tau. \quad (2.3)$$

² Refer to <http://control.ucsd.edu/mauricio/courses/mae280a/lecture5.pdf> for details on how to solve the system trajectory for LTI systems.

The control input can never affect the first state. The system is uncontrollable. In particular, the first state is uncontrollable.

Example 2: Consider another LTI system

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} x + \begin{bmatrix} 0 \\ 1 \end{bmatrix} u. \quad (2.4)$$

The system trajectory is

$$x(t) = \begin{bmatrix} 1 & \exp(t) \\ 0 & 1 \end{bmatrix} x(0) + \int_{\tau=0}^t \begin{bmatrix} \exp(t-\tau) \\ 1 \end{bmatrix} u(\tau) d\tau. \quad (2.5)$$

This system is controllable since the control u can affect both states and can manipulate the two states in different ways.

Example 3: Let us modify the system in the first example to be:

$$\dot{x} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} x + \begin{bmatrix} 1 \\ 1 \end{bmatrix} u. \quad (2.6)$$

The system trajectory now becomes

$$x(t) = \begin{bmatrix} \exp(t) & 0 \\ 0 & \exp(t) \end{bmatrix} x(0) + \int_{\tau=0}^t \begin{bmatrix} \exp(t-\tau) \\ \exp(t-\tau) \end{bmatrix} u(\tau) d\tau. \quad (2.7)$$

Although the control now affects both states, the system is still not controllable since it is not able to apply different amount of control for different states. In this way, the uncontrollable state is neither the first state or the second state in the original state vector x , but is along the direction $[1 \ -1]x$.

In the following discussion, we formally introduce the criteria of controllability for LTI systems. For simplicity, we only introduce discrete time case. The continuous time case can be derived in a similar approach.

2.1.1 Linear Controllability

Consider a discrete time LTI system $x' = Ax + Bu$. The system trajectory can be written as

$$\begin{aligned} x_0 &= x_0 \\ x_1 &= Ax_0 + Bu_0 \\ x_2 &= A^2x_0 + ABu_0 + Bu_1 \\ &\vdots \\ x_t &= A^tx_0 + \sum_{k=0}^{t-1} A^{t-1-k}Bu_k \end{aligned}$$

The feasibility test mentioned earlier involves solving the following equation:

$$x^* = A^tx_0 + \sum_{k=0}^{t-1} A^{t-1-k}Bu_k. \quad (2.8)$$

Rearranging the equation, we get

$$x^* - A^tx_0 = \underbrace{\begin{bmatrix} A^{t-1}B & A^{t-2}B & \cdots & B \end{bmatrix}}_{C_t} \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{t-1} \end{bmatrix} \quad (2.9)$$

In order to get the solution of the sequence of controls for any initial state x_0 and terminal state x^* , the matrix C_t needs to have full row rank. The requirement is formally summarized in the following theorem.

Theorem 2.1 (Controllability for discrete time deterministic LTI systems). *Consider a discrete-time LTI system $x' = Ax + Bu$ where $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times r}$. Define the controllability matrix as*

$$C_n = \begin{bmatrix} A^{n-1}B & A^{n-2}B & \cdots & B \end{bmatrix} \in \mathbb{R}^{n \times nr}. \quad (2.10)$$

The system is controllable if and only if $\text{rank}(C_n) = n$.

Remark 2.1. *The matrix C_t in (2.9) is more likely to have full row rank if we increase t , e.g., adding more columns. However, for any square matrix $A \in \mathbb{R}^{n \times n}$, A^{n+k} can be written as a linear combination of the following matrices $A^{n-1}, A^{n-2}, \dots, A, I$ for any $k \geq 0$. Hence it is useless to add more columns to the matrix C_t if $t = n$. In this way, the controllability criteria only checks C_n .*

Remark 2.2 (Controllability for continuous time deterministic LTI systems). *For a continuous time LTI system $\dot{x} = Ax + Bu$ where $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times r}$, the controllability condition is the same as that*

for the discrete time system. In particular, the controllability matrix is the same as (2.10). The continuous time system is controllable if and only if $\text{rank}(C_n) = n$. Why are they the same? An intuitive explanation is that in order to achieve arbitrary x^* , the local time derivatives $\dot{x}, \ddot{x}, \dots, \dot{x}^{(n)}, \dots$ should be able to be steered by u in all directions. We know that $\dot{x}^{(t)} = A^t x + A^{t-1} B u$. Hence in order to steer the time derivatives in all directions, we need to ensure that the following matrix $\begin{bmatrix} A^{n-1}B & A^{n-2}B & \dots & B \end{bmatrix}$ contains at least n independent column vectors; and this requirement gives us the exact same condition as in the discrete time case.

2.1.2 Uncontrollable Systems

Uncontrollability of a physical state implies that the design of the actuators is deficient. As a result, the effect of control is limited and the system can be unstable. Note that uncontrollability is different from underactuation. A state is uncontrollable means that we can never drive it to a desired value. Underactuated systems are those that cannot be commanded to follow arbitrary trajectories in configuration space, which is usually due to the fact that the system has a lower number of actuators than degrees of freedom. We may still drive an underactuated system to a desired state value, although it cannot follow arbitrary trajectories. Hence, an uncontrollable system is underactuated. But an underactuated system can be controllable.

Nonetheless, if all uncontrollable states in a system are always stable, we can still safely work with the system and try to manipulate the other states to meet the control objective. The notion of stability will be introduced in the following discussion. Here, we introduce a weaker notion of controllability.

Definition 2.2 (Stabilizability). *A system is stabilizable if all its uncontrollable states are stable.*

2.2 Observability

Definition 2.3 (Observability). *A system is observable if for any possible sequence of state x and control u , the initial state x_0 can be determined in finite time using only the outputs y .*

For deterministic ATI systems with trajectory $x(t) = \Phi(x(0), u(0 : t))$, the measurement of the system is $y(t) = h(x(t), u(t))$. Then we can determine the observability of a system by the following uniqueness test:

- For any $u(0 : t)$ and any valid observations $y(0 : t)$, there exists a finite t^* such that there is a unique solution of $x(0)$ in the

following equations

$$y(t) = h(\Phi(x(0), u(0:t)), u(t)), \forall t \in [0, t^*]. \quad (2.11)$$

In other words, if we can always solve an inverse of $h \circ \Phi$, the system is observable.

Example 1: Consider the dynamics in (2.2) with measurement function

$$y = \begin{bmatrix} a & b \end{bmatrix} x. \quad (2.12)$$

Plugging (2.3), the measurement is

$$y(t) = \begin{bmatrix} a \exp(t) & b \exp(t) \end{bmatrix} x(0) + \int_{\tau=0}^t b \exp(t-\tau) u(\tau) d\tau. \quad (2.13)$$

For any $x(0) \propto [b \ -a]$, $y(t)$ does not change. Hence, no matter what a and b are, $x(0)$ is not observable. In particular, the state that is along the direction $[b \ -a]$ is not observable.

Example 2: Consider the dynamics in (2.4) with the same measurement function (2.12). Plugging (2.5), the measurement is

$$y(t) = \begin{bmatrix} a & a \exp(t) + b \end{bmatrix} x(0) + \int_{\tau=0}^t (a \exp(t-\tau) + b) u(\tau) d\tau. \quad (2.14)$$

Then as long as $a \neq 0$, $x(0)$ can be computed from $y(t)$. This new system is observable when $a \neq 0$.

2.2.1 Linear Observability

Consider a discrete time LTI system $x' = Ax + Bu$ with the measurement $y = Cx + Du$. The measurement at different time steps can be computed as

$$\begin{aligned} y_0 &= Cx_0 + Du_0 \\ y_1 &= CAx_0 + CBu_0 + Du_1 \\ y_2 &= CA^2x_0 + CABu_0 + CBu_1 + Du_2 \\ &\vdots \\ y_t &= CA^t x_0 + \sum_{k=0}^{t-1} CA^{t-1-k} Bu_k + Du_t \end{aligned}$$

The uniqueness test mentioned earlier involves solving the above equations to determine x_0 . Rearranging the equations, we get

$$\begin{bmatrix} y_0 - Du_0 \\ y_1 - CBu_0 - Du_1 \\ y_2 - CABu_0 - CBu_1 - Du_2 \\ \vdots \\ y_t - \sum_{k=0}^{t-1} CA^{t-1-k}Bu_k - Du_t \end{bmatrix} = \underbrace{\begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^t \end{bmatrix}}_{\mathcal{O}_t} x_0$$

In order to get a unique solution of x_0 , the matrix $\mathcal{O}_t$ needs to have full column rank. The requirement is formally summarized in the following theorem.

Theorem 2.2 (Observability for discrete time deterministic LTI systems). *Consider a discrete time LTI system $x' = Ax + Bu$ and $y = Cx + Du$ where $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{n \times r}$, $C \in \mathbb{R}^{q \times n}$, and $D \in \mathbb{R}^{q \times r}$. Define the observability matrix as*

$$\mathcal{O}_n = \begin{bmatrix} C \\ CA \\ CA^2 \\ \vdots \\ CA^{n-1} \end{bmatrix} \in \mathbb{R}^{qn \times n}. \quad (2.15)$$

The system is observable if and only if $\text{rank}(\mathcal{O}_n) = n$.

Remark 2.3. The reason that we only check $\mathcal{O}_n$ instead of other $\mathcal{O}_t$ for observability is the same as in the controllability case. It is important to notice that controllability and observability are dual problems. The controllability matrix $\mathcal{C}_n$ for a system $x' = A^T x + C^T u$ is exactly the transpose of the observability matrix $\mathcal{O}_n$ for a system $x' = Ax$ and $y = Cx$. Mathematically, controllability is about existence, i.e., existence of a solution for controls; observability is about uniqueness, i.e., uniqueness of the solution for the initial state. Both controllability and observability are independent from stochasticity. Our analysis so far is based on deterministic systems.

Remark 2.4 (Observability for continuous time deterministic LTI systems). *The observability condition for continuous time systems is exactly the same as that for discrete time systems. That is mainly because observability is the dual problem for controllability; and the controllability condition for continuous time systems is exactly the same as that for discrete time systems.*

2.2.2 Unobservable Systems

If a state is unobservable, we cannot determine its value no matter how long we observe. We need to design more sensors to get addi-

tional information.

Unobservability is different from partial observability in POMDP. A system is partially observable when the agent cannot directly observe the underlying state. Instead, it must maintain a probability distribution over the set of possible states, based on a set of observations and observation probabilities, and the underlying MDP. The uncertainty in partial observability comes from stochasticity. However, observability or unobservability is an intrinsic system property, where the uncertainty does not come from stochasticity.

We may still work with unobservable systems as long as the unobservable states are stable. Here, we introduce a weaker notion of observability.

Definition 2.4 (Detectability). *A system is detectable if all the unobservable states are stable.*

2.3 Equilibrium and Stability

This section formally introduces the notion of stability. We will focus on autonomous systems. An autonomous system is a time-invariant system that does not depend on external inputs. For example, $\dot{x} = f(x)$ and $x' = f(x)$ are both autonomous systems. An autonomous system can be a closed loop system with feedback control, *i.e.*, $\dot{x} = f(x, u)$ and $u = \pi(x)$.

For an autonomous system, we are interested in whether it will rest in a target state. To answer the question, we need to analyze whether the target state is an equilibrium point for the system and whether this equilibrium point is stable.

2.3.1 Equilibrium

Definition 2.5 (System Equilibrium). *For an autonomous system, a system equilibrium is a state that any system trajectory starting from that state can only stay in that state.*

For a continuous time system $\dot{x} = f(x)$, x^* is an equilibrium if $f(x^*) = 0$. For a discrete time system $x' = f(x)$, x^* is an equilibrium if $f(x^*) = x^*$. Note that if we rewrite the discrete time system as $x' = f'(x) + x$ where $f'(x) = f(x) - x$, the equilibrium point satisfies $f'(x^*) = 0$. Then the condition for the discrete time system and the continuous time system is unified. In the following discussion, we use the notation $f(x) = 0$ to denote the condition for the equilibrium point x .

The equilibrium points x such that $f(x) = 0$ can be computed by

solving the following optimization:

$$\min_x \|f(x)\|^2. \quad (2.16)$$

Denote $J(x) := \nabla_x f(x)$. The optimization can be solved iteratively by

- Newton methods: $x_{i+1} = x_i + \alpha J^{-1}(x_i)f(x_i)$;
- Steepest descent: $x_{i+1} = x_i - \beta J^T(x_i)f(x_i)$;

where $\alpha \in \mathbb{R}^+$ and $\beta \in \mathbb{R}^+$ are constants.

Theorem 2.3 (Global Implicit Function Theorem). *The equilibrium point is unique, i.e., $f(x) = 0$ has one unique solution, if*

- $\det [J(x)] \neq 0, \forall x$;
- $\|f(x)\| \rightarrow \infty$ as $\|x\| \rightarrow \infty$.

2.3.2 Stability

In the following discussion, we assume the origin is one system equilibrium, i.e., $f(0) = 0$.

Definition 2.6 (Stability). *A system is stable at origin if and only if for any $\epsilon > 0$, there exist a $\delta > 0$ such that for all initial states at t_0 that satisfy $\|x_0\| < \delta$, $\|x(t)\| < \epsilon$ for all $t > t_0$.*

Definition 2.7 (Asymptotically Stability). *A system is globally asymptotically stable at origin if and only if 1) the system is stable; 2) for any initial condition x_0 and for any $\epsilon > 0$, there exist a time $t^* > t_0$ such that $\|x(t)\| < \epsilon$ for all $t > t^*$.*

If a system is not globally asymptotically stable, it may still be locally converging to the equilibrium. We call the locally converging region as the region of attraction.

Definition 2.8 (Region of Attraction). *Region of attraction of a stable equilibrium is a set of initial states under which the system will eventually reach the equilibrium.*

Remark 2.5. *A stable system may not always converge to an equilibrium. For example, consider a system*

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} x. \quad (2.17)$$

The trajectory of this system starting from any x_0 is on a circle centered at the origin with radius $\|x_0\|$. On the other hand, a system whose trajectories

are all converging to an equilibrium may not be stable according to the above $\epsilon - \delta$ condition. For example, consider a system

$$\dot{x} = \begin{bmatrix} x_1^2 - x_2^2 \\ 2x_1x_2 \end{bmatrix}. \quad (2.18)$$

The trajectory of this system will eventually converge to 0 if we identify $+\infty$ and $-\infty$. But for any $\delta > 0$, there is a trajectory from $x = [\delta; 0]$ that goes to infinity.

2.3.3 Stability for Linear Systems

Consider a continuous time linear system $\dot{x} = Ax$ where $A \in \mathbb{R}^{n \times n}$. The system trajectory is $x(t) = x(0) \exp(At)$. The stability of the system depends on the exponential term $\exp(At)$. Suppose A has n distinct complex eigen values $\lambda_1, \dots, \lambda_n$. And the associated eigen vectors are $v_1, \dots, v_n$.³ By similarity transform, the exponential term can be written as

$$e^{At} = V \text{diag}(e^{\lambda_1 t}, e^{\lambda_2 t}, \dots, e^{\lambda_n t}) V^{-1}, \quad (2.19)$$

where $V = [v_1, \dots, v_n]$. Hence, the stability of the system depends on $e^{\lambda_i t}$. By Euler's formula, $e^{\lambda_i t} = e^{\text{Re}(\lambda_i)t} [\cos(\text{Im}(\lambda_i)t) + i \sin(\text{Im}(\lambda_i)t)]$ where $\text{Re}(\lambda_i)$ takes the real part of λ_i and $\text{Im}(\lambda_i)$ takes the imaginary part. According to definition 2.6, the linear system $\dot{x} = Ax$ is stable if and only if $\text{Re}(\lambda_i) \leq 0$ for all i . Moreover, According to definition 2.7, the linear system $\dot{x} = Ax$ is globally asymptotically stable if and only if $\text{Re}(\lambda_i) < 0$ for all i . Moreover, the system will have exponential delay.

Consider a discrete time linear system $x' = Ax$ where $A \in \mathbb{R}^{n \times n}$. The system trajectory is $x_k = A^k x_0$. The stability of the system depends on A^k . Similarly, we assume that A has n distinct complex eigen values $\lambda_1, \dots, \lambda_n$ with the associated eigen vectors $v_1, \dots, v_n$. Then,

$$A^k = V \text{diag}(\lambda_1^k, \lambda_2^k, \dots, \lambda_n^k) V^{-1}. \quad (2.20)$$

According to definition 2.6, the linear system $x' = Ax$ is stable if and only if $|\lambda_i| \leq 1$ for all i . Moreover, According to definition 2.7, the linear system $x' = Ax$ is globally asymptotically stable if and only if $|\lambda_i| < 1$ for all i .

2.3.4 Stability for Nonlinear Systems

The stability for nonlinear systems can be analyzed using Lyapunov 1st method, which essentially applies linear stability locally on linearized systems. For the nonlinear system $\dot{x} = f(x)$, we linearize the

³ By definition, $Av_i = \lambda_i v_i$ for $i = 1, \dots, n$. Note it is possible that the matrix A has duplicated eigen values. Refer to linear matrix theory for more details of general eigen value decomposition.

dynamics around the equilibrium point x_e :

$$\delta\dot{x} = J(x_e)\delta x + h(x_e, \delta x). \quad (2.21)$$

Then linear stability analyses apply locally if the following two conditions are satisfied:

- All eigen-values of $J(x_e) = \nabla_x f(x_e)$ have non-zero real parts;
- The error term decays faster than first order decay:

$$\lim_{\|\delta x\| \rightarrow 0} \frac{\|h(x_e, \delta x)\|}{\|\delta x\|} \rightarrow 0. \quad (2.22)$$

Example 1: Consider a system $\dot{x} = -\sin(x)$ which has an equilibrium at 0. The linearized dynamics are $\dot{x} \approx -x$. The two conditions above are all satisfied. According to linear stability, the linearized the system is asymptotically stable. Hence, for the nonlinear system, the equilibrium $x = 0$ is locally asymptotically stable.

Example 2: Consider a system $\dot{x} = (x - x_e)^2$. The equilibrium point is x_e . The Jacobian $J(x_e) = 0$ violates the first condition for Lyapunov 1st method. Hence the linear stability condition does not apply. And the system is actually unstable.

Example 3: Consider a system $\dot{x} = \text{mod}(1 - x, 1)$ which has equilibria at any integer $x \in \mathbb{N}$. However, the dynamics are not differentiable at those equilibria, hence violating the second condition in the Lyapunov 1st method. The linear stability condition does not apply.

For those systems that cannot be analyzed by the Lyapunov 1st method, we will introduce nonlinear stability analysis methods that use scalar energy functions, *e.g.*, Lyapunov 2nd method.

2.4 General Properties of Nonlinear Systems

A systems $\dot{x} = f(x)$ is nonlinear if it does not satisfy the following two properties:

- Superposition $f(x + y) = f(x) + f(y)$;
- Homogeneity $f(kx) = kf(x)$.

Typical single-valued time-invariant nonlinearities include

- Saturation: $f(x) = \min(x, \bar{x})$ where $\bar{x} > 0$ is an upper bound;
- Friction: $f(x) = x + b\text{sign}(x)$ where the function is discontinuous at $x = 0$;

- Dead zone: $f(x) = \max(x, 0)$ where the function is zero for all negative x .

One typical time-varying nonlinearity is hysteresis. A hysteresis system has “memory” in the sense that the previous trajectory affects the future evolution of the state. In general, a nonlinearity with “memory” means that we have not included all states that define the nonlinearity.

Another important class of nonlinearity is Chaos: when the present determines the future, the approximate present does not approximately determine the future. A small change in initial conditions can result in huge change in the future. Chaos is different from uncertainty from stochasticity. For example, for a deterministic double-rod pendulum system, starting the pendulum from a slightly different initial condition would result in a completely different trajectory. The double-rod pendulum is one of the simplest dynamical systems with chaotic solutions.

In the real world, nonlinear systems appear in various domains. In mechanical engineering, a pendulum is a nonlinear system. In electrical engineering, circuits can be models as nonlinear systems. In computer science, AI systems are highly nonlinear. In economics, stock market can be analyzed as nonlinear systems. In social science, social networks may also be studied as nonlinear systems. The dimensionality of these systems grows exponentially in these examples.

In general, nonlinear systems are very different from linear systems. A linear system will have one unique equilibrium, while a nonlinear system tends to have multiple equilibrium points. For linear systems, there is no difference between local stability and global stability, *i.e.*, local stability implies global stability. For nonlinear systems, local stability does not imply global stability, and it is very difficult to have global stability guarantees for nonlinear systems. For linear systems, due to superposition and homogeneity, the system response characteristics ($x(t)/x(0)$) are identical to all inputs conditions. However, the system response characteristics of nonlinear systems depend upon initial conditions, input type and input magnitude.

Part II

Optimal Control

3

Optimal Control - Finite Time

This chapter discusses finite time deterministic optimal control in discrete time and continuous time. We will introduce dynamic programming and maximum principle in both cases. We will also derive linear quadratic regulators and the corresponding Riccati Equations in both cases.

3.1 Discrete Time Optimal Control

The problem that we are trying to solve is an optimization problem that tries to minimize the specified cost function over the given finite horizon. Mathematically, this optimization problem may be written as:

$$\begin{aligned} \min_{\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1}} \quad & J = \sum_{k=0}^{N-1} l_k(\mathbf{x}_k, \mathbf{u}_k) + l_N(\mathbf{x}_N), \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = f(\mathbf{x}_k, \mathbf{u}_k) \end{aligned} \quad (3.1)$$

and finding the solution of this problem is the primary motivation of this section. For simplicity, we denote the cost function in (3.1) as $J_0(\mathbf{x}_0; \mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{N-1})$ since the time starts at 0; the initial condition is $\mathbf{x}_0$; and the function depends on the control inputs.

We designate $\{\mathbf{u}_0^*, \mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*\}$ as an **Optimal Control Sequence** if the sequence produces the minimum cost of the defined cost function in (3.1). Given an initial state $\mathbf{x}_0$, this optimal control sequence can be used to form a corresponding **Optimal State Sequence**: $\{\mathbf{x}_0^*, \mathbf{x}_1^*, \dots, \mathbf{x}_N^*\}$. Note that the optimal control sequence only goes up to time step $N - 1$ while the optimal state sequence includes a value for time step N due to the transition function (3.1) solving for the state at time step $k + 1$. These two sequences also have a corresponding **Optimal Cost** denoted as: $J^*(\mathbf{x}_0)$.

Now, let us describe the problem in more detail. Note that for the purposes of this chapter, we make the following assumptions:

- the controller can use full state feedback;

- there is no discount factor, *i.e.*, $\delta = 1$ (or we can say that the discount factor is absorbed into the runtime cost l_k);
- the system is controllable which a designer should check before starting the design process using tools described in earlier chapters.

Information that we know includes the initial state of the system $\mathbf{x}_0$ and the cost associated with the optimization problem (both runtime cost l_k and terminal cost l_n). This information is known due to the fact that a system starts from a specific state, that we assume we know, and that the designer designs these costs (or they can be modeled under principle) to achieve certain system performance.

The solution to the optimization problem, the optimal control sequence, may be encoded in multiple ways. If the sequence is solved directly as numerical values and remains in that form, we refer to this as an **Open-loop solution** as it yields a sequence of controls that must be wholly recomputed if there is any change in the initial state. This differs from the **Closed-loop solution** as in a closed-loop solution, we actually define a control policy, $\mathbf{u}_k = q_k(\mathbf{x}_k)$, $\forall k$ which governs the system response. For those familiar with control theory, the benefits and detriments of both methods correspond directly with open-loop and closed-loop classical control, which is from where these approaches derive their name. For the purposes of this class, note that closed-loop solutions may be more difficult to generate depending on the problem, but are often more robust to disturbances given their continual evaluation at each step.

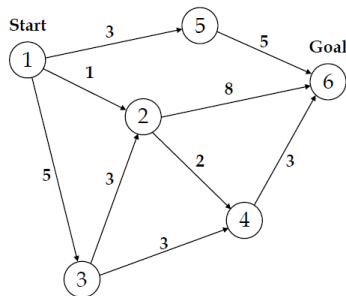


Figure 3.1: An example graph for the shortest path problem.

Example 1: Shortest Path Problem We can frame the task of finding the shortest path as an optimal control problem by accurately modeling the problem states and actions, specifying the costs, and determining an end condition. For the example shortest path problem, consider the following formulation:

1. State Space: Nodes

2. Action Space: Edges
3. Cost: Distance Traveled
4. Termination: Event Triggered (reach the final node)

We will show how to solve this problem in the following discussion.

3.1.1 Dynamic Programming

As our goal is to find a globally optimal solution, should one exist, for our particular problem, let us first consider some implications of what it means for a solution to be "optimal".

We investigate the properties of optimal solutions by first considering a subproblem of our original optimal control problem. Consider that we start at state, $\mathbf{x}_k^*$, which is the state at time k , and we want to minimize the cost-to-go component of our cost, J , from time k , our starting time, to time N , our ending time, using a sequence of controls $\{\mathbf{u}_k, \dots, \mathbf{u}_{N-1}\}$. We can find the minimum cost through the following optimization problem,

$$\min_{\mathbf{u}_k, \dots, \mathbf{u}_{N-1}} l_k(\mathbf{x}_k^*, \mathbf{u}_k) + \sum_{m=k+1}^{N-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + l_N(\mathbf{x}_N) \rightarrow J_k(\mathbf{x}_k^*; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}), \quad (3.2)$$

the solution to which yields a sequence of control that is optimal for this subproblem. We will show in the following discussion that the truncated optimal control sequence $\{\mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*\}$ is one optimal solution.¹ For example, for the shortest path problem, the optimal path sequence when starting at Node 1 and given the goal state of Node 6 is $\{1, 2, 4, 6\}$. This should be read as we are traveling from Node 1 to Node 2 and so on, ending at Node 6. If we start from Node 2, the optimal path is then $\{2, 4, 6\}$ as shown in fig. 3.2.

¹ We say that the truncated optimal control sequence is "one" optimal solution mainly because there may exist multiple optimal solutions.

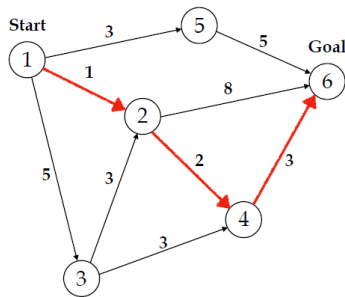


Figure 3.2: The Optimal Solution to the Shortest Path Problem

The fact that any segment of an optimal trajectory is optimal is still optimal is called the **Principle of Optimality**, also known as the **Bellman Principle**. To put this another way, an optimal trajectory will

not have any sub-optimal components. One key word of warning is that the inverse is not true, the summation of individually optimized trajectories does not necessarily yield an optimal trajectory. We now prove the Principle of Optimality via proof by contradiction.

Theorem 3.1 (Principle of Optimality). *Suppose the optimal control sequence for (3.1) is $\{\mathbf{u}_0^*, \mathbf{u}_1^*, \dots, \mathbf{u}_{N-1}^*\}$. Then $\{\mathbf{u}_k^*, \mathbf{u}_{k+1}^*, \dots, \mathbf{u}_{N-1}^*\}$ is the optimal solution for (3.2) starting at $\mathbf{x}_k^*$ for any $k = 1, \dots, N - 1$.*

Proof. Suppose the truncated control sequence $\{\mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*\}$ is not optimal for (3.2). In fact, there exists another control sequence $\{\mathbf{u}_k^0, \dots, \mathbf{u}_{N-1}^0\}$ that results in a lower cost for (3.2). In other words,

$$J_k(\mathbf{x}_k^*; \mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*) > J_k(\mathbf{x}_k^*; \mathbf{u}_k^0, \dots, \mathbf{u}_{N-1}^0).$$

We develop two control sequences by combining the optimal control sequence before time step k with $\{\mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*\}$ and $\{\mathbf{u}_k^0, \dots, \mathbf{u}_{N-1}^0\}$, yielding:

$$\mathbf{u}^1 = \{\mathbf{u}_0^*, \dots, \mathbf{u}_{N-1}^*\},$$

and

$$\mathbf{u}^2 = \{\mathbf{u}_0^*, \dots, \mathbf{u}_{k-1}^*, \mathbf{u}_k^0, \dots, \mathbf{u}_{N-1}^0\}.$$

The total cost of a control sequence is given as

$$J_0(\mathbf{x}_0; \mathbf{u}_0, \dots, \mathbf{u}_{N-1}) = \sum_{m=0}^{k-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}).$$

Then we will have the following relationship between $J_0(\mathbf{x}_0; \mathbf{u}^1)$ and $J_0(\mathbf{x}_0; \mathbf{u}^2)$:

$$\begin{aligned} J_0(\mathbf{x}_0; \mathbf{u}^1) &= \sum_{m=0}^{k-1} l_m(\mathbf{x}_m^*, \mathbf{u}_m^*) + J_k(\mathbf{x}_k^*; \mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*) \\ &> \sum_{m=0}^{k-1} l_m(\mathbf{x}_m^*, \mathbf{u}_m^*) + J_k(\mathbf{x}_k^*; \mathbf{u}_k^0, \dots, \mathbf{u}_{N-1}^0) \\ &= J_0(\mathbf{x}_0; \mathbf{u}^2) \end{aligned}$$

which contradicts the fact that $\mathbf{u}^1$ is an optimal control sequence for the entire problem. Hence the truncated control sequence $\{\mathbf{u}_k^*, \dots, \mathbf{u}_{N-1}^*\}$ must be an optimal solution for (3.2). $\square$

With the principle of optimality, we can now derive the equations for dynamic programming, an approach to solve optimal control problems. Consider, again, the cost function of an optimal control problem that has been broken into two components, one ending at time step $k - 1$ and one beginning at time step k . The cost function for the two components of the optimal control problem is given as

$$J_0(\mathbf{x}_0; \mathbf{u}_0, \dots, \mathbf{u}_{N-1}) = \sum_{m=0}^{k-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}). \quad (3.3)$$

In order to leverage this form for the design of an optimal control system, we consider the problem at hand. As we wish to minimize the total cost with respect to a sequence of control, $\{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}\}$, we take a minimization on both sides of (3.3),

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} J_0(\mathbf{x}_0; \mathbf{u}_0, \dots, \mathbf{u}_{N-1}) = \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} \left\{ \sum_{m=0}^{k-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}) \right\}. \quad (3.4)$$

We can further simplify this expression by recognizing that the first term of the cost function on the left-hand side is independent of control actions which occur after the time step corresponding to a specific element of time. For example, the run-time cost of the system at time step $k-1$ is not affected by the control action taken at time step k . In this way, we can rewrite (3.4) as,

$$\min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} J_0(\mathbf{x}_0; \mathbf{u}_0, \dots, \mathbf{u}_{N-1}) = \min_{\mathbf{u}_0, \dots, \mathbf{u}_{k-1}} \left\{ \sum_{m=0}^{k-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + \min_{\mathbf{u}_k, \dots, \mathbf{u}_{N-1}} J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}) \right\}. \quad (3.5)$$

Note that we cannot separate the minimization problem into two independent minimization problems due to the fact that the starting state in the subproblem is dependent on the control actions taken before it. This is clear through the dynamic equation in (3.1), as the evolution of the state is dependent on the control action taken at earlier steps. While we may not be able to separate (3.5) into two separate minimization problems, we can solve the two minimization problems sequentially. Denoting the minimum cost of the optimization problem as,

$$J_0^*(\mathbf{x}_0) = \min_{\mathbf{u}_0, \dots, \mathbf{u}_{N-1}} J_0(\mathbf{x}_0; \mathbf{u}_0, \dots, \mathbf{u}_{N-1}) \quad (3.6)$$

we may rephrase (3.5) as,

$$J_0^*(\mathbf{x}_0) = \min_{\mathbf{u}_0, \dots, \mathbf{u}_{k-1}} \left\{ \sum_{m=0}^{k-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + J_k^*(\mathbf{x}_k) \right\}, \quad (3.7)$$

where

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k, \dots, \mathbf{u}_{N-1}} J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}). \quad (3.8)$$

We refer to $J_k^*(\mathbf{x}_k)$ as the **optimal cost-to-go function**. It is defined this way as it is the optimal cost to go from state at time step k to the state at the final time step. This allows us to develop a recurrence relation that is used to solve dynamic programming problems. To see this recurrence, consider (3.7) when $k = 1$,

$$J_0^*(\mathbf{x}_0) = \min_{\mathbf{u}_0} \{l_0(\mathbf{x}_0, \mathbf{u}_0) + J_1^*(\mathbf{x}_1)\}, \quad (3.9)$$

and thus by induction

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{x}_{k+1})\} \quad \forall k = 0, 1, \dots, N-1, \quad (3.10)$$

with the terminal condition being,

$$J_N^*(\mathbf{x}_N) = l_N(\mathbf{x}_N). \quad (3.11)$$

Cost-to-go vs Value function We can generalize the optimal cost-to-go expression to a standard **cost-to-go** expression by dropping the requirement that only optimal actions will be taken in the future. To highlight this distinction, and hopefully make referring to the difference between an optimal cost-to-go and a regular cost-to-go expression slightly easier, we rename the optimal cost-to-go as a **value function** at a specific time step, k . As the optimal cost-to-go can only be achieved by taking optimal actions, and these optimal actions are determined solely from a specific starting state, this makes the value function dependent solely on that specific starting state as shown in (3.8). We then choose to define a **cost-to-go function** at time step k as,

$$J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}) = \sum_{m=k}^{N-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + l_N(\mathbf{x}_N), \quad (3.12)$$

where, as we have not chosen to specify that all future control inputs are optimal, this function is still dependent on all future control inputs. We relate the value function to the cost-to-go function by recognizing that the minimum cost-to-go which corresponds to the optimal control sequence yields the value function, thus

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k, \dots, \mathbf{u}_{N-1}} J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}). \quad (3.13)$$

Bellman Equation The Bellman Equation is an update rule for value functions, which is the key idea behind dynamic programming. This relationship is occasionally referred to as the **Bellman update**, and we are going to see a number of applications of the Bellman equation in different scenarios. Mathematically, we express the Bellman Equation as

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{x}_{k+1})\}. \quad (3.14)$$

The general form of the Bellman equation is that we are optimizing some expression by enacting a specific action or control. The expression that we are optimizing, the optimization objective, is the run-time cost at a certain time step plus the value function defined at the next state.

Dynamic Programming for Discrete Finite Time Optimal Control We recursively solve for an optimal control sequence using the Bellman equation. At time step N , we have

$$J_N^*(\mathbf{x}_N) = l_N(\mathbf{x}_N), \quad (3.15)$$

and working backwards in time, $k = N - 1, N - 2, \dots, 1, 0$, we solve the following optimization problem

$$J_k^*(\mathbf{x}_k) = \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{x}_{k+1})\}. \quad (3.16)$$

From this, we can find the optimal control policy

$$q_k^*(\mathbf{x}_k) = \arg \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{x}_{k+1})\}. \quad (3.17)$$

Note that from this backward pass, we only obtain the control policy as a mapping from the state to the control. In order to find the optimal control sequence, we must provide an initial condition (the initial state, $\mathbf{x}_0$), from which we can develop the control sequence.

$$\mathbf{x}_0^* = \mathbf{x}_0 \quad (3.18a)$$

$$\mathbf{u}_k^* = q_k^*(\mathbf{x}_k^*), \forall k = 0, 1, \dots, N - 1 \quad (3.18b)$$

$$\mathbf{x}_{k+1}^* = f(\mathbf{x}_k^*, \mathbf{u}_k^*), \forall k = 0, 1, \dots, N - 1 \quad (3.18c)$$

Example 1: Dynamic Programming for Shortest Path Problem [Check slides]

3.1.2 Maximum Principle

For value functions on continuous state spaces, they usually need to be parameterized before we can run the Bellman equation. Now let us consider the following linear parameterization with a co-state λ_k which is defined to be the gradient of J_k^* , i.e., $\lambda_k := \nabla_x J_k^*(\mathbf{x}_k)$,

$$J_k^*(\mathbf{x}_k) = \lambda_k^T \mathbf{x}_k + \gamma_k, \quad (3.19)$$

where γ_k is the residual term.

Leveraging this parameterization, we can rewrite the Bellman Equation (3.14) using the suggested parameterization:

$$\lambda_k^T \mathbf{x}_k + \gamma_k = \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + \lambda_{k+1}^T \mathbf{x}_{k+1} + \gamma_{k+1}\}, \quad (3.20)$$

which can be further rewritten using the dynamic equation:

$$\lambda_k^T \mathbf{x}_k + \gamma_k = \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + \lambda_{k+1}^T f(\mathbf{x}_k, \mathbf{u}_k) + \gamma_{k+1}\}, \quad (3.21)$$

Hamiltonian We define the **Hamiltonian** as the first two terms in side the minimization in (3.21):

$$H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = l_k(\mathbf{x}_k, \mathbf{u}_k) + \lambda_{k+1}^T f(\mathbf{x}_k, \mathbf{u}_k). \quad (3.22)$$

which is equivalent to the cost-to-go function $J_k(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{N-1}) = \sum_{m=k}^{N-1} l_m(\mathbf{x}_m, \mathbf{u}_m) + l_N(\mathbf{x}_N)$ (by a difference of γ_{k+1}). However, unlike

the cost-to-go function, all the future control (decision) variables are considered by λ_{k+1} , which we called as the **co-state**. It is through an exact parameterization that we can rephrase the dynamic programming problem. Assuming all functions are smooth in the state, the optimal solution of the Bellman equation should satisfy the following relation

$$\lambda_k = \frac{\partial}{\partial x} H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = \frac{\partial}{\partial x} l_k(\mathbf{x}_k, \mathbf{u}_k) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_k, \mathbf{u}_k) \right]^T \lambda_{k+1} \quad (3.23a)$$

$$\mathbf{u}_k = \arg \min_{\mathbf{u}} H_k(\mathbf{x}_k, \mathbf{u}, \lambda_{k+1}) \quad (3.23b)$$

Dynamic Programming with Co-States At time step N , we find a co-state function $\lambda_N(\mathbf{x}_N)$ and a constant γ_N that satisfy

$$\lambda_N(\mathbf{x}_N) + \gamma_N = l_N(\mathbf{x}_N), \quad (3.24)$$

and from there, we solve for all time steps backwards in time, $k = N - 1, N - 2, \dots, 1, 0$, the set of equations given in (3.23). Note that λ_{k+1} itself may be a function of x_{k+1} , which is then a function on $\mathbf{x}_k$ and $\mathbf{u}_k$. When we solve this series of equations for every time step, we obtain the optimal control policy $\mathbf{u}_k = q_k^*(\mathbf{x}_k)$ and a co-state update function $\lambda_k(\mathbf{x}_k)$. We additionally need to recover the constant term, γ_k . The recovery of this term is described more in the Linear Quadratic Regulator section. The optimal control sequence and co-states can thereby be obtained in a forward pass considering the initial state $\mathbf{x}_0$ using the following equations

$$\mathbf{x}_0^* = \mathbf{x}_0, \quad (3.25a)$$

$$\mathbf{u}_k^* = q_k^*(\mathbf{x}_k^*), \quad \forall k = 0, 1, \dots, N - 1 \quad (3.25b)$$

$$\lambda_k^* = \lambda_k(\mathbf{x}_k^*), \quad \forall k = 0, 1, \dots, N \quad (3.25c)$$

$$\mathbf{x}_{k+1}^* = f(\mathbf{x}_k^*, \mathbf{u}_k^*), \quad \forall k = 0, 1, \dots, N - 1. \quad (3.25d)$$

Interpretations of Co-State Now, is there any other interpretation of the co-state? Besides being a way to parameterize the value function, the co-state can be considered to represent a shadow price or a Lagrange multiplier. The shadow price may be thought of as the “unit cost” of a current state value.

Maximum Principle: A Necessary Condition Finally, we must also talk about the Maximum Principle in order to understand the connection between state and co-state trajectories.

Theorem 3.2 (Maximum Principle). *If $\{\mathbf{u}_0^*, \dots, \mathbf{u}_{N-1}^*\}$ and $\{\mathbf{x}_1^*, \dots, \mathbf{x}_N^*\}$ are the optimal state and control trajectories when starting at $\mathbf{x}_0$, then there*

exists a co-state trajectory $\{\lambda_1^*, \dots, \lambda_N^*\}$ with $\lambda_N^* = \frac{\partial}{\partial \mathbf{x}_N} J_N^*(\mathbf{x}_N^*)$ satisfying

$$\mathbf{x}_{k+1}^* = \frac{\partial}{\partial \lambda} H_k(\mathbf{x}_k^*, \mathbf{u}_k^*, \lambda_{k+1}^*) = f(\mathbf{x}_k^*, \mathbf{u}_k^*), \quad (3.26a)$$

$$\lambda_k^* = \frac{\partial}{\partial \mathbf{x}} H_k(\mathbf{x}_k^*, \mathbf{u}_k^*, \lambda_{k+1}^*) = \frac{\partial}{\partial \mathbf{x}} l_k(\mathbf{x}_k^*, \mathbf{u}_k^*) + \left[\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}_k^*, \mathbf{u}_k^*) \right]^T \lambda_{k+1}^*, \quad (3.26b)$$

$$\mathbf{u}_k^* = \arg \min_u H_k(\mathbf{x}_k^*, \mathbf{u}, \lambda_{k+1}^*). \quad (3.26c)$$

The Maximum Principle is a necessary condition, not a sufficient condition and solutions that we obtain in the *Dynamic Programming with Co-States* section always satisfy the maximum principle.

Example: Investment Problem [Check slides]

3.1.3 Linear Quadratic Regulator

In this section, we apply the Maximum Principle to solve the linear quadratic regulator problem. The problem is called linear quadratic mainly because the system dynamics are linear:

$$\mathbf{x}_{k+1} = A_k \mathbf{x}_k + B_k \mathbf{u}_k, \quad (3.27)$$

and the cost function is quadratic:

$$J = \frac{1}{2} \sum_{k=0}^{N-1} [\mathbf{x}_k^T Q_k \mathbf{x}_k + \mathbf{u}_k^T R_k \mathbf{u}_k] + \frac{1}{2} \mathbf{x}_N^T S_N \mathbf{x}_N. \quad (3.28)$$

In equations (3.27) (3.28), $Q_0, \dots, Q_{N-1}, S_N$ are symmetric positive semi-definite, $R_0, \dots, R_{N-1}$ is symmetric positive definite, and A_k, B_k are controllable.

From here, we form the Hamiltonian for the system

$$H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = \frac{1}{2} \mathbf{x}_k^T Q_k \mathbf{x}_k + \frac{1}{2} \mathbf{u}_k^T R_k \mathbf{u}_k + \lambda_{k+1}^T (A_k \mathbf{x}_k + B_k \mathbf{u}_k), \quad (3.29)$$

and then use the Maximum principle to define our co-state dynamics and optimal control values. Note that as this is an unconstrained optimization problem, we can set the left-hand side of the control equation, (3.31), equal to zero, which is not always possible in the constrained control problem.

$$\lambda_k = \frac{\partial}{\partial \mathbf{x}} H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = Q_k \mathbf{x}_k + A_k^T \lambda_{k+1}, \quad (3.30)$$

$$0 = \frac{\partial}{\partial \mathbf{u}} H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = R_k \mathbf{u}_k + B_k^T \lambda_{k+1}. \quad (3.31)$$

Boundary Condition We need to specify a set of boundary conditions to perform our dynamic programming on, thus using a target set for the last state, we specify the terminal cost as

$$\begin{aligned} J_N^*(\mathbf{x}_N) &= \frac{1}{2} \mathbf{x}_N^T S_N \mathbf{x}_N \\ \lambda_N &= \frac{\partial}{\partial \mathbf{x}} J_N^*(\mathbf{x}_N) = S_N \mathbf{x}_N \\ \gamma_N &= -\frac{1}{2} \mathbf{x}_N^T S_N \mathbf{x}_N. \end{aligned} \quad (3.32)$$

From N to $N-1$ As per our discussion over dynamic programming, we start at the last step and solve backwards (in this instance, from the last time step to the first time step). Thus we start with the co-state relation at time $N-1$

$$\lambda_{N-1} = Q_{N-1} \mathbf{x}_{N-1} + A_{N-1}^T \lambda_N. \quad (3.33)$$

We can see that (3.33) is a function of λ_N , which is given by our boundary condition: $\lambda_N = S_N \mathbf{x}_N$. We make the boundary condition replacement in (3.33) and substitute $\mathbf{x}_N$ with $A_{N-1} \mathbf{x}_{N-1} + B_{N-1} \mathbf{u}_{N-1}$:

$$\lambda_{N-1} = Q_{N-1} \mathbf{x}_{N-1} + A_{N-1}^T S_N A_{N-1} \mathbf{x}_{N-1} + A_{N-1}^T S_N B_{N-1} \mathbf{u}_{N-1}. \quad (3.34)$$

Meanwhile, we can use this same substitutions to build an expression determining our control at a given time step

$$\begin{aligned} 0 &= R_{N-1} \mathbf{u}_{N-1} + B_{N-1}^T \lambda_N, \\ &= R_{N-1} \mathbf{u}_{N-1} + B_{N-1}^T S_N A_{N-1} \mathbf{x}_{N-1} + B_{N-1}^T S_N B_{N-1} \mathbf{u}_{N-1}. \end{aligned} \quad (3.35)$$

Let's start with investigating the equation which calculates the control of our system at a specific time step. Rearrange the last line of (3.35) to get

$$0 = [B_{N-1}^T S_N B_{N-1} + R_{N-1}] \mathbf{u}_{N-1} + B_{N-1}^T S_N A_{N-1} \mathbf{x}_{N-1}, \quad (3.36)$$

and now solve for $\mathbf{u}_{N-1}$, yielding

$$\mathbf{u}_{N-1} = - \underbrace{[B_{N-1}^T S_N B_{N-1} + R_{N-1}]^{-1} B_{N-1}^T S_N A_{N-1}}_{K_{N-1}} \mathbf{x}_{N-1}. \quad (3.37)$$

This expression shows that the optimal control for this linear system with a quadratic function is just a linear feedback controller! We are assured to be able to take the inverse in (3.37) due to the positive (semi-)definiteness assumptions on S_N and R_{N-1} .

In order to simplify our co-state update rule, we replace our control term, $\mathbf{u}_{N-1}$ in (3.34), with $-K_{N-1} \mathbf{x}_{N-1}$. This yields

$$\lambda_{N-1} = Q_{N-1} \mathbf{x}_{N-1} + A_{N-1}^T S_N A_{N-1} \mathbf{x}_{N-1} - A_{N-1}^T S_N B_{N-1} K_{N-1} \mathbf{x}_{N-1}. \quad (3.38)$$

We can rearrange (3.38) to

$$\lambda_{N-1} = \underbrace{[Q_{N-1} + A_{N-1}^T S_N A_{N-1} - A_{N-1}^T S_N B_{N-1} K_{N-1}]_{P_{N-1}}}_{P_{N-1}} \mathbf{x}_{N-1}, \quad (3.39)$$

which gives us the relation between state and co-state at time step $N - 1$. To simplify the expression for P_{N-1} , we can expand K_{N-1}

$$P_{N-1} = Q_{N-1} + A_{N-1}^T P_N A_{N-1} - A_{N-1}^T P_N B_{N-1} [B_{N-1}^T P_N B_{N-1} + R_{N-1}]^{-1} B_{N-1}^T P_N A_{N-1}, \quad (3.40)$$

where we define $P_N := S_N$. Thus, we have found an explicit connection between the state and co-state, which allows the user to go between the two easily.

Now, consider the value function update for the LQR problem

$$J_{N-1}^*(\mathbf{x}_{N-1}) = \frac{1}{2} \mathbf{x}_{N-1}^T Q_{N-1} \mathbf{x}_{N-1} + \frac{1}{2} \mathbf{u}_{N-1}^T R_{N-1} \mathbf{u}_{N-1} + \frac{1}{2} \mathbf{x}_{N-1}^T P_N \mathbf{x}_{N-1}. \quad (3.41)$$

We can substitute the expression to calculate control, (3.37), in this expression, as well as substituting the dynamics $\mathbf{x}_N = A_{N-1} \mathbf{x}_{N-1} + B_{N-1} \mathbf{u}_{N-1}$,

$$\begin{aligned} J_{N-1}^*(\mathbf{x}_{N-1}) &= \frac{1}{2} \mathbf{x}_{N-1}^T Q_{N-1} \mathbf{x}_{N-1} + \frac{1}{2} \mathbf{x}_{N-1}^T K_{N-1}^T R_{N-1} K_{N-1} \mathbf{x}_{N-1} \\ &\quad + \frac{1}{2} \mathbf{x}_{N-1}^T [A_{N-1} - B_{N-1} K_{N-1}]^T P_N [A_{N-1} - B_{N-1} K_{N-1}] \mathbf{x}_{N-1}. \end{aligned} \quad (3.42)$$

Finally, we aim to show that $J_{N-1}^*(\mathbf{x}_{N-1}) = \frac{1}{2} \mathbf{x}_{N-1}^T P_{N-1} \mathbf{x}_{N-1}$ by showing that

$$P_{N-1} = Q_{N-1} + K_{N-1}^T R_{N-1} K_{N-1} + [A_{N-1} - B_{N-1} K_{N-1}]^T P_N [A_{N-1} - B_{N-1} K_{N-1}], \quad (3.43)$$

[skipped, check slides]

Thus, we can verify that

$$J_{N-1}^*(\mathbf{x}_{N-1}) = \frac{1}{2} \mathbf{x}_{N-1}^T P_{N-1} \mathbf{x}_{N-1}, \quad (3.44)$$

from which we conclude that

$$\gamma_{N-1} = J_{N-1}^*(\mathbf{x}_{N-1}) - \lambda_{N-1}^T \mathbf{x}_{N-1} = -\frac{1}{2} \mathbf{x}_{N-1}^T P_{N-1} \mathbf{x}_{N-1}. \quad (3.45)$$

From (3.45), we see that the co-state provides first-order information, but to fully describe the value function, we also need to compute the zero-order term.

Induction from $k + 1$ to k The equations used from N to $N - 1$ are summarized below

$$\begin{aligned}
 \mathbf{u}_{N-1} &= -[B_{N-1}^T P_N B_{N-1} + R_{N-1}]^{-1} B_{N-1}^T P_N A_{N-1} \mathbf{x}_{N-1}, \\
 P_{N-1} &= Q_{N-1} + A_{N-1}^T P_N A_{N-1} - A_{N-1}^T P_N B_{N-1} [B_{N-1}^T P_N B_{N-1} + R_{N-1}]^{-1} B_{N-1}^T P_N A_{N-1}, \\
 \lambda_{N-1} &= P_{N-1} \mathbf{x}_{N-1}, \\
 \gamma_{N-1} &= -\frac{1}{2} \mathbf{x}_{N-1}^T P_{N-1} \mathbf{x}_{N-1}, \\
 J_{N-1}^*(\mathbf{x}_{N-1}) &= \frac{1}{2} \mathbf{x}_{N-1}^T P_{N-1} \mathbf{x}_{N-1}.
 \end{aligned} \tag{3.46}$$

Of course, this step generalizes away from the $N - 1$ to N step through induction, a summary of which is included in the figure below.

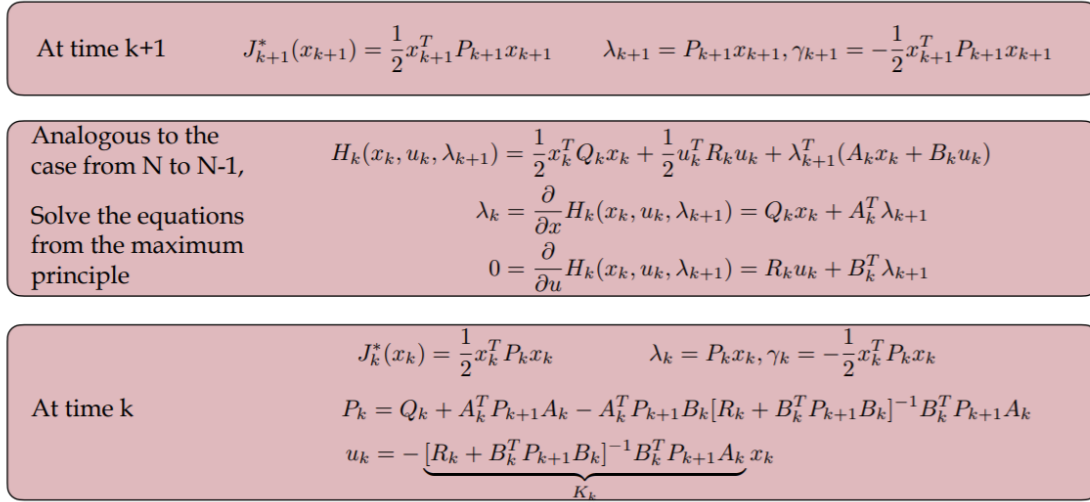


Figure 3.3: Solving LQR from $k + 1$ and k .

Most importantly, the following expression is called the discrete time algebraic Ricatti equation (DARE):

$$P_k = Q_k + A_k^T P_{k+1} A_k - A_k^T P_{k+1} B_k [B_k^T P_{k+1} B_k + R_k]^{-1} B_k^T P_{k+1} A_k, \tag{3.47}$$

which can be solved recursively backward in time without ever computing the control law or synthesizing the actual control input.

Implementation of LQR The following steps summarize how to implement LQR.

1. Obtain the system dynamic model: $\mathbf{x}_k + 1 = A_k \mathbf{x}_k + B_k \mathbf{u}_k$
2. Specify the matrices (design parameters) $Q_0, \dots, Q_{N-1}, S_N, R_0, \dots, R_{N-1}$ in the objective function

3. Solve for $P_0, \dots, P_N$ in the DARE backward in time (offline)
4. Apply real-time control at every time step, one we obtain the state measurement, state in this instance, $\mathbf{x}_k$, which we then apply using (3.37)

3.2 Continuous Time Optimal Control

The problem considered in this section is:

$$\begin{aligned} \min_{\mathbf{u}_t, t \in [0, T)} J &= \int_{t=0}^T l_t(\mathbf{x}_t, \mathbf{u}_t) dt + l_T(\mathbf{x}_T). \\ \text{s.t. } \dot{\mathbf{x}} &= f(\mathbf{x}, \mathbf{u}). \end{aligned} \quad (3.48)$$

Our goal is to decide the control trajectory, $\mathbf{u}_t, t \in [0, T)$, where we expect to know the initial state, $\mathbf{x}_0$, the run-time costs, l_t , and the terminal cost, l_T . Just as in the discrete-time case, we can determine this control trajectory with two different solution types: **open-loop solutions** solve for a trajectory of future controls, $\mathbf{u}_t, t \in [0, T)$, while the closed-loop solution solves for a control policy: $\mathbf{u}_t = q_t(\mathbf{x}_t), \forall t$. The advantages and detriments to each approach are the same as those for the discrete-time case.

Properties of this system that we are assuming are

1. Deterministic continuous time formulation
2. State feedback ($\mathbf{y} = \mathbf{x}$)
3. Finite horizon
4. No discount ($\delta = 1$ or the discount factor is absorbed by l_t)
5. Controllable System

Example: Regulation Problem Consider the regulation problem from the last section. In this problem, we are provided a dynamic model,

$$\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}, \quad (3.49)$$

and a quadratic cost function for the form

$$J = \int_{t=0}^T [\mathbf{x}_t^T Q_t \mathbf{x}_t + \mathbf{u}_t^T R_t \mathbf{u}_t] dt + \mathbf{x}_T^T S_T \mathbf{x}_T. \quad (3.50)$$

This is what we consider a **linear quadratic regulation problem** in continuous time.

Continuous Time vs. Discrete Time The modeling of systems is a complex and thorough field, but for the majority of physical systems, we can more precisely model these systems using continuous time ODE models. Examples of such systems include ground vehicles, aerial vehicles, and robots. One way to conceptualize continuous time models is that they are the extreme case of discrete time models where the sampling time is set to infinitely small. Note, however, that most physical laws derive fundamentally from differential equations, e.g. Newton's Second Law of Motion, and are thus inherently continuous time systems. However, as of the invention of the modern computer, most observation and control systems are implemented in discrete time as that is the very language of the computer. For such inherently discrete time systems, it is advantageous to use direct digital design and start by modeling the system using a discrete time model. Some different equipment associated with implementation include digital signal processors (DSPs) or field-programmable gate arrays (FPGAs).

3.2.1 Dynamic Programming

Similar to the approach taken in the discrete-time optimal control section, we will first start by defining an optimal solution for a continuous time optimal control problem. Let us rewrite the objective function for (3.48) as

$$J_0(\mathbf{x}_0; \mathbf{u}_{0:T}) := \int_{t=0}^T l_t(\mathbf{x}_t, \mathbf{u}_t) dt + l_T(\mathbf{x}_T) \quad (3.51)$$

Note that in the following optimal control problems, we will not be writing the equality constraint in (3.48) if it is clear from the context of the problem. We denote $\mathbf{u}_{0:T}^*$ as the optimal control trajectory. When specified with an initial condition, $\mathbf{x}_0$, we can use the optimal control trajectory through roll-out simulation to determine the corresponding optimal state sequence, $\mathbf{x}_{0:T}^*$. The optimal cost corresponding to the optimal sequence is denoted as $J^*(\mathbf{x}_0)$.

Consider a subproblem: we start at $\mathbf{x}_t^*$ at time t and wish to minimize the "cost-to-go" from time t to time T over $\mathbf{u}_{t:T}$, mathematically

$$\min_{\mathbf{u}_{t:T}} \underbrace{\int_{\tau=t}^T l_{\tau}(\mathbf{x}_{\tau}, \mathbf{u}_{\tau}) d\tau + l_T(\mathbf{x}_T)}_{J_t(\mathbf{x}_t, \mathbf{u}_{t:T})}. \quad (3.52)$$

Just as in the discrete time case, the truncated optimal control sequence, $\mathbf{u}_{t:T}^*$ is optimal for this subproblem via the principle of optimality.² To leverage the principle of optimality in determining this control sequence, consider the following cost-to-go

$$J_t(\mathbf{x}_t, \mathbf{u}_{t:T}) = \int_t^{t'} l_{\tau}(\mathbf{x}_{\tau}, \mathbf{u}_{\tau}) d\tau + J_{t'}(\mathbf{x}_{t'}, \mathbf{u}_{t':T}). \quad (3.53)$$

² The principle of optimality for the continuous time case can be proved in a similar way as we did for the discrete time case.

When we take the minimization on both sides, just like the discrete time case, we note that the cost-to-go for future time steps is still dependent on $\mathbf{u}_{t:t'}$, and thus the problem is not separable. Therefore, we rephrase (3.53) using the principle of optimality to

$$\min_{\mathbf{u}_{t:T}} J_t(\mathbf{x}_t; \mathbf{u}_{t:T}) = \min_{\mathbf{u}_{t:t'}} \left\{ \int_t^{t'} l_\tau(\mathbf{x}_\tau, \mathbf{u}_\tau) d\tau + \min_{\mathbf{u}_{t':T}} J_{t'}(\mathbf{x}_{t'}; \mathbf{u}_{t':T}) \right\}, \quad (3.54)$$

where the term that depends on $\mathbf{u}_{t:t'}$ is $\mathbf{x}_{t'}$. Just as in the discrete time case, we will solve the problem in (3.54) sequentially.

Cost-to-go and Value Function The **Cost-to-go** function at time t solely depends on the current state and all future control inputs. This is again analogous to the discrete-time case. The cost-to-go function is given as

$$J_t(\mathbf{x}_t; \mathbf{u}_{t:T}) = \int_{\tau=t}^T l_\tau(\mathbf{x}_\tau, \mathbf{u}_\tau) d\tau + l_T(\mathbf{x}_T), \quad (3.55)$$

and we likewise choose to define the optimal cost-to-go, the continuous time **value function**, as

$$V(t, \mathbf{x}_t) = \min_{\mathbf{u}_{t:T}} J_t(\mathbf{x}_t; \mathbf{u}_{t:T}). \quad (3.56)$$

Remember that the value function assumes optimal actions will be taken, hence the reason why it is not a function of $u_{t:T}$.

The Hamilton-Jacobi-Bellman Equation Now that we have developed a number of tools that drive the comparison between discrete-time optimal control and continuous-time optimal control, our next goal is to build a tool analogous to the discrete-time **Bellman Equation** for use in continuous time optimal control. That tool is known as the **Hamilton-Jacobi-Bellman Equation**, and it is a nonlinear partial differential equation of the form:

$$0 = \frac{\partial}{\partial t} V(t, \mathbf{x}_t) + \min_{\mathbf{u}_t} \{ l_t(\mathbf{x}_t, \mathbf{u}_t) + \frac{\partial}{\partial \mathbf{x}} V(t, \mathbf{x}_t) \cdot f(\mathbf{x}_t, \mathbf{u}_t) \}. \quad (3.57)$$

We start by recalling the principle of optimality in (3.54). We make the value function substitutions noted above to notationally simplify (3.54) into

$$V(t, \mathbf{x}_t) = \min_{\mathbf{u}_{t:t'}} \left\{ \int_t^{t'} l_\tau(\mathbf{x}_\tau, \mathbf{u}_\tau) d\tau + V(t', \mathbf{x}_{t'}) \right\}. \quad (3.58)$$

In order to get rid of t' in the expression, we first move $V(t, \mathbf{x}_t)$ to the right-hand-side (it can direct go into the minimization since it does not depend on the decision variables $u_{t:t'}$), then divide $(t' - t)$ on both sides, finally take the limit by requiring $t' \rightarrow t^+$:

$$0 = \lim_{t' \rightarrow t^+} \min_{\mathbf{u}_{t:t'}} \left\{ \frac{1}{t' - t} \int_t^{t'} l_\tau(\mathbf{x}_\tau, \mathbf{u}_\tau) d\tau + \frac{V(t', \mathbf{x}_{t'}) - V(t, \mathbf{x}_t)}{t' - t} \right\}. \quad (3.59)$$

Under a smoothness assumption of all functions, we can make the following simplifications

$$\lim_{t' \rightarrow t^+} \frac{1}{t' - t} \int_t^{t'} l_\tau(\mathbf{x}_\tau, \mathbf{u}_\tau) d\tau = l_t(\mathbf{x}_t, \mathbf{u}_t), \quad (3.60)$$

and

$$\lim_{t' \rightarrow t^+} \frac{V(t', \mathbf{x}_{t'}) - V(t, \mathbf{x}_t)}{t' - t} = \frac{\partial}{\partial t} V(t, \mathbf{x}_t) + \frac{\partial}{\partial \mathbf{x}} V(t, \mathbf{x}_t) \cdot \dot{\mathbf{x}}_t. \quad (3.61)$$

which generates the form of (3.57) when we sub in our dynamic equation $\dot{\mathbf{x}} = f(\mathbf{x}_t, \mathbf{u}_t)$. Note that the Hamilton-Jacobi-Bellman (HJB) Equation is a partial differential equation regarding the value function, which can also be written as:

$$0 = \dot{V}(t, \mathbf{x}_t) + \min_{\mathbf{u}_t} \{l_t(\mathbf{x}_t, \mathbf{u}_t) + \nabla_{\mathbf{x}} V(t, \mathbf{x}_t) \cdot f(\mathbf{x}_t, \mathbf{u}_t)\}. \quad (3.62)$$

Dynamic Programming for Continuous Time Optimal Control The question now becomes: *How do we solve the continuous-time optimal control problem* posed above. We do this by solving for the optimal control recursively by solving the Hamilton-Jacobi-Bellman Equation. Consider, at time T we have

$$V(T, \mathbf{x}_T) = l_T(\mathbf{x}_T), \quad (3.63)$$

which we can use an initial condition to solve the following optimization problem backward in time (this is just the Hamilton-Jacobi-Bellman rewritten)

$$-\dot{V}(t, \mathbf{x}_t) = \min_{\mathbf{u}_t} \{l_t(\mathbf{x}_t, \mathbf{u}_t) + \nabla_{\mathbf{x}} V(t, \mathbf{x}_t) \cdot f(\mathbf{x}_t, \mathbf{u}_t)\}, \quad (3.64)$$

yielding the optimal control policy

$$q_t^*(\mathbf{x}_t) = \arg \min_{\mathbf{u}_t} \{l_t(\mathbf{x}_t, \mathbf{u}_t) + \nabla_{\mathbf{x}} V(t, \mathbf{x}_t) \cdot f(\mathbf{x}_t, \mathbf{u}_t)\}. \quad (3.65)$$

Through the backward pass, we only obtain the control policy as a mapping from the state to the control, so we must obtain the optimal control sequence in the forward pass given the initial condition, $\mathbf{x}_0$:

$$\begin{aligned} \mathbf{x}_0^* &= \mathbf{x}_0 \\ \mathbf{u}_t^* &= q_t^*(\mathbf{x}_t^*), \quad \forall t \in [0, T) \\ \dot{\mathbf{x}}_t^* &= f(\mathbf{x}_t^*, \mathbf{u}_t^*), \quad \forall t \in [0, T). \end{aligned}$$

3.2.2 Maximum Principle

The Hamiltonian in the continuous time case is defined as follows

$$H(t, \mathbf{x}_t, \mathbf{u}_t, \boldsymbol{\lambda}_t) := l_t(\mathbf{x}_t, \mathbf{u}_t) + \boldsymbol{\lambda}_t^T f(\mathbf{x}_t, \mathbf{u}_t), \quad (3.66)$$

where the co-state λ_t satisfies

$$\lambda_t = \nabla_{\mathbf{x}} V(t, \mathbf{x}_t). \quad (3.67)$$

Comparing this expression with (3.64), we see that

$$-\dot{V}(t, \mathbf{x}_t) = \min_{\mathbf{u}_t} H(t, \mathbf{x}_t, \mathbf{u}_t, \lambda_t). \quad (3.68)$$

Although the Hamiltonian and its relationship with the Bellman principle in the continuous time case is similar to the discrete time case, there are several distinctions. First, in the continuous time, the Hamiltonian at time t depends on λ_t , while in the discrete time, the Hamiltonian at time k depends on λ_{k+1} . Second, the Hamiltonian in the continuous time case is exactly the term inside the minimization for the HJB equation, while the Hamiltonian in the discrete time case may be different from the term inside the minimization for the Bellman equation due to the residual term γ_k . Note that the reason behind the second difference is that the HJB equation is a first-order condition (which is concerned with derivatives of the value function), while the Bellman equation is a zeroth-order condition (which is concerned with the value function itself).

Next, we need to find the update rule for the co-state. Consider the Hamilton-Jacobi-Bellman Equation (3.64), let us take partial derivatives with respect to x on both sides. Note that due to the Partial Derivative Lemma,³ we have the following equality with respect to the optimal control $\mathbf{u}_t$,

³ Consider the following equality $V^*(a) = \min_b V(a, b)$, then the partial derivatives satisfy $\frac{d}{da} V^*(a) = \frac{\partial}{\partial a} V(a, b^*)$.

$$-\frac{\partial^2}{\partial x \partial t} V(t, \mathbf{x}_t) = \frac{\partial}{\partial x} l_t(\mathbf{x}_t, \mathbf{u}_t) + \frac{\partial^2}{\partial x^2} V(t, \mathbf{x}_t) f(\mathbf{x}_t, \mathbf{u}_t) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_t, t) \right]^T \frac{\partial}{\partial x} V(t, \mathbf{x}_t). \quad (3.69)$$

Using the following identity

$$-\frac{d}{dt} \frac{\partial}{\partial x} V(t, \mathbf{x}_t) = \frac{\partial^2}{\partial t \partial x} V(t, \mathbf{x}_t) + \frac{\partial^2}{\partial x^2} V(t, \mathbf{x}_t) f(\mathbf{x}_t, \mathbf{u}_t), \quad (3.70)$$

to substitute the term on the left-hand-side and the second term on the right-hand-side, we have

$$-\frac{d}{dt} \underbrace{\frac{\partial}{\partial x} V(t, \mathbf{x}_t)}_{\lambda_t} = \frac{\partial}{\partial x} l_t(\mathbf{x}_t, \mathbf{u}_t) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_t, \mathbf{u}_t) \right]^T \underbrace{\frac{\partial}{\partial x} V(t, \mathbf{x}_t)}_{\lambda_t}. \quad (3.71)$$

Finally, we are led to **Pontryagin's Maximum Principle** which states that for optimal state trajectory $\mathbf{x}_{0:T}^*$ and optimal control trajectory $\mathbf{u}_{0:T}^*$ starting at initial condition $\mathbf{x}_0$, then there exists a co-state trajectory $\lambda_{0:T}^*$ with $\lambda_T^* = \frac{\partial}{\partial x} l_T(\mathbf{x}_T^*)$ satisfying

	Continuous Time	Discrete Time
Hamiltonian	$H(t, \mathbf{x}_t, \mathbf{u}_t, \lambda_t) := l_t(\mathbf{x}_t, \mathbf{u}_t) + \lambda_t^T f(\mathbf{x}_t, \mathbf{u}_t)$	$H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = l_k(\mathbf{x}_k, \mathbf{u}_k) + \lambda_{k+1}^T f(\mathbf{x}_k, \mathbf{u}_k)$
State	$\dot{\mathbf{x}}_t^* = \frac{\partial}{\partial \lambda} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = f(\mathbf{x}_t^*, \mathbf{u}_t^*)$	$\mathbf{x}_{k+1}^* = \frac{\partial}{\partial \lambda} H_k(\mathbf{x}_k^*, \mathbf{u}_k^*, \lambda_{k+1}^*) = f(\mathbf{x}_k^*, \mathbf{u}_k^*)$
Co-State	$-\dot{\lambda}_t^* = \frac{\partial}{\partial x} H = \frac{\partial}{\partial x} l_t(\mathbf{x}_t^*, \mathbf{u}_t^*) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_t^*, \mathbf{u}_t^*) \right]^T \lambda_t^*$	$\lambda_k^* = \frac{\partial}{\partial x} H_k = \frac{\partial}{\partial x} l_k(\mathbf{x}_k^*, \mathbf{u}_k^*) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_k^*, \mathbf{u}_k^*) \right]^T \lambda_{k+1}^*$
Control	$\mathbf{u}_t^* = \arg \min_{\mathbf{u}} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*)$	$\mathbf{u}_k^* = \arg \min_{\mathbf{u}} H_k(\mathbf{x}_k^*, \mathbf{u}_k^*, \lambda_{k+1}^*)$

Table 3.1: Comparison between the Maximum Principle in the continuous time case and the discrete time case.

$$\begin{aligned}
\dot{\mathbf{x}}_t^* &= \frac{\partial}{\partial \lambda} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = f(\mathbf{x}_t^*, \mathbf{u}_t^*) \\
-\dot{\lambda}_t^* &= \frac{\partial}{\partial x} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = \frac{\partial}{\partial x} l_t(\mathbf{x}_t^*, \mathbf{u}_t^*) + \left[\frac{\partial}{\partial x} f(\mathbf{x}_t^*, \mathbf{u}_t^*) \right]^T \lambda_t^* \\
\mathbf{u}_t^* &= \arg \min_{\mathbf{u}} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*)
\end{aligned}$$

3.2.3 Linear Quadratic Regulator

Now we show how to use the Maximum Principle to solve the continuous time Linear Quadratic Regulator problem. The cost function and the dynamics are

$$\begin{aligned}
J &= \frac{1}{2} \int_{t=0}^T [\mathbf{x}_t^T Q_t \mathbf{x}_t + \mathbf{u}_t^T R_t \mathbf{u}_t] dt + \frac{1}{2} \mathbf{x}_T^T S_T \mathbf{x}_T \\
\dot{\mathbf{x}} &= A_t \mathbf{x} + B_t \mathbf{u},
\end{aligned} \tag{3.72}$$

where Q_t, S_T must be symmetric positive semi-definite, R_t must be symmetric positive definite, and A_t, B_t are at least stabilizable. We build the Hamiltonian as

$$H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = \frac{1}{2} [\mathbf{x}_t^T Q_t \mathbf{x}_t + \mathbf{u}_t^T R_t \mathbf{u}_t] + \lambda_t^T (A_t \mathbf{x}_t + B_t \mathbf{u}_t). \tag{3.73}$$

Using the Maximum Principle, we find

$$\begin{aligned}
-\dot{\lambda}_t &= \frac{\partial}{\partial x} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = Q_t \mathbf{x}_t + A_t^T \lambda_t \\
0 &= \frac{\partial}{\partial u} H(t, \mathbf{x}_t^*, \mathbf{u}_t^*, \lambda_t^*) = R_t \mathbf{u}_t + B_t^T \lambda_t \iff \mathbf{u}_t = -R_t^{-1} B_t^T \lambda_t
\end{aligned}$$

Substituting the control in the dynamic equation using $\mathbf{u}_t = -R_t^{-1} B_t^T \lambda_t$, the optimal trajectory satisfies the following ordinary differential equation

$$\begin{aligned}
\dot{\mathbf{x}} &= A_t \mathbf{x}_t - B_t R_t^{-1} B_t^T \lambda_t \\
\dot{\lambda} &= -Q_t \mathbf{x}_t - A_t^T \lambda_t,
\end{aligned} \tag{3.74}$$

with the boundary conditions $\mathbf{x}_0$ (for the forward pass) and λ_T (for the backward pass) where

$$\lambda_T = \frac{\partial}{\partial \mathbf{x}} l_T(\mathbf{x}_T) = S_T \mathbf{x}_T. \tag{3.75}$$

Assuming that $\lambda_t = P_t \mathbf{x}_t$ and $P_T = S_T$, then we have

$$\begin{aligned} 0 &= \dot{\lambda} + Q_t \mathbf{x}_t + A_t^T \lambda_t \\ &= \dot{P}_t \mathbf{x}_t + P_t \dot{\mathbf{x}}_t + Q_t \mathbf{x}_t + A_t^T \lambda_t \\ &= \dot{P}_t \mathbf{x}_t + P_t A_t \mathbf{x}_t - P_t B_t R_t^{-1} B_t^T P_t \mathbf{x}_t + Q_t \mathbf{x}_t + A_t^T P_t \mathbf{x}_t. \end{aligned}$$

which, removing the state $\mathbf{x}_t$, gives us an Algebraic Riccati Equation (ARE),

$$\dot{P}_t + P_t A_t + A_t^T P_t - P_t B_t R_t^{-1} B_t^T P_t + Q_t = 0. \quad (3.76)$$

Thus to implement LQR in continuous time, one must

1. Obtain a linear system dynamic model, $\dot{\mathbf{x}} = A_t \mathbf{x} + B_t \mathbf{u}$
2. Specify the matrices, Q_t , S_T , and R_t in the objective function in (3.72)
3. Solve for P_t in the ARE backward in time (offline)
4. Real-time control: at time t , we apply the following optimal control: $\mathbf{u}_t = -R_t^{-1} B_t^T P_t \mathbf{x}_t$

Finally, to end this series of notes let us compare the discrete-time and the continuous-time formulations as shown in the figure below. Generally speaking, it is easier to find the control law for continuous time systems than for discrete time systems; while it is easier to perform inference (an inverse problem of control) for discrete time systems than for continuous time systems.

	Continuous Time	Discrete Time
System Dynamics	$\dot{x} = A_t x + B_t u$	$x_{k+1} = A_k x_k + B_k u_k$
Cost Function	$J = \frac{1}{2} \int_{t=0}^T [x_t^T Q_t x_t + u_t^T R_t u_t] dt + \frac{1}{2} x_T^T S_T x_T$	$J = \frac{1}{2} \sum_{k=0}^{N-1} [x_k^T Q_k x_k + u_k^T R_k u_k] + \frac{1}{2} x_N^T S_N x_N$
Optimal Control Law	$u_t = -R_t^{-1} B_t^T P_t x_t$	$u_k = -\underbrace{[R_k + B_k^T P_{k+1} B_k]^{-1} B_k^T P_{k+1} A_k}_{K_k} x_k$
Riccati Equation	$\dot{P}_t + P_t A_t + A_t^T P_t - P_t B_t R_t^{-1} B_t^T P_t + Q_t = 0$	$P_k = Q_k + A_k^T P_{k+1} A_k - A_k^T P_{k+1} B_k [R_k + B_k^T P_{k+1} B_k]^{-1} B_k^T P_{k+1} A_k$

Figure 3.4: A comparison between discrete-time LQR and continuous-time LQR.

4

Optimal Control - Infinite Time

This chapter discusses the optimal control strategy when we set the horizon to infinity. We will first introduce the concept of stationarity, then discuss stationary Bellman equations. Finally, we will end the discussion by showing the convergence and stability conditions for infinite-horizon LQR.

4.1 Stationarity

For this course, we are going to make two key distinctions between the infinite horizon and the finite horizon optimal control problems. Note that in the real world, these distinctions may not hold, but for the purposes of the studies in this class, they will hold unless explicitly stated that they will not.

1. The number of time steps is infinite (infinite horizon)
2. The system is **stationary**, which means that the system equation (dynamics) and the run-time cost does not change from time to time. Note as an infinite horizon problem, we do not have a terminal cost.

As you will see in this section, infinite horizon policies are generally simpler than their finite-time counterparts, but often require much more sophisticated mathematical treatment. Given that our run-time cost and our system dynamics are stationary, that same stationarity is preserved in the value function. Why? As the dynamics and the run-time cost are stationary, solving the infinite-time optimal control problem and starting from an initial state $\mathbf{x}^*$, when considered at any time step k , will yield the same optimal control sequence, which is why we have stationary value function. In this section, we write the value function as $V : \mathbb{T} \times \mathcal{X} \rightarrow \mathbb{R}$ for both continuous time ($\mathbb{T} = \mathbb{R}^+$) and discrete time ($\mathbb{T} = \{0, 1, 2, \dots\}$). The following derivation shows that if we have the stationarity condition, then the value

function remains the same at different time:

$$V(k, \mathbf{x}^*) = \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \dots} \{l(\mathbf{x}^*, \mathbf{u}_k) + \sum_{k'=k+1}^{\infty} \delta^{k'-k} l(\mathbf{x}_{k'}, \mathbf{u}_{k'})\} \quad (4.1)$$

$$= \min_{\mathbf{u}_0, \mathbf{u}_1, \dots} \{l(\mathbf{x}^*, \mathbf{u}_0) + \sum_{k'=1}^{\infty} \delta^{k'} l(\mathbf{x}_{k'}, \mathbf{u}_{k'})\} \quad (4.2)$$

$$= V(0, \mathbf{x}^*) \quad (4.3)$$

Discrete Time	Continuous Time
$J = \sum_{k=0}^{\infty} \delta^k l(x_k, u_k)$	$J = \int_{t=0}^{\infty} \delta^t l(x, u) dt$
$x' = f(x, u)$	$\dot{x} = f(x, u)$
$V(k, x_k)$	$V(t, x_t)$
$= \min_{u_k, u_{k+1}, \dots} \sum_{k'=k}^{\infty} \delta^{k'-k} l(x_{k'}, u_{k'})$	$= \min_{u_{t:\infty}} \int_{\tau=t}^{\infty} \delta^{\tau-t} l(x, u) d\tau$

Table 4.1: A listing of the elements of discrete-time and continuous-time infinite horizon control that are stationary.

Since the value function does not depend on time, in the following discussion, we will drop the time dependency in the value function.

4.1.1 Bellman Equation in Infinite Horizon

The derivation of the Bellman Equation for Infinite Horizon control is similar to that of finite-horizon control, observe. We start by defining the value function

$$V(\mathbf{x}_k) = \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \dots} \sum_{k'=k}^{\infty} \delta^{k'-k} l(\mathbf{x}_{k'}, \mathbf{u}_{k'}) \quad (4.4)$$

$$= \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \dots} \{l(\mathbf{x}_k, \mathbf{u}_k) + \sum_{k'=k+1}^{\infty} \delta^{k'-k} l(\mathbf{x}_{k'}, \mathbf{u}_{k'})\} \quad (4.5)$$

$$= \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \dots} \{l(\mathbf{x}_k, \mathbf{u}_k) + \delta \sum_{k'=k+1}^{\infty} \delta^{k'-k-1} l(\mathbf{x}_{k'}, \mathbf{u}_{k'})\} \quad (4.6)$$

$$= \min_{\mathbf{u}_k} \{l(\mathbf{x}_k, \mathbf{u}_k) + \delta \min_{\mathbf{u}_{k+1}, \dots} \sum_{k'=k+1}^{\infty} \delta^{k'-k-1} l(\mathbf{x}_{k'}, \mathbf{u}_{k'})\} \quad (4.7)$$

Now, what is happening at each of these steps? In (4.5), what we have done is pulled out the run-time cost at time-step k from the summation. Next, we note that $l(\mathbf{x}_k, \mathbf{u}_k)$ is not a function of any higher control than $\mathbf{x}_k$, thus we can split the minimization into two sequential minimization problems, which is what we are doing between (4.6) and (4.7). Noticing the change of indices on the summation term, we see that we should pullout a δ in order to establish a pattern consistent with the indices on the summation. What this means is that, as the summation begins with $k' = k + 1$, then plugging this value in for k in (4.6) yields a δ^1 term, which means there is an applied discount. Should we remove this discount, we can establish a “form” of $V(\mathbf{x}_k)$ that can be evaluated iteratively. This is how

we arrive at (4.7). Note, from this definition of value function, we can immediately state the Bellman Equation:

$$V(\mathbf{x}_k) = \min_{\mathbf{u}_k} \{l(\mathbf{x}_k, \mathbf{u}_k) + \delta V(\mathbf{x}_{k+1})\} \quad (4.8)$$

4.1.2 Hamilton-Jacobi-Bellman Equation in Infinite Horizon

Just as what we did with the discrete-time case, we begin from the definition of the value function

$$V(\mathbf{x}_t) = \min_{\mathbf{u}_{t:\infty}} \int_{\tau=t}^{\infty} \delta^{\tau-t} l(\mathbf{x}, \mathbf{u}) d\tau, \quad (4.9)$$

$$= \min_{\mathbf{u}_{t:t'}} \left\{ \int_{\tau=t}^{t'} \delta^{\tau-t} l(\mathbf{x}, \mathbf{u}) d\tau + \delta^{t'-t} V(\mathbf{x}_{t'}) \right\}, \quad (4.10)$$

By moving $V(\mathbf{x}_t)$ to the right-hand-side, dividing $(t' - t)$ on both sides, and taking the limit by requiring $t' \rightarrow t^+$, we have

$$0 = \lim_{t' \rightarrow t^+} \min_{\mathbf{u}_{t:t'}} \left\{ \frac{1}{t' - t} \int_{\tau=t}^{t'} \delta^{\tau-t} l(\mathbf{x}, \mathbf{u}) d\tau + \frac{\delta^{t'-t} V(\mathbf{x}_{t'}) - V(\mathbf{x}_t)}{t' - t} \right\}. \quad (4.11)$$

The first term inside the minimization will be $l(\mathbf{x}_t, \mathbf{u}_t)$ after taking the limit. The second term inside the minimization will be $\frac{d}{dt} [\delta^{\tau-t} V(\mathbf{x}_\tau)]$ after taking the limit, which then equals $\ln(\delta)V(\mathbf{x}_t) + \nabla_{\mathbf{x}} V(\mathbf{x}_t) \cdot \dot{\mathbf{x}}_t$. Hence, the Hamilton-Jacobi-Bellman equation for infinite horizon problems with discount becomes

$$-\ln(\delta)V(\mathbf{x}_t) = \min_{\mathbf{u}_t} \{l(\mathbf{x}_t, \mathbf{u}_t) + \nabla_{\mathbf{x}} V(\mathbf{x}_t) f(\mathbf{x}_t, \mathbf{u}_t)\}. \quad (4.12)$$

4.2 Solving the Stationary Bellman Equation

The discrete-time, we recall that the Bellman Equation is given by

$$V(\mathbf{x}) = \min_{\mathbf{u}} \{l(\mathbf{x}, \mathbf{u}) + \delta V(f(\mathbf{x}, \mathbf{u}))\} \quad (4.13)$$

We then choose to define an Operator, known as the **Bellman operator**, $\mathcal{T}$, as

$$\mathcal{T}(V) := \min_{\mathbf{u}} \{l(\mathbf{x}, \mathbf{u}) + \delta V(f(\mathbf{x}, \mathbf{u}))\} \quad (4.14)$$

We do this as we are interested in finding the optimal value function to solve the infinite-time optimal control problem. This optimal value function is a fixed point of the Bellman operator, and thus to find it we need perform **value iteration**: starting from an initial estimation of the value function, V_0 , we iteratively apply the Bellman operator:

$$V_{i+1} = \mathcal{T}(V_i), \quad (4.15)$$

until we get convergence of V_i . The question really is: *Are V_i 's guaranteed to converge under the Bellman Operator?* It turns out that if we can show the Bellman Operator is a **Contracting Operator**, then we can show the sequence of V_i will converge to a fixed point. A function operator, $\mathcal{F}$, is said to have α -contraction with respect to some norm $\|\cdot\|$ if the following condition holds:

$$\forall X_1, X_2 : \|\mathcal{F}X_1 - \mathcal{F}X_2\| \leq \alpha \|X_1 - X_2\|. \quad (4.16)$$

Theorem 4.1. *For an α -contracting operator $\mathcal{F}$ with $\alpha < 1$, there is a unique fixed point X^* which satisfies $\mathcal{F}X^* = X^*$. Moreover, all sequences $X, \mathcal{F}X, \mathcal{F}^2X, \dots$ converge to this unique fixed point X^* .*

Now the problem becomes *how to shall that the Bellman Operator is contracting?* We provide the proof for a simple case where we have a finite state space with a discount factor $\delta < 1$. We will show that the corresponding Bellman Operator is contracting with respect to the L_∞ norm: $\|V\|_\infty = \max_{\mathbf{x}} |V(\mathbf{x})|$. To show that the Bellman Operator is contracting, we directly use the definition in (4.16). For any two functions $V_1 : \mathcal{X} \rightarrow \mathbb{R}$ and $V_2 : \mathcal{X} \rightarrow \mathbb{R}$, we have

$$\begin{aligned} & \|\mathcal{T}(V_1) - \mathcal{T}(V_2)\|_\infty \\ &= \left\| \min_{\mathbf{u}} \{l(\mathbf{x}, \mathbf{u}) + \delta V_1(f(\mathbf{x}, \mathbf{u}))\} - \min_{\mathbf{u}} \{l(\mathbf{x}, \mathbf{u}) + \delta V_2(f(\mathbf{x}, \mathbf{u}))\} \right\|_\infty \\ &\leq \left\| \max_{\mathbf{u}} \{l(\mathbf{x}, \mathbf{u}) + \delta V_1(f(\mathbf{x}, \mathbf{u}))\} - \{l(\mathbf{x}, \mathbf{u}) + \delta V_2(f(\mathbf{x}, \mathbf{u}))\} \right\|_\infty \\ &= \left\| \delta \max_{\mathbf{u}} |V_1(f(\mathbf{x}, \mathbf{u})) - V_2(f(\mathbf{x}, \mathbf{u}))| \right\|_\infty \\ &\leq \delta \|V_1 - V_2\|_\infty \end{aligned}$$

The second first inequality is due to the fact that $|\min A(u) - \min B(u)| \leq \max\{|A(u_A) - B(u_A)|, |A(u_B) - B(u_B)|\} \leq \max |A(u) - B(u)|$ where $u_A = \arg \min A(u)$ and $u_B = \arg \min B(u)$. In our case, $A(u) = l(\mathbf{x}, \mathbf{u}) + \delta V_1(f(\mathbf{x}, \mathbf{u}))$ and $B(u) = l(\mathbf{x}, \mathbf{u}) + \delta V_2(f(\mathbf{x}, \mathbf{u}))$. Moreover, since the state space is finite, this inequality is preserved under L_∞ norm over $\mathbf{x}$.

Hence we have shown that the Bellman Operator is contracting with $\alpha = \delta$ in the case that we have a finite state space and $\delta < 1$. Since the Bellman operator is contracting, value iteration will generate a converging sequence to the fixed point. After confirming convergence will occur, we are also interested in the rate of convergence. Additionally, the underlying policy may converge before the value function. This will be discussed more when we talk about **policy iteration**.

4.3 Linear Quadratic Regulator (Infinite Continuous Time)

This Linear Quadratic Regulator problem for infinite continuous time is given below

$$\begin{aligned} J &= \frac{1}{2} \int_{t=0}^{\infty} [\mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u}] dt \\ \dot{\mathbf{x}} &= A\mathbf{x} + B\mathbf{u}, \end{aligned} \quad (4.17)$$

where Q must be symmetric positive semi-definite, R must be symmetric positive definite, and A, B are at least stabilizable. We build the Hamiltonian

$$H(\mathbf{x}, \mathbf{u}, \lambda) = \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + \frac{1}{2} \mathbf{u}^T R \mathbf{u} + \lambda^T (A\mathbf{x} + B\mathbf{u}), \quad (4.18)$$

and leveraging the maximum principle, we find

$$-\dot{\lambda} = \frac{\partial}{\partial \mathbf{x}} H(\mathbf{x}, \mathbf{u}, \lambda) = Q\mathbf{x} + A^T \lambda \quad (4.19)$$

$$0 = \frac{\partial}{\partial \mathbf{u}} H(\mathbf{x}, \mathbf{u}, \lambda) = R\mathbf{u} + B^T \lambda. \quad (4.20)$$

Using the second equation, we can solve for $\mathbf{u}$:

$$\mathbf{u} = -R^{-1} B^T \lambda. \quad (4.21)$$

Plugging the expression of $\mathbf{u}$ back to the dynamic equation, we have the following ODE that needs to be satisfied by the optimal trajectory:

$$\begin{aligned} \dot{\mathbf{x}} &= A\mathbf{x} - BR^{-1}B^T \lambda \\ \dot{\lambda} &= -Q\mathbf{x} - A^T \lambda, \end{aligned}$$

Assuming that $\lambda = P\mathbf{x}$, then we have

$$\begin{aligned} 0 &= \dot{\lambda} + Q\mathbf{x} + A^T \lambda \\ &= P\dot{\mathbf{x}} + Q\mathbf{x} + A^T \lambda \\ &= PA\mathbf{x} - PBR^{-1}B^T P\mathbf{x} + Q\mathbf{x} + A^T P\mathbf{x}. \end{aligned}$$

which, can be seen to be equivalent to a form of the Algebraic Riccati Equation (by removing $\mathbf{x}$):

$$PA + A^T P - PBR^{-1}B^T P + Q = 0. \quad (4.22)$$

This infinite horizon Algebraic Riccati Equation is the steady state solution to the ODE that we introduced earlier for the finite horizon continuous time optimal control:

$$\dot{P}_t + P_t A + A^T P_t - P_t B R^{-1} B^T P_t + Q = 0 \quad (4.23)$$

We can solve the infinite horizon Algebraic Riccati Equation by integrating along the ODE (4.23), which is another form of value iteration given that the value function is $V(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T P \mathbf{x}$. Yet, it remains unclear as to whether the "value iteration" with respect to the Algebraic Riccati Equation will converge. We have already shown that the Bellman operator is contracting in a finite state space with a discount factor $\delta < 1$, but now we have an infinite state space and no discount factor.

Indeed, if (A, B) is controllable or stabilizable, then the solution to (4.23) is guaranteed to converge to a bounded P_∞ when starting from a positive definite P_0 . The intuition behind this claim is that if (A, B) is not stabilizable, then there are unstable states that may cause the cost function to go to infinity as t goes to infinity. It implies that the value function is unbounded, hence P_∞ .

Getting a converging solution for P_∞ is not enough, the next question is whether the resulting control law $\mathbf{u} = -R^{-1}B^T P \mathbf{x}$ stabilizes the system, i.e., $\mathbf{x}_t \rightarrow 0$ as $t \rightarrow \infty$. We have the following proposition: the resulting system will be asymptotically stable if $(A, Q^{\frac{1}{2}})$ is detectable for $Q^{\frac{1}{2}} = \sqrt{Q}$. This $Q^{\frac{1}{2}}$ is a design parameter selected by the user. The intuition behind this claim is that, if $(A, Q^{\frac{1}{2}})$ is not detectable, then there are unstable states that the cost function fails to penalize, thus the closed-loop system can be unstable but still minimizes the cost function.

4.4 Linear Quadratic Regulator (Infinite Discrete Time)

Consider the infinite-horizon discrete time Linear Quadratic Regulator optimal control problem, mathematically expressed as

$$J = \frac{1}{2} \sum_{k=0}^{\infty} [\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k]$$

$$\mathbf{x}_{k+1} = A \mathbf{x}_k + B \mathbf{u}_k,$$

where Q is symmetric positive semi-definite, R is symmetric positive definite, and A, B are controllable. From here, we form the Hamiltonian for the system

$$H(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = \frac{1}{2} \mathbf{x}_k^T Q \mathbf{x}_k + \frac{1}{2} \mathbf{u}_k^T R \mathbf{u}_k + \lambda_{k+1}^T (A \mathbf{x}_k + B \mathbf{u}_k). \quad (4.24)$$

Applying the maximum principle, we have

$$\lambda_k = \frac{\partial}{\partial \mathbf{x}} H(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = Q \mathbf{x}_k + A^T \lambda_{k+1}$$

$$0 = \frac{\partial}{\partial \mathbf{u}} H(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) = R \mathbf{u}_k + B^T \lambda_{k+1}$$

By assuming that $\lambda_k = P\mathbf{x}_k$, we can rearrange the second equation to get

$$\mathbf{u} = -[B^T P B + R]^{-1} B^T P A \mathbf{x}. \quad (4.25)$$

By manipulating the co-state update rule, we find

$$P = Q + A^T P A - A^T P B [B^T P B + R]^{-1} B^T P A, \quad (4.26)$$

which, similar to the continuous case, we can iteratively solve using the DARE. The convergence conditions are that (A, B) is controllable or stabilizable, and the closed-loop stability condition is that $(A, Q^{\frac{1}{2}})$ is observable or detectable.

Thus, we have shown that the Bellman update is a contracting operator if the state space is finite and there is a discount factor and if the system is controllable or stabilizable.

Final Remarks These two chapters have captured exact solutions of optimal control problems. Unfortunately, we may not always be able to get an exact solution to an optimal control problem due to a number of reasons, including if there are unknown parameters in the dynamic equations or the objective function or if the problem is computationally intractable. In these cases, we either need to identify the system parameters or need to find approximate solutions, which will be discussed in the following chapters.

5

Model Predictive Control

So far, we have studied unconstrained optimal control problems (meaning that there is no additional state or control constraint except for the dynamic constraint). However, in many optimal control problems, we have more than just dynamic constraints, but also state and control constraints appearing as inequality constraints. Constrained optimal control problems may also be solved via the methods we have learned in earlier lectures, but the previous approaches may be computationally intractable. To properly solve those problems, we introduce **Model Predictive Control** (MPC), which turns an infinite horizon problem into a sequence of finite horizon problems. In the following discussion, we briefly discuss constrained optimal control problems in section 5.1. Then we will provide an overview of MPC in section 5.2, followed by the discussion of unconstrained linear quadratic MPC in section 5.3 and the constrained linear quadratic MPC in section 5.4. Finally, we will discuss how to ensure feasibility of the MPC solutions in section 5.5 as well as how to ensure MPC stability in section 5.6.

5.1 Constrained Optimal Control

State-Constrained Optimal Control Suppose that we have a state constraint given as $\phi(\mathbf{x}) \leq 0$. We consider a discrete time optimal control problem, which will be the context for most of this chapter. To solve the state-constrained optimal control, we need to reconsider the techniques we learnt from previous chapters. The Bellman Principle still holds, but we must now introduce the state constraint $\phi(\mathbf{x}_k) \leq 0$:

$$\begin{aligned} J_k^*(\mathbf{x}_k) &= \min_{\mathbf{u}_k} \{l_k(\mathbf{x}_k, \mathbf{u}_k) + J_{k+1}^*(\mathbf{x}_{k+1})\}, \\ \text{s.t. } \mathbf{x}_{k+1} &= f(\mathbf{x}_k, \mathbf{u}_k), \\ \phi(\mathbf{x}_k) &\leq 0. \end{aligned} \tag{5.1}$$

On the other hand, the following equation in the maximum principle no longer holds:

$$\lambda_k = \frac{\partial}{\partial \mathbf{x}} H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}).$$

That is because we now need to consider this additional state constraints in the Hamiltonian. The Hamiltonian needs to be extended to the following form:

$$\tilde{H}_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}, \eta_{k+1}) = H_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}) + \eta_{k+1} \phi(f(\mathbf{x}_k, \mathbf{u}_k)),$$

where $\eta_{k+1} \geq 0$ is the Lagrangian multiplier for the state constraint which satisfies the complementary slackness condition: $\eta_{k+1} \phi(f(\mathbf{x}_k, \mathbf{u}_k)) = 0$. Thus the equation for the co-state becomes

$$\lambda_k = \frac{\partial}{\partial \mathbf{x}} \tilde{H}_k(\mathbf{x}_k, \mathbf{u}_k, \lambda_{k+1}, \eta_{k+1}).$$

The detailed derivation is omitted here for simplicity. Interested readers can derive the equation using the Lagrangian dual method.

Control-Constrained Optimal Control State-independent control constraints (in the form of $g(u) \leq 0$) are easier to deal with, we only need to add the constraints when computing the optimal control. The dynamics for the co-state remains the same as the unconstrained case (recall that we have used the control constraint in the investment problem).

5.2 The Big Picture about Model Predictive Control

The key idea behind MPC is that we are designing an infinite horizon sub-optimal controller by repeatedly solving a series of finite time optimal control problems in a receding horizon fashion. We only consider a portion of the original cost in the finite horizon problem that we solve at every time step; and we use the terminal cost to approximate the value function of the remainder of the infinite horizon optimal control problem. While this is one way to solve the dynamic programming problem, it does come with challenges, notably those associated with computational complexity, run time requirements, feasibility, and stability.

Receding Horizon Control The philosophy of MPC is built on the idea of **Receding Horizon Control**. In receding horizon control, at every time step k , we solve the open-loop discrete time optimal control problem by using the current state as the initial condition and solving for the control for future N steps. The control that is solved is then applied between the time interval $[k, k + 1]$, and at the next time step,

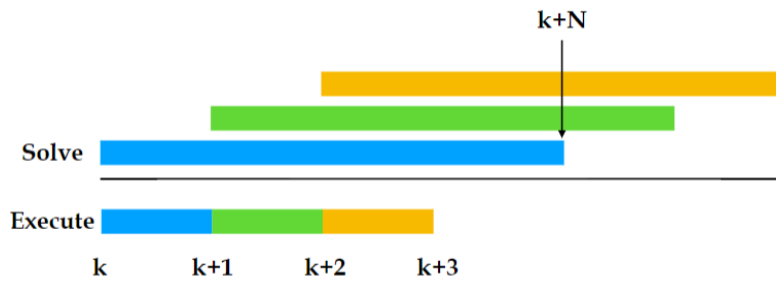


Figure 5.1: Receding Horizon Control with three planned control sequences.

$k + 1$, the process repeats. The idea is illustrated in fig. 5.1. In this figure, it is important to note that the solved optimal control sequence has multiple control actions, but only the first one is taken before the next optimal control problem is solved. In the figure, this is seen by the different colors in the execute line. At time step k , the solved finite-time horizon optimal control problem yields a sequence of controls for the next couple time steps, yielding the blue bar. When the optimal control problem is then solved again at time step $k + 1$, we discard the remainder of the solved for blue control sequence and implement the newly solved for green control sequence. This process repeats until the entire optimal control problem is solved. What is the relationship between receding horizon control and MPC? A receding horizon controller where the finite time optimal control law is computed by solving an optimization problem online is usually referred to as a Model Predictive Control (MPC) problem. Generally, as hinted at earlier in this chapter, MPC usually considers constrained optimization problems. MPC has been used in a number of applications, ranging from chemical processes to aircraft to production planning. An image from the ME290J course from Berkeley is included to describe a number of different MPC applications and their relative timescales.

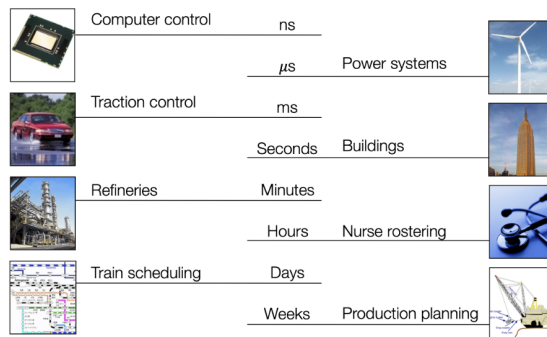


Figure 5.2: A series of different applications that may employ MPC controllers and their relative timescales.

Classical Control vs MPC How does classical control compare to MPC? In the classical control, the major consideration is how to derive an analytical control law to reject process disturbance or measurement noise while achieving the control objective. In an MPC problem, the major consideration is how to satisfy the constraints while optimally achieving the control objective. A repeated optimization problem is solved in real-time to choose the next control command. This optimization problem usually encodes constraints on the control (actuator or control limits) and constraints on the system state (safety constraints).



Figure 5.3: Comparison between Classical Control (left) and MPC Control (right).

Constrained Control All physical systems have constraints. These constraints may occur due to physical limitations of the system, e.g. control constraints in the form of actuator limits. Another type of constraint could be the performance constraints or requirements levied on the system, e.g. maximum system overshoot and specified rise or settling time, which constrain the available set of states that the system may evolve into. Additionally, we have safety constraints, which we will consider to be state constraints for the purposes of this class, e.g. collision avoidance tasks, but could also include artificial actuator limits in the case that a system is interacting with human agents in its environment. Often times, optimal operating points are near constraints.

Classical methods to handle constraints have done so with varying levels of success. Generally, ad hoc constraint management through saturation or setting references that are sufficiently far from constraints have been fairly popular, but these approaches will result in sub-optimal plant operation such as inefficiencies and/or deadlocks, or even lead to safety hazards.

The MPC Problem Mathematically, the MPC problem at time k can be written as the following discrete finite time optimal control prob-

lem:

$$\begin{aligned}
 \min_{\mathbf{u}_k, \dots, \mathbf{u}_{k+N-1}} J_{k:k+N}(\mathbf{x}_k; \mathbf{u}_k, \dots, \mathbf{u}_{k+N-1}) &= \sum_{k'=k}^{k+N-1} l_{k'-k}(\mathbf{x}_{k'}, \mathbf{u}_{k'}) + l_N(\mathbf{x}_{k+N}), \\
 \text{s.t. } \mathbf{x}_{k'+1} &= f(\mathbf{x}_{k'}, \mathbf{u}_{k'}), \forall k' = k, \dots, k + (N - 1), \\
 \mathbf{x}_{k'+1} &\in \mathcal{X}, \forall k' = k, \dots, k + (N - 1), \\
 \mathbf{u}_{k'} &\in \mathcal{U}, \forall k' = k, \dots, k + (N - 1).
 \end{aligned} \tag{5.2}$$

where l_k for $k = 0, 1, \dots, N - 1$ are run-time costs; l_N is the terminal cost; $\mathcal{X}$ is the state constraint; $\mathcal{U}$ is the control constraint.

This problem is difficult (if not impossible) to solve analytically; but we can solve it numerically to obtain an open-loop optimal control sequence. The optimal control sequence obtained at time step k is denoted: $\{\mathbf{u}_{k|k}^*, \dots, \mathbf{u}_{k+N-1|k}^*\}$. Note that we have indicated a certain optimality here with the $*$, and this optimality is that the control sequence obtained is optimal for the discrete finite time optimal control problem solved at k , not necessarily the infinite horizon control problem that this discrete finite time optimal control problem is approximating. Noticeably, from solving this discrete time finite optimization problem over and over, we get a stair step model similar to fig. 5.1. The obtained optimal control sequences at every time step in shown in fig. 5.4.

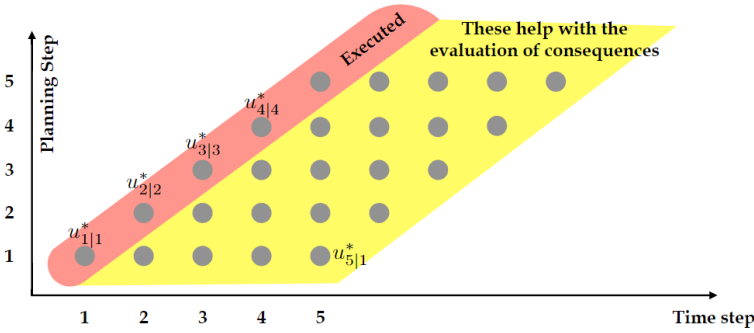


Figure 5.4: Illustration of the optimal control sequences obtained at every time step. Only the first optimal control is executed at every time step.

In fig. 5.5, we see the planned trajectories that correspond with the control sequences planned at the different time steps in fig. 5.4. Note that, as only the first action is taken at each time step, the actual executed trajectory (red line) is the the concatenation of the evolution of the dynamic system with respect to each of these first actions, assuming there is no external disturbance or model mismatch.

It should be noted that the optimal control problem solved at time $k + 1$ is not a subproblem or cost-to-go of the optimal control

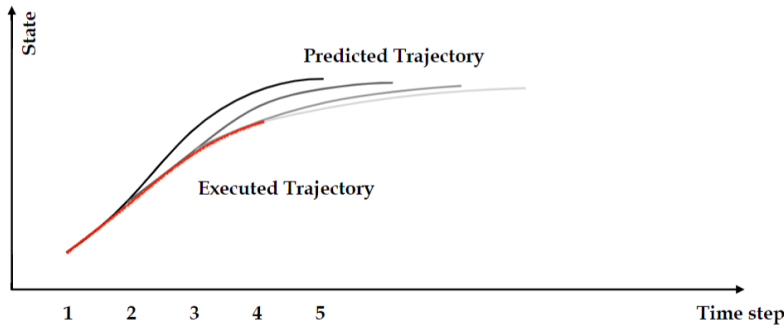


Figure 5.5: Trajectories associated with fig. 5.4.

problem solved at time k . There are two reasons. First, if there are disturbances or model mismatch, the actual evolution of the system may not be exactly the same as predicted. Hence the initial state for the problem at $k + 1$ may not be exactly the state $\mathbf{x}_{k+1|k}^*$ computed at time k . Second, even if the system evolves according to the prediction, the optimal control may be still be different due to the *change of control objectives*. Note that at time k , the optimization only considers costs from k to $k + N$. At time $k + 1$, the optimization considers costs from $k + 1$ to $k + 1 + N$. As a result, it is possible that

$$\mathbf{u}_{k+1|k+1}^* \neq \mathbf{u}_{k+1|k}^*. \quad (5.3)$$

Since the horizon for each MPC step is shorter than that of the original infinite horizon problem, it is challenging for MPC solutions to approximate the original infinite horizon problem. In order to make MPC work, it is ideal to let the terminal cost approximate the value function, i.e., the optimal cost-to-go for the original infinite horizon problem.

$$J_{k:k+N}(x_k; u_k, \dots, u_{k+N-1}) = \sum_{k'=k}^{k+N-1} l_{k'-k}(x_{k'}, u_{k'}) + \underline{l_N(x_{k+N})}$$

$$J_{k:\infty}(x_k; u_k, \dots) = \sum_{k'=k}^{k+N-1} l_{k'-k}(x_{k'}, u_{k'}) + \underline{\sum_{k'=k+N}^{\infty} l_{k'-k}(x_{k'}, u_{k'})}$$

Figure 5.6: Comparing the MPC cost function to the infinite horizon optimal control problem cost function. We are attempting to approximate the underlined term in the second equation with the underlined term in the first equation.

Two Ways to Implement MPC There are generally two methods to solve and implement MPC problems. The **Open-loop approach** repeatedly solves a mathematical program for the current initial state.

This approach yields an **implicit control law**, and no offline computation is required. Furthermore, the open-loop approach is able to account for time-varying dynamics and time-varying cost functions. The other approach is the **Closed-loop approach**, which pre-computes the optimal actions for each state offline and composes a control law from the entire set of optimal actions. This approach yields an **Explicit Control Law**, which is easy to verify and compute during run-time as it only requires online function evaluation instead of online optimization. A comparison between the open-loop approach and the closed-loop approach is illustrated in fig. 5.7.

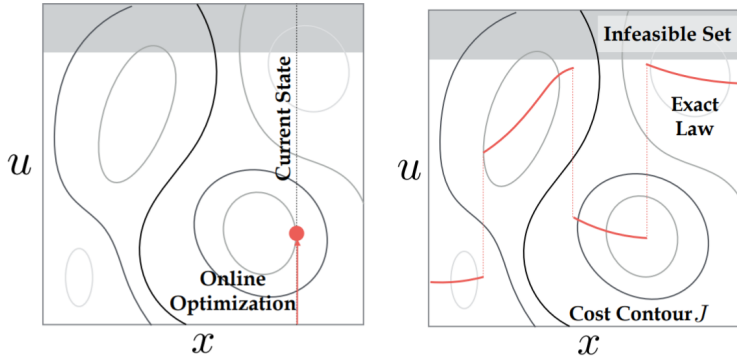


Figure 5.7: Comparison between the open-loop approach (left) and the closed-loop approach (right) to solve the MPC problems. The contours represent the cost contours for the specific (state, control) pairs. In the open-loop approach, the online optimization occurs along the dashed-line, which solves for the lowest cost action sequence, corresponding to the red dot. That action is then taken. In the closed-loop approach, for any given state, the system takes the action suggested by the red line, which corresponds to our offline computed control law. Note that by calculating the control law offline, we can avoid the infeasible set given in the state-control space.

5.3 Unconstrained Linear Quadratic MPC

We start this section by comparing the three optimal control formulations (finite horizon LQR, infinite horizon LQR, MPC) for linear systems with dynamics of the form:

$$\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k. \quad (5.4)$$

The run-time costs are written as $\frac{1}{2}[\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k]$, while the terminal cost is $\frac{1}{2}\mathbf{x}_N^T S \mathbf{x}_N$.

Finite-time LQR solves the following cost function J and obtains the following optimal control law.

$$J = \frac{1}{2} \sum_{k=0}^{N-1} [\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k] + \frac{1}{2} \mathbf{x}_N^T S \mathbf{x}_N$$

$$\mathbf{u}_k = - \underbrace{[R + B^T P_{k+1} B]^{-1} B^T P_{k+1} A}_{K_k} \mathbf{x}_k$$

Infinite-time LQR solves the following cost function J and obtains

the following optimal control law.

$$J = \frac{1}{2} \sum_{k=0}^{\infty} [\mathbf{x}_k^T Q \mathbf{x}_k + \mathbf{u}_k^T R \mathbf{u}_k]$$

$$\mathbf{u}_k = - \underbrace{[B^T P_{\infty} B + R]^{-1} B^T P_{\infty} A}_{K_{\infty}} \mathbf{x}_k$$

MPC solves the following cost function J at every time step k and obtains the following optimal control law.

$$J_{k:k+N} = \frac{1}{2} \sum_{k'=k}^{k+N-1} [\mathbf{x}_{k'}^T Q \mathbf{x}_{k'} + \mathbf{u}_{k'}^T R \mathbf{u}_{k'}] + \frac{1}{2} \mathbf{x}_{k+N}^T S \mathbf{x}_{k+N}$$

$$\mathbf{u}_k = - \underbrace{[R + B^T P_1 B]^{-1} B^T P_1 A}_{K_0} \mathbf{x}_k$$

The relationship among K_k , K_{∞} , and K_0 as well as the relationship among the corresponding P matrices in the Riccati equation are illustrated in fig. 5.8.

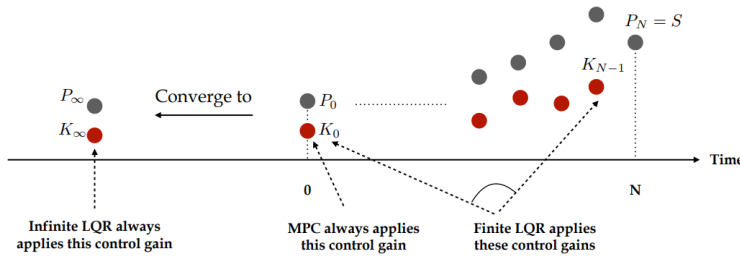


Figure 5.8: A comparison of the optimal control gains in finite-time, infinite-time, and MPC.

Example Consider a one-dimensional double integrator with sampling time T ,

$$x_{k+1} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} \frac{T^2}{2} \\ T \end{bmatrix} u_k, \quad (5.5)$$

with a cost-function

$$J = \frac{1}{2} \sum_{k=0}^{N-1} [x_k^T Q x_k + u_k^T R u_k] + \frac{1}{2} x_N^T S x_N. \quad (5.6)$$

Let us now illustrate the optimal trajectories computed at every MPC step k by simulating this system using $N = 10$, $T = 0.5$,

$Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, $R = 10$, and $S = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$. Note that we choose a

zero terminal cost in order to make the optimal trajectories at different time steps more “distinguishable”. The resulting trajectories are shown in fig. 5.9. These trajectories are “computed trajectories”

not the “executed trajectories”. Only the first step of each computed trajectory is being executed. In other words, the executed trajectory is the concatenation of the first steps of the computed trajectories.

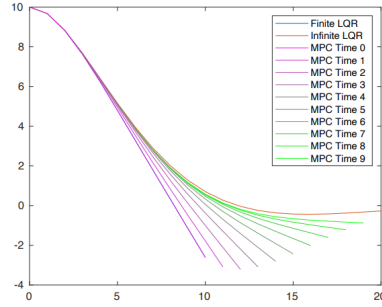


Figure 5.9: Illustration of the optimal trajectories obtained at different MPC steps, and the optimal trajectories for Finite and Infinite Horizon LQRs.

Let us now compare the executed MPC trajectory with the optimal trajectories from finite horizon LQR and infinite horizon LQR. We perform the simulation with the following parameters $N = 10$, $T = 0.5$, $Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$, $R = 1$, and $S = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix}$. The resulting trajectories are shown in fig. 5.10. One important observation is that the trajectories for MPC and infinite horizon LQR overlap. Why does it happen? That is because N is long enough such that the control gain K_0 for MPC has already converged to the control gain K_∞ for infinite horizon LQR. Another observation is that the trajectory for finite horizon LQR has a smaller overshoot compared to the other two trajectories. That is mainly because $S \succ P_\infty$, i.e., the penalty on the final state is bigger in the finite horizon LQR than in other two cases.

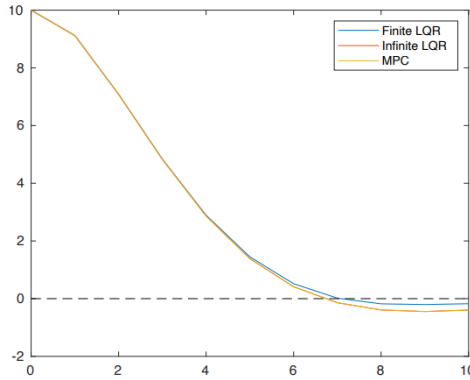


Figure 5.10: Simulated trajectories for finite time LQR, infinite time LQR, and MPC. Note that the finite LQR controller has smaller overshoot due to the big penalty on S .

Now under the same parameters, we investigate how the preview horizon N will affect the MPC trajectory. Note that the control gains are different when N changes, which will then result in different trajectories. As we increase the preview horizon N , we start to approach the trajectory for infinite horizon LQR (however, this does depend on

the convergence rate of the Riccati equation). Note that one step MPC is extremely sensitive to the terminal condition S . These different trajectories are shown in 5.11.

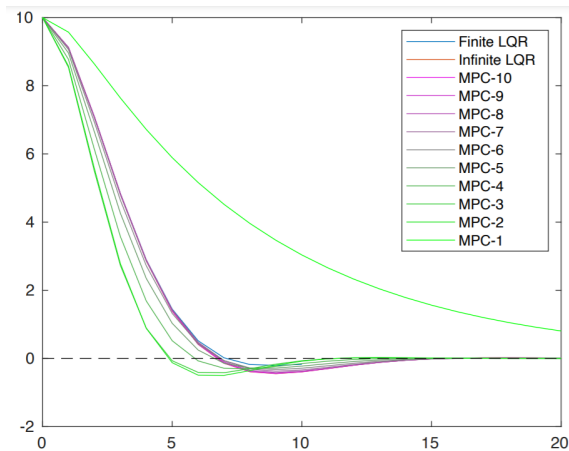


Figure 5.11: MPC Problem solved with different preview horizons.

The choice of the terminal cost S , a design parameter available to the control engineer, also affects the control gain in MPC. Moreover, the impact of the terminal state cost is smaller with a longer preview horizon. Should we choose to set the terminal cost to the value function of the infinite horizon LQR optimal control problem, i.e., $S = P_\infty$, then MPC (as well as finite horizon LQR) will perform the same as the infinite LQR as shown in fig. 5.12. In this case, it does not matter what preview horizon N we choose. That is because $P_k \equiv P_\infty$ in the Riccati equation for all k . Under a better terminal cost (closer to the value function), MPC can perform well with a shorter preview horizon.

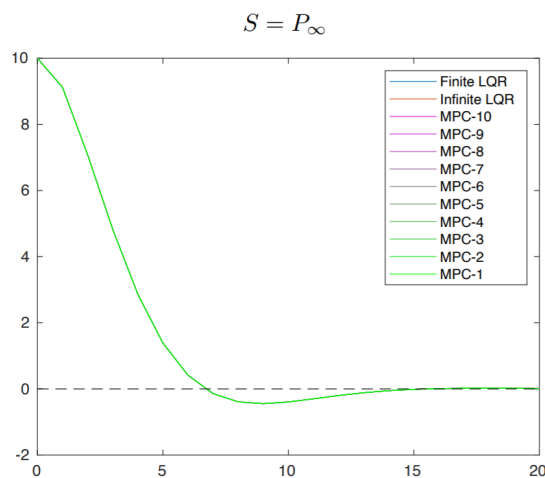


Figure 5.12: When $S = P_\infty$, we notice that preview horizon N does not affect the control gain and thus trajectory for the system.

The trouble is, we do not know the true value function or how close our estimate is. If we did, there is no reason to do MPC because we would have just solved the optimal control problem out right. Thus, tuning the terminal cost and preview horizon constraints is a bit of an art, as one needs to find the desired hyper-parameters, the trade-offs being shown in 5.13.

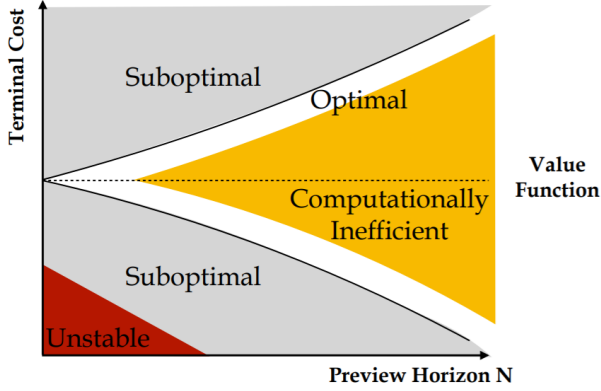


Figure 5.13: The challenges when selecting the preview horizon and the terminal cost constraints. Note that you want to remain as close as possible to the optimal curve (the bold lines). Moving too far past the optimal curve to the right indicates that you have too long of a preview horizon for your approximated value function and you should decrease the used time horizon. Finally, while sub-optimality is not preferred, it is extremely important to ensure the stability of the closed-loop system. Avoid the red area at all costs!

5.4 Constrained Linear Quadratic MPC

This section studies the constrained version of the linear quadratic MPC. Consider the set, $\mathcal{X}$, which represents our state constraint. Furthermore, consider set, $\mathcal{U}$, which represents our control constraint. Additionally, assume that sets $\mathcal{X}$ and $\mathcal{U}$ are polyhedra, that is that they may be represented as linear inequality constraints. The current state x_k at time k is also expected to be known. Our goal is to solve for the control variables needed to minimize the following quadratic cost function on linear dynamics, considering all constraints. Mathematically, the above-described problem can be written as

$$\begin{aligned} \min_{\mathbf{u}_k, \dots, \mathbf{u}_{k+N-1}} J_{k:k+N} &= \frac{1}{2} \sum_{k'=k}^{k+N-1} [\mathbf{x}_{k'}^T Q \mathbf{x}_{k'} + \mathbf{u}_{k'}^T R \mathbf{u}_{k'}] + \frac{1}{2} \mathbf{x}_{k+N}^T S \mathbf{x}_{k+N} \\ \text{s.t. } \mathbf{x}_{k'+1} &= A \mathbf{x}_{k'} + B \mathbf{u}_{k'}, \forall k' = k, \dots, k+N-1 \\ \mathbf{x}_{k'} &\in \mathcal{X}, \mathbf{u}_{k'} \in \mathcal{U}, \forall k' = k, \dots, k+N-1 \\ \mathbf{x}_{k+N} &\in \mathcal{X}_f. \end{aligned} \quad (5.7)$$

where $\mathcal{X}_f$ is the terminal constraint, which is a design parameter that we choose to ensure the feasibility of the subsequent MPC problems; S is the terminal cost matrix, which is a design parameter that we choose to ensure stability of the MPC trajectory. The problem (5.7) can be solved by quadratic programming.

The standard form of a quadratic programming problem is:

$$\begin{aligned} \min_U &= \frac{1}{2} U^T Q U + C^T U \\ \text{s.t. } & AU \leq B, \end{aligned} \quad (5.8)$$

where $U \in \mathbb{R}^n$; $C \in \mathbb{R}^n$; $A \in \mathbb{R}^{m \times n}$; $b \in \mathbb{R}^m$; and $Q \in \mathbb{R}^{n \times n}$ is symmetric positive semi-definite. In order to get the problem into the standard form, we can lift the dynamics. To do so, we first define:

$$X_k = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{x}_{k+1} \\ \vdots \\ \mathbf{x}_{k+N} \end{bmatrix}, \quad U_k = \begin{bmatrix} \mathbf{u}_k \\ \mathbf{u}_{k+1} \\ \vdots \\ \mathbf{u}_{k+N-1} \end{bmatrix}. \quad (5.9)$$

Then we lift the dynamics:

$$X_k = \underbrace{\begin{bmatrix} I \\ A \\ A^2 \\ \vdots \\ A^N \end{bmatrix}}_{\tilde{f}} \mathbf{x}_k + \underbrace{\begin{bmatrix} 0 & 0 & \cdots & 0 \\ B & 0 & \cdots & 0 \\ AB & B & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}}_{\tilde{B}} U_k. \quad (5.10)$$

When we lift the dynamics, we must also lift the cost function:

$$J_{k:k+N} = \frac{1}{2} \sum_{k'=k}^{k+N-1} [\mathbf{x}_{k'}^T Q \mathbf{x}_{k'} + \mathbf{u}_{k'}^T R \mathbf{u}_{k'}] + \frac{1}{2} \mathbf{x}_{k+N}^T S \mathbf{x}_{k+N}. \quad (5.11)$$

Given the lifted system dynamics $X_k = \tilde{f} + \tilde{B}U_k$, we can rewrite the cost function as

$$\begin{aligned} J_{k:k+N} &= \frac{1}{2} X_k^T \underbrace{\begin{bmatrix} Q & 0 & \cdots & 0 & 0 \\ 0 & Q & \ddots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \ddots & Q & 0 \\ 0 & 0 & \cdots & 0 & S \end{bmatrix}}_{\tilde{Q}} X_k + \frac{1}{2} U_k^T \underbrace{\begin{bmatrix} R & 0 & \cdots & 0 \\ 0 & R & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & R \end{bmatrix}}_{\tilde{R}} U_k \\ &= \frac{1}{2} U_k^T [\tilde{B}^T \tilde{Q} \tilde{B} + \tilde{R}] U_k + \tilde{f}^T \tilde{Q} \tilde{B} U_k + \frac{1}{2} \tilde{f}^T \tilde{Q} \tilde{f}. \end{aligned} \quad (5.12)$$

Furthermore, we must consider the system constraints. Suppose the constraints (including the terminal constraint) are written in the form:

$$L_x \mathbf{x} \leq \mathbf{b}_x, \quad L_u \mathbf{u} \leq \mathbf{b}_u. \quad (5.13)$$

We see that we can combine the constraints together to make a large constraint matrix, shown below

$$\begin{aligned}
 & \underbrace{\begin{bmatrix} L_u & 0 & \cdots & 0 \\ 0 & L_u & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & L_u \end{bmatrix}}_{\bar{L}_u} U_k \leq \underbrace{\begin{bmatrix} b_u \\ b_u \\ \vdots \\ b_u \end{bmatrix}}_{\bar{b}_u}, \quad \underbrace{\begin{bmatrix} L_x & 0 & \cdots & 0 \\ 0 & L_x & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & L_x \end{bmatrix}}_{\bar{L}_x} X_k \leq \underbrace{\begin{bmatrix} b_x \\ b_x \\ \vdots \\ b_x \end{bmatrix}}_{\bar{b}_x} \\
 & \Rightarrow \begin{bmatrix} \bar{L}_u \\ \bar{L}_x \bar{B} \end{bmatrix} U_k \leq \begin{bmatrix} \bar{b}_u \\ \bar{b}_x - \bar{L}_x \bar{f} \end{bmatrix}. \quad (5.14)
 \end{aligned}$$

These allow us to arrive at the formulation of the MPC problem as a quadratic program:

$$\min_{U_k} \quad \frac{1}{2} U_k^T \left[\bar{B}^T \bar{Q} \bar{B} + \bar{R} \right] U_k + \bar{f}^T \bar{Q} \bar{B} U_k \quad (5.15)$$

$$\text{s.t.} \quad \begin{bmatrix} \bar{L}_u \\ \bar{L}_x \bar{B} \end{bmatrix} U_k \leq \begin{bmatrix} \bar{b}_u \\ \bar{b}_x - \bar{L}_x \bar{f} \end{bmatrix} \quad (5.16)$$

However, we still have questions of whether this quadratic program is even feasible to solve. That question will be discussed in the next section.

5.5 Feasibility and Persistent Feasibility

We say the following optimization problem at time k starting from $\mathbf{x}_k$ is *feasible* if there exists a nontrivial solution for the control sequence:

$$\begin{aligned}
 & \min_{\mathbf{u}_k, \mathbf{u}_{k+1}, \dots, \mathbf{u}_{k+N-1}} \quad J_{k:k+N}(\mathbf{x}_k; \mathbf{u}_k, \mathbf{u}_{k+1}, \dots, \mathbf{u}_{k+N-1}), \\
 & \text{s.t.} \quad \mathbf{x}_{k'+1} = f(\mathbf{x}_{k'}, \mathbf{u}_{k'}), \forall k' = k, \dots, k + (N-1) \\
 & \quad \mathbf{x}_{k'+1} \in \mathcal{X}, \forall k' = k, \dots, k + (N-1) \\
 & \quad \mathbf{u}_{k'} \in \mathcal{U}, \forall k' = k, \dots, k + (N-1)
 \end{aligned} \quad (5.17)$$

The set of all possible $\mathbf{x}_k$ such that the optimization problem is feasible is denoted $\mathcal{X}_0$. However, $\mathbf{x}_k \in \mathcal{X}_0$ does not necessarily imply that the optimization problem is feasible for all future times $k+1, k+2, \dots$. It is desirable to design a form of MPC such that our optimization will be feasible for all future time steps, not just at the current time step. We refer to this property as **Persistent Feasibility**.

Let us look at this concept in more detail in fig. 5.14. At time k , the system finds a feasible trajectory; but when the system execute that trajectory, it ends up at a state $\mathbf{x}_{k+1}$ that is no longer $\mathcal{X}_0$; hence the optimization problem at $k+1$ is no longer feasible. This implies that we need some form of set invariance to ensure persistent feasibility for all future time steps.

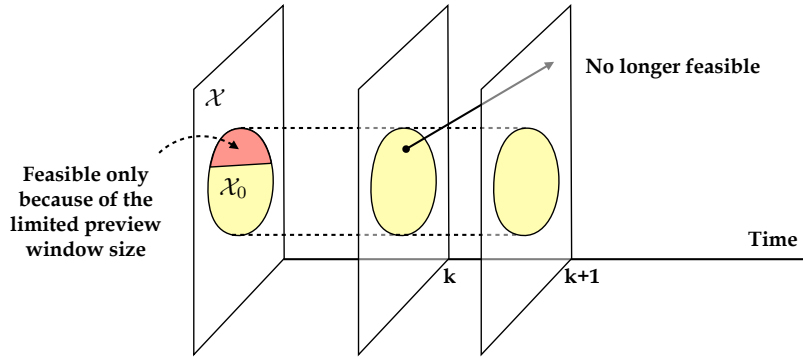


Figure 5.14: Feasible states in multi-time step planning.

Controllable Sets and Invariant Sets In the following discussion, we will formalize this invariance by defining a **Controllable Set** and an **Invariant Set** over the following system:

$$\begin{aligned} \mathbf{x}_{k+1} &= f(\mathbf{x}_k, \mathbf{u}_k), \forall \{k = 0, 1, \dots\}, \\ \mathbf{x}_k &\in \mathcal{X}, \forall \{k = 0, 1, \dots\} \\ \mathbf{u}_k &\in \mathcal{U}, \forall \{k = 0, 1, \dots\} \end{aligned} \quad (5.18)$$

where the system dynamics and the sets $\mathcal{X}$ and $\mathcal{U}$ are all time-invariant.

The **one-step controllable set** is defined as

$$\text{Pre}(\mathcal{S}) := \{\mathbf{x} \in \mathbb{R}^n : \exists \mathbf{u} \in \mathcal{U} \text{ s.t. } f(\mathbf{x}, \mathbf{u}) \in \mathcal{S}\}. \quad (5.19)$$

The **N-step controllable set** from set $\mathcal{S}$, denoted $\mathcal{K}_N(\mathcal{S})$, can be computed recursively as

$$\mathcal{K}_j(\mathcal{S}) = \text{Pre}(\mathcal{K}_{j-1}(\mathcal{S})) \cap \mathcal{X}, \mathcal{K}_0(\mathcal{S}) = \mathcal{S}, \forall j = 1, \dots, N. \quad (5.20)$$

The next few figures (figs. 5.15 to 5.17) pictorially show how to construct $\mathcal{X}_0$ from the controllable sets. What is occurring is that we are propagating backward in time the different controllable sets required at that instance. We then take the intersection of all these controllable sets to determine $\mathcal{X}_0$. For the N step optimization problem eq. (5.17) to be feasible, the initial state should lie in $\mathcal{X}_0 = \bigcap_{i=0}^N \mathcal{K}_i(\mathcal{X})$.

Next, we introduce the concept of the control invariant set. A set $\mathcal{C} \subseteq \mathcal{X}$ is said to be a **control invariant set** if

$$\mathbf{x}_k \in \mathcal{C} \implies \exists \mathbf{u}_k \in \mathcal{U} \text{ s.t. } f(\mathbf{x}_k, \mathbf{u}_k) \in \mathcal{C}, \forall k. \quad (5.21)$$

Moreover, a set $\mathcal{C}_\infty$ is said to be the **maximal control invariant set** if it is both control invariant and it contains all control invariant sets

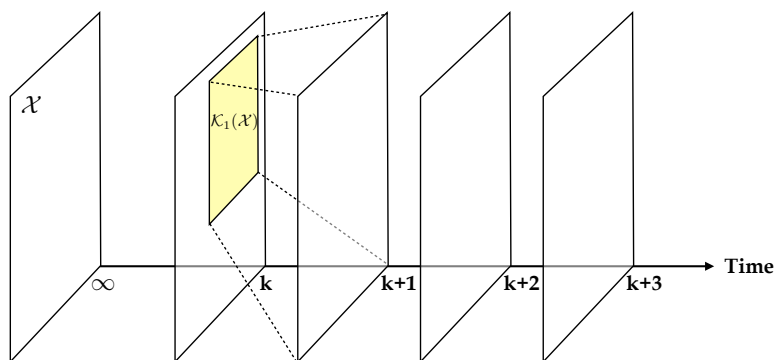


Figure 5.15: The required feasible set at time $k + 1$ projected onto all sets in time step k .

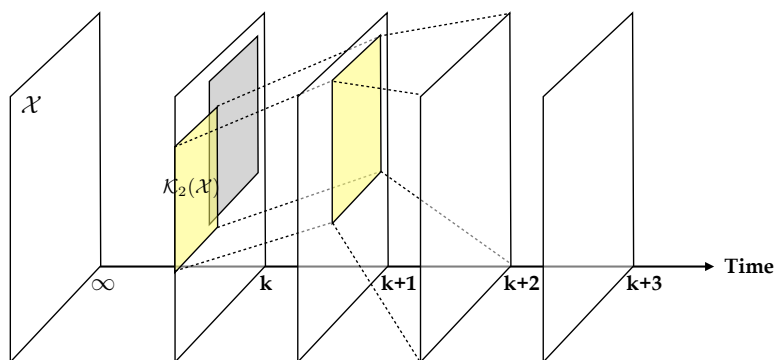


Figure 5.16: The required feasible set at time $k + 2$ projected onto all sets in time step k .

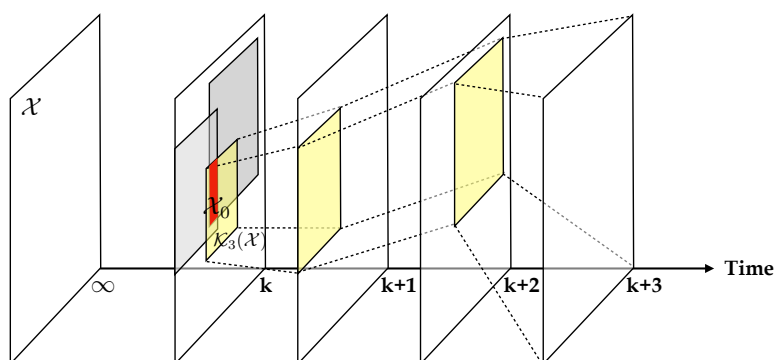


Figure 5.17: The required feasible set at time $k + 3$ projected onto all sets in time step k . The red section is the set required for persistent feasibility.

contained in $\mathcal{X}$. We have a general theorem about the geometric condition for invariance.

Theorem 5.1. *A set $\mathcal{C} \subseteq \mathcal{X}$ is a control invariant set if and only if $\mathcal{C} \subseteq \text{Pre}(\mathcal{C})$.*

Proof. If $\mathcal{C} \subseteq \text{Pre}(\mathcal{C})$, we want to show that $\mathcal{C}$ is a control invariant set. For any $x \in \mathcal{C}$, we have $x \in \text{Pre}(\mathcal{C})$. According to the definition of the controllable set, we have

$$\exists u \in \mathcal{U}, \text{ s.t. } f(x, u) \in \mathcal{C}.$$

Then the definition of control invariant set is satisfied. Hence $\mathcal{C}$ is control invariant.

If $\mathcal{C}$ is a control invariant set, we want to show $\mathcal{C} \subseteq \text{Pre}(\mathcal{C})$. This can be directly obtained from the definitions. Since $\forall x \in \mathcal{C}, \exists u \in \mathcal{U}, \text{ s.t. } f(x, u) \in \mathcal{C}$, then $\mathcal{C}$ must be a subset of $\text{Pre}(\mathcal{C})$. $\square$

Corollary. *$\mathcal{C} \subseteq \mathcal{X}$ is control invariant implies that $\text{Pre}(\mathcal{C}) \cap \mathcal{X}$ is also control invariant.*

Proof. According to the theorem, $\mathcal{C} \subseteq \mathcal{X}$ is control invariant implies that $\mathcal{C} \subseteq \text{Pre}(\mathcal{C})$. By definition of controllable set, $\forall x \in \text{Pre}(\mathcal{C})$,

$$\exists u \in \mathcal{U}, \text{ s.t. } f(x, u) \in \mathcal{C} \subseteq \text{Pre}(\mathcal{C}).$$

Hence $\text{Pre}(\mathcal{C}) \cap \mathcal{X}$ is also control invariant. $\square$

The relationship between $\mathcal{C}$ and $\text{Pre}\mathcal{C}$ is illustrated in fig. 5.18.

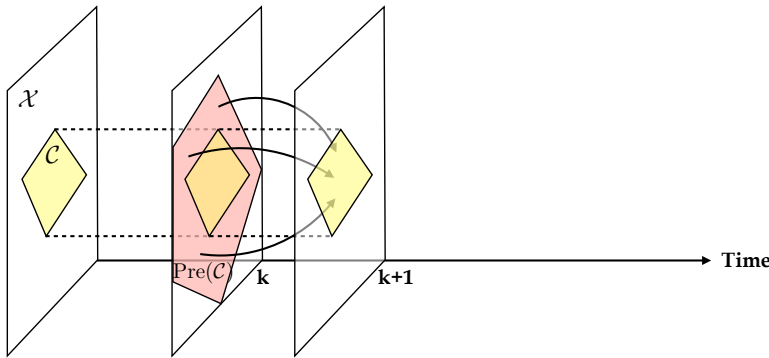


Figure 5.18: A figure capturing the idea of control invariant set. Notice how $\text{Pre}(\mathcal{C})$ maps to $\mathcal{C}$ in the next time step and $\mathcal{C}$ is consistent across time steps.

Computing the Maximum Control Invariant Set Starting from a control invariant set, $\mathcal{C} \subseteq \mathcal{X}$, the theorem implies that

$$\mathcal{C} \subseteq \mathcal{K}_1(\mathcal{C}) \subseteq \mathcal{K}_2(\mathcal{C}) \subseteq \dots \quad (5.22)$$

This sequence may or may not converge to the maximum control invariant set, depending on the choice of the initial $\mathcal{C}$. For example, as shown in fig. 5.19, if the maximum control invariant set contains two disjoint components and if we start only from one of the disjoint components, then we will never be able to recover the other disjoint components by gradually expanding the control invariant set.

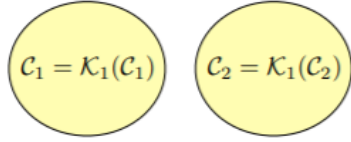


Figure 5.19: Starting from one of the disjoint control invariant sets and gradually expanding the control invariant set are not guaranteed to converge to the maximum control invariant set.

If iterative expansion may not work, how about the other way around – iterative reduction? According to the theorem, for any set $\mathcal{S}$, the points in $\mathcal{S} \setminus \text{Pre}(\mathcal{S})$ should not belong to the control invariant set. Hence we can start from $\mathcal{S} := \mathcal{X}$ and iteratively remove points that do not belong to $\text{Pre}(\mathcal{S})$. The general process is outlined below. The inputs are the dynamics f and the constraints $\mathcal{X}$ and $\mathcal{U}$. The output from the algorithm is $\mathcal{C}_\infty$. We take the following steps:

1. Let $\Omega_0 \leftarrow \mathcal{X}$
2. Repeat:
3. $k = k + 1$
4. $\Omega_{k+1} \leftarrow \text{Pre}(\Omega_k) \cap \Omega_k$
5. until $\Omega_{k+1} = \Omega_k$
6. $\mathcal{C}_\infty \leftarrow \Omega_{k+1}$

This algorithm will converge if the dynamics are continuous and the control and state set are bounded.

With that, we can actually characterize the control invariant set as the intersection of the n -step controllable sets for $\mathcal{X}$ for any n . Let us define that intersection as $\mathcal{K}_\infty$ which is controllable for any step:

$$\mathcal{K}_\infty = \bigcap_{i=0}^{\infty} \mathcal{K}_i(\mathcal{X}) \subseteq \mathcal{X}_0. \quad (5.23)$$

Now we show $\mathcal{K}_\infty = \mathcal{C}_\infty$. It is apparent that $\mathcal{K}_\infty \subseteq \text{Pre}(\mathcal{K}_\infty)$, hence $\mathcal{K}_\infty$ is a control invariant set according to the theorem. Additionally, for any $x \notin \mathcal{K}_\infty$, there exists some i such that $x \notin \mathcal{K}_i(\mathcal{X})$, which implies that there does not exist any control sequence that will keep the state inside $\mathcal{X}$ at i steps later. Then $x \notin \mathcal{C}_\infty$. Hence $\mathcal{C}_\infty = \mathcal{K}_\infty$.

Persistent Feasibility With the notion of control invariance, then we can state the following theorem about persistent feasibility.

Theorem 5.2. *If $\mathcal{X}_f$ is a control invariant set, then the MPC problem (5.17) is persistently feasible.*

The proof of this statement comes from a series of intuitive points:

- Since $\mathcal{X}_f$ is control invariant, then $\mathcal{K}_i(\mathcal{X}_f)$ are control invariant for all i .
- In particular, we are interested in $\mathcal{K}_{N-1}(\mathcal{X}_f)$, since the next state lies in this set.
- Since $\mathcal{K}_{N-1}(\mathcal{X}_f)$ is control invariant, then the optimization problem with an initial state from $\mathcal{K}_{N-1}(\mathcal{X}_f)$ and a terminal constraint $\mathcal{X}_f$ is always feasible.

Pictorially, we can see this as the progression of n -step controllable sets either remaining the same shape or collapsing in future time steps.

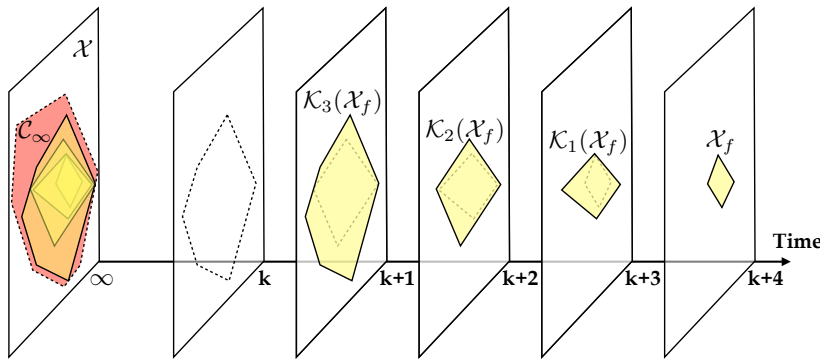


Figure 5.20: Illustration of persistent feasibility of MPC using control invariant sets.

Note that persistent feasibility is not dependent on the cost function. Furthermore, the control-invariant constraint can be not only added to the terminal step, but also any other time step with $i > 0$.

5.6 Stability of Constrained Linear Quadratic MPC

This section discusses the stability of closed-loop MPC trajectories. Consider the following constrained linear quadratic MPC problem.

$$\begin{aligned}
\min_{\mathbf{u}_k, \dots, \mathbf{u}_{k+N-1}} J_{k:k+N} &= \frac{1}{2} \sum_{k'=k}^{k+N-1} [\mathbf{x}_{k'}^T Q \mathbf{x}_{k'} + \mathbf{u}_{k'}^T R \mathbf{u}_{k'}] + \frac{1}{2} \mathbf{x}_{k+N}^T S \mathbf{x}_{k+N} \\
\text{s.t. } \mathbf{x}_{k'+1} &= A \mathbf{x}_{k'} + B \mathbf{u}_{k'}, \forall k' = k, \dots, k+N-1 \\
\mathbf{x}_{k'} &\in \mathcal{X}, \mathbf{u}_{k'} \in \mathcal{U}, \forall k' = k, \dots, k+N-1 \\
\mathbf{x}_{k+N} &\in \mathcal{X}_f,
\end{aligned} \tag{5.24}$$

with the following assumptions:

(Ao) Q, R, S are all symmetric positive definite. This is to ensure strong convexity and asymptotic convergence.

(A1) The sets $\mathcal{X}, \mathcal{X}_f, \mathcal{U}$ contain the origin in their interior and are closed, which is made to ensure that the origin is the closed-loop equilibrium.

(A2) The set $\mathcal{X}_f$ is control invariant, which implies that we have persistent feasibility.

(A3) We have the following inequality

$$\min_{\mathbf{u} \in \mathcal{U}, A\mathbf{x} + B\mathbf{u} \in \mathcal{X}_f} [-\mathbf{x}^T S \mathbf{x} + \mathbf{x}^T Q \mathbf{x} + \mathbf{u}^T R \mathbf{u} + (A\mathbf{x} + B\mathbf{u})^T S (A\mathbf{x} + B\mathbf{u})] \leq 0, \forall \mathbf{x} \in \mathcal{X}_f, \tag{5.25}$$

which ensures that the cost can be decreased at two consecutive time steps. With that assumption, we can always find a smaller cost at the next planning step since $\forall \mathbf{x}_k$,

$$\begin{aligned}
& 2J_{k+1:k+N+1}^*(\mathbf{x}_{k+1}) - 2J_{k:k+N}^*(\mathbf{x}_k) \\
& \leq \sum_{k'=k+1}^{k+N} [\mathbf{x}_{k'|k+1}^{*T} Q \mathbf{x}_{k'|k+1}^* + \mathbf{u}_{k'|k+1}^{*T} R \mathbf{u}_{k'|k+1}^*] + \mathbf{x}_{k+N+1|k+1}^{*T} S \mathbf{x}_{k+N+1|k+1}^* \\
& \quad - \mathbf{x}_k^T Q \mathbf{x}_k - \mathbf{u}_{k|k}^{*T} R \mathbf{u}_{k|k}^* - \sum_{k'=k+1}^{k+N-1} [\mathbf{x}_{k'|k+1}^{*T} Q \mathbf{x}_{k'|k+1}^* + \mathbf{u}_{k'|k+1}^{*T} R \mathbf{u}_{k'|k+1}^*] - \mathbf{x}_{k+N|k+1}^{*T} S \mathbf{x}_{k+N|k+1}^* \\
& = -\mathbf{x}_k^T Q \mathbf{x}_k - \mathbf{u}_{k|k}^{*T} R \mathbf{u}_{k|k}^* - \mathbf{x}_{k+N|k+1}^{*T} S \mathbf{x}_{k+N|k+1}^* + \mathbf{x}_{k+N|k+1}^{*T} Q \mathbf{x}_{k+N|k+1}^* + \mathbf{u}_{k+N|k+1}^{*T} R \mathbf{u}_{k+N|k+1}^* + \mathbf{x}_{k+N+1|k+1}^{*T} S \mathbf{x}_{k+N+1|k+1}^* \\
& \leq -\mathbf{x}_k^T Q \mathbf{x}_k - \mathbf{u}_{k|k}^{*T} R \mathbf{u}_{k|k}^*,
\end{aligned}$$

where the last inequality uses assumption A3.

Theorem 5.3. *If all assumptions (Ao-A3) holds, the state of the closed-loop MPC system converges to the origin. Moreover, the origin is asymptotically stable with domain of attraction $\mathcal{K}_N(\mathcal{X}_f)$.*

The proof is sketched below. By (A2), the MPC problem is persistently feasible. By (Ao) and (A1), the origin is the only closed-loop equilibrium. By (A3), the optimal cost $J_{k:k+N}^*(\mathbf{x}_k)$ is strictly decreasing. Hence the closed-loop system converges to the origin.

To prove that the origin is asymptotically stable with domain of attraction $\mathcal{K}_N(\mathcal{X}_f)$ requires Lyapunov analysis, which will be covered later.

Examples For unconstrained linear quadratic MPC, we have the control law $u_k = -K_0 x_k$. Then by plugging the control law to the left-hand-side of (5.25), we have the following expression

$$-x^T S x + x^T Q x + x^T K_0 R K_0 x + x^T (A - B K_0)^T S (A - B K_0) x. \quad (5.26)$$

The above expression will be smaller than 0 for all x if and only if

$$S \succeq Q + K_0^T R K_0 + (A - B K_0)^T S (A - B K_0). \quad (5.27)$$

By letting $S = P_\infty$ where P_∞ is the solution of the steady state Riccati equation, then K_0 becomes the steady state control gain K_∞ . Then the above inequality holds (it actually achieved equality). Hence the unconstrained linear quadratic MPC with $S = P_\infty$ is stable.

For constrained problems, it is also a good idea to choose $S = P_\infty$. Meanwhile, we should set $\mathcal{X}_f$ to be the maximum invariant set under control law $u = -K_\infty x$ with the constraint $u \in \mathcal{U}$. Then (5.25) holds (it achieves the equality under control law $u = -K_\infty x$). Hence, the constrained system is asymptotically stable around the origin by the stability theorem.

Final Remark What we have discussed so far are set-point regulation problems. MPC is used in a variety of scenarios, such as trajectory generation for autonomous driving, trajectory tracking with state constraints, setc. The challenge is that theoretical analysis of feasibility and stability become very difficult if we relax our linearity assumptions on the system dynamics or the convexity assumptions on the costs and constraints. There are still many open research problems.

6

Iterative Learning Control

We have studied optimal control using exact dynamic programming and model predictive control using “approximated” dynamic programming. They all result in feedback control laws. The future information is encoded in the co-states, hence considered in the control gain. However, if there are disturbances and model mismatch, the future may be incorrectly computed in the design phase, which results in problematic control gains.

This chapter first discusses model mismatch and overviews adaptation methods that can be used to handle model mismatch (section 6.1). Then we introduce Iterative Learning Control (ILC) as a feedforward adaptation method (section 6.2). The basic principles of ILC is introduced followed by the detailed discussion on lifted ILC (section 6.3).

6.1 Model Mismatch and Adaptation

Definition 6.1 (Model Mismatch). *Model mismatch refers to the situation that the model used for controller synthesis does not align with the actual system when the controller is applied.*

Model mismatch can come from errors introduced by linearization during controller synthesis, unmodeled dynamics, or unanticipated time-varying dynamics. We may use adaptive control to compensate model mismatch. There are different ways to perform the adaptation. We can either perform indirect or direct adaptation.

- Indirect adaptation directly adapts the system model. Consequently, the control law is changed, since the control gain depends on model parameters.
- Direct adaptation adapts the control law directly.

We may use either feedback or feedforward adaptation.

- Feedback adaptation modifies the feedback control law either directly or indirectly.
- Feedforward adaptation modifies the feedforward control law or the reference signal.

Moreover, it is worth noting that the adaptation frequency does not need to be synchronized with the control frequency. Figure 6.1 shows one case that the adaptation runs at a different frequency from the control loop. The adaptation takes information from time step $k - 2$ to $k + 2$ to update the control law at $k + 2$ step. On the other hand, the control only takes information from its current step.

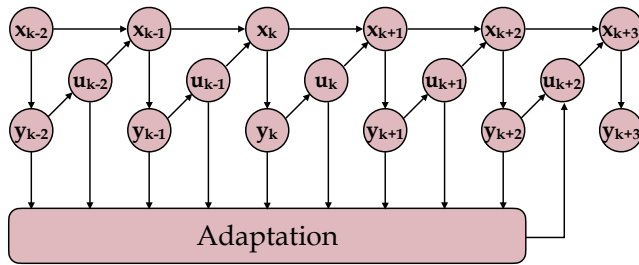


Figure 6.1: Illustration of an adaptation which runs at a different frequency from the control loop.

6.2 Iterative Learning Control: Basic Principle

Iterative Learning Control (ILC) learns to improve and eventually perfect its policy on repetitive tasks from previous experience. It belongs to direct feedforward adaptation. ILC only adapts locally to the specific task. An illustration of the basic principles in ILC is shown in fig. 6.2. The system is supposed to track the computed optimal trajectory shown as the thick red curve. However, due to model mismatch or unknown disturbances, the executed trajectory deviates from the desired optimal trajectory. Then ILC can learn the error from previous iterations to better track the optimal trajectory. After a few iterations, the tracking error becomes smaller. However, the feedforward signal learned by ILC may not generalize to other trajectories.

The initial ideas of ILC appear in the 1967 US patent “Learning control of actuators in control systems” and the 1984 Paper “Bettering operation of robots by learning” by Suguro Arimoto. Starting from 1990s, there are many new results, applications, and well-established theories.

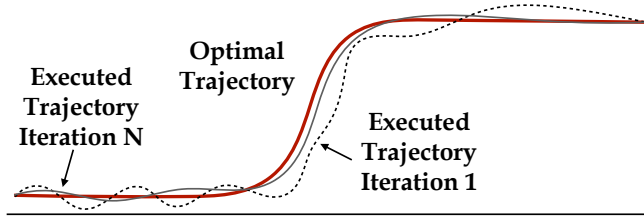


Figure 6.2: Illustration of basic principles of ILC.

6.2.1 Repetitive Tasks

Repetitive tasks are those with repeating trajectories. For example, for basketball players, the shooting trajectories are similar to each other; for industrial robots on production lines, their motion trajectories are exactly the same for individual workpieces of the same kind; for race cars, they are looping on the race track. Hence all of them can be regarded repetitive tasks.

Among repetitive tasks, there are two distinctions:

- Iteratively static task: where systems operate repetitively with states reset for every iteration;
- Iteratively dynamic task: where systems operate repetitively without state reset.

ILC only handles iteratively static tasks, since 1) ILC requires that the same initial conditions apply for every iteration; and 2) ILC may need the system to pause between two iterations in order to analyze the error from the last iteration and then compute the feedforward signal for the next iteration. On the other hand, iteratively dynamic tasks are also well studied in control theory, especially under repetitive control.

6.2.2 Limitations on Feedback Control

For iteratively static repetitive tasks, the errors are highly reproducible in each iteration. Hence, we can modify the system trajectory for the next iteration using the errors we received from previous iterations. The system trajectory can be modified either through feedback approaches or feedforward approaches. In a feedback approach, we need to analyze the characteristics of the error, and then adjust the feedback control gain. In a feedforward approach, we need to modify the reference signal or the control signal directly.

Feedback control has its own limitations. Notably, for a linear time-invariant system with linear time-invariant control law, we have the *waterbed effect* that limits the system performance. Waterbed effect refers to the phenomenon that if the sensitivity to disturbance is

suppressed at some frequency range, it is necessarily increased at some other range. As a result, we cannot get uniform improvement on control performance.

6.2.3 Sensitivity For Linear Systems

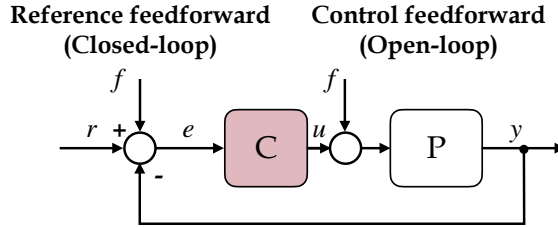


Figure 6.3: Illustration of a closed loop system with plant dynamics P and controller C , and the two places to inject feedforward signals.

Consider the closed-loop system shown in fig. 6.3. Assume the system is linear, where P denotes the transfer function for the plant dynamics (or we can call it as open-loop dynamics); C denotes the transfer function for the controller; r is the reference signal; e is the error; u is the feedback control signal to the plant; y is the output of the plant; f is the feedforward signal. For simplicity, there is no additional disturbance to the closed-loop system, and we treat the (unknown) reference r as the disturbance. Since the closed-loop system is linear, we have $y = Pu$ and $u = Ce$.¹

We define the sensitivity function as the closed-loop transfer function from the reference signal r to the error e . When there is no feedforward signal, the error can be represented as $e = r - y$ where $y = Pu = PCe$. Then we have $e = r - PCe$. Hence,

$$e = \underbrace{\frac{1}{1 + PC}}_S r, \quad (6.1)$$

where S is the sensitivity function. By only adjusting the feedback control C , it is impossible to get uniform decrease of the sensitivity across all frequencies, according to the waterbed effect.² To properly attenuate the error, feedforward control is needed.

For feedforward control, there are two places to inject feedforward signals as shown in fig. 6.3:

- Reference feedforward, which injects the feedforward f in the same channel as the reference r . Reference feedforward is also called closed-loop feedforward since the internal signal u in the loop is not directly modified.
- Control feedforward, which injects the feedforward f in the control channel. Control feedforward is also called open-loop feedforward since the internal signal u is directly modified.

¹ These two equalities should be interpreted in the frequency domain. In the time domain, P and C become two convolution operators. Nonetheless, these two functions satisfy basic linear operations. Hence our interpretation will not affect the following derivation. In the following derivation, these two functions P and C can be simply viewed as two linear operators (e.g., linear matrices).

² The integration of the sensitivity function across all frequencies should be constant under any control gain.

In the following discussion, we will focus on control feedforward. And the analysis can be easily transferred to reference feedforward.

With the control feedforward design, the error dynamics (6.1) will be different. Now since $y = P(u + f)$, then we have $e = r - PCe - Pf$. Rearranging the terms in the equation, we get³

$$e = Sr - PSf. \quad (6.2)$$

If we properly choose the feedforward signal, we can make the error uniformly decrease at all frequencies. However, since r is assumed to be unknown, how shall we choose f ? We will discuss the learning approach in the following sections.

³ The sensitivity function is the intrinsic property of a closed-loop system. The transfer function from any external signal to the error e equals to the open loop transfer function from that signal to e multiplying the sensitivity function. For example, for the control feedforward signal, its open-loop transfer function to e is P . Hence the closed-loop transfer function from f to e is PS .

6.2.4 Feedforward Adaptation

The fundamental problem that ILC addresses is specified below: Consider a time-invariant system P , a model of the system $\hat{P}$, a feedback controller C designed w.r.t. $\hat{P}$, an unknown signal r that is iteratively static, how shall we design a feedforward control law to attenuate the error?

With the fundamental problem in mind, let us see what happens at different iterations of ILC. We use superscript to denote the iteration number, i.e., $e^{(1)}$ denotes the error from the first iteration.

First Iteration In the first iteration, there is no feedforward signal, we simply measure the errors $e^{(1)} = Sr$.

Second Iteration In the second iteration, we include the feedforward signal, then error becomes $e^{(2)} = Sr - PSf^{(2)} = e^{(1)} - PSf^{(2)}$. Since the goal is to attenuate the error, the feedforward control should depend on the previous error. Let us assume that $f^{(2)} = Le^{(1)}$, where L is the transfer function from the error to the feedforward signal. Then the error dynamics become

$$e^{(2)} = e^{(1)} - PSLe^{(1)}. \quad (6.3)$$

Ideally, we should design L to be the inverse of PS . However, since we do not have the actual model, we choose

$$L = (\hat{P}\hat{S})^{-1}, \quad (6.4)$$

where $\hat{S} = 1/(1 + \hat{P}C)$. Note that the computation of the learning gain requires model inverse, which may not always be feasible. Moreover, due to model mismatch, $e^{(2)} = (1 - PSL)e^{(1)}$ may still be nonzero. Hence we shall do more learning.

Iteration $j + 1$ In order to leverage the previously learned information and to incorporate new errors, the feedforward control law is designed as $f^{(j+1)} = f^{(j)} + Le^{(j)}$. However, not all error signal contains useful information. Some of them may come from non iteratively static disturbances, which we should not learn. In order to improve robustness of the learning process, we should add a filter (denoted Q) to filter out the undesired signals from the past. As a result, the feedforward control law becomes

$$f^{(j+1)} = Qf^{(j)} + QLe^{(j)}. \quad (6.5)$$

Figure 6.4 illustrates the iteration-domain updates of ILC.

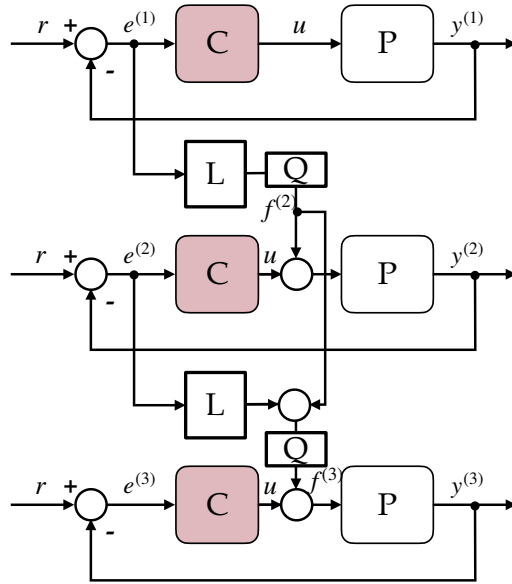


Figure 6.4: The block diagram of ILC. There are three copies of the time-domain closed-loop system. And the three copies correspond to three different iterations in ILC. The feedforward control law (6.5) feeds the errors from previous iterations to the future. There are two design parameters: L and Q . L is the learning gain. Q is the filter.

Now let us compute the error dynamics in ILC. The error in one iteration satisfies (6.2). We need to compute the relationship between errors in consecutive iterations. According to (6.2) and (6.5),

$$e^{(j+1)} = Sr - PSf^{(j+1)} \quad (6.6a)$$

$$= Sr - PSQf^{(j)} - PSQLe^{(j)} \quad (6.6b)$$

$$= (1 - Q)Sr + QSr - QPSf^{(j)} - QPSLe^{(j)} \quad (6.6c)$$

$$= (1 - Q)Sr + Qe^{(j)} - QPSLe^{(j)} \quad (6.6d)$$

$$= (1 - Q)Sr + Q(1 - PSL)e^{(j)} \quad (6.6e)$$

Hence the error at $(j + 1)$ th iteration is a weighted sum of errors directly coming from the reference r and the error in the previous

iteration. And the weights depend on the Q filter. To ensure stability across different iterations, we need to ensure that⁴

$$\|Q(1 - PSL)\|_\infty < 1. \quad (6.7)$$

⁴ Recall that this condition comes from linear stability analysis.

In the case that we do not have exact model of the system, as long as the above condition is satisfied, the error can be attenuated. Moreover, we have flexibility to design the Q filter for frequency-selective learning. The learning is turned on when we set $Q = 1$ and off when we set $Q = 0$. We should turn on the learning to improve performance and turn off the learning to improve robustness.

6.2.5 Different Types of ILC

The approaches introduced above can be either performed in frequency domain or time domain. Frequency domain ILC can handle infinite time problems or problems that have different time scales at different iterations. However, the drawback of frequency domain ILC is that it can only be applied to linear systems. Time domain ILC is also called lifted ILC, which can handle nonlinear systems. But lifted ILC requires the time steps at each iteration to be fixed. The introduction of ILC above is based on frequency domain ILC. To obtain the equations for time domain ILC, we just need to substitute all signals and functions with their time domain forms. In the following discussion, we will focus on lifted ILC.

6.3 Lifted Iterative Learning Control

Consider a time-domain closed-loop system where the state is taken as the tracking error

$$e_{k+1} = A_k e_k + B_k f_k + C_k r_k, \quad (6.8)$$

where f_k is the feedforward signal at time k ; r_k is the reference at time k . Since it is a closed-loop system model, we no longer have control input u . The initial error is set to $e_0 = 0$. Then let us lift the system in the iteration domain. Define the lifted reference R , the lifted feedforward signal $F^{(j)}$ at j th iteration, and the lifted tracking error $E^{(j)}$ at j th iteration as

$$R = \begin{bmatrix} r_0 \\ r_1 \\ \vdots \\ r_{N-1} \end{bmatrix}, F^{(j)} = \begin{bmatrix} f_0^{(j)} \\ f_1^{(j)} \\ \vdots \\ f_{N-1}^{(j)} \end{bmatrix}, E^{(j)} = \begin{bmatrix} e_1^{(j)} \\ e_2^{(j)} \\ \vdots \\ e_N^{(j)} \end{bmatrix} \quad (6.9)$$

Then the lifted system in the iteration domain can be written as

$$\begin{aligned}
 E^{(j)} = & \underbrace{\begin{bmatrix} A_0 \\ A_1 A_0 \\ \vdots \\ \Pi_{i=0}^{N-1} A_i \end{bmatrix}}_{\bar{A}} e_0 + \underbrace{\begin{bmatrix} B_0 & 0 & \dots & 0 \\ A_1 B_0 & B_1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \Pi_{i=0}^{N-2} A_i B_0 & \Pi_{i=0}^{N-3} A_i B_1 & \dots & B_{N-1} \end{bmatrix}}_{\bar{B}} F^{(j)} \\
 & + \underbrace{\begin{bmatrix} C_0 & 0 & \dots & 0 \\ A_1 C_0 & C_1 & \ddots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ \Pi_{i=0}^{N-2} A_i C_0 & \Pi_{i=0}^{N-3} A_i C_1 & \dots & C_{N-1} \end{bmatrix}}_{\bar{C}} R
 \end{aligned} \quad (6.10)$$

Since $e_0 = 0$, then the lifted dynamics can be compactly written as

$$E^{(j)} = \bar{B}F^{(j)} + \bar{C}R. \quad (6.11)$$

Note that (6.11) is equivalent to (6.2), and is static in the iteration-domain. There is no connection to signals in other iterations. Such connect will be introduced if we consider the feedforward control law. The feedforward control law in the lifted system can be written as

$$F^{(j+1)} = F^{(j)} + LE^{(j)}, \quad (6.12)$$

where L is the learning matrix for the lifted system. For simplicity, we assume that the Q filter is identity. Then (6.11) and (6.12) forms a closed-loop system in the iteration domain. To simplify the dynamics in the iteration domain, let us consider $E^{(j+1)}$ and substitute (6.12) into (6.11), then we have:

$$E^{(j+1)} = \bar{B}F^{(j+1)} + \bar{C}R \quad (6.13a)$$

$$= \bar{B}F^{(j)} + \bar{B}LE^{(j)} + \bar{C}R \quad (6.13b)$$

$$= E^{(j)} + \underbrace{\bar{B}LE^{(j)}}_{U^{(j)}}. \quad (6.13c)$$

The above equations capture the iteration-domain dynamics with state being $E^{(j)}$. In the last equation, we denote $LE^{(j)}$ as a control input $U^{(j)}$ in the iteration-domain dynamics.⁵ As a result, the learning gain L is then equivalent to the state-feedback control gain in the iteration-domain dynamics. And we can use our linear control design tools to obtain L . For example, we can pose the problem as a discrete-time LQR by specifying run-time cost at iteration j to be

$$l^{(j)}(E^{(j)}, U^{(j)}) = (E^{(j)})^T Q E^{(j)} + (U^{(j)})^T R U^{(j)} \quad (6.14)$$

And by applying the formula for LQR, we can obtain the optimal control gain L .

⁵ Note the control input $U^{(j)}$ is applied to change the feedforward signal according to (6.12), which is different from the control input u in the time-domain dynamics.

Examples We will show a numerical example for the lifted ILC. Consider the following time-domain dynamic system

$$e_{k+1} = 0.5e_k + f_k + c_k r_k, \quad (6.15)$$

where $c_k = \cos(k\pi/2)$. The horizon in each iteration is chosen to be $N = 4$. The reference signal is $R = [1; -1; 1; -1]$. In the first iteration, there is no feedforward signal, *i.e.*, $f_k \equiv 0$. After simulating the dynamics (6.15), we get the resulting error profile

$$E^{(0)} = [1; 0.5; -0.75; -0.375]. \quad (6.16)$$

According to the parameters specified above, the $\bar{B}$ and $\bar{C}$ matrices in the lifted system are

$$\bar{B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ 0.25 & 0.5 & 1 & 0 \\ 0.125 & 0.25 & 0.5 & 1 \end{bmatrix}, \bar{C} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 0 & 0 & 0 \\ 0.25 & 0 & -1 & 0 \\ 0.125 & 0 & -0.5 & 0 \end{bmatrix}. \quad (6.17)$$

Let us define the feedforward control gain as

$$L = \begin{bmatrix} -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}. \quad (6.18)$$

Simulating the system in the iteration domain according to (6.11) and (6.12), we get the following results.

At iteration 0,

$$F^{(0)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}, E^{(0)} = \begin{bmatrix} 1 \\ 1.5 \\ -0.75 \\ -0.375 \end{bmatrix}. \quad (6.19)$$

At iteration 1,

$$F^{(1)} = \begin{bmatrix} -1 \\ -1.5 \\ 0.75 \\ 0.375 \end{bmatrix}, E^{(1)} = \begin{bmatrix} 0 \\ -0.5 \\ -0.5 \\ 0.125 \end{bmatrix}. \quad (6.20)$$

At iteration 2,

$$F^{(2)} = \begin{bmatrix} -1 \\ 0 \\ 1.25 \\ 0.25 \end{bmatrix}, E^{(2)} = \begin{bmatrix} 0 \\ 0 \\ 0.25 \\ 0.375 \end{bmatrix}. \quad (6.21)$$

At iteration 3,

$$F^{(3)} = \begin{bmatrix} -1 \\ 0 \\ 1 \\ -0.125 \end{bmatrix}, E^{(3)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -0.125 \end{bmatrix}. \quad (6.22)$$

At iteration 4,

$$F^{(4)} = \begin{bmatrix} -1 \\ 0 \\ 1 \\ 0 \end{bmatrix}, E^{(4)} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}. \quad (6.23)$$

After four iterations, the errors all drop to zero. One interesting fact is that the k th error becomes zero at the k th iteration. In general, it is easier to make the time domain error in the earlier time steps drop to zero than the error in the later time steps. The reason is that the later in the time steps, the more influence that error term will receive from other signals (such as the feedforward signal, the feedback signal, and the reference), hence more difficult to balance out all different factors.

Part III

State Estimation and Model Adaptation

7

Probability and Least Squares

Starting from this chapter, we are going to deal with stochastic problems. The stochasticity may come from process disturbance, measurement noise, or dynamic uncertainty. To deal with stochastic problems, we may derive a stochastic policy directly, or decouple the problem into two parts: parameter estimation and then control with the estimated parameter in deterministic policies. We will first discuss the decoupled approach. And the first step in that approach is to perform parameter estimation. In this chapter, we will first review some basic principles in probability theory that are useful for parameter estimation, then discuss least square methods for parameter estimation, which will be used in the following chapter on Kalman Filters.

7.1 Introduction of Probability Theory

Random Variables A random variable V is a variable whose values depend on outcomes of a random phenomenon. For example, tomorrow's temperature is random variable to us today. We consider real-valued random variables. The (cumulative) probability distribution associated with a random variable is denoted $\mathcal{P}_V : \mathbb{R} \rightarrow [0, 1]$, where $\mathcal{P}_V(v)$ denotes the likelihood that $V \leq v$. The probability distribution is a left-continuous non-decreasing function which satisfies $\lim_{v \rightarrow -\infty} \mathcal{P}_V(v) = 0$ and $\lim_{v \rightarrow \infty} \mathcal{P}_V(v) = 1$. The probability density of the random variable V is defined as $p_V(v) = \frac{d\mathcal{P}_V(v)}{dv}$. The expectation (*i.e.*, first moment) of the random variable V is defined as

$$\mathbb{E}[V] = \int_{-\infty}^{\infty} v p_V(v) dv. \quad (7.1)$$

The variance (*i.e.*, second moment) of the random variable V is defined as

$$\text{VAR}[V] = \mathbb{E}[(V - \mathbb{E}[V])^2] = \int_{-\infty}^{\infty} (v - \mathbb{E}[V])^2 p_V(v) dv. \quad (7.2)$$

The standard deviation is then defined as $\sigma = \sqrt{\mathbb{V}\mathbb{A}\mathbb{R}[V]}$.

Random Vectors A random vector is a collection of random variables. For example, $V = [V_1, V_2, \dots, V_n]^T$ is a random vector where V_i 's are all random variables. The value space of V is in $\mathbb{R}^n$. For the random vector V , we can define its joint probability distribution $\mathcal{P}_V : \mathbb{R}^n \rightarrow [0, 1]$, where $\mathcal{P}_V(v)$ is the likelihood that $V \leq v$ point-wise. Then we can define a joint probability density for the random vector V as $p_V(v) = \frac{\partial^n}{\partial v_1 \partial v_2 \dots \partial v_n} \mathcal{P}_V(v)$. The expectation of the random vector equals to the collection of the expectations of the random variables in that random vector:

$$\mathbb{E}[V] = \begin{bmatrix} \mathbb{E}[V_1] \\ \vdots \\ \mathbb{E}[V_n] \end{bmatrix}. \quad (7.3)$$

For simplicity, we denote the expectation $\mathbb{E}[V]$ as m_V . And the expectation can also be called as the "mean". The variance of the random vector V is defined as

$$\mathbb{V}\mathbb{A}\mathbb{R}(V) = \mathbb{E}[(V - m_V)(V - m_V)^T] = \begin{bmatrix} \mathbb{V}\mathbb{A}\mathbb{R}(V_1) & \mathbb{C}\mathbb{O}\mathbb{V}(V_1, V_2) & \cdots & \mathbb{C}\mathbb{O}\mathbb{V}(V_1, V_n) \\ \mathbb{C}\mathbb{O}\mathbb{V}(V_2, V_1) & \mathbb{V}\mathbb{A}\mathbb{R}(V_2) & \ddots & \mathbb{C}\mathbb{O}\mathbb{V}(V_2, V_n) \\ \vdots & \ddots & \ddots & \vdots \\ \mathbb{C}\mathbb{O}\mathbb{V}(V_n, V_1) & \mathbb{C}\mathbb{O}\mathbb{V}(V_n, V_2) & \cdots & \mathbb{V}\mathbb{A}\mathbb{R}(V_n) \end{bmatrix}. \quad (7.4)$$

where $\mathbb{C}\mathbb{O}\mathbb{V}(V_i, V_j)$ denotes the covariance between two random variables which is defined as

$$\mathbb{C}\mathbb{O}\mathbb{V}(V_i, V_j) = \mathbb{E}[(V_i - m_{V_i})(V_j - m_{V_j})] = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} (v_i - m_{V_i})(v_j - m_{V_j}) p_V(v_i, v_j) dv_i dv_j. \quad (7.5)$$

The covariance between two random vectors can be defined similarly. By definition, the variance of a random variable or a random vector equals to the covariance between itself, i.e., $\mathbb{V}\mathbb{A}\mathbb{R}(V) = \mathbb{C}\mathbb{O}\mathbb{V}(V, V)$.

From a joint density, we can recover the density for a single random variable:

$$p_{V_i}(v_i) = \int_{-\infty}^{\infty} \cdots \int_{-\infty}^{\infty} p_V(v_1, \dots, v_i, \dots, v_n) dv_1 \dots dv_{i-1} dv_{i+1} \dots dv_n \quad (7.6)$$

Correlation and Independence We say two random variables are uncorrelated if $\mathbb{C}\mathbb{O}\mathbb{V}(V_1, V_2) = 0$. We say two random variables are independent if their joint distribution / density can be decoupled: $\mathcal{P}_V(v_1, v_2) = \mathcal{P}_{V_1}(v_1)\mathcal{P}_{V_2}(v_2)$ or $p_V(v_1, v_2) = p_{V_1}(v_1)p_{V_2}(v_2)$.

Using the definition of covariance, we have the following relation-

ship

$$\text{COV}(V_1, V_2) = \mathbb{E}[(V_1 - m_{V_1})(V_2 - m_{V_2})] \quad (7.7a)$$

$$= \mathbb{E}[V_1 V_2 - V_1 m_{V_2} - V_2 m_{V_1} + m_{V_1} m_{V_2}] \quad (7.7b)$$

$$= \mathbb{E}[V_1 V_2] - \mathbb{E}[V_1] \mathbb{E}[V_2]. \quad (7.7c)$$

From the equality, we see that uncorrelation only requires decoupling of the first moments, while independence requires decoupling of the entire distribution or density. Hence independence implies uncorrelation, while uncorrelation does not imply independence.

Example: Let V_1 be uniformly distributed on $[-1, 1]$. Construct another random variable $V_2 = \text{sign}(V_1)V_1$. It is clear that V_1 and V_2 are not independent. However, they are uncorrelated. From the definition, we know that $p_{V_1}(v_1) = 0.5$ for $v_1 \in [-1, 1]$ and 0 otherwise; $p_{V_2}(v_2) = 1$ for $v_2 \in [0, 1]$ and 0 otherwise. Then $\mathbb{E}[V_1 V_2] = \mathbb{E}[\text{sign}(V_1)V_1^2] = \int_{-1}^1 \text{sign}(v_1)v_1^2 p_{V_1}(v_1) dv_1 = 0.5 \int_0^1 v_1^2 dv_1 - 0.5 \int_{-1}^0 v_1^2 dv_1 = 0$ and $\mathbb{E}[V_1] = 0$. Hence $\mathbb{E}[V_1 V_2] = \mathbb{E}[V_1] \mathbb{E}[V_2]$.

Conditional Probability When two variables are not independent, knowing the value of one random variable provide us information on the other variable. We denote the conditional probability density of a random variable V_1 given a random variable V_2 as $p_{V_1}(v_1 | v_2)$. According to the Bayesian rule, we have

$$p_{V_1}(v_1 | v_2) = \frac{p_V(v_1, v_2)}{p_{V_2}(v_2)}. \quad (7.8)$$

If V_1 and V_2 are independent, then

$$p_{V_1}(v_1 | v_2) = p_{V_1}(v_1). \quad (7.9)$$

With the conditional probability density, we can define the conditional expectation as

$$\mathbb{E}[V_1 | v_2] = \int_{-\infty}^{\infty} v_1 p_{V_1}(v_1 | v_2) dv_1. \quad (7.10)$$

Lemma 7.1. *The conditional mean satisfies the following equality:*

$$\mathbb{E}[\mathbb{E}[V_1 | V_2]] = \mathbb{E}[V_1], \quad (7.11)$$

where the outer expectation on the left hand side is taken with respect to V_2 .

Proof. By definition,

$$\mathbb{E}[\mathbb{E}[V_1 | V_2]] = \int_{-\infty}^{\infty} \left[\int_{-\infty}^{\infty} v_1 p_{V_1}(v_1 | v_2) dv_1 \right] p_{V_2}(v_2) dv_2 \quad (7.12a)$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} v_1 p_{V_1}(v_1 | v_2) p_{V_2}(v_2) dv_1 dv_2 \quad (7.12b)$$

$$= \int_{-\infty}^{\infty} v_1 \left[\int_{-\infty}^{\infty} p_V(v_1, v_2) dv_2 \right] dv_1 \quad (7.12c)$$

$$= \int_{-\infty}^{\infty} v_1 p_{V_1}(v_1) dv_1. \quad (7.12d)$$

The third equality uses the Bayesian rule and the last equality uses the relationship between joint density and individual density. The last expression is exactly the expectation of V_1 . Hence the equality in (7.11) is verified. $\square$

Gaussian (Normal) Distributions If a random variable V is a Gaussian random variable, denoted by $V \sim \mathcal{N}(m_V, \sigma_V^2)$, then it has the following probability density

$$p_V(v) = \frac{1}{\sigma_V \sqrt{2\pi}} \exp\left(-\frac{(v - m_V)^2}{2\sigma_V^2}\right), \quad (7.13)$$

where m_V is its mean and σ_V is its standard deviation.

For an n -dimensional jointly distributed Gaussian random vector V , we denote it as $V \sim \mathcal{N}(m_V, \Sigma_V)$ where the joint probability density is

$$p_V(v) = \frac{1}{\sqrt{|\Sigma_V|}(\sqrt{2\pi})^n} \exp\left(-\frac{1}{2}[v - m_V]^T \Sigma_V^{-1}[v - m_V]\right). \quad (7.14)$$

The parameter m_V is still the mean, *i.e.*, $\mathbb{E}(V) = m_V$, while Σ_V is the covariance matrix, *i.e.*, $\mathbb{VAR}[V] = \Sigma_V$.

Given two Gaussian random variables V_1 and V_2 . The conditional random variable $V_1 | v_2$ is also Gaussian with mean

$$\mathbb{E}[V_1 | v_2] = \mathbb{E}[V_1] + \text{COV}(V_1, V_2) \text{VAR}(V_2)^{-1} [v_2 - \mathbb{E}[V_2]], \quad (7.15)$$

and variance

$$\text{VAR}[V_1 | v_2] = \text{VAR}[V_1] - \text{COV}(V_1, V_2) \text{VAR}(V_2)^{-1} \text{COV}(V_2, V_1). \quad (7.16)$$

These two equations can be obtained using the Bayesian rule.

The importance of the Gaussian distribution is stated in the central limit theorem.

Theorem 7.1 (Central Limit Theorem). *A normalized sum of independent random variables tends toward a Gaussian distribution even if the original variables themselves are not normally distributed.*

The theorem implies that the probabilistic and statistical methods that work for Gaussian distributions can be applicable to many problems involving other types of distributions.

In addition, Gaussian distributions have two important features.

- Under Gaussian distribution, independence is equivalent to uncorrelation.
- Gaussian distribution is invariant under linear transformations.

7.2 Random Process

A random variable V_k that evolves with time step k is called a discrete-time random process. For example, the process noises, the system states in stochastic models, and the noisy measurements of system states can all be viewed as random processes. The complete probabilistic properties of a random process can be defined by the joint distribution or density, which is usually difficult to get. In most cases, it is sufficient to know the mean $m_V(k)$ and auto-covariance $\Sigma_V(j, k)$ of a random process which are defined as

$$\mathbb{E}[V_k] = m_V(k) \quad (7.17)$$

$$\mathbb{E}[(V_j - m_V(j))(V_k - m_V(k))] = \Sigma_V(j, k). \quad (7.18)$$

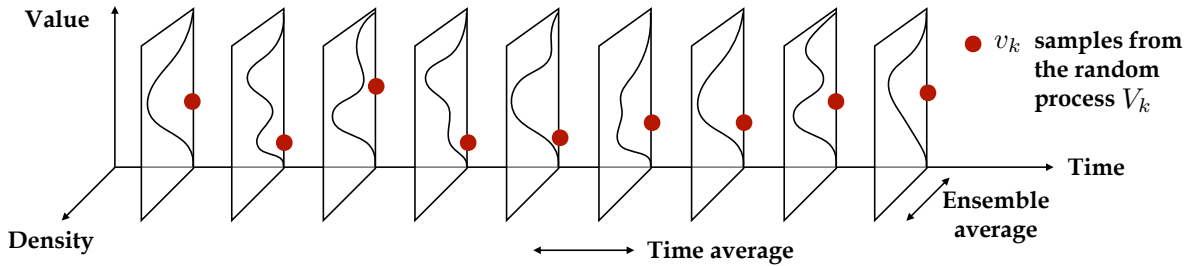


Figure 7.1: Illustrate of a random process.

Figure 7.1 illustrates a random process. At each time step, there is one random variable. That random variable has its own probability density. The sample we observe at time k is denoted v_k which follows the distribution of the random variable V_k . The expectation of V_k is computed with respect to that distribution, which is called as an *ensemble average*. We may also take a time average of a random process as

$$\bar{V} = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{k=1}^N v_k. \quad (7.19)$$

The time average is different from the ensemble average in the sense that the time average is taken with respect to samples for different random variables in the random process; and the ensemble average is the expectation of one single random variable. According to the central limit theorem, the distribution of time average following a Gaussian distribution.

Ergodicity and Stationarity A random process is called an ergodic random process if for all moments of the distribution, the ensemble averages equal the time averages. In practice, it is difficult to test ergodicity but many processes in the real world are ergodic. For example, tossing a normal coin multiple times will give us an ergodic process. With ergodicity, we can use time average to compute the mean and variance of the random process, *i.e.*, computing the expected outcome from the coin by averaging our all samples at different time steps. Otherwise, it is difficult, if not impossible, to draw large amount of samples for a single random variable at each time step.

One necessary condition for ergodicity is stationarity. In the strict sense, a random process is stationary if its joint probability distribution does not change with time, *i.e.*,

$$\mathcal{P}_{V_{k_1}, V_{k_2}, \dots, V_{k_n}}(v_1, \dots, v_n) = \mathcal{P}_{V_{k_1+i}, V_{k_2+i}, \dots, V_{k_n+i}}(v_1, \dots, v_n), \forall n, k_1, \dots, k_n, i, v_1, \dots, v_n. \quad (7.20)$$

In the weak sense, a random process is stationary if its mean does not depend on time, *i.e.*,

$$\mathbb{E}[V_k] = m_V, \forall k, \quad (7.21)$$

and its auto-covariance depends only on the time difference, *i.e.*,

$$\mathbb{E}[(V_j - m_V(j))(V_k - m_V(k))] = \Sigma_V(j - k) \quad (7.22)$$

In this course, we will always assume stationarity and ergodicity, unless otherwise specified. Typical random processes include:

- White noise where there is no correlation across different time steps, *i.e.*, $\Sigma_V(i) = \begin{cases} 0 & i \neq 0 \\ \text{VAR}(V_k) & i = 0 \end{cases}$.
- Identical independent distribution (i.i.d.) where the joint distribution satisfies $\mathcal{P}_{V_{k_1}, V_{k_2}, \dots, V_{k_n}}(v_1, \dots, v_n) = \mathcal{P}_{V_{k_1}}(v_1) \cdots \mathcal{P}_{V_{k_n}}(v_n)$ and $\mathcal{P}_{V_k}$ is identical among one another for all k .

7.3 Least Square

Consider a parameter estimation problem below. We have two correlated random vectors X and Y . Given observation of $Y = y$, what is

the best estimation of X ? We may do maximum likelihood estimation by

$$\hat{x} = \arg \max_x p_{X|Y}(x|y), \quad (7.23)$$

or minimum variance estimation by

$$\hat{x} = \arg \min_x \mathbb{E}[\|X - x\|^2 | y]. \quad (7.24)$$

For Gaussian distributions, the two estimates are the same. Both equal to

$$\mathbb{E}[X|y] = \mathbb{E}[X] + \text{COV}(X, Y) \text{VAR}(Y)^{-1} [y - \mathbb{E}[Y]]. \quad (7.25)$$

Before we observe the sample y of Y , the estimate $\mathbb{E}[X|Y]$ is a random variable that depends on Y . In the following discussion, we denote the estimate as $\hat{X} := \mathbb{E}[X|Y]$. According to lemma 7.1, $\mathbb{E}[\hat{X}] = \mathbb{E}[X]$. Moreover, since $\hat{X}$ depends linearly on Y and Gaussian distributions are invariant under linear transformations, $\hat{X}$ follows a Gaussian distribution. The estimation error is defined as $\tilde{X} = X - \hat{X}$, which is also Gaussian with mean and variance:

$$\mathbb{E}[\tilde{X}] = \mathbb{E}[X] - \mathbb{E}[\hat{X}] = 0 \quad (7.26)$$

$$\text{VAR}[\tilde{X}] = \text{VAR}[X | Y] = \text{VAR}[X] - \text{COV}(X, Y) \text{VAR}(Y)^{-1} \text{COV}(Y, X). \quad (7.27)$$

The relationship between $\tilde{X}$ and Y is summarized in the following lemma.

Lemma 7.2. *The estimation error $\tilde{X} = X - \hat{X}$ is uncorrelated with Y , i.e.,*

$$\mathbb{E}[\tilde{X}(Y - \mathbb{E}[Y])^T] = 0 \quad (7.28)$$

Proof. Using the formula for $\hat{X}$, we have

$$\mathbb{E}[\tilde{X}(Y - \mathbb{E}[Y])^T] \quad (7.29a)$$

$$= \mathbb{E} \left[(X - \mathbb{E}[X] - \text{COV}(X, Y) \text{VAR}(Y)^{-1} [Y - \mathbb{E}[Y]]) (Y - \mathbb{E}[Y])^T \right]$$

$$= \text{COV}(X, Y) - \text{COV}(X, Y) \text{VAR}(Y)^{-1} \text{VAR}(Y) \quad (7.29b)$$

$$= 0 \quad (7.29c)$$

□

The above discussion only considers one information source, i.e., Y . If we have multiple information source, the resulting optimal estimate of X is summarized in the following two theorems. In the following discussion, to distinguish between the estimates from different information sources, we use subscripts to denote the random variable that is conditioned on, e.g., $\hat{X}_{|Y}$ refers to the estimate that is conditioned on Y .

Theorem 7.2. Let Y and Z be Gaussian and uncorrelated, then the optimal estimate of X and its error covariance are

$$\mathbb{E}[X | Y, Z] = \mathbb{E}[X | Y] + \mathbb{E}[\tilde{X}_{|Y} | Z] \quad (7.30)$$

$$\mathbb{VAR}[X | Y, Z] = \mathbb{VAR}[\tilde{X}_{|Y}] - \mathbf{COV}[\tilde{X}_{|Y}, Z] \mathbb{VAR}[Z]^{-1} \mathbf{COV}[Z, \tilde{X}_{|Y}]. \quad (7.31)$$

Proof. We combine Y and Z as one big random vector, then apply the conditional expectation formula:

$$\mathbb{E}[X | Y, Z] = \mathbb{E}[X] + [\mathbf{COV}(X, Y) \ \mathbf{COV}(X, Z)] \begin{bmatrix} \mathbb{VAR}(Y) & \mathbf{COV}(Y, Z) \\ \mathbf{COV}(Z, Y) & \mathbb{VAR}(Z) \end{bmatrix}^{-1} \begin{bmatrix} Y - \mathbb{E}[Y] \\ Z - \mathbb{E}[Z] \end{bmatrix} \quad (7.32)$$

Note that $\mathbf{COV}(Y, Z) = 0$. Rearranging the equation above, we have

$$\mathbb{E}[X | Y, Z] = \underbrace{\mathbb{E}[X] + \mathbf{COV}(X, Y) \mathbb{VAR}(Y)^{-1} [Y - \mathbb{E}[Y]]}_{\mathbb{E}[X|Y]} + \underbrace{\mathbf{COV}(X, Z) \mathbb{VAR}(Z)^{-1} [Z - \mathbb{E}[Z]]}_{\mathbb{E}[X|Z] - \mathbb{E}[X]} \quad (7.33)$$

Notice that $X = \hat{X}_{|Y} + \tilde{X}_{|Y}$ and $\mathbb{E}[\hat{X}_{|Y} | Z] = \mathbb{E}[\mathbb{E}[X | Y] | Z] = \mathbb{E}[X]$.

Then we have

$$\mathbb{E}[X | Y, Z] = \mathbb{E}[X | Y] + \mathbb{E}[(\hat{X}_{|Y} + \tilde{X}_{|Y}) | Z] - \mathbb{E}[X] \quad (7.34)$$

$$= \mathbb{E}[X | Y] + \mathbb{E}[\tilde{X}_{|Y} | Z] \quad (7.35)$$

Hence (7.30) is verified.

Similarly, we apply the conditional variance formula:

$$\mathbb{VAR}[X | Y, Z] = \mathbb{VAR}[X] - [\mathbf{COV}(X, Y) \ \mathbf{COV}(X, Z)] \begin{bmatrix} \mathbb{VAR}(Y) & \mathbf{COV}(Y, Z) \\ \mathbf{COV}(Z, Y) & \mathbb{VAR}(Z) \end{bmatrix}^{-1} \begin{bmatrix} \mathbf{COV}(Y, X) \\ \mathbf{COV}(Y, Z) \end{bmatrix} \quad (7.36)$$

Rearranging the equation above, we have

$$\mathbb{VAR}[X | Y, Z] = \underbrace{\mathbb{VAR}[X] - \mathbf{COV}(X, Y) \mathbb{VAR}(Y)^{-1} \mathbf{COV}(Y, X)}_{\mathbb{VAR}[X|Y]} - \mathbf{COV}(X, Z) \mathbb{VAR}(Z)^{-1} \mathbf{COV}(Z, X) \quad (7.37)$$

Since Y and Z are independent, then $\mathbf{COV}[\hat{X}_{|Y}, Z] = 0$, and

$$\mathbf{COV}[X, Z] = \mathbf{COV}[(\hat{X}_{|Y} + \tilde{X}_{|Y}), Z] = \mathbf{COV}[\tilde{X}_{|Y}, Z]. \quad (7.38)$$

Hence we get the result

$$\mathbb{VAR}[X | Y, Z] = \mathbb{VAR}[\tilde{X}_{|Y}] - \mathbf{COV}[\tilde{X}_{|Y}, Z] \mathbb{VAR}[Z]^{-1} \mathbf{COV}[Z, \tilde{X}_{|Y}]. \quad (7.39)$$

□

Theorem 7.3. Let Y and Z be Gaussian and correlated, then the optimal estimate of X and its error covariance are

$$\mathbb{E}[X | Y, Z] = \mathbb{E}[X | Y] + \mathbb{E}[\tilde{X}_{|Y} | \tilde{Z}_{|Y}] \quad (7.40)$$

$$\mathbb{VAR}[X | Y, Z] = \mathbb{VAR}[\tilde{X}_{|Y}] - \mathbf{COV}[\tilde{X}_{|Y}, \tilde{Z}_{|Y}] \mathbb{VAR}[\tilde{Z}_{|Y}]^{-1} \mathbf{COV}[\tilde{Z}_{|Y}, \tilde{X}_{|Y}]. \quad (7.41)$$

This theorem can be proved in a similar approach as we did for the previous theorem. The intuition behind is that: if we are going to estimate X using Y and Z , then we can do the following two steps. First, we can exploit all information from Y , which leaves us $\tilde{X}_{|Y}$ and $\tilde{Z}_{|Y}$. Then we exploit all information from $\tilde{Z}_{|Y}$.

8

Kalman Filter, Extended Kalman Filter and Unscented Kalman Filter

This chapter discusses how to apply least square methods to estimate system states. We will first introduce Kalman Filter which is able to estimate states in linear dynamic systems section 8.1. Then we discuss extended Kalman Filter and Unscented Kalman Filter which handle nonlinear dynamics.

8.1 Kalman Filter

One methodology for estimating parameters which is of particular interest is the Kalman Filter. Kalman Filter can be defined for both discrete time and continuous time systems, but for this introduction we shall focus on the discrete time case. Consider a discrete time linear system model of the following form

$$\mathbf{x}_{k+1} = A_k \mathbf{x}_k + B_k \mathbf{u}_k + B_k^{wv} \mathbf{w}_k, \quad (8.1a)$$

$$\mathbf{y}_k = C_k \mathbf{x}_k + \mathbf{v}_k, \quad (8.1b)$$

where the initial state $\mathbf{x}_0$, the process noise $\mathbf{w}_k$, and the measurement noise $\mathbf{v}_k$ are Gaussian and all independent from each other. Additionally, $\mathbf{w}_k$ and $\mathbf{v}_k$ are white random processes. The initial state and the two noises take on the following distributions: $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_0^*, X_0)$, $\mathbf{w}_k \sim \mathcal{N}(0, W_k)$, $\mathbf{v}_k \sim \mathcal{N}(0, V_k)$, where $\mathbf{x}_0^*$ is the expectation of the initial state. Note that our goal is, given a series of measurements of the above system

$$Y_j = \{\mathbf{y}_0, \mathbf{y}_1, \mathbf{y}_2, \dots, \mathbf{y}_j\}, \quad (8.2)$$

to minimize the residual error of the difference between the current state and our predicted state. Mathematically, this can be declared as

$$\min_{\hat{\mathbf{x}}_k} \mathbb{E} [\|\mathbf{x}_k - \hat{\mathbf{x}}_k\|^2 \mid Y_j], \quad (8.3)$$

which is done by modifying our estimate of the state

$$\hat{\mathbf{x}}_k = \mathbb{E}[\mathbf{x}_k | Y_j]. \quad (8.4)$$

This is effectively equivalent to searching for the minimum variance expectation of the given quantity.

Depending on how much information the designer uses to update the estimate of the given quantity, the outcome from the estimation process and thus the name of the corresponding estimate are different. In eq. (8.3), if $k > j$, the estimation process is referred to as a prediction problem as the designer is attempting to estimate a mathematical quantity at a future time step. Should $k < j$, then the estimation process is referred to as a smoothing problem. Finally, should $k = j$, then the estimation process is referred to as a filtering problem. This is where the “filter” component of the Kalman Filter name comes from. Note that as filtering processes have this one-to-one correspondence between when the time the estimate of the quantity and the latest measurement line up, these filtering processes are also occasionally known as sequential estimators. This is why, when a Kalman Filter is used to predict system state, it is occasionally referred to as a sequential state estimator as the Kalman Filter is employed to reconstruct the state variable of interest by filtering out noisy measurements.

Table 8.1 is a list of notations that we will be using for the remaining discussion of the Kalman Filter.

a priori estimation	$\hat{\mathbf{x}}_{k k-1} = \mathbb{E}[\mathbf{x}_k Y_{k-1}]$	$\tilde{\mathbf{x}}_{k k-1} = \mathbf{x}_k - \hat{\mathbf{x}}_{k k-1}$
a posteriori estimation	$\hat{\mathbf{x}}_{k k} = \mathbb{E}[\mathbf{x}_k Y_k]$	$\tilde{\mathbf{x}}_{k k} = \mathbf{x}_k - \hat{\mathbf{x}}_{k k}$
a priori covariance	$M_k = \mathbb{E}[\tilde{\mathbf{x}}_{k k-1} \tilde{\mathbf{x}}_{k k-1}^T]$	
a posteriori covariance	$Z_k = \mathbb{E}[\tilde{\mathbf{x}}_{k k} \tilde{\mathbf{x}}_{k k}^T]$	

Table 8.1: Notation for use in the remainder of the chapter

To start, let us consider a point in time, $k - 1$, where we have the following estimate of the quantity we wish to estimate, denoted $\hat{\mathbf{x}}_{k-1|k-1}$, and an error covariance denoted Z_{k-1} . At time k , we receive a new measurement, y_k , which we want to use in order to improve our estimate of x_k . Figure 8.1 shows this one time step update in the Kalman Filter, which consists of two steps: the Dynamic Update and the Measurement Update. The first step corresponds to propagating forward the system model one time step with our current estimate (hence leveraging our understanding of the process defining the dynamical system) and the second update corresponds to integrating the available information from the measurement of the system.

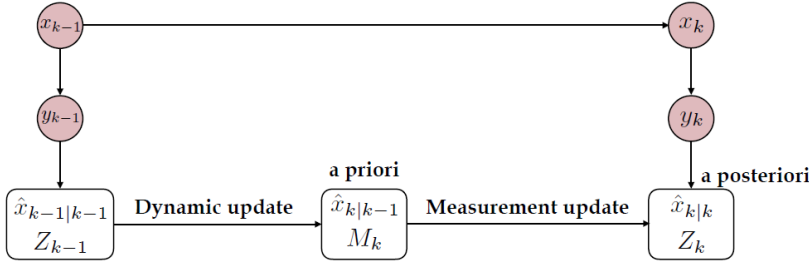


Figure 8.1: One time step view of Kalman Filter update, which consists of the dynamic update and the measurement update.

8.1.1 Dynamic Update

Let us first consider the Dynamic Update in more detail. Given (8.1a) and our a posteriori estimate and covariance from the last step, which correspond to $\hat{\mathbf{x}}_{k-1|k-1}$ and Z_{k-1} , respectively, we want to estimate how our system will evolve, so we plug in the previous estimate to the dynamic equation (8.1a) and yield

$$\hat{\mathbf{x}}_{k|k-1} = \mathbb{E}[\mathbf{x}_k | Y_{k-1}] \quad (8.5a)$$

$$= \mathbb{E}[A_{k-1}\mathbf{x}_{k-1} + B_{k-1}\mathbf{u}_{k-1} + B_{k-1}^w \mathbf{w}_{k-1} | Y_{k-1}] \quad (8.5b)$$

$$= A_{k-1}\mathbb{E}[\mathbf{x}_{k-1} | Y_{k-1}] + B_{k-1}\mathbf{u}_{k-1} + \mathbb{E}[B_{k-1}^w \mathbf{w}_{k-1} | Y_{k-1}] \quad (8.5c)$$

$$= A_{k-1}\hat{\mathbf{x}}_{k-1|k-1} + B_{k-1}\mathbf{u}_{k-1}. \quad (8.5d)$$

As you can see from (8.5), we first start by substituting the dynamics. In the second step, we leverage the linearity of the Expectation operator and take expectation of the individual terms. Finally, we note that the noise is zero mean; hence it is dropped in the third equation. Then we need to evaluate the error covariance associated with this estimate. The estimation error takes the form

$$\tilde{\mathbf{x}}_{k|k-1} = \mathbf{x}_k - \hat{\mathbf{x}}_{k|k-1}.$$

If we substitute (8.1a) for $\mathbf{x}_k$ and (8.5d) for $\hat{\mathbf{x}}_{k|k-1}$ respectively, we get the form of the a priori estimation error as

$$\tilde{\mathbf{x}}_{k|k-1} = A_{k-1}\tilde{\mathbf{x}}_{k-1|k-1} + B_{k-1}^w \mathbf{w}_{k-1}. \quad (8.6)$$

In (8.6), the error no longer depends on the control input, but the process noise. To compute the a priori error covariance M_k , we just need to use the definition provided in 8.1 and plugging in (8.6), which yield

$$M_k = A_{k-1}Z_{k-1}A_{k-1}^T + B_{k-1}^w W_{k-1} (B_{k-1}^w)^T. \quad (8.7)$$

Thus the Dynamic Update is completed. The inputs to the Dynamic Update are the a posteriori estimate and covariance from the last step and the dynamic model; the outputs of the Dynamic Update is the a priori estimate and covariance at the current time step.

8.1.2 Measurement Update

The next major component that we need to consider is how to incorporate new information, which is done craftily via the Measurement Update. We start first by considering Proposition 2 from the previous chapter, repeated here for convenience:

$$\mathbb{E}[\mathbf{x}_k | Y_k] = \mathbb{E}[\mathbf{x}_k | Y_{k-1}] + \mathbb{E}[\tilde{\mathbf{x}}_k |_{k-1} | (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}], \quad (8.8)$$

where we can expand the second term using the conditional expectation relations discussed earlier:

$$\mathbb{E}[\tilde{\mathbf{x}}_k |_{k-1} | (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] = \mathbb{E}[\tilde{\mathbf{x}}_k |_{k-1}] + \text{COV}[\tilde{\mathbf{x}}_k |_{k-1}, (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \text{VAR}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]^{-1} [(\tilde{\mathbf{y}}_k)_{|Y_{k-1}} - \mathbb{E}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]] \quad (8.9)$$

Moreover, the corresponding conditional variance is:

$$\text{VAR}[\mathbf{x}_k | Y_k] = \text{VAR}[\mathbf{x}_k | Y_{k-1}] - \text{COV}[\tilde{\mathbf{x}}_k |_{k-1}, (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \text{VAR}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]^{-1} \text{COV}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}, \tilde{\mathbf{x}}_k |_{k-1}]. \quad (8.10)$$

Note that $\mathbb{E}[\tilde{\mathbf{x}}_k |_{k-1}] = 0$. Using the notations in table 8.1, we can rewrite the conditional mean and variance as

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \text{COV}[\tilde{\mathbf{x}}_k |_{k-1}, (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \text{VAR}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]^{-1} [(\tilde{\mathbf{y}}_k)_{|Y_{k-1}} - \mathbb{E}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]] \quad (8.11)$$

$$\mathbf{Z}_k = \mathbf{M}_k - \text{COV}[\tilde{\mathbf{x}}_k |_{k-1}, (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \text{VAR}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}]^{-1} \text{COV}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}, \tilde{\mathbf{x}}_k |_{k-1}] \quad (8.12)$$

In order to obtain the Measurement Update, the next step is to figure out the closed-form expression of

$$\begin{aligned} & (\tilde{\mathbf{y}}_k)_{|Y_{k-1}} \\ & \text{COV}[\tilde{\mathbf{x}}_k |_{k-1}, (\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \\ & \text{VAR}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \\ & \mathbb{E}[(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}] \end{aligned}$$

Let us compute these terms one-by-one.

For $(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}$, by definition we have the following relationship

$$(\tilde{\mathbf{y}}_k)_{|Y_{k-1}} = \mathbf{y}_k - (\hat{\mathbf{y}}_k)_{|Y_{k-1}} = \mathbf{y}_k - \mathbf{C}_k \hat{\mathbf{x}}_{k|k-1}. \quad (8.13)$$

The above equation will be used to compute $(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}$. Moreover, for analysis purpose, we can expand $\mathbf{y}_k$, $(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}$ and obtain the following relationship:

$$(\tilde{\mathbf{y}}_k)_{|Y_{k-1}} = \mathbf{C}_k \mathbf{x}_k + \mathbf{v}_k - \mathbf{C}_k \hat{\mathbf{x}}_{k|k-1} = \mathbf{C}_k \tilde{\mathbf{x}}_{k|k-1} + \mathbf{v}_k. \quad (8.14)$$

Note that (8.14) cannot be used to compute $(\tilde{\mathbf{y}}_k)_{|Y_{k-1}}$ since we do not have access to $\tilde{\mathbf{x}}_{k|k-1}$ and $\mathbf{v}_k$. But (8.14) can help us compute

the following terms $\text{COV}[\tilde{\mathbf{x}}_k | k-1, (\tilde{\mathbf{y}}_k) | Y_{k-1}]$, $\text{VAR}[(\tilde{\mathbf{y}}_k) | Y_{k-1}]$, and $\mathbb{E}[(\tilde{\mathbf{y}}_k) | Y_{k-1}]$:

$$\mathbb{E}[(\tilde{\mathbf{y}}_k) | Y_{k-1}] = C_k \mathbb{E}[\tilde{\mathbf{x}}_{k|k-1}] + \mathbb{E}[\mathbf{v}_k] = 0 \quad (8.15)$$

$$\text{COV}[\tilde{\mathbf{x}}_k | k-1, (\tilde{\mathbf{y}}_k) | Y_{k-1}] = \text{VAR}[\tilde{\mathbf{x}}_k | k-1] C_k^T + \text{COV}[\tilde{\mathbf{x}}_k | k-1, \mathbf{v}_k] = M_k C_k^T \quad (8.16)$$

$$\text{VAR}[(\tilde{\mathbf{y}}_k) | Y_{k-1}] = C_k M_k C_k^T + V_k \quad (8.17)$$

Hence, the conditional mean and variance for the Measurement Update becomes:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + M_k C_k^T (C_k M_k C_k^T + V_k)^{-1} [\mathbf{y}_k - C_k \hat{\mathbf{x}}_{k|k-1}], \quad (8.18)$$

$$Z_k = M_k - M_k C_k^T (C_k M_k C_k^T + V_k)^{-1} C_k M_k. \quad (8.19)$$

The coefficient before the term $\mathbf{y}_k - C_k \hat{\mathbf{x}}_{k|k-1}$ is called Kalman Gain F_k with

$$F_k := M_k C_k^T (C_k M_k C_k^T + V_k)^{-1}. \quad (8.20)$$

We summarize the entire Kalman Filter process is summarized in fig. 8.2.

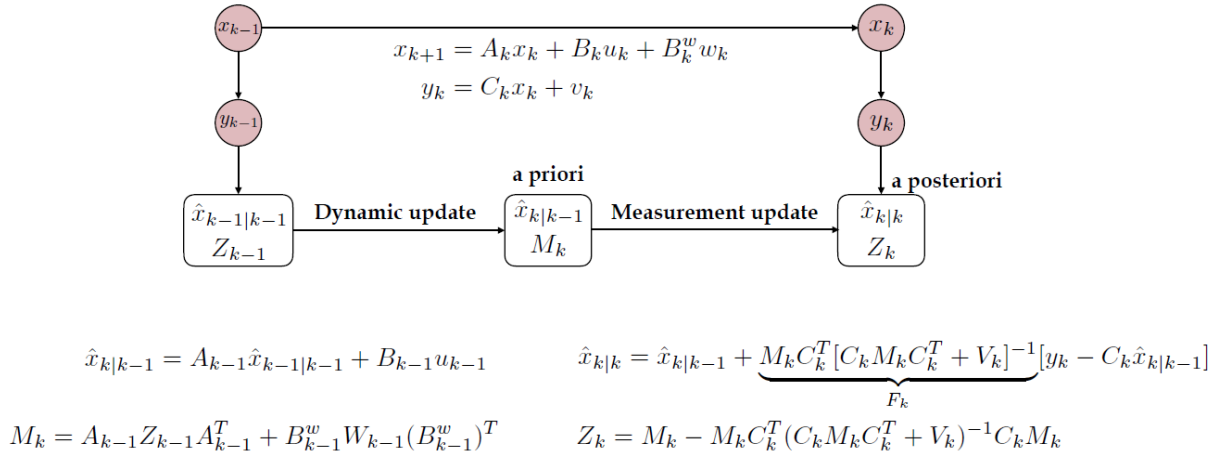


Figure 8.2: An overview of the Kalman Filter Algorithm.

8.1.3 Steady State Kalman Filter

Note that for a linear system with Gaussian noise, a Kalman filter is optimal. In other words, "fancier" nonlinear estimation techniques will not improve the results of the Kalman Filter. A Kalman Filter works for time-varying systems and can be used for parameter estimation, not only state estimation. The Kalman gain (F_k) and the

covariance matrices (M_k, Z_k) may be computed offline, and the update equations for the covariance matrices is in the form of the discrete algebraic Riccati equation (DARE). This form is evident whenever we combine the covariance update from both the Dynamic and Measurement update, yielding the following expression:

$$M_{k+1} = A_k M_k A_k^T + B^w W_k (B^w)^T - A_k M_k C_k^T [V_k + C_k M_k C_k^T]^{-1} C_k M_k A_k^T. \quad (8.21)$$

This connection with the discrete algebraic Riccati equation brings up another interesting point. Just as the discrete algebraic Riccati equation may converge under certain conditions to an optimal value when solving the DARE during LQR control, it may also do so when implementing a Kalman Filter. These conditions are,

1. The system is time-invariant: A, B, B^w, C are constant
2. The system noise is stationary: $V \succ 0, W \succ 0$ and do not depend on time
3. The pair, (A, C) , are observable or detectable
4. The pair, (A, B^w) , are controllable or stabilizable

If so, then M_k in the DARE will converge to a steady-state value M_s . A steady-state Kalman Filter may be implemented using the following equations:

$$\hat{\mathbf{x}}_{k|k-1} = A \hat{\mathbf{x}}_{k-1|k-1} + B \mathbf{u}_{k-1}, \quad (8.22a)$$

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + M_s C^T (C M_s C^T + V)^{-1} [\mathbf{y}_k - C \hat{\mathbf{x}}_{k|k-1}], \quad (8.22b)$$

$$F_s = M_s C^T (C M_s C^T + V)^{-1}, \quad (8.22c)$$

where M_s is the positive definite solution of the DARE, given by:

$$M_s = A M_s A^T + B^w W (B^w)^T - A M_s C^T [C M_s C^T + V]^{-1} C M_s A^T. \quad (8.23)$$

8.1.4 Duality with LQR

Kalman Filter shares many similarity with LQR as evident from previous discussion. This similarity arises from the fact that the quadratic cost function in LQR and Least-Squares cost function in KF are similar (both are quadratic). To describe further the duality with LQR, let's directly build a table that will hopefully be useful to users when trying to code a Kalman Filter algorithm. Please note that in $Q = C^T C$, C here is not C_k , rather it is a design parameter. First, let us visualize the DARE that must be solved in the discrete-time LQR control problem,

$$P_k = A_k^T P_{k+1} A_k + Q_k - A_k^T P_{k+1} B_k [R_k + B_k^T P_{k+1} B_k]^{-1} B_k^T P_{k+1} A_k, \quad (8.24)$$

which is remarkably similar to the DARE (8.21) that must be solved in the discrete-time Kalman Filter problem. To help remember these similarities, the following table was generated.

Discrete-time LQR	Discrete-time Kalman Filter
A	A^T
B	C^T
C	B^w
R	V
$Q = C^T C$	$B^w W (B^w)^T$
P	M
Backward recursion	Forward recursion

Table 8.2: Table comparing the similarities between LQR and Kalman Filter algorithm design problems.

8.2 Nonlinear Kalman Filtering

We have discussed how to obtain optimal state estimate for linear systems. Now the question arises: "How do we handle nonlinear dynamics?" In other words, we are looking at system dynamics of the form:

$$\begin{aligned} \mathbf{x}_{k+1} &= f_k(\mathbf{x}_k, \mathbf{u}_k) + B_k^w \mathbf{w}_k \\ \mathbf{y}_k &= h_k(\mathbf{x}_k) + \mathbf{v}_k. \end{aligned} \quad (8.25)$$

We maintain the assumptions that $\mathbf{x}_0, \mathbf{w}_k, \mathbf{v}_k$ are independent and Gaussian. Additionally, $\mathbf{w}_k$ and $\mathbf{v}_k$ are white. Furthermore, $\mathbf{x}_0 \sim \mathcal{N}(\mathbf{x}_0^*, X_0)$, $\mathbf{w}_k \sim \mathcal{N}(0, W_k)$, $\mathbf{v}_k \sim \mathcal{N}(0, V_k)$. We know that Gaussian distributions are preserved under linear transformations, as we see from the transformation in the figure below (from random variable x to y).

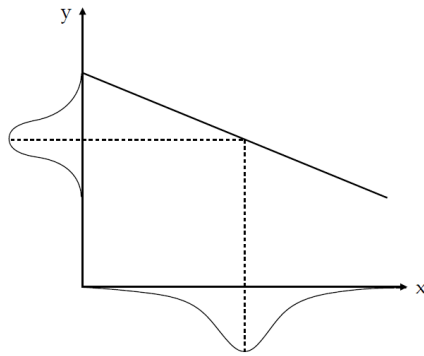


Figure 8.3: A Gaussian Distribution remains Gaussian under linear transformations.

However, challenge arises when we have a nonlinear transformation of a Gaussian as our distribution no longer remains Gaussian. This is shown in the figure below

We have two ways to describe the distribution y .

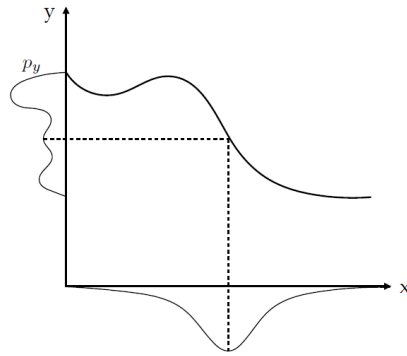


Figure 8.4: A Gaussian Distribution is no longer Gaussian under a nonlinear transformations.

1. Directly approximate p_y ;
2. Linearize the nonlinear function.

The first way to describe the distribution y is visualized in fig. 8.5. This is done by constructing a Gaussian approximation of p_y after the nonlinear transformation acts on distribution x . We leverage the mean and variance of p_y to characterize this approximated Gaussian distribution. The other approach is directly constructing p_y after linearizing the nonlinear mapping as shown in fig. 8.6.

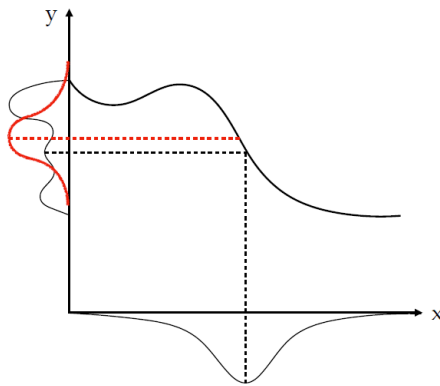


Figure 8.5: A Gaussian approximation of p_y using the mean and variance of p_y .

The direct approximation method is known as an **Unscented Kalman Filter** (to be discussed in section 8.4) and the linearization approach to nonlinear Kalman Filtering is known as **Extended Kalman Filter** (to be discussed in section 8.3).

8.3 Extended Kalman Filter

Conceptually, the Extended Kalman Filter is fairly easy to incorporate into our current understanding of Kalman Filter. Let's take a view of a Kalman Filter update step to get a feel for how the system works:

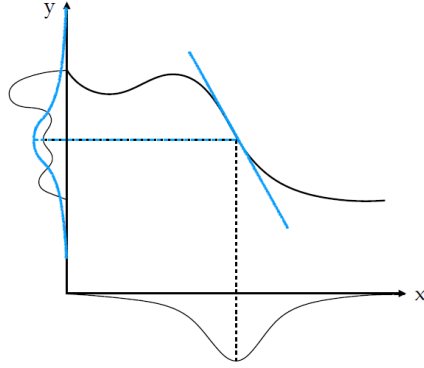


Figure 8.6: Approximating the non-linear mapping by linearizing the transformation and then constructing the distribution from the linearized mapping.

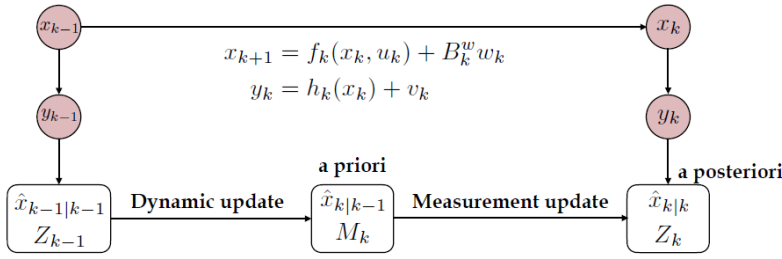


Figure 8.7: A step view of the Extended Kalman Filter algorithm

We know that the first order Taylor series expansion at $\hat{\mathbf{x}}_{k-1|k-1}$ is given as:

$$f_{k-1}(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) \approx f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}) + \underbrace{\frac{\partial}{\partial \mathbf{x}} f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1})}_{\bar{A}_{k-1}} (\mathbf{x}_{k-1} - \hat{\mathbf{x}}_{k-1|k-1}). \quad (8.26)$$

From this expression, we can state a dynamic update for the linearized dynamics as:

$$\hat{\mathbf{x}}_{k|k-1} = f_{k-1}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}), \quad (8.27)$$

$$M_k = \bar{A}_{k-1} Z_{k-1} \bar{A}_{k-1}^T + B_{k-1}^w W_{k-1} (B_{k-1}^w)^T, \quad (8.28)$$

which are similar to those taken in the linear Kalman Filter.

Likewise for the Measurement Update, we can generate a linearized approximation via a first order Taylor series expansion at $\hat{\mathbf{x}}_{k|k-1}$. That first-order approximation is:

$$h_k(\mathbf{x}_k) \approx h_k(\hat{\mathbf{x}}_{k|k-1}) + \underbrace{\frac{\partial}{\partial \mathbf{x}} h_k(\hat{\mathbf{x}}_{k|k-1})}_{\bar{C}_k} (\mathbf{x}_k - \hat{\mathbf{x}}_{k|k-1}). \quad (8.29)$$

This leads to a measurement update in the style of the linear Kalman

filter:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + M_k \bar{C}_k^T [\bar{C}_k M_k \bar{C}_k^T + V_k]^{-1} [\mathbf{y}_k - h_k(\hat{\mathbf{x}}_{k|k-1})], \quad (8.30)$$

$$Z_k = M_k - M_k \bar{C}_k^T (\bar{C}_k M_k \bar{C}_k^T + V_k)^{-1} \bar{C}_k M_k. \quad (8.31)$$

Now the Kalman Filter gain takes the following form $F_k = M_k \bar{C}_k^T [\bar{C}_k M_k \bar{C}_k^T + V_k]^{-1}$.

Quality of Approximation The quality of our approximation through linearization is of great importance. The First order Taylor expansion does not consider the variance of the input, which implies that for processes with large variance, such linearization may result in a bad approximation in the sense that the approximated variance significantly deviates from the actual variance of the output, which is shown in fig. 8.8.

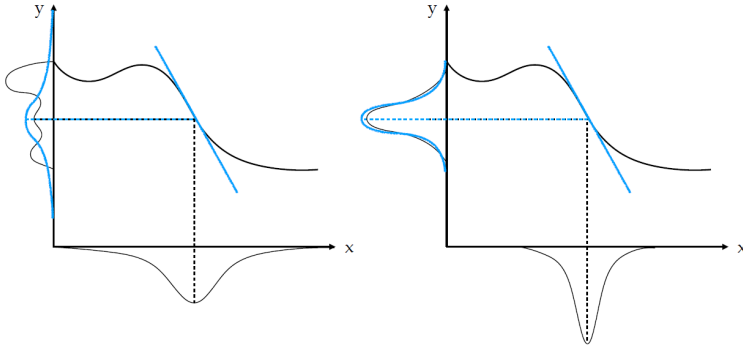


Figure 8.8: Illustration of the quality of approximation using first-order Taylor expansion in EKF. We consider two different input distributions. As shown in the figures, a larger input variance leads to a worse approximation of the distribution of y .

Optimal Linear Approximation Naturally, we begin to wonder how we can ensure better linear approximations for our Extended Kalman Filters. The optimality criteria here is to not only minimize the error in the estimate of the mean but also the estimate of the variance. For this, we may consider solving the approximation problem as an additional estimation problem, and then use least square to find the linearization that results in minimal error variance. In the following discussion, we call this type of linearization as minimum variance linearization or least square linearization. Consider the nonlinear function $y = f(x)$, the minimum variance linearization of f as a function $y = ax + b$ can be solved by the following minimization:

$$\min_{a,b} \mathbb{E}_{x \sim \mathcal{N}(m_x, \Sigma_x)} [\|f(x) - ax - b\|_2^2]. \quad (8.32)$$

If we drew samples $(x^{(i)}, y^{(i)})$ for $i = 1, \dots, n$ from function $y = f(x)$ according to distribution $x \sim \mathcal{N}(m_x, \Sigma_x)$, the above optimization

problem is equivalent to:

$$\min_{a,b} \sum_{i=1}^n \|y^{(i)} - ax^{(i)} - b\|_2^2. \quad (8.33)$$

There exists analytical solution for a and b given all samples. Now to obtain the analytical solution, let us first stacking all the sample points:

$$\begin{aligned} \bar{X} &= \begin{bmatrix} 1 & 1 & \dots & 1 \\ x^{(1)} & x^{(2)} & \dots & x^{(n)} \end{bmatrix} \\ Y &= \begin{bmatrix} y^{(1)} & y^{(2)} & \dots & y^{(n)} \end{bmatrix}, \end{aligned} \quad (8.34)$$

Define $\bar{a} = [b \ a]$. Then the approximation error on all sample points can be written as the following matrix

$$Y - \bar{a}\bar{X} = \begin{bmatrix} y^{(1)} - ax^{(1)} - b & y^{(2)} - ax^{(2)} - b & \dots & y^{(n)} - ax^{(n)} - b \end{bmatrix}. \quad (8.35)$$

Then we can rewrite the objective for the minimization problem as

$$\sum_{i=1}^n \|y^{(i)} - ax^{(i)} - b\|_2^2 = \text{Tr}[(Y - \bar{a}\bar{X})(Y - \bar{a}\bar{X})^T], \quad (8.36)$$

with $\bar{a} = [b \ a]$. Note that if y is a scalar, then we do not need the trace. Apply the following first order optimality condition on the objective function, we find the optimal solution for $\bar{a}$:

$$\begin{aligned} 0 &= \nabla_{\bar{a}} \text{Tr}[(Y - \bar{a}\bar{X})(Y - \bar{a}\bar{X})^T] \\ &= \nabla_{\bar{a}} \text{Tr}[\bar{a}\bar{X}\bar{X}^T\bar{a}^T - \bar{a}\bar{X}Y^T - Y\bar{X}^T\bar{a}^T] \\ &= 2\bar{a}\bar{X}\bar{X}^T - 2Y\bar{X}^T \\ &\implies \bar{a} = Y\bar{X}^T(\bar{X}\bar{X}^T)^{-1} \end{aligned}$$

The minimum variance linearization will result in a better approximation on both mean and variance as shown in fig. 8.9. It is worth noting that least square linearization is not about getting a better approximation of the density function, but rather about getting a better mean and variance approximation (the first two moments of the distribution).

Sampling Strategy Now, we need to develop a sampling strategy for points when using our least square analysis. A natural choice is to sample along the ellipse of standard deviation, using points

$$m_x, m_x \pm \sqrt{\Sigma_x} e_i, \quad (8.37)$$

where $e_i = [0, \dots, 0, 1, 0, \dots, 0]$ is a unit vector whose entries are all zero except the i th entry. This sampled ellipse is shown in fig. 8.10.

For $x \in \mathbb{R}^n$, this will result in $2n + 1$ samples.

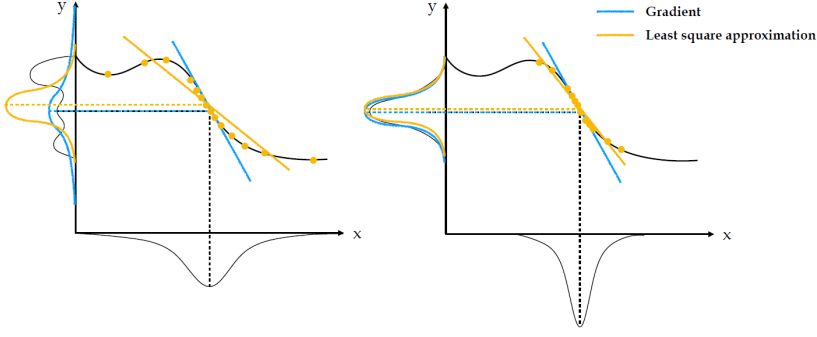


Figure 8.9: Comparison of quality of approximation under the gradient-based linearization and the least square linearization.

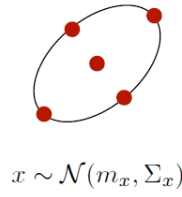


Figure 8.10: A sample ellipse diagram for use in EKF with least square linearization.

Final Remarks While EKF is computationally efficient, it is not always an optimal estimator. It can diverge if the system is highly nonlinear, but it usually works well in practice after some tuning.

8.4 Unscented Kalman Filter

The Unscented transform (UT) is a mathematical function used to estimate the result of applying a given nonlinear transformation to a probability distribution that is characterized only in terms of a finite set of statistics. The key idea is to keep the nonlinear transformation and approximate the distribution directly. This is done by finding a minimal set of sample points that match the first, the second, and the third moments of a Gaussian. A picture of this approach is given in fig. 8.11.

A convenient choice for these sigma points (sample, weight) is:

$$m_x \pm \sqrt{(n + \kappa) \Sigma_x} e_i \quad \frac{\kappa}{n + \kappa} \quad \frac{1}{2(n + \kappa)} \quad (8.38)$$

where $e_i = [0, \dots, 0, 1, 0, \dots, 0]$ is a unit vector whose entries are all zero except the i th entry (same as defined above). The variable κ is an extra degree of freedom to fine-tune the higher order moments of the approximation. For $x \in \mathbb{R}^n$, this results in $2n + 1$ samples.

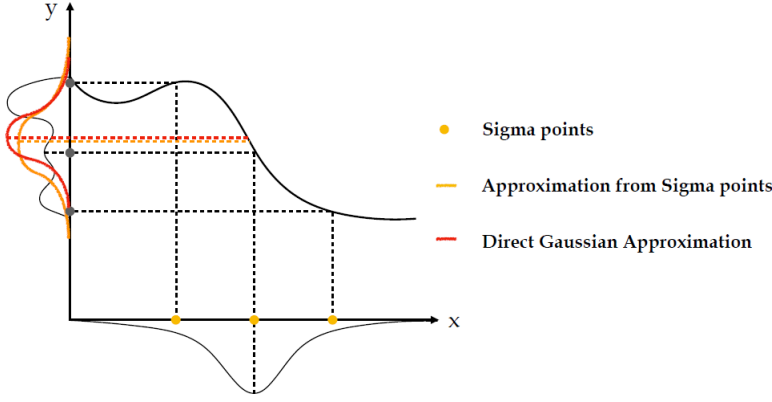


Figure 8.11: Illustration of the UKF approach for approximating the non-Gaussian distribution after a nonlinear transformation. The actual y distribution is shown in black. The direct Gaussian approximation of the y distribution is shown in red. However, the direct Gaussian approximation requires the access of the full distribution of y , which is very challenging to obtain. In practice, the UKF approach takes several samples to approximate the y distribution. These samples are called the sigma points. The approximated y distribution is shown in yellow, which can at least approximate the first three moments of the y distribution.

UKF vs EKF You might have noticed that both EKF and UKF uses samples for certain computation step. However, the usage of these samples are different. EKF uses these samples to linearizes the non-linear function through least square linearization, while UKF keeps the function as a nonlinear function and only uses these samples to “linearizes” the distribution after the nonlinear transformation. Moreover, even if we use the same samples in both EKF and UKF, UKF will still out perform the EKF. That is because UKF contains higher order information than EKF as it does not linearize the original nonlinear function. In addition, UKF will significantly outperform EKF when the analytical derivatives, finite-difference derivatives, and least squares all end up with a horizontal linearization (zero gradient), which leads to a prediction of zero variance using EKF.

Dynamic Update in UKF The inputs to the dynamic update are the a posteriori estimate $\hat{\mathbf{x}}_{k-1|k-1}$ and the covariance from the last step $\mathbf{Z}_{k-1}$. We first collect a number of samples of the distribution

$$\mathcal{X}_{k-1} = \text{Sample}(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{Z}_{k-1}), \quad (8.39)$$

and then we propagate these samples forward in time using the dynamic function (without noise):

$$\mathcal{X}_k = f_{k-1}(\mathcal{X}_{k-1}, \mathbf{u}_k). \quad (8.40)$$

We then calculate the mean via:

$$\hat{\mathbf{x}}_{k|k-1} = \text{mean}(\mathcal{X}_k), \quad (8.41)$$

and the covariance as

$$\mathbf{M}_k = \text{Variance}(\mathcal{X}_k) + \mathbf{B}^w \mathbf{W} (\mathbf{B}^w)^T. \quad (8.42)$$

Measurement Update in UKF From the previous discussion of Measurement Update in Kalman Filter, we have

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \text{COV}(\tilde{\mathbf{x}}_{k|k-1}, (\tilde{\mathbf{y}}_k)_{Y_{k-1}}) \text{VAR}((\tilde{\mathbf{y}}_k)_{Y_{k-1}})^{-1} (\tilde{\mathbf{y}}_k)_{Y_{k-1}}, \quad (8.43)$$

$$Z_k = M_k - \text{COV}[\tilde{\mathbf{x}}_{k|k-1}, (\tilde{\mathbf{y}}_k)_{Y_{k-1}}] \text{VAR}[(\tilde{\mathbf{y}}_k)_{Y_{k-1}}, \tilde{\mathbf{x}}_{k|k-1}] \text{COV}[(\tilde{\mathbf{y}}_k)_{Y_{k-1}}, \tilde{\mathbf{x}}_{k|k-1}]. \quad (8.44)$$

Previously, we derived the error, the variance, and the covariance terms using linear or linearized measurement equations. Now, in UKF, we need to use the samples to estimate these terms, in particular, computing $(\tilde{\mathbf{y}}_k)_{Y_{k-1}}$ using the statistic error, $\text{COV}(\tilde{\mathbf{x}}_{k|k-1}, (\tilde{\mathbf{y}}_k)_{Y_{k-1}})$ using the statistic covariance $\Sigma_k^{x,y}$, and $\text{VAR}((\tilde{\mathbf{y}}_k)_{Y_{k-1}})$ using the statistic variance Σ_k^y .

We first take a number of samples from the a prior estimate:

$$\tilde{\mathcal{X}}_k = \text{Sample}(\hat{\mathbf{x}}_{k|k-1}, M_k), \quad (8.45)$$

and then propagating them through the nonlinear measurement equation:

$$\mathcal{Y}_k = h_k(\tilde{\mathcal{X}}_k). \quad (8.46)$$

The statistics from the samples are

$$\Sigma_k^{x,y} = \text{Covariance}(\tilde{\mathcal{X}}_k, \mathcal{Y}_k), \quad (8.47)$$

$$\Sigma_k^y = \text{Variance}(\mathcal{Y}_k) + V_k, \quad (8.48)$$

$$\tilde{\mathbf{y}}_{k|k-1} = \mathbf{y}_k - \text{Mean}(\mathcal{Y}_k), \quad (8.49)$$

which allows us to rewrite our earlier measurement update equations to

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + \Sigma_k^{x,y} (\Sigma_k^y)^{-1} (\mathbf{y}_k - \hat{\mathbf{y}}_{k|k-1}), \quad (8.50)$$

$$Z_k = M_k - \Sigma_k^{x,y} (\Sigma_k^y)^{-1} (\Sigma_k^{x,y})^T. \quad (8.51)$$

Final Remarks A UKF has the same complexity as an EKF and has a better “linearization” than an EKF. This better linearization arises from the fact that the UKF captures the first two terms of the Taylor expansion. Furthermore, it’s derivative-free, which is super-helpful as that means we do not need to calculate any Jacobian.

Recursive Least Square and Stochastic Gradient Descent

This chapter will discuss methods for online adaptation. Online adaptation can be applied to 1) offline obtained control policies (either designed or learned), which corresponds to direct adaptive control; or 2) offline obtained dynamic models (either designed or learned), which corresponds to indirect adaptive control. In the second case, the model-based controller will be updated once the model is adapted.

Mathematically, online adaptation and online filtering solve the same problem, *i.e.*, inferring certain parameters from the historical observation. In practice, people use the term “filtering” for system states and the term “adaptation” for system parameters. Later in this chapter, we will see that the widely used recursive least square algorithms for online adaptation is just another form of the Kalman Filter.

9.1 Model Adaptation

9.1.1 Prediction versus Control

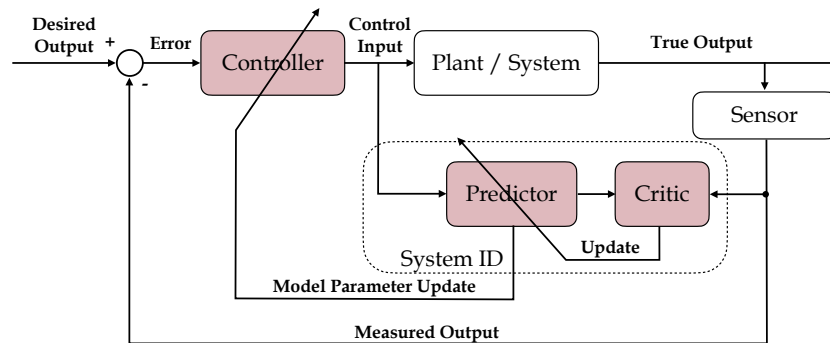


Figure 9.1: Block diagram for indirect adaptive control.

The block diagram of an indirect adaptive control system is shown

in fig. 9.1. The dashed box contains modules for model adaptation, where the predictor contains the model parameters that describe the plant/system dynamics; the critic compares the predicted output with the measured output and updates the model parameters in the predictor; finally, the updated model parameter will then be incorporated in the controller. The model adaptation solves similar problems as an system identification process, both of which compute system parameters from system inputs and system outputs. Conventionally, system identification refers to an offline process where the data (system inputs and outputs) can be shuffled, while model adaptation refers to an online process where the data is received chronically in time.

For model adaptation, it is important to distinguish between pure prediction problem versus control problem that uses prediction as a sub-module. The key difference between the two lies in whether the output of the plant/system will be affected by the prediction, in other words, whether the predictor is a pure observer. For a pure prediction problem, the predictor functions as a pure observer in the sense that the output of the predictor will not affect the plant's output, as shown in the left figure of fig. 9.2. For a control problem that involves prediction, the output of the predictor will enter affect the plant's output, as shown in the right figure of fig. 9.2. For example, in the case of match prediction in a ball game, an ordinary audience is a pure observer, while the player in the game is a controller since he or she can affect the true outcome of the game. Similarly, the predictor in fig. 9.1 is not facing a pure prediction problem. When the prediction enters in to the control loop, we need to consider the consequence generated by a prediction, which adds another layer of complexity. In this chapter, we are focused on the prediction problem only.

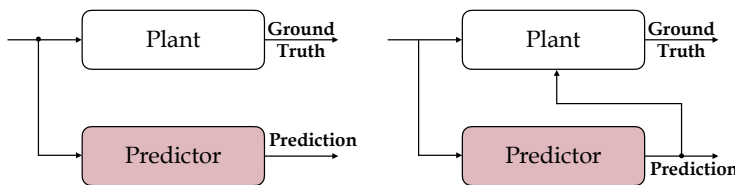


Figure 9.2: Prediction problem (left) versus control problem (right).

9.1.2 Model Learning versus Model Adaptation

Model adaptation is different from the case where learn the model from scratch. We call the method that learn a model from scratch as *model learning*. We treat model learning as equivalent to system identification in this course. For model learning, we need to obtain

a function approximator from a data sequence as shown in fig. 9.3. In the figure, we used a first order model to approximate the data sequence, which can be computed from least square methods as introduced in earlier chapters. The learned model gives good approximation globally but may do poorly locally. Model adaptation

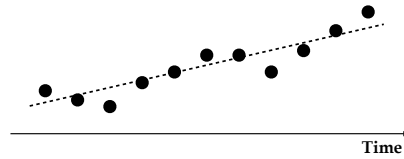


Figure 9.3: Illustration of model learning from a few samples.

is to locally modify the global model to fit the local data in order to improve the local prediction accuracy. The model adaptation process is shown in fig. 9.4. At the first step, there is no extra information, so the future is predicted following the global model, as shown in the red dashed curve. In the second step, the model is adapted based on the prediction error we measured, resulting in a change in the slope of the prediction. This adaptation reduces the prediction error in the next time step compared to the global model. In the third time step, the model is further adapted. However, this time, the adapted model results in a larger error compared the global model. This is because there is a sudden change in the data samples considering the first order model, and model adaptation amplifies the sudden change. In the fourth step, the model adaptation corrects the error and improves the prediction again.

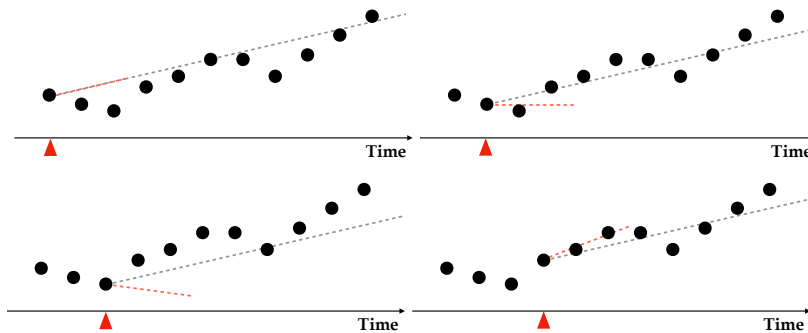


Figure 9.4: Illustration of model adaptation. The current time is noted by the red arrow. The prediction done by the nominal model is shown as the gray dashed line. The prediction done by the adapted model is shown as the red dashed line.

In summary, the objective for model learning is to minimize the fitting error on all data points, while the objective for model adaptation is to minimize local prediction error. Model learning emphasizes generalizability of the model and assumes that the data is homogeneous. Model adaptation exploits local over-fitting to minimize prediction error, which works when the model is time-invariant or with slow time-varying dynamics.

The process of the model adaptation is illustrated in fig. 9.5. The ground truth system is assumed to be $x_{k+1} = f(x_k)$. We approximate the ground truth using a model f_θ which is parameterized by θ . At different time step, we have different estimate of the model parameter, denoted θ_k . Using the model parameter, we will generate a prediction of the future state $\hat{x}_{k+1} = f_{\theta_k}(x_k)$. Now let us see what is going on at time k . Once we get a measurement x_k , we compare it with the prediction $\hat{x}_k = f_{\theta_{k-1}}(x_{k-1})$ made at last step. Then the adaptation mechanism will turn the error in to a signal to update θ_{k-1} to θ_k . Then using the new model parameter θ_k , we will generate a prediction $\hat{x}_{k+1}$. And this process will be repeated.

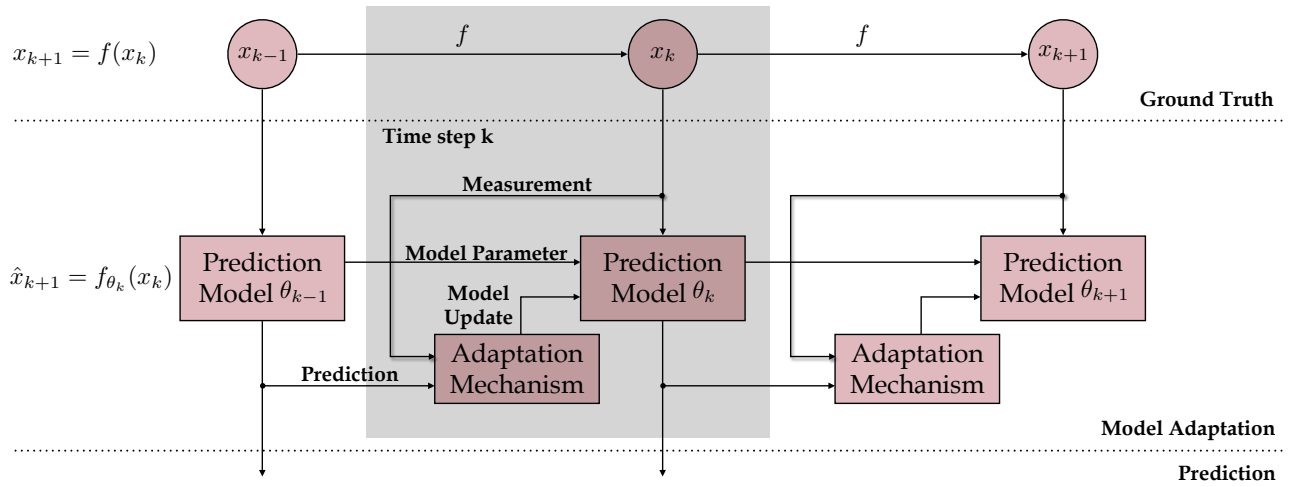


Figure 9.5: Illustration of model adaptation in prediction problems.

9.2 Recursive Least Square

This section introduces the recursive least square algorithm for model adaptation. Note that we mentioned earlier that we can use least square methods for model learning or system identification. Recursive least square is a way to solve least square problems online, *i.e.*, when the problem received new data points at every time step.

At time k , we want to have a minimum variance prediction of the future

$$\min_{\theta} \mathbb{E}_{x_{k+1}} \|x_{k+1} - f_{\theta}(x_k)\|^2. \quad (9.1)$$

We assume that the parameters change slowly, *i.e.*, $\dot{\theta} \approx 0$. Then the parameter that results in smallest estimation error can be approximated by θ_k which solves the following least square fitting problem

$$\theta_k = \min_{\theta} L_k, \quad (9.2a)$$

$$L_k = \frac{1}{2} \|x_k - f_{\theta}(x_{k-1})\|^2 + \lambda L_{k-1} = \frac{1}{2} \sum_{i=0}^k \lambda^{k-i} \|x_i - f_{\theta}(x_{i-1})\|^2. \quad (9.2b)$$

Note that L_k is defined recursively. The parameter $\lambda \in [0, 1]$ is a forgetting factor. When $\lambda = 0$, the formulation will forget all past information. As a result, the true parameter may be better approximated, at the expense of stability and robustness issues (recall the example in fig. 9.4). When $\lambda = 1$, the formulation will memorize all past information. Memorizing past information will improve stability and robustness with reduced accuracy, *i.e.*, the learning process may be saturated quickly.

To solve (9.2), we need to find the parameter θ_k such that

$$\nabla_{\theta} L_k = -\nabla_{\theta} f_{\theta}(x_{k-1})^T [x_k - f_{\theta}(x_{k-1})] + \lambda \nabla_{\theta} L_{k-1} = 0. \quad (9.3)$$

Since we need to solve the problem for every time step k , it is better to save some computation by reusing the result from previous steps. Ideally, we should find a relationship between θ_k and θ_{k-1} by solving (9.3). Let us now analyze the terms in (9.3) at θ_k one by one. First, with slow changing dynamics assumption, the gradient from the model f to its parameter at θ_k can be approximated by the gradient at θ_{k-1} and we denote the second gradient evaluated at x_{k-1} as G_{k-1} :

$$\nabla_{\theta|_{\theta_k}} f_{\theta}(x_{k-1}) \approx \underbrace{\nabla_{\theta|_{\theta_{k-1}}} f_{\theta}(x_{k-1})}_{G_{k-1}}. \quad (9.4)$$

Then we have that the a posteriori prediction error can be approximated by the a priori prediction error e_k plus the gradient multiplying the difference between two parameter estimates in the two consecutive time steps:

$$x_k - f_{\theta_k}(x_{k-1}) \quad (9.5a)$$

$$= x_k - f_{\theta_{k-1}}(x_{k-1}) + f_{\theta_{k-1}}(x_{k-1}) - f_{\theta_k}(x_{k-1}) \quad (9.5b)$$

$$\approx e_k + \underbrace{\nabla_{\theta|_{\theta_{k-1}}} f_{\theta}(x_{k-1})}_{G_{k-1}} [\theta_{k-1} - \theta_k]. \quad (9.5c)$$

Finally, the gradient of the loss function L_{k-1} at θ_k can be approximated by the gradient at θ_{k-1} plus the second-order gradient (*i.e.*, Hessian) at θ_{k-1} multiplying the difference between two parameter estimates in the two consecutive time steps. Since θ_{k-1} is the optimal

solution of L_{k-1} , then we have

$$\nabla_{\theta|_{\theta_k}} L_{k-1} \approx \nabla_{\theta|_{\theta_{k-1}}} L_{k-1} + \underbrace{\nabla_{\theta|_{\theta_{k-1}}}^2 L_{k-1}}_{H_{k-1}} \cdot [\theta_k - \theta_{k-1}] = H_{k-1} [\theta_k - \theta_{k-1}]. \quad (9.6)$$

The Hessian of the loss function L_k evaluated at θ_k is denoted as H_k .

Plugging (9.4), (9.5), and (9.6) into (9.3), we have

$$\nabla_{\theta} L_k = G_{k-1}^T [G_{k-1}(\theta_k - \theta_{k-1}) - e_k] + \lambda H_{k-1}(\theta_k - \theta_{k-1}) = 0. \quad (9.7)$$

Rearranging the above equation, we find the relationship between two consecutive estimates

$$\theta_k = \theta_{k-1} + [G_{k-1}^T G_{k-1} + \lambda H_{k-1}]^{-1} G_{k-1}^T e_k. \quad (9.8)$$

We can also get the approximate update rule of the Hessian:

$$\nabla_{\theta|_{\theta_k}}^2 L_k = \frac{1}{2} \nabla_{\theta|_{\theta_k}}^2 \|x_k - f_{\theta}(x_{k-1})\|^2 + \lambda \nabla_{\theta|_{\theta_k}}^2 L_{k-1} \quad (9.9a)$$

$$\approx [\nabla_{\theta|_{\theta_k}} f_{\theta}(x_{k-1})]^T \nabla_{\theta|_{\theta_k}} f_{\theta}(x_{k-1}) + \lambda \nabla_{\theta|_{\theta_k}}^2 L_{k-1} \quad (9.9b)$$

$$\approx [\nabla_{\theta|_{\theta_{k-1}}} f_{\theta}(x_{k-1})]^T \nabla_{\theta|_{\theta_{k-1}}} f_{\theta}(x_{k-1}) + \lambda \nabla_{\theta|_{\theta_{k-1}}}^2 L_{k-1} \quad (9.9c)$$

$$= G_{k-1}^T G_{k-1} + \lambda H_{k-1} \quad (9.9d)$$

Hence the update rule of the RLS can be rewritten as

$$\theta_k = \theta_{k-1} + [H_k]^{-1} G_{k-1}^T e_k \quad (9.10a)$$

$$H_k = G_{k-1}^T G_{k-1} + \lambda H_{k-1} \quad (9.10b)$$

$$e_k = x_k - f_{\theta_{k-1}}(x_{k-1}), \quad (9.10c)$$

where $G_k = \nabla_{\theta|_{\theta_k}} f_{\theta}(x_k)$.

Note that the RLS algorithm does not require that the model input and output are from the same source. The model can be generalized to be any $y_{k+1} = f_{\theta}(x_k)$. There is no notion of “noise” in the derivation, unlike in the Kalman filter. However, in order for the estimated parameter to converge to the true value (not stuck in local optima), it is necessary for the ground truth system to have persistent disturbances across all spectrum, e.g., white noise.

When the system is linear,

$$y_{k+1} = \theta^T x_k, \quad (9.11)$$

where $y_{k+1} \in \mathbb{R}$ and $\theta, x_k \in \mathbb{R}^n$, then all the above computations are exact, *i.e.*, no approximation. In this case, $G_k = x_k^T$, $H_k = \sum_{i=0}^{k-1} \lambda^{k-1-i} x_i x_i^T$ and the update rule is:

$$\theta_k = \theta_{k-1} + \left[\sum_{i=0}^{k-1} \lambda^{k-1-i} x_i x_i^T \right]^{-1} x_{k-1} e_k, \quad (9.12)$$

$$e_k = y_k - \theta_{k-1}^T x_{k-1}. \quad (9.13)$$

Let us now denote the inverse of Hessian as the learning gain $F_k := H_k^{-1}$. By Matrix Inverse Indentity, we have

$$F_k = H_k^{-1} = (G_{k-1}^T G_{k-1} + \lambda H_{k-1})^{-1} \quad (9.14a)$$

$$= [\lambda H_{k-1}]^{-1} - [\lambda H_{k-1}]^{-1} G_{k-1}^T [I + G_{k-1} [\lambda H_{k-1}]^{-1} G_{k-1}^T]^{-1} G_{k-1} [\lambda H_{k-1}]^{-1} \quad (9.14b)$$

$$= \frac{1}{\lambda} \left\{ F_{k-1} - F_{k-1} G_{k-1}^T [\lambda I + G_{k-1} F_{k-1} G_{k-1}^T]^{-1} G_{k-1} F_{k-1} \right\}. \quad (9.14c)$$

In the linear case in (9.11), we have

$$F_k = \frac{1}{\lambda} \left\{ F_{k-1} - \frac{F_{k-1} x_{k-1} x_{k-1}^T F_{k-1}}{\lambda + x_{k-1}^T F_{k-1} x_{k-1}} \right\}. \quad (9.15)$$

This new update rule is easier to compute than taking the matrix inverse of H_k .

9.3 RLS versus KF

Here we are going to show that RLS can be viewed as a special case of KF. Let us first reformulate the parameter estimation problem in the standard KF form with the dynamic equation and the measurement equation. Since we are estimating parameter θ , the dynamic equation now describes the evolution of θ . With dynamic noise and measurement noise, we have:

$$\theta_k = \theta_{k-1} + w_k \quad (9.16)$$

$$y_k = x_{k-1}^T \theta_k + v_k \quad (9.17)$$

Then apply the standard KF on the above dynamic systems, we have following update rules.

- Dynamic update

$$\hat{\theta}_{k|k-1} = \hat{\theta}_{k-1|k-1} \quad (9.18)$$

$$M_k = Z_{k-1} + W_k. \quad (9.19)$$

- Measurement update

$$\hat{\theta}_{k|k} = \hat{\theta}_{k|k-1} + M_k x_{k-1} [x_{k-1}^T M_k x_{k-1} + V_k]^{-1} [y_k - x_{k-1}^T \hat{\theta}_{k|k-1}] \quad (9.20)$$

$$Z_k = M_k - M_k x_{k-1} (x_{k-1}^T M_k x_{k-1} + V_k)^{-1} x_{k-1}^T M_k \quad (9.21)$$

Combining these two updates, we have the following update rule for the a posterior estimate (which equals to the a priori estimate)

$$\hat{\theta}_k = \hat{\theta}_{k-1} + \underbrace{(Z_{k-1} + W_k) x_{k-1} [x_{k-1}^T (Z_{k-1} + W_k) x_{k-1} + V_k]^{-1}}_{KF_k} [y_k - x_{k-1}^T \hat{\theta}_{k-1}]. \quad (9.22)$$

The Kalman gain is denoted as KF_k . Here it refers to the Kalman gain. The update rule for the a posterior variance is

$$Z_k = (Z_{k-1} + W_k) - (Z_{k-1} + W_k)x_{k-1}(x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} + V_k)^{-1}x_{k-1}^T(Z_{k-1} + W_k). \quad (9.23)$$

Now the question becomes: how are these two equations relate to the RLS? Let us first try to put the two equations into a similar form as in RLS. We first want to find a simplified expression for the Kalman gain KF_k . To simplify KF_k , let us look at the update rule for the variance in (9.23) and multiply x_{k-1} on both sides:

$$\begin{aligned} Z_k x_{k-1} &= (Z_{k-1} + W_k)x_{k-1} - (Z_{k-1} + W_k)x_{k-1}(x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} + V_k)^{-1}x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} \\ &= (Z_{k-1} + W_k)x_{k-1} \left[I - (x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} + V_k)^{-1}x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} \right] \\ &= \underbrace{(Z_{k-1} + W_k)x_{k-1}(x_{k-1}^T(Z_{k-1} + W_k)x_{k-1} + V_k)^{-1}}_{KF_k} V_k \quad (9.24a) \end{aligned}$$

Hence we have a simplified expression for the Kalman gain:

$$KF_k = Z_k x_{k-1} V_k^{-1} \quad (9.25)$$

Then the update rule for the a posterior estimate can be rewritten as

$$\hat{\theta}_k = \hat{\theta}_{k-1} + Z_k x_{k-1} V_k^{-1} [y_k - x_{k-1}^T \hat{\theta}_{k-1}], \quad (9.26)$$

which is getting closer to the expression in RLS.

Now let's make the following assumptions on the process noise and measurement noise: $W_k = (\lambda_1 - 1)Z_{k-1}$ and $V_k = \lambda_2 I$, and define $\bar{Z}_k = \frac{Z_k}{\lambda_2}$. As a result, the update rule becomes

$$\hat{\theta}_k = \hat{\theta}_{k-1} + \bar{Z}_k x_{k-1} [y_k - x_{k-1}^T \hat{\theta}_{k-1}] \quad (9.27a)$$

$$\bar{Z}_k = \lambda_1 \left\{ \bar{Z}_{k-1} - \frac{\bar{Z}_{k-1} x_{k-1} x_{k-1}^T \bar{Z}_{k-1}}{x_{k-1}^T \bar{Z}_{k-1} x_{k-1} + \frac{1}{\lambda_1}} \right\} \quad (9.27b)$$

Let us rewrite the update rule in RLS below.

$$\hat{\theta}_k = \hat{\theta}_{k-1} + F_k x_{k-1} [y_k - x_{k-1}^T \hat{\theta}_{k-1}] \quad (9.28a)$$

$$F_k = \frac{1}{\lambda} \left\{ F_{k-1} - \frac{F_{k-1} x_{k-1} x_{k-1}^T F_{k-1}}{\lambda + x_{k-1}^T F_{k-1} x_{k-1}} \right\} \quad (9.28b)$$

By comparing KF and RLS, we find that $\bar{Z}_k$ in KF is equivalent to the learning gain F_k in RLS, and λ_1 in KF is equivalent to λ^{-1} in RLS.

Note that in KF, the parameter for the measurement noise λ_2 actually does not enter into the final expression. Hence we conclude that the measurement noise will not affect the update of the learning gain

if its covariance matrix is proportional to the identity matrix. The dynamic noise does affect the learning gain. Essentially we have

$$F_k = \bar{Z}_k = \frac{Z_k}{\lambda_2} = \frac{W_{k+1}}{(\lambda_1 - 1)\lambda_2} = \frac{\lambda}{1 - \lambda} W_{k+1} V_{k+1}^{-1}. \quad (9.29)$$

The bigger the ratio between the dynamic noise and the measurement noise, the smaller the forgetting rate should be, in order to keep the same learning rate. Although the mathematical forms of the update rules are equivalent in KF and RLS, the physical interpretations of the learning gains are different. The learning gain $\bar{Z}_k$ in KF represents the ratio between the error covariance of the a posteriori estimate and the covariance of the measurement noise. The learning gain F_k in RLS represents the inverse of the Hessian of the objective function. For an initial estimate of the parameter $\theta_0 \sim \mathcal{N}(\bar{\theta}, \Sigma)$, KF will initialize the algorithm by $\hat{\theta}_0 = \bar{\theta}$ and $Z_0 = \Sigma$, while RLS will initialize the algorithm by $\hat{\theta}_0 = \bar{\theta}$ and $F_0 \rightarrow \infty$ since the initial Hessian is zero.

In summary, both KF and RLS are least square estimators. In general, KF offers more design freedoms than RLS since the two covariance matrices can be non-diagonal and can encode prior information. For general nonlinear cases, we just need to replace x_k with gradient G_k^T of the measurement function $y_{k+1} = f_\theta(x_k)$.

9.4 Generalization of RLS

As we discussed earlier, RLS has learning gain F_k which is the inverse of the Hessian $H_k = \lambda H_{k-1} + G_{k-1}^T G_{k-1}$. We can introduce a more general way to update the Hessian and the learning gain where

$$H_k = \lambda_{1,k-1} H_{k-1} + \lambda_{2,k-1} G_{k-1}^T G_{k-1}, \quad (9.30)$$

is a general weighted sum between two terms: the Hessian at the previous step and the gradients at the current step. This formulation covers several types of problems

- $\lambda_{1,k} \equiv 1$ and $\lambda_{2,k} \equiv 0$.

Then the learning rate is constant. The algorithm becomes stochastic gradient descent (SGD), which will be discussed shortly after.

- $\lambda_{1,k} \equiv 1$ and $\lambda_{2,k} \equiv 1$.

Then we have the least square problem without forgetting where the learning rate equals to the Hessian.

- $\lambda_{1,k} < 1$ and $\lambda_{2,k} \equiv 1$.

Then we have the least square problem with forgetting factor where the learning rate equals to the Hessian.

The stochastic gradient descent algorithm solves an optimization problem whose objective function can be decomposed as a summation

$$J(\theta) = \sum_k J_k(\theta). \quad (9.31)$$

We may directly apply gradient descent on $J(\theta)$, which gives us the following update rule

$$\theta^i = \theta^{i-1} - F \nabla_{\theta} J(\theta) = \theta^{i-1} - F \sum_k \nabla_{\theta} J_k(\theta), \quad (9.32)$$

which requires us to take the gradient of every J_k at every iteration i . Stochastic gradient descent allows us to take fewer gradients by learning on each J_k instead of learning J as a whole. The update rule of SGD is

$$\theta_k = \theta_{k-1} - F \nabla_{\theta} J_{k-1}(\theta). \quad (9.33)$$

In our parameter estimation problem, the optimization objective is a (weighted) summation of $J_k(\theta) = \frac{1}{2} \|y_k - f_{\theta_{k-1}}(x_{k-1})\|^2$ for all k . The gradient of $J_k(\theta)$ is $G_{k-1}^T (f_{\theta_{k-1}}(x_{k-1}) - y_k)$. Hence the update rule under SGD becomes

$$\theta_k = \theta_{k-1} - F G_{k-1}^T (f_{\theta_{k-1}}(x_{k-1}) - y_k). \quad (9.34)$$

This equation is exactly the same as what we will have in our RLS update rule when we set the learning rate to be constant. Note that in RLS, we flipped the sign of the second term on the right hand side and the sign of the a priori error in the parenthesis.

Both RLS and SGD are optimization algorithms that can be applied to either offline learning or online adaptation. RLS is more suitable for online adaptation due to its adaptive learning rate, which results in faster convergence as well as more robust learning.

Example Consider a time-varying nonlinear system $x_{k+1} = a(k) \sin(x_k)$ where $a(k) = 0.99 + 0.01k$ and $x_0 = 1$. Figure 9.6 shows the system trajectory and the predictions at different time steps when there is no adaptation of the system parameter. The prediction at time k is

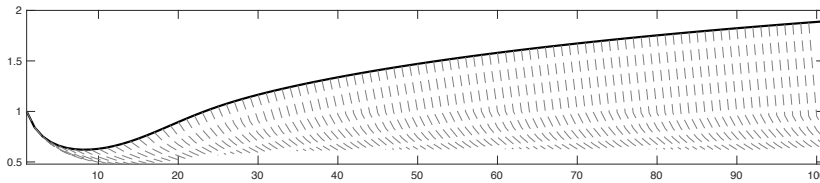
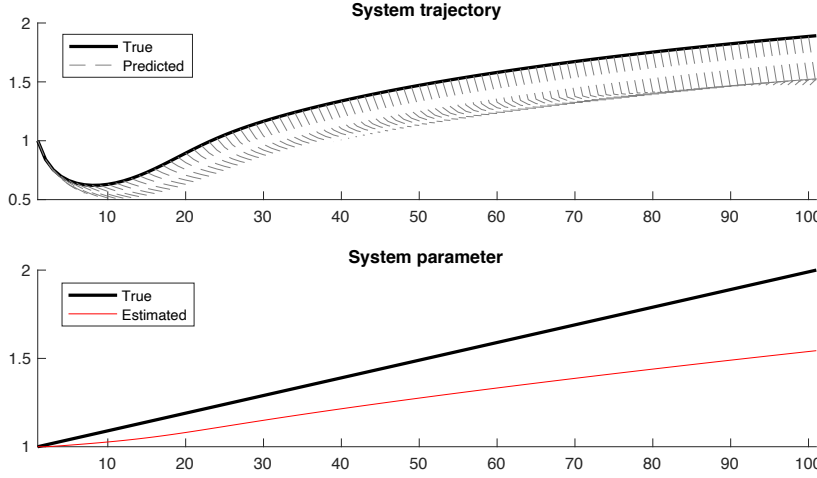
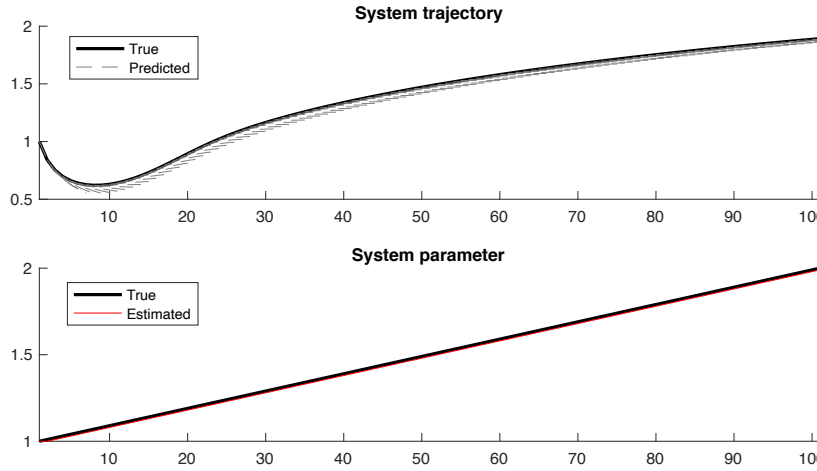


Figure 9.6: The ground truth trajectory (solid curve) and the predictions (dashed curves) using a non-adaptive model in the example.

made by simulating $\hat{x}_{j+1} = \hat{a}_k \sin(\hat{x}_j)$ for $j = k, k+1, \dots, k+4$ from the measured state x_k , i.e., $\hat{x}_k = x_k$. When there is no adaptation, $\hat{a}_k \equiv 1$. Figure 9.7 shows the predictions when we use RLS to adapt $\hat{a}_k$, as well as $\hat{a}_k$ for different time j . When we choose the forgetting factor $\lambda = 1$, $\hat{a}_j$ can slowly track the true parameter. When we choose the forgetting factor $\lambda = 0$, $\hat{a}_j$ is almost identical to the ground truth. There is still a one-step delay in the estimate compared with the ground truth since we are using “previous” data to infer the “current” parameter. The learning rate F_k under different forgetting



(a) Forgetting factor $\lambda = 1$.



(b) Forgetting factor $\lambda = 0$.

factor is shown in fig. 9.8. The learning rate is higher when the forgetting factor is smaller. The learning rate is also time varying, which is bigger when the trajectory changes more rapidly and smaller when the trajectory is smoother.

For comparison, the performance of SGD is shown in fig. 9.9.

Figure 9.7: The performance of RLS under different forgetting factor λ .

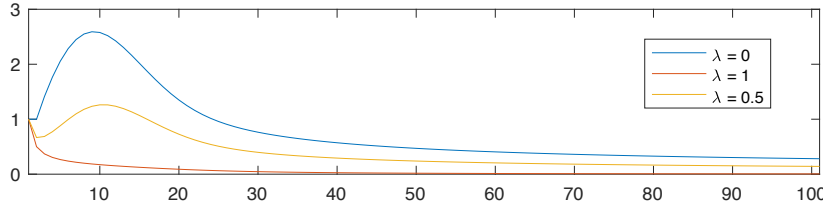


Figure 9.8: The effective learning rate F_k in RLS under different forgetting factor λ .

When the learning rate in SGD is set to be too small ($F = 0.1$), there is a phase lag in the parameter estimate compared to the ground truth. When the learning rate in SGD is set to be too big ($F = 1.7$), the estimate becomes unstable. The system performs desirably when the learning rate is properly chosen ($F = 1$). However, the desired learning rate is problem-specific, which depends both on the changing rate of the parameter and the current input.

9.5 Parameter Convergence

With the parameter adaptation algorithms, we are interested in 1) whether the adaptation is always stable, 2) whether it will always converge to the true value, and 3) whether the prediction error always goes to zero? We are going to analyze these properties formally in a linear time-invariant problem. The ground truth function is $y_{k+1} = \theta^T x_k$ where $y_{k+1} \in \mathbb{R}$ and $\theta, x_k \in \mathbb{R}^n$. The parameter update law is

$$\theta_k = \theta_{k-1} + F_k x_{k-1} e_k \quad (9.35)$$

$$e_k = y_k - \theta_{k-1}^T x_{k-1}. \quad (9.36)$$

In the case of RLS, the learning gain F_k is updated using (9.15). In the case of SGD, the learning gain F_k is a constant.

The parameter estimation error is defined as $\tilde{\theta}_k = \theta - \theta_k$. According to the update law, the error dynamics can be derived as

$$\tilde{\theta}_k = \tilde{\theta}_{k-1} - F_k x_{k-1} e_k \quad (9.37a)$$

$$= \tilde{\theta}_{k-1} - F_k x_{k-1} [y_k - \theta_{k-1}^T x_{k-1}] \quad (9.37b)$$

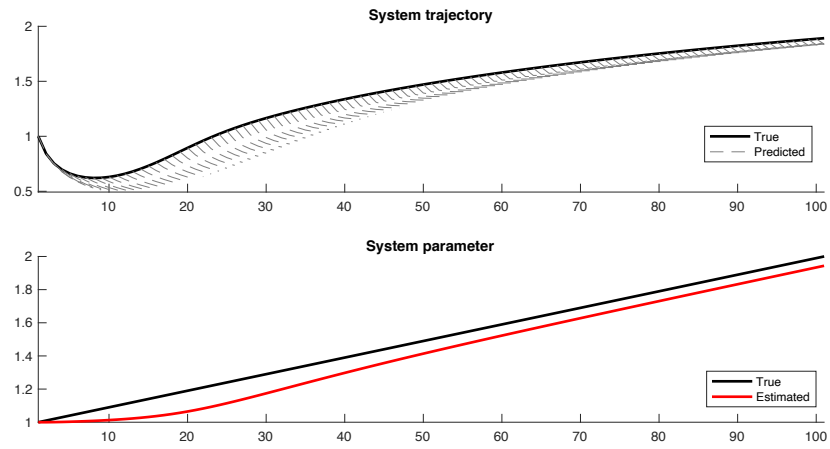
$$= \tilde{\theta}_{k-1} - F_k x_{k-1} \tilde{\theta}_{k-1}^T x_{k-1} \quad (9.37c)$$

$$= \tilde{\theta}_{k-1} - F_k x_{k-1} x_{k-1}^T \tilde{\theta}_{k-1} \quad (9.37d)$$

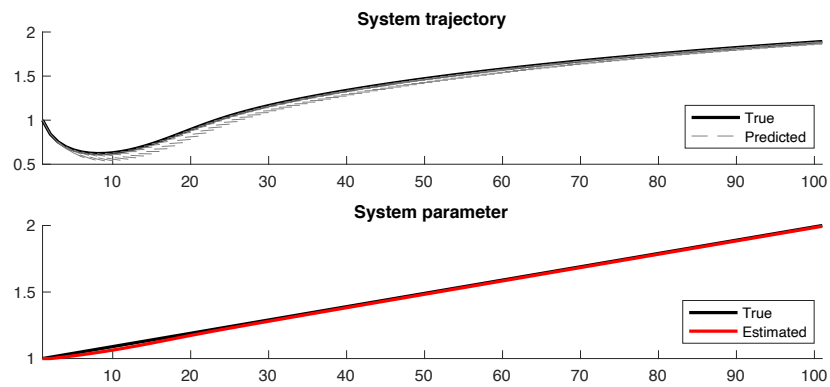
$$= (I - F_k x_{k-1} x_{k-1}^T) \tilde{\theta}_{k-1}. \quad (9.37e)$$

The error dynamics are linear.¹ To answer the questions above, we just need to investigate the stability of the matrix $I - F_k x_{k-1} x_{k-1}^T$, i.e., when the eigen values of the matrix are always smaller than or equal to one, the adaptation is always stable; when the eigen values

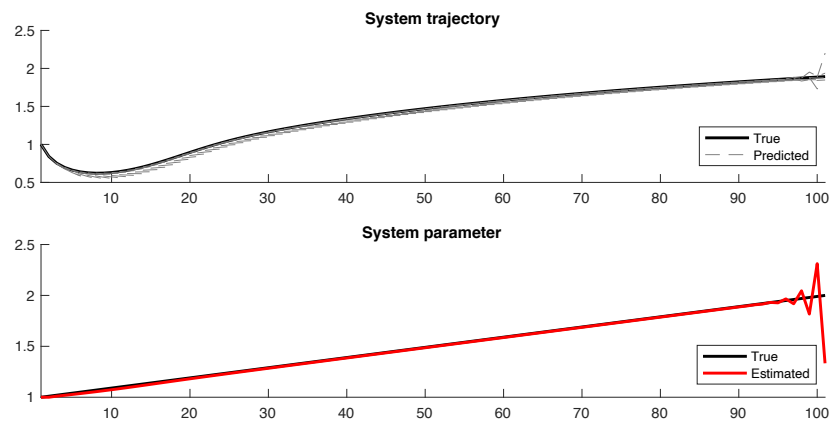
¹ Note that the error dynamics are linear since this is a prediction problem in the sense that the current estimation will not affect the future data x_k . In the case of a control problem where x_k depends on previous parameter estimates, the error dynamics are nonlinear.



(a) Learning rate 0.1.



(b) Learning rate 1.



(c) Learning rate 1.7.

Figure 9.9: Illustration of the performance of SGD under different learning rates.

of the matrix are always smaller than or equal to some $\delta < 1$, then the estimate will always converge to the true value and hence the prediction error will go to zero. Under SGD, the learning gain is a constant, then $I - F_k x_{k-1} x_{k-1}^T$ can have eigenvalues bigger than one. As a result, the error $\tilde{\theta}_k$ may be diverging, as we have seen in the example. In the following discussion, we will focus on the analysis of RLS. We first analyze a scalar case where $\theta \in \mathbb{R}$ and then a vector case where $\theta \in \mathbb{R}^n$ for $n > 1$.

9.5.1 Scalar Case

Under RLS, the learning gain is chosen as $F_k = \left[\sum_{i=0}^{k-1} \lambda^{k-1-i} x_i x_i^T \right]^{-1}$, which implies that

$$1 - F_k x_{k-1}^2 = 1 - \frac{x_{k-1}^2}{\sum_{i=0}^{k-1} \lambda^{k-1-i} x_i^2} \in [0, 1]. \quad (9.38)$$

The equality at 0 is achieved when $\lambda = 0$. The equality at 1 is achieved when $x_{k-1} = 0$. Hence we conclude that the error dynamics are stable since $\|\tilde{\theta}_k\| \leq \|\tilde{\theta}_{k-1}\|$. However, it is not guaranteed that the estimation error will go to zero, since we may always have the equality $\|\tilde{\theta}_k\| = \|\tilde{\theta}_{k-1}\|$. One necessary condition for the parameter to converge to the true value is to have *persistent excitation*, i.e., $x_k \neq 0$ as $k \rightarrow \infty$. The persistent excitation condition implies that $1 - F_k x_{k-1}^2 \neq 1$ when $k \rightarrow \infty$, which breaks the previous equality. And the rate of convergence is affected by the forgetting factor λ . When λ is smaller, then F_k will be bigger and $1 - F_k x_{k-1}^2$ will be smaller, which will lead to faster convergence.

9.5.2 Vector Case

Lemma 9.1 (Eigen Values of the Learning Gain in RLS). *For $F_k = \left[\sum_{i=0}^{k-1} \lambda^{k-1-i} x_i x_i^T \right]^{-1}$, all eigen values of $I - F_k x_{k-1} x_{k-1}^T$ belongs to $[0, 1]$.*

Proof. Let η be an eigen value and μ the associated eigen vector for $I - F_k x_{k-1} x_{k-1}^T$. We have

$$v - F_k x_{k-1} x_{k-1}^T v = \eta v. \quad (9.39)$$

Multiplying H_k on both sides, we have

$$H_k v - x_{k-1} x_{k-1}^T v = \eta H_k v. \quad (9.40)$$

Then multiplying v^T on both sides, we have

$$v^T H_k v - v^T x_{k-1} x_{k-1}^T v = \eta v^T H_k v. \quad (9.41)$$

Since $0 \leq v^T H_k v - v^T x_{k-1} x_{k-1}^T v \leq v^T H_k v$, then we conclude that $\eta \in [0, 1]$. The above analysis does not depend on which eigen value we choose. Hence all eigen values of $I - F_k x_{k-1} x_{k-1}^T$ should belong to $[0, 1]$. $\square$

With the lemma, we can have the following theorem.

Theorem 9.1 (Stability of RLS). *Under RLS parameter adaptation for $y_{k+1} = \theta^T x_k$ where $y_{k+1} \in \mathbb{R}$ and $\theta, x_k \in \mathbb{R}^n$, the error dynamics $\tilde{\theta}_k = \theta - \theta_k$ is stable and $\|\tilde{\theta}_k\| \leq \|\tilde{\theta}_{k-1}\|$ for all k . One necessary condition for $\lim_{k \rightarrow \infty} \tilde{\theta}_k = 0$ is persistent excitation, i.e., $\|x_k\| \not\equiv 0$ as $k \rightarrow \infty$.*

The proof is straightforward and is skipped. One important aspect to note is that persistent excitation is not a sufficient condition for the parameter estimate to converge to the true parameter.

9.5.3 Prediction Error

We have analyzed the dynamics of the parameter estimation error. In certain applications, we care more about the prediction error e_k . Moreover, in the objective function for parameter estimation, it is the prediction error that is being penalized. From the expression $e_k = \tilde{\theta}_{k-1}^T x_{k-1}$, we can see that the prediction error highly depends on the input data x_k . Since the inner product of two vectors is bounded by the product of the norm of the two vectors, we have the following inequality

$$e_k \leq \|\tilde{\theta}_{k-1}\| \|x_{k-1}\|. \quad (9.42)$$

With the inequality, we can conclude that

- If the parameter error converges to zero, then the prediction error converges to zero.
- If the prediction error converges to zero, the parameter error may not converge.
- $\frac{|e_k|}{\|x_{k-1}\|}$ is bounded.

For online adaptation, the input data is not random, but follows its own dynamics. The analysis of the prediction error should take that dynamics into consideration. In practice, for linear time-invariant problem, the prediction error converges to zero in most cases.

9.5.4 Uncertainty Estimation

We have shown that RLS can be viewed as a special case of KF. In KF, we have propagate the covariance Z_k of the parameter error together with the parameter estimate $\hat{\theta}_k$. And that covariance can be used as a

way to quantify uncertainties of the estimation and prediction in real time. The variance of the prediction error e_k is

$$\text{VAR}(e_k) = x_{k-1}^T \text{VAR}(\tilde{\theta}_{k-1}) x_{k-1} + V_k \quad (9.43a)$$

$$= x_{k-1}^T Z_{k-1} x_{k-1} + V_k. \quad (9.43b)$$

Example Figure 9.10 illustrates the uncertainty range of an adaptation process. The system follows the dynamics $x_{k+1} = \theta x_k + w_k$ where $\theta = 1$, $x_0 = 1$, and $w_k \sim \mathcal{N}(0, 0.25)$. The parameter estimate is initialized as $\hat{\theta}_0 = 1.4$ and the forgetting factor is chosen as $\lambda = 0.8$. In the bottom figure in fig. 9.10, we show both the σ and the 3σ uncertainty bounds where $\sigma_k = \sqrt{Z_k}$. The estimate $\hat{\theta}_k$ in this example is properly bounded by the 3σ bounds.

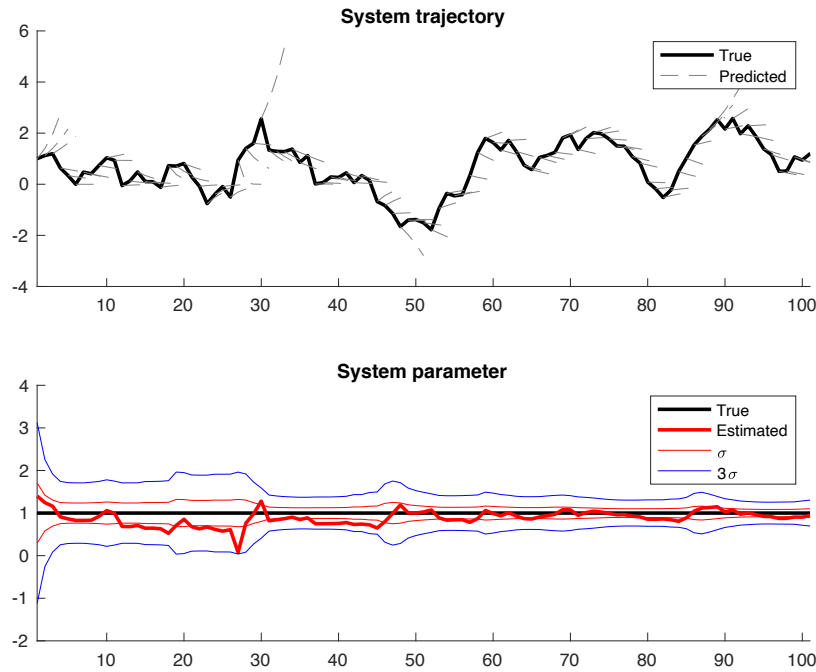


Figure 9.10: Illustration of the uncertainty estimation.

Part IV

Indirect Adaptive Control

Passive Learning and Separation Principle

We have studied model adaptation methods in prediction problems. From now on, we are going to discuss adaptation methods in control problems. As we discussed earlier, in prediction problems, how we predict does not affect the future data that we receive for adaptation. The adaptation process is decoupled from the dynamics that we are adapting (called the target dynamic system). This feature makes it easier to design the adaptation algorithms as we do not need to worry about exciting unstable modes in the target dynamic system. However, in control problems, how we predict will affect how we act, hence affects the evolution of the target dynamic system. As a result, it will also affect the future data we receive for adaptation. The control problem becomes more complicated than a pure prediction problem, since we not only need to improve the estimation but we also need to stabilize the target dynamic system. This chapter considers indirect adaptive control where we obtain the control law by combining a model-based controller with a model adaptation algorithm. The error signal for model adaptation is affected by the model-based controller. We can do the model adaptation in two different approaches

- Passive learning

The controller does not actively take actions to minimize the parametric uncertainty in the future.

- Active learning

The controller actively takes actions to minimize the parametric uncertainty in the future. The action that minimizes the parametric uncertainty in the future is also called *information gathering* or *exploration*.

This chapter deals with passive learning, while active learning will be addressed in the next chapter. In the following sections, we first discuss passive learning with more details, then introduce the

separation principle where passive learning is optimal, followed by the introduction of the model reference adaptive control which deals with systems that the separation principle does not apply.

10.1 Passive Learning

In passive learning, we do not need to modify the model-based controller (obtained for the deterministic system) other than plugging in the newly estimated parameters. Consider the homework problem where we have the following dynamic system with unknown parameters a, b, c :

$$x_{k+1} = \begin{bmatrix} 1 & a \\ 0 & 1 \end{bmatrix} x_k + \begin{bmatrix} b \\ c \end{bmatrix} (u_k + w_k) \quad (10.1)$$

$$y_k = x_k + v_k \quad (10.2)$$

The control objective is to minimize a quadratic cost function. In the deterministic case (no noise and we know a, b, c), we can compute the optimal control law $u_k = g(x \mid a, b, c)$ by solving LQR. The control law is conditioned of the system parameter a, b, c . In the stochastic case, if we know the control signals, we can run a state or parameter estimation algorithm to estimate the state and the parameters: $\hat{x}_k, \hat{a}_k, \hat{b}_k, \hat{c}_k = h(\hat{x}_{k-1}, \hat{a}_{k-1}, \hat{b}_{k-1}, \hat{c}_{k-1}, u_{k-1}, y_k)$. In passive learning, we construct the control law for the stochastic problem by combining the two functions g and h together

$$u_k = g(\hat{x}_k, \hat{a}_k, \hat{b}_k, \hat{c}_k) \quad (10.3a)$$

$$\hat{x}_k, \hat{a}_k, \hat{b}_k, \hat{c}_k = h(\hat{x}_{k-1}, \hat{a}_{k-1}, \hat{b}_{k-1}, \hat{c}_{k-1}, u_{k-1}, y_k) \quad (10.3b)$$

Will this naive combination be a problem? The answer is yes. Since the control law does not consider information gathering, there may not be enough exploration in parameter adaptation, which may lead to biased parameter estimates, hence suboptimal control. Nonetheless, will this approach ever work (achieve optimality in the stochastic sense)? The answer is also yes. This approach works (is optimal) if the problem satisfies the separation principle.

10.2 Separation Principle

Separation principle is a principle of separation of estimation and control. An optimal stochastic control law usually consists of two parts: an optimal control law with given state and parameter estimates and an optimal state and parameter estimator. Under the separation principle, the naive combination of the two parts as shown in (10.3) is optimal. In the following discussion, we will show that

Linear Quadratic Gaussian (LQG) problems satisfy separation principle.

10.2.1 Linear Quadratic Gaussian

An LQG problem is formulated as the following. We have a linear stochastic system

$$x_{k+1} = A_k x_k + B_k u_k + B_k^w w_k \quad (10.4)$$

$$y_k = C_k x_k + v_k, \quad (10.5)$$

where $x_0 \sim \mathcal{N}(x_0^*, X_0)$, $w_k \sim \mathcal{N}(0, W_k)$, $v_k \sim \mathcal{N}(0, V_k)$ are Gaussian independent random variables. The random processes w_k and v_k are white. We have a quadratic cost function

$$J = \mathbb{E} \left[\frac{1}{2} \sum_{k=0}^{N-1} [x_k^T Q_k x_k + u_k^T R_k u_k] + \frac{1}{2} x_N^T S_N x_N \right], \quad (10.6)$$

where Q_k and S_N are positive semi-definite and R is positive definite.

There is no parametric uncertainty in LQG, *i.e.*, A_k, B_k, B_k^w, C_k are all known. But as we discussed earlier, we can turn any system with parametric uncertainty to a system without parametric uncertainty by augmenting the states to include the unknown parameters. If such augmented system is still linear, we can apply the analysis of LQG on the augmented system.

To derive the separation principle under LQG, we first consider a simpler problem where we can observe the state without any uncertainty, *i.e.*, $y_k = x_k$. Then we will consider the case that we only have noisy readings of the state.

10.2.2 LQG with Fully Observed States

If we can fully observe the states, then the expectation in (10.6) is essentially taken over all the process noises w_k . Since the process noises are independent from each other, the expectation can be decoupled if a state value is given, *i.e.*, the expectation will not depend on previous states or control. Hence we can define the cost-to-go as

$$J_k(x_k; u_k, \dots, u_{N-1}) = \mathbb{E}_{w_k, \dots, w_{N-1}} \left[\frac{1}{2} \sum_{j=k}^{N-1} [x_j^T Q_j x_j + u_j^T R_j u_j] + \frac{1}{2} x_N^T S_N x_N \right]. \quad (10.7)$$

Then the value function can be defined as

$$J_k^*(x_k) = \min_{u_k, \dots, u_{N-1}} J_k(x_k; u_k, \dots, u_{N-1}). \quad (10.8)$$

Recall that in the deterministic case, we apply the principle of optimality to derive the Bellman equation regarding the value function, *i.e.*,

$$J_k^*(x_k) = \min_{u_k} \{l_k(x_k, u_k) + J_{k+1}^*(x_{k+1})\}. \quad (10.9)$$

In the stochastic case, does the principle of optimality hold? The answer is yes. Then what is the resulting Bellman equation?

Let us now derive the Bellman Equation from the value function at time k . By definition, we have

$$J_k^*(x_k) = \min_{u_k, \dots, u_{N-1}} \mathbb{E}_{w_k, \dots, w_{N-1}} \left[\frac{1}{2} \sum_{j=k}^{N-1} [x_j^\top Q_j x_j + u_j^\top R_j u_j] + \frac{1}{2} x_N^\top S_N x_N \right]. \quad (10.10)$$

Since x_k and u_k does not depend on any of the process noise in the expectation, we can rewrite the equation as

$$J_k^*(x_k) = \min_{u_k, \dots, u_{N-1}} \left\{ \frac{1}{2} (x_k^\top Q_k x_k + u_k^\top R_k u_k) + \mathbb{E}_{w_k, \dots, w_{N-1}} \left[\frac{1}{2} \sum_{j=k+1}^{N-1} [x_j^\top Q_j x_j + u_j^\top R_j u_j] + \frac{1}{2} x_N^\top S_N x_N \right] \right\}. \quad (10.11)$$

Note that the second term in the minimization is exactly the expectation of the cost-to-go at $k+1$, *i.e.*, $\mathbb{E}_{w_k} [J_{k+1}(x_{k+1}; u_{k+1}, \dots, u_{N-1})]$. Moreover, we can turn the minimization into two cascaded minimization problems:

$$J_k^*(x_k) = \min_{u_k} \left\{ \frac{1}{2} (x_k^\top Q_k x_k + u_k^\top R_k u_k) + \min_{u_{k+1}, \dots, u_{N-1}} \mathbb{E}_{w_k} [J_{k+1}(x_{k+1}; u_{k+1}, \dots, u_{N-1})] \right\}. \quad (10.12)$$

Since the expectation is a linear operation, we can switch the order between minimization and expectation, which implies that

$$J_k^*(x_k) = \min_{u_k} \left\{ \frac{1}{2} (x_k^\top Q_k x_k + u_k^\top R_k u_k) + \mathbb{E}_{w_k} \min_{u_{k+1}, \dots, u_{N-1}} [J_{k+1}(x_{k+1}; u_{k+1}, \dots, u_{N-1})] \right\}. \quad (10.13)$$

Hence the Bellman equation in the stochastic case is

$$J_k^*(x_k) = \min_{u_k} \left\{ \frac{1}{2} (x_k^\top Q_k x_k + u_k^\top R_k u_k) + \mathbb{E}_{w_k} J_{k+1}^*(x_{k+1}) \right\}. \quad (10.14)$$

With the Bellman equation, we can solve for the value function by dynamic programming through backward propagation. At time N , we have $J_N^*(x_N) = \frac{1}{2} x_N^\top S_N x_N$. Assume that at time $k+1$, the value function satisfies the following form

$$J_{k+1}^*(x_{k+1}) = \frac{1}{2} (x_{k+1}^\top P_{k+1} x_{k+1} + b_{k+1}), \quad (10.15)$$

where the first term is exactly the same as the value function in the deterministic LQR problems and the second term is new in LQG problems, which is introduced to account for noises.

At time k , applying the Bellman equation, we have

$$J_k^*(x_k) = \frac{1}{2} \min_{u_k} \left\{ x_k^\top Q_k x_k + u_k^\top R_k u_k + \mathbb{E}_{w_k} \left[x_{k+1}^\top P_{k+1} x_{k+1} + b_{k+1} \right] \right\}. \quad (10.16)$$

Let us now compute the expectation inside the minimization. By directly applying the dynamic equation (10.4), we have

$$\mathbb{E}_{w_k} \left[x_{k+1}^\top P_{k+1} x_{k+1} + b_{k+1} \right] \quad (10.17a)$$

$$= \mathbb{E}_{w_k} \left[x_k^\top A_k^\top P_{k+1} A_k x_k + u_k^\top B_k^\top P_{k+1} B_k u_k + 2x_k^\top A_k^\top P_{k+1} B_k u_k + 2(A_k x_k + B_k u_k)^\top P_{k+1} B_k^w w_k + w_k^\top (B_k^w)^\top P_{k+1} B_k^w w_k + b_{k+1} \right] \quad (10.17b)$$

$$= x_k^\top A_k^\top P_{k+1} A_k x_k + u_k^\top B_k^\top P_{k+1} B_k u_k + 2x_k^\top A_k^\top P_{k+1} B_k u_k + b_{k+1} + \mathbb{E}_{w_k} \left[w_k^\top (B_k^w)^\top P_{k+1} B_k^w w_k \right] \quad (10.17c)$$

$$= x_k^\top A_k^\top P_{k+1} A_k x_k + u_k^\top B_k^\top P_{k+1} B_k u_k + 2x_k^\top A_k^\top P_{k+1} B_k u_k + b_{k+1} + \text{Tr} \left[(B_k^w)^\top P_{k+1} B_k^w W_k \right] \quad (10.17d)$$

Hence (10.16) becomes

$$J_k^*(x_k) = \frac{1}{2} \min_{u_k} \left\{ \underbrace{x_k^\top (Q_k + A_k^\top P_{k+1} A_k) x_k + u_k^\top (R_k + B_k^\top P_{k+1} B_k) u_k + 2x_k^\top A_k^\top P_{k+1} B_k u_k}_{J_1} + \underbrace{b_{k+1} + \text{Tr} \left[(B_k^w)^\top P_{k+1} B_k^w W_k \right]}_{J_2} \right\}. \quad (10.18)$$

The term J_1 is exactly the same as what appears in deterministic LQR, while the term J_2 is the additional terms due to noises. Since J_2 does not contain the control u_k , the result of the minimization is still determined by J_1 . Consequently, the control law is exactly the same as what we have in the deterministic LQR:

$$u_k = -(R_k + B_k^\top P_{k+1} B_k)^{-1} P_{k+1} A_k x_k. \quad (10.19)$$

Plugging the optimal control law back to (10.18), we then obtain the expression for the value function at time k :

$$J_k^*(x_k) = \frac{1}{2} \left\{ x_k^\top P_k x_k + b_k \right\}, \quad (10.20)$$

where

$$P_k = Q_k + A_k^\top P_{k+1} A_k - A_k^\top P_{k+1} B_k [R_k + B_k^\top P_{k+1} B_k]^{-1} B_k^\top P_{k+1} A_k \quad (10.21a)$$

$$b_k = b_{k+1} + \text{Tr} \left[(B_k^w)^\top P_{k+1} B_k^w W_k \right] \quad (10.21b)$$

By induction, we can obtain the value function at time $N-1, N-2, \dots, 1$. At time 0, we have

$$J_0^*(x_0) = \mathbb{E}_{x_0} [x_0^\top P_0 x_0 + b_0] \quad (10.22a)$$

$$= \mathbb{E}_{x_0} [(x_0^*)^\top P_0 x_0^* + 2\tilde{x}_0^\top P_0 x_0^* + \tilde{x}_0^\top P_0 \tilde{x}_0 + b_0] \quad (10.22b)$$

$$= (x_0^*)^\top P_0 x_0^* + \text{Tr}(P_0 X_0) + b_0 \quad (10.22c)$$

$$= (x_0^*)^\top P_0 x_0^* + \text{Tr}(P_0 X_0) + \sum_{k=0}^{N-1} \text{Tr} [(B_k^w)^\top P_{k+1} B_k^w W_k] \quad (10.22d)$$

10.2.3 LQG with Inexactly Known States

When we do not have exactly known states, we should not only consider the process noise, but also the measurement noise, since the control input is a function of the measurement. The cost-to-go now becomes

$$J_k(x_k; u_k, \dots, u_{N-1}) = \mathbb{E}_{w_k, \dots, w_{N-1}, v_k, \dots, v_{N-1}} \left[\frac{1}{2} \sum_{j=k}^{N-1} [x_j^\top Q_j x_j + u_j^\top R_j u_j] + \frac{1}{2} x_N^\top S_N x_N \right]. \quad (10.23)$$

The value function can still be defined as in (10.8). However, in either the cost-to-go or the value function, we do not explicitly have the measurement y . How can we incorporate y ? We do so through the conditional expectation as what we did in the Kalman filter. Let us denote Y_k as the collection of all measurements up to time k , i.e., $Y_k = \{y_0, y_1, \dots, y_k\}$. The conditional expectation satisfies that

$$\mathbb{E} [x_k^\top Q_k x_k] = \mathbb{E} \left\{ \mathbb{E} [x_k^\top Q_k x_k \mid Y_k] \right\}. \quad (10.24)$$

where the outer expectation on the right-hand side is taken with respect to Y_k . Let $\hat{x}_{k|k}$ be the optimal least square estimate of x_k given Y_k . Recall that we can compute $\hat{x}_{k|k}$ (a posterior estimate) by Kalman filter. Then we have $x_k = \hat{x}_{k|k} + \tilde{x}_{k|k}$ and (10.24) can be rewritten as

$$\mathbb{E} [x_k^\top Q_k x_k] = \mathbb{E} \left\{ \mathbb{E} [\hat{x}_{k|k}^\top Q_k \hat{x}_{k|k} + 2\tilde{x}_{k|k}^\top Q_k \hat{x}_{k|k} + \tilde{x}_{k|k}^\top Q_k \tilde{x}_{k|k} \mid Y_k] \right\} \quad (10.25a)$$

$$= \mathbb{E} [\hat{x}_{k|k}^\top Q_k \hat{x}_{k|k} + 2\text{Tr}(Q_k \hat{x}_{k|k} \tilde{x}_{k|k}^\top) + \text{Tr}(Q_k \tilde{x}_{k|k} \tilde{x}_{k|k}^\top)] \quad (10.25b)$$

$$= \mathbb{E} [\hat{x}_{k|k}^\top Q_k \hat{x}_{k|k}] + \text{Tr}(Q_k Z_k). \quad (10.25c)$$

Hence we can rewrite the cost-to-go as

$$J_k(x_k; u_k, \dots, u_{N-1}) = \mathbb{E} \left[\frac{1}{2} \sum_{j=k}^{N-1} [\hat{x}_{j|j}^\top Q_j \hat{x}_{j|j} + u_j^\top R_j u_j] + \frac{1}{2} \hat{x}_{N|N}^\top S_N \hat{x}_{N|N} \right] + \sum_{j=k}^{N-1} \text{Tr}(Q_j Z_j) + \text{Tr}(S_N Z_N), \quad (10.26)$$

where the first term is a cost-to-go function with exactly known state as we discussed earlier and the second term is a bias caused by the noise(s) which is independent from the control. As a result, the conclusions we made for the case with exactly known state can still be applied here. But now the dynamics become the dynamics of the state estimate:

$$\hat{x}_{k|k} = A_{k-1}\hat{x}_{k-1|k-1} + B_{k-1}u_{k-1} + F_k e_k \quad (10.27)$$

$$e_k = y_k - C_k(A_{k-1}\hat{x}_{k-1|k-1} + B_{k-1}u_{k-1}), e_k \sim \mathcal{N}(0, V_k + C_k M_k C_k^T). \quad (10.28)$$

The control law remains the same, although it now takes the estimate of the state instead of the true state:

$$u_k = -(R_k + B_k^T P_{k+1} B_k)^{-1} P_{k+1} A_k \hat{x}_{k|k}. \quad (10.29)$$

The Riccati equation remains the same. The value function becomes

$$J_k^*(x_k) = \frac{1}{2} \left\{ \hat{x}_{k|k}^T P_k \hat{x}_{k|k} + b_k \right\}, \quad (10.30)$$

where

$$b_k = b_{k+1} + \text{Tr} \left[(F_{k+1})^T P_{k+1} F_{k+1} [V_{k+1} + C_{k+1} M_{k+1} C_{k+1}^T] \right] + \text{Tr} [Q_k Z_k]. \quad (10.31)$$

The boundary condition is $P_N = S, b_N = \text{Tr} [S_N Z_N]$.

And the optimal cost at time 0 becomes

$$J_0^*(x_0) = \mathbb{E}_{\hat{x}_{0|0}} \left[\hat{x}_{0|0}^T P_0 \hat{x}_{0|0} + b_0 \right] \quad (10.32)$$

$$= (x_0^*)^T P_0 x_0^* + \text{Tr}(P_0 Z_0) + b_0, \quad (10.33)$$

where

$$b_0 = \sum_{k=0}^{N-1} \text{Tr} \left[F_{k+1}^T P_{k+1} F_{k+1} [V_{k+1} + C_{k+1} M_{k+1} C_{k+1}^T] \right] + \sum_{k=0}^{N-1} \text{Tr} [Q_k Z_k] + \text{Tr} [S_N Z_N]. \quad (10.34)$$

In summary, the optimal control gains are the same for LQR and LQG, although LQG (with inexactly known state) takes the estimated state in control while LQR takes the true state in control. Hence the separation principle holds for LQG.

10.2.4 Separation of Dynamics

Now we are going to show that the closed loop dynamics of LQG contain two separated parts: LQ dynamics and KF dynamics. The

closed loop dynamics of LQG can be written as

$$x_{k+1} = A_k x_k - B_k K_k \hat{x}_{k|k} + B_k^w w_k \quad (10.35)$$

$$\hat{x}_{k+1|k+1} = A_k \hat{x}_{k|k} + B_k u_k + F_{k+1} C_{k+1} (A_k x_k - A_k \hat{x}_{k|k} + B_k^w w_k) + F_{k+1} v_{k+1}. \quad (10.36)$$

The augmented system (ignoring the noise terms) is

$$\begin{bmatrix} x_{k+1} \\ \hat{x}_{k+1|k+1} \end{bmatrix} = \begin{bmatrix} A_k & -B_k K_k \\ F_{k+1} C_{k+1} A_k & A_k - B_k K_k - F_{k+1} C_{k+1} A_k \end{bmatrix} \begin{bmatrix} x_k \\ \hat{x}_{k|k} \end{bmatrix}. \quad (10.37)$$

We can apply a similarity transform to the augmented system to obtain

$$\underbrace{\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x_{k+1} \\ \hat{x}_{k+1|k+1} \end{bmatrix}}_{\begin{bmatrix} x_{k+1} \\ -\tilde{x}_{k+1|k+1} \end{bmatrix}} = \begin{bmatrix} A_k - B_k K_k & -B_k K_k \\ 0 & A_k - F_{k+1} C_{k+1} A_k \end{bmatrix} \underbrace{\begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} \begin{bmatrix} x_k \\ \hat{x}_{k|k} \end{bmatrix}}_{\begin{bmatrix} x_k \\ -\tilde{x}_{k|k} \end{bmatrix}}. \quad (10.38)$$

Hence, the eigen values of the closed loop system of LQG consists of two parts

- Eigenvalues from LQ: $A_k - B_k K_k$
- Eigenvalues from KF: $A_k - F_{k+1} C_{k+1} A_k$

10.3 Model Reference Adaptive Control

When separation principle holds, the optimal control law has the same structure as in the deterministic case - just plug in estimated parameters. However, when the augmented system is nonlinear, the separation principle may not hold, and as such the optimal control law may no longer be optimal or even stable. In this case, we can use model reference adaptive control.

Model reference adaptive control creates a reference for both control and estimation. The system parameter is adapted to the error signal measured between the true output and the desired output from the reference system. This approach is different from model adaptation, where we use the error between the true output and the estimated output. The control law is designed to ensure that the closed loop system behaves like the reference system. We will show that the closed-loop system under MRAC is stable and is converging to the reference system. Nonetheless, the learning is still passive.

10.3.1 MRAC for Discrete Time LTI Systems

For simplicity, we now assume that we have the following noise-free dynamics

$$x_{k+1} = Ax_k + Bu_k \quad (10.39)$$

$$y_k = x_k. \quad (10.40)$$

But the parameters A and B are unknown. The goal is to design a control law to track the reference dynamics

$$y_k^* = A^* y_{k-1}^*. \quad (10.41)$$

The initial condition is $y_0^* = x_0 = x_0^*$.

The control law is designed as follows:

- error computation:

$$e_k = y_k - y_k^*. \quad (10.42)$$

- parameter estimation (e.g., RLS):

$$[\hat{A}_k \ \hat{B}_k] = [\hat{A}_{k-1} \ \hat{B}_{k-1}] + e_k [x_{k-1}^T \ u_{k-1}^T] F_k, \quad (10.43)$$

where F_k is the learning gain.

- control gain computation:

$$\hat{A}_k - \hat{B}_k K(\hat{A}_k, \hat{B}_k) = A^*. \quad (10.44)$$

- control synthesis:

$$u_k = -K(\hat{A}_k, \hat{B}_k) y_k. \quad (10.45)$$

The block diagram of MRAC is shown in fig. 10.1.

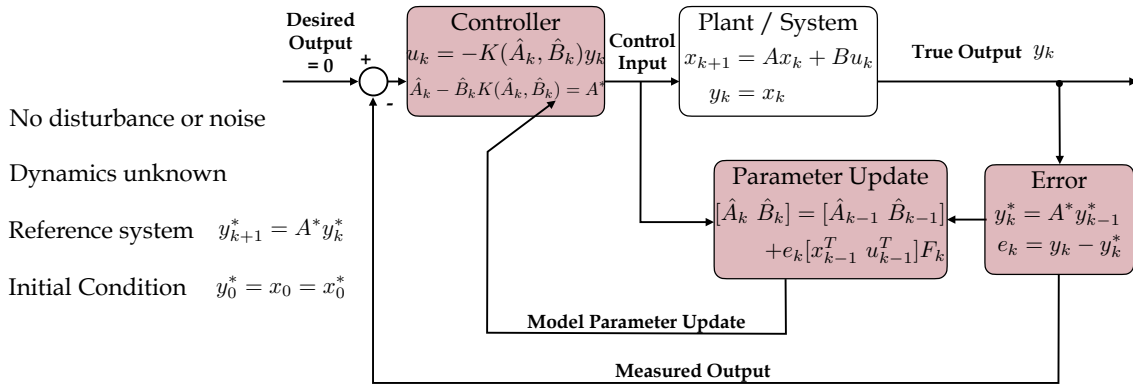


Figure 10.1: The block diagram for model reference adaptive control.

In the following discussion, we will show that if we use RLS for parameter estimation, then the closed-loop system is stable. Before we proceed with stability proof, let us first analyze the error dynamics.

10.3.2 Error Dynamics

There are two kinds of errors in an adaptive system: state error and the parameter error. State error is directly used in parameter adaptation, while parameter error affects the computation of the control signal.

For the control law (10.42) to (10.45), the state error is

$$e_{k+1} = y_{k+1} - y_{k+1}^* \quad (10.46a)$$

$$= [A - BK(\hat{A}_k, \hat{B}_k)]y_k - A^*y_k^* \quad (10.46b)$$

$$= [\hat{A}_k + \tilde{A}_k - (\hat{B}_k + \tilde{B}_k)K(\hat{A}_k, \hat{B}_k)]y_k - A^*y_k^* \quad (10.46c)$$

$$= [\hat{A}_k - \hat{B}_kK(\hat{A}_k, \hat{B}_k) + \tilde{A}_k - \tilde{B}_kK(\hat{A}_k, \hat{B}_k)]y_k - A^*y_k^* \quad (10.46d)$$

$$= A^*y_k + \tilde{A}_kx_k + \tilde{B}_ku_k - A^*y_k^* \quad (10.46e)$$

$$= A^*e_k + \tilde{A}_kx_k + \tilde{B}_ku_k. \quad (10.46f)$$

Note that we used the equality (10.44) to simplify (10.46d). For simplicity, let us write the unknown parameters more compactly and introduce the following notations

$$C_k = [A_k \ B_k], \phi_k = \begin{bmatrix} x_k \\ u_k \end{bmatrix}. \quad (10.47)$$

Hence the state error can be compactly written as

$$e_{k+1} = A^*e_k + \tilde{C}_k\phi_k. \quad (10.48)$$

The parameter error can be computed by subtracting the true parameters on both sides of (10.43):

$$\tilde{C}_k = \tilde{C}_{k-1} - e_k\phi_{k-1}^T F_k. \quad (10.49)$$

Using the state error in (10.48), we have

$$\tilde{C}_{k+1} = \tilde{C}_k - e_{k+1}\phi_k^T F_{k+1} \quad (10.50)$$

$$= \tilde{C}_k[I - \phi_k\phi_k^T F_{k+1}] - A^*e_k\phi_k^T F_{k+1} \quad (10.51)$$

With the error dynamics, now we can prove the stability of MRAC.

10.3.3 Stability of MRAC

For simplicity, assume that $A^* = 0$. Hence the error dynamics can be further simplified as

$$e_{k+1} = \tilde{C}_k\phi_k \quad (10.52a)$$

$$\tilde{C}_{k+1} = \tilde{C}_k[I - \phi_k\phi_k^T F_{k+1}] \quad (10.52b)$$

To prove stability, we want to show that the following value function (Lyapunov function) is non-increasing throughout time

$$V_k = (1 - \phi_{k-1}^T F_k \phi_{k-1})^2 e_k^T e_k + \text{Tr} [\tilde{C}_k F_k^{-1} \tilde{C}_k], \quad (10.53)$$

where F_k is the RLS learning gain where

$$F_{k+1}^{-1} = F_k^{-1} + \phi_k \phi_k^T. \quad (10.54)$$

Note that e_k is the a priori state error and $(1 - \phi_{k-1}^T F_k \phi_{k-1})e_k$ is the a posteriori state error. It can be seen from the following equations

$$e_{k+1} = \tilde{C}_k \phi_k \quad (10.55)$$

$$(1 - \phi_k^T F_{k+1} \phi_k) e_{k+1} = \tilde{C}_k \phi_k (1 - \phi_k^T F_{k+1} \phi_k) = \tilde{C}_k (I - \phi_k \phi_k^T F_{k+1}) \phi_k = \tilde{C}_{k+1} \phi_k. \quad (10.56)$$

At time $k + 1$, we have the following value function

$$\begin{aligned} V_{k+1} &= (1 - \phi_k^T F_{k+1} \phi_k)^2 e_{k+1}^T e_{k+1} + \text{Tr} \left[\tilde{C}_{k+1} F_{k+1}^{-1} \tilde{C}_{k+1}^T \right] \\ &= (\tilde{C}_k \phi_k)^T \tilde{C}_k \phi_k (1 - \phi_k^T F_{k+1} \phi_k)^2 + \text{Tr} \left[\tilde{C}_k [I - \phi_k \phi_k^T F_{k+1}] F_{k+1}^{-1} [I - \phi_k \phi_k^T F_{k+1}]^T \tilde{C}_k^T \right] \\ &= \text{Tr} \left[\tilde{C}_k \phi_k \phi_k^T (1 - \phi_k^T F_{k+1} \phi_k)^2 \tilde{C}_k^T \right] + \text{Tr} \left[\tilde{C}_k [F_{k+1}^{-1} - \phi_k \phi_k^T] F_{k+1} [F_{k+1} - \phi_k \phi_k^T]^T \tilde{C}_k^T \right] \\ &= \text{Tr} \left[\tilde{C}_k [\phi_k \phi_k^T - 2\phi_k \phi_k^T F_{k+1} \phi_k \phi_k^T + \phi_k \phi_k^T (\phi_k^T F_{k+1} \phi_k)^2] \tilde{C}_k^T \right] + \text{Tr} \left[\tilde{C}_k F_k^{-1} F_{k+1} F_k^{-1} \tilde{C}_k^T \right] \\ &\leq \text{Tr} \left[\tilde{C}_k [\phi_k \phi_k^T - \phi_k \phi_k^T F_{k+1} \phi_k \phi_k^T + F_k^{-1} F_{k+1} F_k^{-1}] \tilde{C}_k^T \right] \\ &= \text{Tr} \left[\tilde{C}_k [F_{k+1}^{-1} - F_k^{-1} - (F_{k+1}^{-1} - F_k^{-1}) F_{k+1} (F_{k+1}^{-1} - F_k^{-1}) + F_k^{-1} F_{k+1} F_k^{-1}] \tilde{C}_k^T \right] \\ &= \text{Tr} \left[\tilde{C}_k F_k^{-1} \tilde{C}_k^T \right] \\ &= V_k - (1 - \phi_{k-1}^T F_k \phi_{k-1})^2 e_k^2 \\ &\leq V_k. \end{aligned}$$

The first inequality uses the following relationship $\phi_k^T F_{k+1} \phi_k \leq 1$.

The last inequality verifies that the value function is non decreasing.

Hence we can make the following two conclusions:

- Conclusion 1: The a posteriori error will converge to zero, since

$$V_{k+1} = V_k - (1 - \phi_{k-1}^T F_k \phi_{k-1})^2 e_k^2. \quad (10.58)$$

- Conclusion 2: The a priori error and the parameter estimation error are bounded, but may not converge to zero.

Active Learning and Dual Control

In passive learning, there is no action for information gathering or exploration. It is fine with LQG, *i.e.*, the control law is still optimal, due to the separation principle. But for nonlinear non-Gaussian cases, the separation principle no longer holds. The control law under passive learning will be sub-optimal in these cases. To deal with this problem, we need to introduce active learning and dual control. In the following discussion, we first take a deeper look at the problems with passive learning, then introduce active learning and dual control.

11.1 Problems with Passive Learning

When the separation principle holds, the design of control and estimation methods can be separated. If we have an optimal control law $g_1 : x \rightarrow u$ for the deterministic system and an optimal estimator $g_2 : y \rightarrow \hat{x}$, then the combined control law $g_1 \circ g_2 : y \rightarrow u$ for the stochastic system is still optimal. Under separation principle, the value function in the stochastic case should equal to the value function in the deterministic case plus some constant bias with respect to the noises. The constant bias will not affect the optimal control gain. That is because we use the first order optimality condition of the value function to compute the control gain and the constant bias has zero gradient in the first order optimality condition. Moreover, since the bias with respect to the noises does not change for different states, all future measurements indeed carry the same amount of information. Hence, there is no need for active information gathering.

In LQG, it is shown in (10.18) and (10.26) that the cost-to-go function is indeed the cost-to-go function of LQR plus a constant bias term corresponding to the noises. Since the bias term can be totally separated from the control, essentially all control actions will result in the same amount of information gain. Hence the LQR control law plus KF results in the optimal LQG control law. However, let us consider the case that the bias term is no longer constant, but changes

with respect to the state or control variables. Then the separation principle no longer holds. How we act will then affect the future uncertainties in the cost-to-go or value function. The control law in the deterministic case will no longer be optimal with respect to the cost-to-go in the stochastic case. To better see such effect, let us consider the following example.

Suppose we have a simple lottery system with one state representing the cost, and n control inputs

$$x_{k+1} = [l^1 \ l^2 \ \dots \ l^n] u_k, \quad (11.1)$$

where $u_k \in \{[1 \ 0 \ \dots \ 0]^T, [0 \ 1 \ 0 \ \dots \ 0]^T, \dots, [0 \ \dots \ 0 \ 1]^T\}$. The value of l^i does not change over time, but is unknown before we sample it. The only thing we know is the distribution $l^i \sim \mathcal{N}(\mu^i, \Sigma^i)$. We are going to play this lottery repeatedly, *i.e.*, this is an infinite horizon problem. We want to minimize the cumulative cost $\sum_k x_k$. When the system is deterministic, we just need to choose the smallest l^i for all time steps. And solving the infinite horizon problem is equivalent to solving the following single-step MPC $\min_{u_k} x_{k+1}$ at every time k .

Then what is the best strategy when the system is stochastic? If we follow passive learning, we can decompose the problem into an estimation problem and a control problem. The estimation problem uses previous samples to estimate l^i . The control problem just minimizes the following expected cost

$$\min_{u_k} \mathbb{E}[x_{k+1}], \quad (11.2)$$

where $\mathbb{E}[x_{k+1}]$ is taken with respect to the estimates.

At time 0, since we have no additional information, the expected cost is

$$\mathbb{E}[x_0] \in \{\mu^1, \dots, \mu^n\}. \quad (11.3)$$

Then the best strategy is to choose the control that corresponds to the smallest μ^i , denoted as i^* . Then we get to observe the true cost $l_{i^*}^*$.

At time 1, since we already observed the true cost $l_{i^*}^*$, the expected cost is

$$\mathbb{E}[x_1] \in \{\mu^1, \dots, l_{i^*}^*, \dots, \mu^n\}. \quad (11.4)$$

Then the best strategy is to choose the control that corresponds to the smallest entry in $\{\mu^1, \dots, l_{i^*}^*, \dots, \mu^n\}$.

Consider an extreme case where $l_{i^*}^*$ is the smallest in the list. Then we will keep choosing $l_{i^*}^*$ again and again. However, it is possible that some l is essentially smaller than $l_{i^*}^*$, but is just not observed so far.

As a result, we get stuck in the local optimum with $l_{i^*}^*$.

Note this lottery problem is not a multi-arm bandit problem, since the ground truth dynamic model in this problem is deterministic while the ground truth dynamic model in multi-arm bandit problem is stochastic. But this example can be viewed as a special case of the multi-arm bandit problem. In the previous example, the learning is passive in the sense that the cost function is indifferent whether we get new information or not. However, since the problem is nonlinear, separation principle does not hold. Strategies work in the deterministic case may fail in the stochastic case. We may do better if we introduce active learning.

11.2 Dual Control and Active Learning

11.2.1 Exploitation vs Exploration

The concept of dual control is developed by Alexander Aronovich Fel'dbaum in 1960. It refers to a balance between two objectives

- Exploitation: to control the system as well as possible based on current system knowledge
- Exploration: to experiment with the system so as to learn about its behavior and control it better in the future

In passive learning, only exploitation is considered. In active learning, both exploitation and exploration are considered. In the case of LQG, J_1 in (10.18) corresponds to the exploitation cost while, J_2 in (10.18) corresponds to the exploration cost. Since J_2 is constant, the exploration cost does not affect the choice of the action.

Let us now go back to the lottery problem. The exploration cost will show up if we minimize the expectation of the cumulative cost below instead of the single step cost in (11.2) (although minimizing cumulative cost is equivalent to minimizing single step cost in the deterministic case),

$$\min_{u_k} \mathbb{E} \left[\sum_{t=k+1}^{\infty} x_t \right]. \quad (11.5)$$

However, unlike in LQG, the closed-form solution of the above minimization will be difficult to obtain. In the following discussion, we discuss a heuristic method to encourage exploration by adding a cost with respect to information gain to the single step optimization (11.2). The goal is for the new cost function to mimic (11.5).

11.2.2 Information Gain for Exploration

To make the estimation more challenging, we now assume that we only have a noisy measurement of l^i through x_{k+1} . The dynamic

equation becomes

$$x_{k+1} = [l^1 \ l^2 \ \dots \ l^n] u_k + w_k, \quad (11.6)$$

where u_k is the same as above, $l^i \sim \mathcal{N}(\mu^i, \Sigma^i)$ and $w_k \sim \mathcal{N}(0, W)$. The estimate of l^i given $u_0, x_1, u_1, x_2, \dots, u_{k-1}, x_k$ is denoted $\hat{l}_k^i$ with $\hat{l}_k^i \sim \mathcal{N}(\hat{\mu}_k^i, \hat{\Sigma}_k^i)$.

We may quantify information gain in different ways. Here we introduce two approaches: reduction of variance and entropy.

Reduction of variance At time k , if we choose the i -th control, then we get a new observation on l^i through $x_{k+1} = l^i + w_k$. According to the conditional variance formula

$$\hat{\Sigma}_{k+1}^i = \hat{\Sigma}_k^i - \text{COV}(l^i, x_{k+1}) \text{VAR}(x_{k+1})^{-1} \text{COV}(x_{k+1}, l^i), \quad (11.7)$$

we know that by choosing the i -th control, we will be able to reduce the variance of the estimate of l^i by

$$\hat{\Sigma}_{k+1}^i - \hat{\Sigma}_k^i = -\frac{(\hat{\Sigma}_k^i)^2}{\hat{\Sigma}_k^i + W}. \quad (11.8)$$

When W is very small, we can approximate the information gain as

$$\hat{\Sigma}_{k+1}^i - \hat{\Sigma}_k^i \approx -\hat{\Sigma}_k^i. \quad (11.9)$$

Entropy Entropy describes the average level of “information” or “uncertainty” inherent in the possible outcomes of a random variable. For random variables that take discrete values (with probability $p_1, p_2, \dots$), the entropy is defined as $H(X) = -\sum_i p_i \ln p_i$. For random variables that take continuous values (with density function $p(x)$), the (differential) entropy is defined as

$$h(X) = -\int p(x) \ln p(x) dx. \quad (11.10)$$

The entropy for a Gaussian distribution with variance Σ is¹

$$h(X) = \ln(\sqrt{2\pi\Sigma e}). \quad (11.11)$$

¹ Interesting fact: for all random variables with the same variance, the variables with Gaussian distribution have the largest (differential) entropy.

The change of the variance and the entropy for a Gaussian variable after multiple measurements are illustrated in fig. 11.1 and fig. 11.2. The variance changes more rapidly than the entropy. Both of them demonstrates diminishing returns in the sense the more measurements we get (*i.e.*, increasing k), the less the information gain is.

Let us now use reduction of variance to quantify the information gain. The new cost function at time k is defined as

$$\min_{u_k} \mathbb{E}[x_{k+1} \mid \hat{l}_k^1, \dots, \hat{l}_k^n] - \alpha \left[\hat{\Sigma}_k^1 \hat{\Sigma}_k^2 \dots \hat{\Sigma}_k^n \right] u_k, \quad (11.12)$$

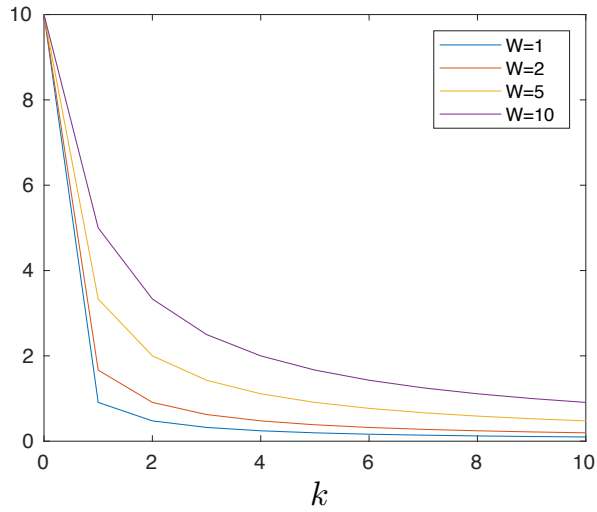


Figure 11.1: Illustration of the variance of a Gaussian random variable after k measurements. The initial variance $\text{Sigma} = 10$. Measurement noises W takes different values.

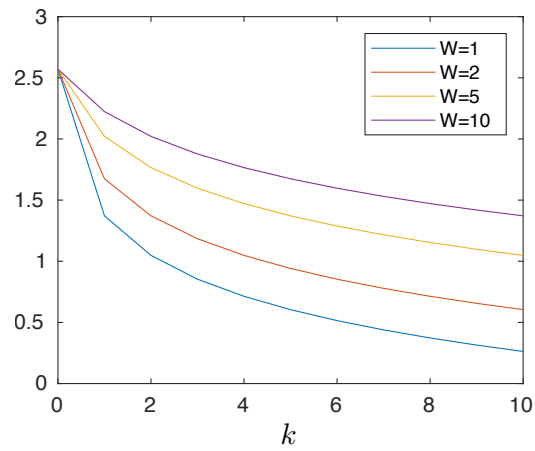


Figure 11.2: Illustration of the entropy of a Gaussian random variable after k measurements. The initial variance $\text{Sigma} = 10$. Measurement noises W takes different values.

where the first term is the exploitation cost, *i.e.*, expected cost under the current uncertainty model. The second term is the exploration cost, *i.e.*, how much variance we can reduce in the next uncertainty model. The constant α is a weighting factor to balance exploration and exploitation. Rearranging the optimization, we get

$$\min_{u_k} \left[\hat{\mu}_k^1 - \alpha \hat{\Sigma}_k^1 \quad \hat{\mu}_k^2 - \alpha \hat{\Sigma}_k^2 \quad \cdots \quad \hat{\mu}_k^n - \alpha \hat{\Sigma}_k^n \right] u_k. \quad (11.13)$$

When $\alpha = 0$, the formulation reduces to passive learning. When $\alpha > 0$, we have active learning: when choosing the next control input, we not only consider the expectation but also favor choices that have bigger variances – since it will result in more information gain. When $\alpha < 0$, the control decision will favor smaller variances in the sense that it will afford an increase in the mean cost for a smaller variance.

The choice of α depends on the agent's risk preferences. The behavior of exploration is risk seeking: higher risk may lead to higher return or lower cost. When $\alpha = 0$, the agent is risk neutral. When $\alpha > 0$, the agent is risk seeking. When $\alpha < 0$, the agent is risk averse.

11.2.3 Gaussian Process

We can extend the lottery problem to a continuous control problem

$$x_{k+1} = f(u_k), \quad (11.14)$$

where the function f is Gaussian at every point

$$x_{k+1} \sim \mathcal{N}(\mu(u_k), \Sigma(u_k)). \quad (11.15)$$

Any finite number of these random variables have a joint Gaussian distribution. Since we have a continuous problem, we can assume that the measurement of $f(u)$ can tell us something about $f(u')$. Let's define a covariance function (kernel) $\kappa(u, u')$, which encodes the covariance of any two function values $f(u)$ and $f(u')$. By definition, $\Sigma(u_k) = \kappa(u_k, u_k)$. In the discrete example above, we have $\kappa(u, u') = \mathbf{1}(u = u')$. Such a model f with the mean function μ and kernel κ is called a *Gaussian process*.

At every step n , we need to solve the following problem: Given n past observations $\{\hat{f}(u_i)\}_{i=1}^n$ where $\hat{f}(u) = f(u) + w$, $w \sim \mathcal{N}(0, \sigma^2)$. What is the best choice of u^* to minimize $f(u)$? For simplicity, let us now assume the mean function $\mu(u) \equiv 0$.

To simplify the notation, let us define the list of the past ground truths as

$$\mathbf{f}_n = [f(u_1), \dots, f(u_n)]^\top. \quad (11.16)$$

Define the list of the past measurements as

$$\hat{\mathbf{f}}_n = [\hat{f}(u_1), \dots, \hat{f}(u_n)]^\top. \quad (11.17)$$

Define the kernel matrix of past inputs as

$$[\mathbf{K}_n]_{i,j} = k(u_i, u_j). \quad (11.18)$$

Then the mean and variance of the observations are

$$\mathbb{E} [\hat{\mathbf{f}}_n] = \mathbf{0} \quad (11.19)$$

$$\mathbb{V}\text{AR} [\hat{\mathbf{f}}_n] = \mathbf{K}_n + \mathbf{I}_n \sigma^2 \quad (11.20)$$

Then the covariance between $f(u^*)$ and $\hat{\mathbf{f}}_n$ satisfies

$$\text{COV}[f(u^*), \hat{\mathbf{f}}_n] = \text{COV}[f(u^*), \mathbf{f}_n] = [k(u^*, u_1), \dots, k(u^*, u_n)] =: \mathbf{k}_n(u^*). \quad (11.21)$$

The mean and variance of an arbitrary $f(u^*)$ conditioned on the observations satisfy:

$$\begin{aligned} \mathbb{E} [f(u^*) | \hat{\mathbf{f}}_n] &= \mathbb{E} [f(u^*)] + \text{COV}[f(u^*), \hat{\mathbf{f}}_n] \mathbb{V}\text{AR} [\hat{\mathbf{f}}_n]^{-1} [\hat{\mathbf{f}}_n - \mathbb{E}[\hat{\mathbf{f}}_n]] \\ &= \mathbf{k}_n(u^*) [\mathbf{K}_n + \mathbf{I}_n \sigma^2]^{-1} \hat{\mathbf{f}}_n \end{aligned} \quad (11.22)$$

$$\begin{aligned} \mathbb{V}\text{AR} [f(u^*) | \hat{\mathbf{f}}_n] &= \mathbb{V}\text{AR} [f(u^*)] - \text{COV}[f(u^*), \hat{\mathbf{f}}_n] \mathbb{V}\text{AR} [\hat{\mathbf{f}}_n]^{-1} \text{COV}[\hat{\mathbf{f}}_n, f(u^*)] \\ &= k(u^*, u^*) - \mathbf{k}_n(u^*) [\mathbf{K}_n + \mathbf{I}_n \sigma^2]^{-1} \mathbf{k}_n(u^*)^\top \end{aligned} \quad (11.23)$$

If we are going to minimize $f(u)$, then the next sample location can be chosen as

$$u_{n+1} = \arg \min_{u \in \mathcal{U}} \mathbb{E} [f(u) | \hat{\mathbf{f}}_n] - \beta_n^{1/2} \mathbb{V}\text{AR} [f(u) | \hat{\mathbf{f}}_n] \quad (11.24)$$

where β_n reflects the confidence interval of the GP. This process is called *Bayesian Optimization*.

11.3 Randomization in Control

In the exploration with GPs, we need to maintain a probability model of the unexplored area, and use the model to optimize the information gain. However, in many cases, it is difficult to obtain or maintain such a model to quantify the information gain. Then how can we avoid the local optima in passive learning? We can do randomly exploration.

Until now, we have only introduced deterministic control policy in the form of $u = g(x)$. Actually, we can make the control policy

stochastic denoted *i.e.*, $\pi(u \mid x)$. The function π is a probability density function with respect to the random control u conditioned a specific state x such that $\int_u \pi(u \mid x) = 1$. Then every time when we evaluate the stochastic control policy, we need to sample from the function $\pi(\cdot \mid x)$.

Epsilon greedy is a stochastic policy that has been used a lot in reinforcement learning to encourage exploration. Under epsilon greedy, we will select a random action under uniform distribution from a set of available actions under probability ϵ ; and will choose the action that minimizes the expected cost under probability $1 - \epsilon$.

Part V

Value Approximation and Policy Learning

Temporal Difference Learning and Value Approximation

Instead of the learning approaches that we have already covered, where we learn the dynamic parameters and then apply optimal control, you may be curious if we can directly learn the value function or the control policy to control our system of interest. Over the next couple chapters, we will discuss how to accomplish such a task. We first consider approaches that approximately learn the value function, then approaches to approximately the policy, both of which are called **approximate dynamic programming**. And more generally, they belong to the direct adaptation approaches. The key difference between direct adaptation using approximate dynamic programming and the indirect adaptation approaches that we discussed earlier is that

1. We no longer estimate the system dynamics.
2. We no longer explicitly solve the optimal control problem.

In the following discussion, we first introduce the notations in the stochastic optimal control problem. Then we discuss the dynamic programming approaches for the stochastic optimal control problem, which is still model-based and exact. Then we approximate the steps in the exact dynamic programming (namely policy evaluation and policy improvement) using samples and derive the associated approximate dynamic dynamics methods. We will introduce both Monte Carlo methods and temporal difference learning methods. To deal with continuous state space, we introduce methods to learn parameterized value functions and show how to perform approximated policy evaluation and policy improvement using parameterized value functions.

12.1 Problem Formulation

We first consider a series of problems that are elements of the class of stochastic discrete-time dynamics. This is not to say that the methods

discussed in this chapter are solely applicable to such problems, but we start here as a means for easy derivation. Consider a stochastic dynamic system with state-update equation $x_{k+1} = f(x_k, u_k, w_k)$ and output equation $y_k = x_k$, with a cost structure including run-time cost $l(x_k, u_k, x_{k+1})$ and discount factor δ , controlled by a stochastic policy $\pi(u|x)$. Note that these stochastic control tasks are episodic in nature, this is that they can be repeated again and again after the system reaches some terminal event. We reformulate this stochastic dynamic system into a set of generalized dynamics

$$p(x', l|x, u), \quad (12.1)$$

where $p(\cdot)$ is the probability given a specific state x and control value u that we will achieve state x' with reward l . If we integrate over all possible next states and rewards, from the rules of probability we recover a probability of 1. In other words,

$$\int_{x'} \int_l p(x', l|x, u) = 1.$$

As this is a discrete time system, the system trajectory under a specific policy π , is:

$$X_0, U_0, L_0, X_1, U_1, L_1, X_2, \dots$$

where $U_k \sim \pi(\cdot|X_k)$ and $X_{k+1}, L_k \sim p(\cdot, \cdot|X_k, U_k)$ are random variables. Note that the process given above is stationary as you can shift the time index without changing the statistical features of the sequence.

It is worth noting that we use the capital letter notation of states, controls, and costs to denote random samples drawn from stochastic dynamics and stochastic policy. With these samples, we define the return of the system, the cost-to-go associated with the random variables, given discount δ as

$$G_k := L_k + \delta L_{k+1} + \delta^2 L_{k+2} + \dots$$

We can also reformulate this definition to return a more useful recursive form as

$$G_k = L_k + \delta G_{k+1}.$$

Under different random seeds, this random cost-to-go may take different values. We are mostly interested in the expected return or the expected cost-to-go, defined as

$$V_\pi(x) := \mathbb{E}_\pi[G_k|X_k = x], \quad (12.2)$$

which is a function on the current state value x and is implicitly dependent on the dynamics and the policy π . The expectation is essentially taken over the dynamics and the policy π , although we

usually only explicitly write out the policy π in the subscript of the expectation. In addition, we define a **Q Function** which is dependent on both the state and the control

$$Q_\pi(x, u) = \mathbb{E}_\pi[G_k | X_k = x, U_k = u]. \quad (12.3)$$

The only difference between the Q function and the expected cost-to-go is that U_k in the Q function is taken as u , while U_k in the expected cost-to-go is not fixed but sampled from the policy $\pi(\cdot | X_k)$.

Furthermore, we can expand the expected cost-to-go:

$$V_\pi = \mathbb{E}_\pi[G_k | X_k = x] \quad (12.4a)$$

$$= \mathbb{E}_\pi[L_k + \delta G_{k+1} | X_k = x] \quad (12.4b)$$

$$= \int_u \pi(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta \mathbb{E}_\pi[G_{k+1} | X_{k+1} = x']] \quad (12.4c)$$

$$= \int_u \pi(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V_\pi(x')] \quad (12.4d)$$

where a difference arises between this expected cost-to-go and the Bellman Equation due to the fact that we are not computing/using an optimal control policy, but rather that denoted by π . We can define an optimal value function as

$$V_*(x) := \min_\pi V_\pi(x), \quad (12.5)$$

which is also referenced as the optimal state-value function. We can define an equivalent optimal Q-function as

$$Q_*(x, u) = \min_\pi Q_\pi(x, u), \quad (12.6)$$

where in both cases the optimal policy, π^* is the minimizer of the minimization problems posed in the optimal value function or optimal Q function. Just as we rephrased the expected cost to go, we can rephrase to optimal value function (or Q function) to recover the Bellman Equation.

$$V_*(x) = \min_u Q_*(x, u) \quad (12.7a)$$

$$= \min_u \mathbb{E}_{\pi^*}[G_k | X_k = x, U_k = u] \quad (12.7b)$$

$$= \min_u \mathbb{E}_{\pi^*}[L_k + \delta G_{k+1} | X_k = x, U_k = u] \quad (12.7c)$$

$$= \min_u \mathbb{E}_{\pi^*}[L_k + \delta V_*(X_{k+1}) | X_k = x, U_k = u] \quad (12.7d)$$

$$= \min_u \int_{x'} \int_l p(x', l|x, u) [l + \delta V_*(x')] \quad (12.7e)$$

Similarly, we can derive a recursive expression for the optimal Q function:

$$Q_*(x, u) = \mathbb{E}[L_k + \delta \min_{u'} Q_*(X_{k+1}, u') | X_k = x, U_k = u] \quad (12.8a)$$

$$= \int_{x'} \int_l p(x', l|x, u) [l + \delta \min_{u'} Q_*(x', u')] \quad (12.8b)$$

The major difference between (12.7) and (12.8) is where the minimization takes place. For (12.7), the minimization takes place after the integration over the generalized dynamics. Hence we need to have a model of the dynamics in order to obtain the optimal control input. On the other hand, for (12.8), the minimization is directly taken at Q before the integration over the generalized dynamics. Hence it is much easier to extract the optimal policy as we do not need to know (and often do not know) the dynamics, which is very advantageous as we will see in the following discussion.

12.2 Dynamic Programming for Stochastic Systems

We have discussed dynamic programming for deterministic systems, but how do we do it for stochastic systems? We have two major approaches: value iteration and policy iteration.

Value Iteration Let us discuss the Bellman Operator for the Stochastic Case. Recalling the definition of the optimal value function in (12.7). We can define the Bellman operator $\mathcal{T}$ as

$$\mathcal{T}(V) := \min_u \int_{x'} \int_l p(x', u | x, u) [l + \delta V(x')]. \quad (12.9)$$

The optimal value function is a fixed point of the Bellman operator. And it should be noted that the deterministic Bellman operator is a special case of the stochastic Bellman operator, in which the probability distribution of x', l is all concentrated to a point mass. We will use this Bellman operator to perform the Value Iteration, which has the following steps:

1. Starting from an initial estimate of the optimal value function V_0 ;
2. Iterative apply the Bellman operator, $V_{i+1} = \mathcal{T}(V_i)$;
3. Return V_{i+1} if it converges; otherwise go back to step 2.

We will have assured convergence if the Bellman operator is contracting. In practice, under most circumstances, the Bellman operator is contracting. However the major issue with the Bellman operator is that the policy may converge way faster than the value function. In other words, the minimization step performed in the Bellman operator may not change very frequently.

Policy Iteration Let us now break the Bellman operator into two operators. The first is the operator for policy improvement, $\mathcal{Z} : V \rightarrow \pi$, defined as

$$\mathcal{Z}(V) := \arg \min_u \int_{x'} \int_l p(x', l | x, u) [l + \delta V(x')], \quad (12.10)$$

and the second is the operator for policy evaluation, $\mathcal{T}_{(\cdot)} : \pi \times V \rightarrow V$, defined as

$$\mathcal{T}_{\pi}(V) := \int_u \pi(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V(x')]. \quad (12.11)$$

The Bellman operator can be written as $\mathcal{T}(V) = \mathcal{T}_{\mathcal{Z}(V)}(V)$. Hence in the value iteration, the two operators are used sequentially as shown in the left subfigure in fig. 12.1. Since the outcome of the $\mathcal{Z}$ operator does not change frequently, we can simply lower the frequency that we apply the $\mathcal{Z}$ operator.



Figure 12.1: The Bellman Operators update step. Generalized Policy Iteration is shown in the right hand figure

Thus for policy iteration, we follow a fairly straightforward process. We first calculate the expected cost-to-go for some fixed policy using the $\mathcal{T}_{\pi}$ operator. After calculating the expected cost-to-go, then we update the policy using one-step look-ahead with the $\mathcal{Z}$ operator. We continue doing these two steps until the policy converges.

To compute the expected cost-to-go $V_{\pi}(x)$ for a given policy π , we can use a (model-based) fixed-point approach where we iteratively update $V_{i+1} \leftarrow \mathcal{T}_{\pi}(V_i)$ where

$$V_{i+1} \leftarrow \int_u \pi(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V_i(x')]. \quad (12.12)$$

Then the policy evaluation is followed by the policy improvement step via one-step look-ahead where

$$\pi = \mathcal{Z}(V) = \arg \min_u \int_{x'} \int_l p(x', l|x, u) [l + \delta V(x')]. \quad (12.13)$$

Policy Improvement We can show that the new policy generated in the policy improvement phase is guaranteed to have a smaller value. Let us now assume that the improved policy π_1 is the policy π_2 :

$$\pi_2 = \mathcal{Z}(V_{\pi_1}) = \arg \min_u \int_{x'} \int_l p(x', l|x, u) [l + \delta V_{\pi_1}(x')]. \quad (12.14)$$

Moreover, we have $V_{\pi_i} = \int_u \pi_i(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V_{\pi_i}(x')]$ for $i = 1, 2$. In the following discussion, we will show that $V_{\pi_2} \leq V_{\pi_1}$. By

definition, we have

$$V_{\pi_1}(x) = \int_u \pi_1(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V_{\pi_1}(x')] \quad (12.15a)$$

$$\geq \int_u \pi_2(u|x) \int_{x'} \int_l p(x', l|x, u) [l + \delta V_{\pi_1}(x')] \quad (12.15b)$$

$$= V_{\pi_2}(x) + \delta \int_u \pi_2(u|x) \int_{x'} p(x', l|x, u) [V_{\pi_1}(x') - V_{\pi_2}(x')] \quad (12.15c)$$

$$= V_{\pi_2}(x) + \delta \int_{x'} \mathcal{P}(x \rightarrow x', 1, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] \quad (12.15d)$$

where the inequality in the second line follows from (12.14); the equality in the third line is obtained by substituting $V_{\pi_1}(x')$ with $V_{\pi_1}(x') + V_{\pi_2}(x') - V_{\pi_2}(x')$ and then rearranging the terms; in the last equality, we have $\mathcal{P}(x \rightarrow x', 1, \pi_2) = \int_u \pi_2(u|x) p(x', l|x, u)$. Then we can rearrange the above inequality into the following form:

$$V_{\pi_1}(x) - V_{\pi_2}(x) \geq \delta \int_{x'} \mathcal{P}(x \rightarrow x', 1, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')].$$

If $\delta < 1$, by induction, we have

$$\begin{aligned} V_{\pi_1}(x) - V_{\pi_2}(x) &\geq \delta^2 \int_{x'} \mathcal{P}(x \rightarrow x', 2, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] \\ &\geq \lim_{n \rightarrow \infty} \delta^n \int_{x'} \mathcal{P}(x \rightarrow x', n, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] = 0. \end{aligned}$$

If $\delta = 1$, and under the assumption that $V_{\pi_1}(x^*) = V_{\pi_2}(x^*)$ for all terminal states $x^* \in \mathcal{X}^*$, we have

$$\begin{aligned} V_{\pi_1}(x) - V_{\pi_2}(x) &\geq \int_{x' \in \mathcal{X}^*} \mathcal{P}(x \rightarrow x', 1, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] + \int_{x'} \mathcal{P}(x \rightarrow x', 2, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] \\ &\geq \sum_{n=1}^{\infty} \int_{x' \in \mathcal{X}^*} \mathcal{P}(x \rightarrow x', n, \pi_2) [V_{\pi_1}(x') - V_{\pi_2}(x')] = 0. \end{aligned}$$

Hence in both cases, we have $V_{\pi_1}(x) - V_{\pi_2}(x) \geq 0$, which completes our proof.

Generalized Policy Iteration In addition to policy iteration which requires the value function to converge at the policy evaluation step, we can indeed apply the two operators Z and $\mathcal{T}_\pi$ iteratively at any frequency. This is shown in 12.1 on the right-hand side. The advantage of this approach is that we do not need to visit all states or wait for the convergence of the policy evaluation.

12.3 Temporal Difference Learning

In dynamic programming, we used a model-based approach for both policy evaluation and policy improvement, but what if we do not have access to the model? We will need to resort to model-free approaches to perform policy evaluation and policy improvement.

Model-Free Policy Evaluation One model-free policy evaluation approach is by leveraging the Monte Carlo (MC) method, where we generate many system trajectories using policy π and compute the value function by averaging all the returns:

$$V_\pi(x) = \text{Average}(G_k) \text{ for } k \text{ s. t. } X_k = x.$$

This method is also called *every-visit MC method*. The problem with this approach is that the value function can only be computed offline after all the trajectories are obtained. To address this problem, we can perform an exponential moving average over the value function so that the value function can be updated whenever a return G_k is obtained:

$$\hat{V}_\pi(X_k) \leftarrow \hat{V}_\pi(X_k) + \alpha[G_k - \hat{V}_\pi(X_k)]$$

Nevertheless, in this approach, we still need to wait until a trajectory is finished to compute the return. Notice that we have shown that $G_k = L_k + \delta G_{k+1}$ and $V_\pi(X_{k+1}) = \mathbb{E}_\pi[G_{k+1} \mid X_{k+1}]$, hence we can use the current estimate of the value function to replace G_{k+1} in the computation. As a result, we no longer need to wait until we have the full trajectory. The update rule becomes:

$$\hat{V}_\pi(X_k) \leftarrow \hat{V}_\pi(X_k) + \alpha[L_k + \delta \hat{V}_\pi(X_{k+1}) - \hat{V}_\pi(X_k)], \quad (12.16)$$

where the term $L_k + \delta \hat{V}_\pi(X_{k+1}) - \hat{V}_\pi(X_k)$ is called the Temporal Difference (ED) error. And this approach is known as one-step temporal difference learning, TD(o). Since TD(o) bases its update in part on an existing estimate of the value function, it is a bootstrapping method. Similarly, the generalized policy iteration in dynamic programming is also a bootstrapping method. Furthermore, let us compare the form of the TD error and the a priori estimation error in the online parameter adaptation. For TD error, we have

$$\Delta_k = L_k + \delta \hat{V}_\pi(X_{k+1}) - \hat{V}_\pi(X_k) \quad (12.17)$$

where a priori estimation error in online parameter estimation is

$$e_k = x_k - \hat{x}_k \text{ where } \hat{x}_k = f_{\theta_{k-1}}(x_{k-1}) \quad (12.18)$$

The ground truth x_k is equivalent to the ground truth L_k , while the estimate $\hat{x}_k$ is equivalent to the estimate $\delta \hat{V}_\pi(X_{k+1}) - \hat{V}_\pi(X_k)$. Hence the TD error can also be viewed as a form of a priori estimation error w.r.t. the run-time cost.

Generally speaking, the Monte Carlo method generates a minimum variance estimate and TD(o) generates a maximum likelihood estimate. We can also do n-Step Bootstrapping where the new up-

dated value function is given as

$$\hat{V}_\pi(X_k) \leftarrow \hat{V}_\pi(X_k) + \alpha \left[\sum_{t=k}^{k+n-1} \delta^{t-k} L_t + \delta^n \hat{V}_\pi(X_{k+1}) - \hat{V}_\pi(X_k) \right]. \quad (12.19)$$

The formulation of n-step look-ahead is similarly to n-step MPC.

With n-step look-ahead, we can decouple the action frequency from the adaptation frequency.

Model-Free Policy Improvement If we only learn the cost-to-go V_π , then we still need to have a model of the system to find the optimal action and to improve the policy:

$$\pi = \mathcal{Z}(V) = \arg \min_u \int_{x'} \int_l p(x', l | x, u) [l + \delta V(x')] \quad (12.20)$$

However, there is a problem with the above approach. In the model-free setting, we do not have access to the generalized dynamics.

The way we get around this problem is by directly learning the Q function. Rephrasing TD(o) using the Q function, we have

$$Q(X_k, U_k) \leftarrow Q(X_k, U_k) + \alpha [L_k + \delta Q(X_{k+1}, U_{k+1}) - Q(X_k, U_k)], \quad (12.21)$$

where $U_k \sim \pi(\cdot, X_k)$.

To choose the next action, we need to balance exploration and exploitation in the sense that we should minimize Q while exploring the policy space. To achieve that, we can use the ϵ -greedy method where with $1 - \epsilon$ probability, we choose the action that minimizes the current Q value; and with ϵ probability, we choose a random action. With that, we know how to choose U_k in (12.21). But there is another action term U_{k+1} on the right-hand side of the equation. Then the question becomes: how shall we choose that term? And shall we actually apply the chosen U_{k+1} at the next time step even if it is not minimizing our updated Q function? Indeed, we have two choices for U_{k+1} . Option 1 is to use the same ϵ -greedy method to sample U_{k+1} and use the sampled U_{k+1} both for the update of Q in (12.21) and the actual action at the next time step. This method corresponds to the SARSA algorithm. Option 2 is to choose the optimal U_{k+1} that minimizes the current Q in the update (12.21), but use a different U_{k+1} for action in the next time step. This method corresponds to the Q learning algorithm.

Since the U_{k+1} for computation and the U_{k+1} for action are aligned in SARSA, we call it as an on-policy learning approach. On the other hand, Q learning is an off-policy approach since the U_{k+1} for computation and the U_{k+1} for action are different.

SARSA The main procedure of SARSA is summarized below.

- Algorithm parameters: α, ϵ
- Initialize $Q(x, u)$
- Loop for each episode
 - Initialize X
 - Choose action U from X using policy derived from Q (e.g. ϵ -greedy)
 - Loop for each time step
 - * Take action U and observe L, X'
 - * Choose action U' from X' using policy derived from Q (e.g. ϵ -greedy)
 - * $Q(X, U) \leftarrow Q(X, U) + \alpha[L + \delta Q(X', U') - Q(X, U)]$
 - * $X \leftarrow X', U \leftarrow U'$
 - Until we reach the terminal state

Q Learning The main procedure of Q Learning is summarized below:

- Algorithm parameters: α, ϵ
- Initialize $Q(x, u)$
- Loop for each episode
 - Initialize X
 - Loop for each time step:
 - * Choose action U from X using policy derived from Q (e.g. ϵ -greedy)
 - * Take action U and observe L, X'
 - * $Q(X, U) \leftarrow Q(X, U) + \alpha[L + \delta \max_u \{Q(X', u) - Q(X, U)\}]$
 - * $X \leftarrow X'$
 - Until we reach the terminal state

12.4 Value Approximation

All the methods that we have discussed so far only update one point of the estimated value function $V(x)$ at a time. Point-wise updates work well for tabular solutions of finite state problems, but do not generalize to continuous state problems since the probability of visiting any particular state is 0. We need to parameterize the value function for efficient learning in a continuous state space.

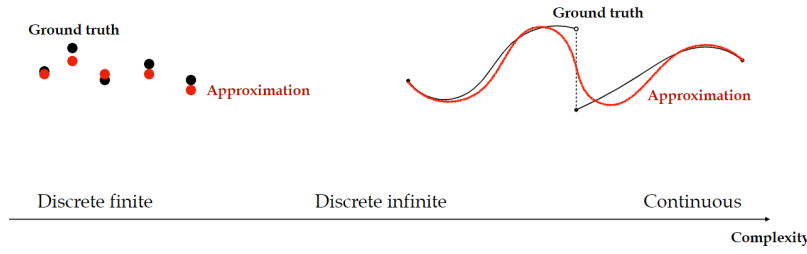


Figure 12.2: A figure comparing the complexity between determining what a “good” approximation of a function is in discrete spaces and in continuous spaces.

Quantification of Value Approximation We have a larger question of how to quantify if a function approximation is “good” or not. Consider the following cases. For discrete functions, point-wise matching is an easy approach so long as we can collect data for every point, but it is extremely difficult to extend that argument to a continuous state system as one cannot collect data for every point in the domain. To quantify the quality of approximation in continuous state systems, we need a notion of “almost everywhere”, meaning that certain property holds at almost all points except for some points that have measure zero in the space. With this notion, we can set the objective for the parameterization of the value function $\hat{V}_w(x)$ to be minimizing the mean-squared value error:

$$J(w) = \int_x \mu(x) [V_\pi(x) - \hat{V}_w(x)]^2, \quad (12.22)$$

where $\mu(x)$ denotes the state distribution under the policy. The value approximation should be more accurate for states that are visited more often. $J(w)$ achieve zero if $\hat{V}_w(x)$ equals to $V_\pi(x)$ almost everywhere under the measure $\mu(x)$.

With this objective function, we now need to answer the following question: how can we learn a parameterized function to minimize $J(w)$, especially when we do not explicitly know $\mu(x)$ and $V_\pi(x)$? To answer this question, we will investigate stochastic gradient descent algorithms and least squares methods again.

Stochastic Gradient Descent Since $\mu(x)$ is the probability density for state distribution under policy π , we can rewrite (12.22) as

$$J(w) = \mathbb{E}_\pi [V_\pi(x) - \hat{V}_w(x)]^2. \quad (12.23)$$

We can perform stochastic gradient descent on this expected error to try to recover the optimal parameters that minimize the mean-square value error. The stochastic gradient descent rule on the parameters is

$$w_{k+1} = w_k + \frac{\alpha}{2} \nabla_w [V_\pi(X_k) - \hat{V}_{w_k}(X_k)]^2 \quad (12.24)$$

which can be expanded to

$$w_{k+1} = w_k + \alpha[V_\pi(X_k) - \hat{V}_{w_k}(X_k)]\nabla_w \hat{V}_{w_k}(X_k). \quad (12.25)$$

However, in (12.25), we actually do not know $V_\pi(X_k)$. The most common ways to compute $V_\pi(X_k)$ is either through the Monte Carlo method using the return G_k or via bootstrapping using the current run-time cost and the approximated function $L_k + \delta \hat{V}_{w_k}(X_k)$. Bootstrapping with stochastic gradient descent is also called a *semi-gradient* method since the gradient is now being approximated. In the following discussion, we provide the pseudo code for the gradient Monte Carlo method which uses G_k for $V_\pi(X_k)$ and the semi-gradient TD(0) which uses bootstrapping. These two algorithms are used to evaluate a policy π .

Gradient Monte Carlo Algorithm

- Input: the policy π to be evaluated; a differentiable function $\hat{V}_w$
- Algorithm parameters: learning rate $\alpha > 0$
- Initialize weights w in $\hat{V}_w$
- Loop for each episode:
 - Generate an episode $X_0, U_0, L_0, X_1, U_1, L_1, X_2, \dots, L_{T-1}, X_T$ using π
 - Loop for each step $k = 0, 1, \dots, T-1$
 - * Compute G_k
 - * Update, $w \leftarrow w + \alpha[G_k - \hat{V}_w(X_k)]\nabla_w \hat{V}_w(X_k)$

Semi-Gradient TD(0)

- Input: the policy π to be evaluated; a differentiable function $\hat{V}_w$
- Algorithm parameters: learning rate $\alpha > 0$
- Initialize weights w in $\hat{V}_w$
- Loop for each episode:
 - Initialize X
 - Loop for each step
 - * Choose $U \sim \pi(\cdot|X)$
 - * Take action U , observe L, X'
 - * Update, $w \leftarrow w + \alpha[L + \delta \hat{V}_w(X') - \hat{V}_w(X)]\nabla_w \hat{V}_w(X)$
 - * $X \leftarrow X'$
 - Until X is terminal

How to do Control? The above methods can only be used for policy evaluation. Then with the learned parameterized value function for a given policy, how can we perform policy improvement and select next optimal control action? In order to perform model-free control, we need to learn a parameterized Q function $\hat{Q}_w(x, u)$ instead of the parameterized value function. Nonetheless, the update rule is the same:

$$w \leftarrow w + \alpha [Q(X_k, U_k) - \hat{Q}_w(X_k, U_k)] \nabla_w \hat{Q}_w(X_k, U_k), \quad (12.26)$$

where we again can estimate $Q(X_k, U_k)$ via Monte Carlo or Bootstrapping. This leads us to the continuous-state forms of SARSA and Q-Learning that we introduced in section 12.3.

Episodic Semi-Gradient SARSA

- Input: a differentiable function $\hat{Q}_w$
- Algorithm parameters: α, ϵ
- Initialize weights w in $\hat{Q}_w$
- Loop for each episode:
 - Initialize X ; Choose action U from X using the policy derived from Q (e.g., ϵ -greedy)
 - Loop for each time step:
 - * Take action U , and observe L, X'
 - * Choose action U' from X' using policy derived from Q (e.g., ϵ -greedy)
 - * Update, $w \leftarrow w + \alpha [L + \delta \hat{Q}_w(X', U') - \hat{Q}_w(X, U)] \nabla_w \hat{Q}_w(X, U)$
 - * $X \leftarrow X', U \leftarrow U'$
 - Until we reach the terminal state

Episodic Semi-Gradient Q-Learning

- Input: a differentiable function $\hat{Q}_w$
- Algorithm parameter: α, ϵ
- Initialize weights w in $\hat{Q}_w$
- Loop for each episode:
 - Initialize X
 - Loop for each time step:
 - * Choose action U from X using policy derived from Q (e.g., ϵ -greedy)

- * Take action U and observe L, X'
- * Update, $w \leftarrow w + \alpha[L + \delta \min_u \hat{Q}_w(X', u) - \hat{Q}_w(X, U)] \nabla_w \hat{Q}_w(X, U)$
- * $X \leftarrow X', U \leftarrow U'$
- Until we reach the terminal state

From SGD to RLS We can also replace all SGD algorithms with RLS. In the case of TD(0), the objective function may be written as:

$$J(w) = \mathbb{E}_\pi [L_k + \delta \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)]^2, \quad (12.27)$$

from which we can apply RLS as

$$w \leftarrow w + [H_{k+1}]^{-1} \nabla_w [\delta \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)]^T [L_k + \delta \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)], \quad (12.28)$$

where $H_{k+1} = \nabla_w [\delta \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)]^T \nabla_w [\delta \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)] + \lambda H_k$. The advantage of RLS over SGD follows from our discussion in earlier chapters. The major drawback of RLS is that it is more computationally expensive than SGD since we need to compute the Hessian and its inverse.

12.5 Choice of Functions

There are a couple of different ways to parameterize a value function. We introduce a number of them here.

Linear Models For an n -dimensional system, we can construct a number of linear models which are easier to obtain convergence guarantees using SGD or RLS than using more complicated nonlinear models (e.g. a neural network). We define a linear model estimate of a value function as:

$$\hat{V}_w(x) = w^T \begin{bmatrix} x \\ 1 \end{bmatrix} \quad (12.29)$$

and the gradient with respect to the parameters is easily calculated as

$$\nabla_w \hat{V}_w(x) = \begin{bmatrix} x \\ 1 \end{bmatrix} \quad (12.30)$$

Feature-based Models We can further extend the idea of linear models to constructing a linear model with (nonlinear) features

$$\hat{V}_w(x) = \sum_i w_i f_i(x) \quad (12.31)$$

where the number of parameters equals to the number of features that we have. This helps to improve the expressiveness of the model

over just doing a linear combination with the states. The gradient is still easy to recover:

$$\nabla_w \hat{Y}_w(x) = [f_1(x), f_2(x), \dots, f_m(x)]^T. \quad (12.32)$$

Now the question becomes, how do we construct these features?

We may construct them in a number of different ways, but a simple way might be just through the use of polynomials. If we have an n dimensional state, $x = [x_1, x_2, x_3, \dots, x_n]^T$, we can build a second order polynomial by using the following feature vector $[1, x_1, x_2, \dots, x_n, x_1^2, x_1x_2, \dots, x_1x_n, \dots, x_n^2]$. This is effectively a special Sum of Square (SOS) programming approach to construct the polynomial

$$f^T W f, \quad (12.33)$$

which we note is also the form of the function that we used in LQR, and where f contains the polynomial features and W is the parameter matrix. Another means that we can use to construct our feature vectors is the Fourier Basis. For scalar states, the Fourier Basis returns a combination of feature vectors in the following form:

$$f_i = \cos(i\pi x). \quad (12.34)$$

For vector states, the Fourier Basis may be recovered as

$$f_i = \cos(\pi c_i^T x). \quad (12.35)$$

Another valid choice one could leverage is a set of Radial basis functions, which take the form

$$f_i = \exp\left(-\frac{\|x - c_i\|^2}{2\sigma_i^2}\right). \quad (12.36)$$

In case we want to have bounded support for the basis function, we can use compact basis functions:

$$f_i = \begin{cases} \exp\left(\frac{1}{\|x - c_i\|^2 - r_i} + \frac{1}{r_i}\right) & \|x - c_i\|^2 \leq r_i \\ 0 & \|x - c_i\|^2 > r_i \end{cases} \quad (12.37)$$

You can also go all out and use a nonlinear function approximators, but that will not be covered in much detail here. To select the correct parameter functions is not a easy task. The general guidance is to ensure that the function space you choose covers the target function that you are approximating (for example, you can never well approximate a quadratic function using a linear model) and to select the minimal set of parameters that you need to get the desired behavior you want (the more parameters we have, the more data we need for learning).

Policy Gradient and Actor Critic

While value approximation is a useful concept, what if instead of trying to approximate the value function, we directly learn/approximate the policy? We will introduce methods for directly policy learning in this chapter.

First, we need a parameterized policy, $\pi_\theta(u|x)$. This policy should be a continuous and differentiable function, where $\int_u \pi_\theta(u|x) = 1, \forall x$. The parameterized policy is only an approximation of the true stochastic policy, as the choice of parameterization constrains us to a specific function space, which means we might not be able to recover the full $\pi(\cdot)$. Constraining the problem to estimating θ gives us a viable approach as it shrinks the infinite-dimensional function space $\pi(\cdot)$ to a finite-dimensional parameter space θ .

Why might we choose to use a stochastic policy instead of a deterministic one? There are a number of benefits. First, a stochastic policy automatically ensures exploration as we have a distribution of the action u ; and thus does not over commit to any particular local optima. Furthermore, as we learn our policy, the amount of exploration may be automatically decreased over time if we get a more concentrated probability distribution for the stochastic policy (shown in the dash black curve in fig. 13.1) as we learn the policy.

In the following discussion, we will first introduce several typical parameterization of stochastic policies. Then we introduce the policy gradient theorem, which is the core theorem that specifies how the gradient w.r.t. the policy can be computed. Combining the policy gradient theorem with stochastic learning, we introduce several algorithms: REINFORCE, REINFORCE with baseline, and actor critic. Toward the end of this chapter, we will briefly discuss the off-policy gradients.

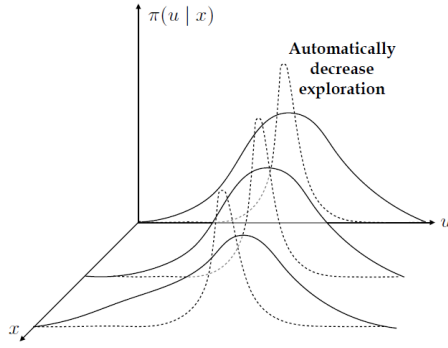


Figure 13.1: A stochastic policy defined over the state and control space after policy learning. Note the concentration on the optimal action after the policy has been learned.

13.1 Policy Parameterizations

There are a couple of well-known and popular parameterizations for policies. Here we introduce the soft-max parameterization for discrete action space and the Gaussian parameterization for continuous action space.

Soft-Max for Discrete Action Space The formula for a soft-max is given as

$$\pi_{\theta}(u|x) = \frac{e^{h_{\theta}(x,u)}}{\sum_{u'} e^{h_{\theta}(x,u')}}, \quad (13.1)$$

where h_{θ} is a preference function which assigns higher weights to actions that are more preferred. The preference function can be encoded in a deep neural network or simply via a linear function

$$h_{\theta}(x, u) = \theta^T f(x, u). \quad (13.2)$$

Gaussian for Continuous Action Space Suppose $u \in \mathbb{R}$, then we need a continuous parameterization. We generally use a Gaussian to parameterize the stochastic policy with continuous action spaces. We are assigning the policy $\pi(\cdot)$ to a Gaussian kernel,

$$\pi_{\theta}(u|x) = \frac{1}{\sigma_{\theta}(x)\sqrt{2\pi}} \exp\left(-\frac{(u - m_{\theta}(x))^2}{2\sigma_{\theta}(x)^2}\right)$$

where we can parameterize the mean function m_{θ} and the covariance function σ_{θ} , both of which are functions of state. One possible parameterization is

$$\begin{aligned} \sigma_{\theta}(x) &= \exp[\theta_1^T f_1(x)] \\ m_{\theta} &= \theta_2^T f_2(x) \end{aligned}$$

the exponential on $\sigma_{\theta}(x)$ is so that $\sigma_{\theta}(x)$ is positive, which is in line with the definition of covariance.

13.2 Policy Gradient Theorem (PGT)

As usual, we need an objective function/performance measure to relate the quality of our learning/approximation. The performance measure we select to evaluate the quality of the parameterized policy is the value function, V_{π_θ} , where we define the performance measure as

$$J(\theta) := V_{\pi_\theta}(x_0), \quad (13.3)$$

which is the value function given a specific starting state x_0 . We know that the minimum value of this performance measure is obtained when we have the optimal π_θ policy. To find the optimal parameter (θ) that minimizes $J(\theta)$, we can use gradient descent

$$\theta_{k+1} = \theta_k - \alpha \nabla_\theta J(\theta_k). \quad (13.4)$$

However, how can we compute the gradient over $J(\theta_k)$ with respect to the parameter, θ ? Recall that V_π can be written as

$$V_\pi(x) = \int_u \pi(u|x) Q_\pi(x, u) \quad (13.5)$$

For this expression, we will derive the Policy Gradient Theorem in the following discussion, which specifies that:

$$\nabla_\theta V_{\pi_\theta}(x_0) = \int_x \eta(x) \int_u \nabla_\theta \pi_\theta(u|x) Q_{\pi_\theta}(x, u) \quad (13.6)$$

where $\eta(x) = \sum_{k=0}^{\infty} \mathcal{P}(x_0 \rightarrow x, k, \pi_\theta)$. The intuition behind $\eta(x)$ is that it is the total probability we will enter state x in k time steps under policy π_θ . Note $\eta(x)$ is not a probability measure itself since $\int_x \eta(x) \neq 1$. We can normalize $\eta(x)$, which will give us the probability measure $\mu(x) = \eta(x) / \int_x \eta(x)$ that appeared earlier in the performance index for value approximation.

Proof of Policy Gradient Theorem Now we prove (13.6) holds. Let us start from (13.5) and take gradient w.r.t. θ on both sides:

$$\begin{aligned} V_{\pi_\theta}(x) &= \int_u \pi_\theta(u|x) Q_{\pi_\theta}(x, u) \\ \nabla_\theta V_{\pi_\theta}(x) &= \int_u \nabla_\theta [\pi_\theta(u|x) Q_{\pi_\theta}(x, u)] \\ &= \int_u [\nabla_\theta \pi_\theta(u|x) Q_{\pi_\theta}(x, u) + \pi_\theta(u|x) \nabla_\theta Q_{\pi_\theta}(x, u)] \end{aligned}$$

The last equation uses the chain rule.

We note that the first term in the last expression is similar to what we have in the target equation, but the second term is not. By definition, we have

$$Q_{\pi_\theta}(x, u) = \int_{x'} \int_l p(x', l|x, u) [l + V_{\pi_\theta}(x')].$$

By taking gradient w.r.t. θ on both sides, we see that l drops out:

$$\begin{aligned}\nabla_{\theta} Q_{\pi_{\theta}}(x, u) &= \int_{x'} \int_l p(x', l | x, u) [\nabla_{\theta} V_{\pi_{\theta}}(x')] \\ &= \int_{x'} p(x' | x, u) [\nabla_{\theta} V_{\pi_{\theta}}(x')]\end{aligned}$$

Substituting $\nabla_{\theta} Q_{\pi_{\theta}}(x, u)$ in the gradient of the value function, we get

$$\nabla_{\theta} V_{\pi_{\theta}}(x) = \int_u [\nabla_{\theta} \pi_{\theta}(u | x) Q_{\pi_{\theta}}(x, u) + \pi_{\theta}(u | x) \int_{x'} p(x' | x, u) \nabla_{\theta} V_{\pi_{\theta}}(x')]. \quad (13.7)$$

We notice on the left-hand side of (13.7) we have gradient of $V_{\pi_{\theta}}$ taken at x and on the right-hand side we have gradient of $V_{\pi_{\theta}}$ taken at x' . To simplify (13.7), we break the right-hand-side into two parts:

$$\nabla_{\theta} V_{\pi_{\theta}}(x) = \int_u \nabla_{\theta} \pi_{\theta}(u | x) Q_{\pi_{\theta}}(x, u) + \int_u \pi_{\theta}(u | x) \int_{x'} p(x' | x, u) \nabla_{\theta} V_{\pi_{\theta}}(x') \quad (13.8a)$$

$$= A(x) + \int_{x'} \mathcal{P}(x \rightarrow x', 1, \pi_{\theta}) \nabla_{\theta} V_{\pi_{\theta}}(x') \quad (13.8b)$$

where $A(x) = \int_u \nabla_{\theta} \pi_{\theta}(u | x) Q_{\pi_{\theta}}(x, u)$ and $\mathcal{P}(x \rightarrow x', 1, \pi_{\theta}) = \int_u \pi_{\theta}(u | x) p(x' | x, u)$.

With (13.8), we can perform inductions by substituting $V_{\pi_{\theta}}(x')$ on the right-hand-side with the same expression:

$$\nabla_{\theta} V_{\pi_{\theta}}(x) = A(x) + \int_{x'} \mathcal{P}(x \rightarrow x', 1, \pi_{\theta}) A(x') + \int_{x''} \mathcal{P}(x \rightarrow x'', 2, \pi_{\theta}) \nabla_{\theta} V_{\pi_{\theta}}(x'') \quad (13.9)$$

$$\rightarrow \sum_{k=0}^{\infty} \int_{x'} \mathcal{P}(x \rightarrow x', k, \pi_{\theta}) A(x') \quad (13.10)$$

Because all the terms in this expression are bounded, we can switch the order of the summation and integration.

$$\begin{aligned}\nabla_{\theta} V_{\pi_{\theta}}(x) &= \int_{x'} \sum_{k=0}^{\infty} \mathcal{P}(x \rightarrow x', k, \pi_{\theta}) A(x') \\ &= \int_{x'} \eta(x') A(x')\end{aligned}$$

which finishes our proof, yielding

$$\nabla_{\theta} V_{\pi_{\theta}}(x_0) = \int_x \eta(x) \int_u \nabla_{\theta} \pi_{\theta}(u | x) Q_{\pi_{\theta}}(x, u) \quad (13.11)$$

where we have switched the parameters, x is now x_0 and x' is x .

Interpreting Policy Gradient Theorem How can we understanding this PGT? Let us consider fig. 13.2. In this figure, what do $V_{\pi}(x)$ and $A(x)$ mean? We know $V_{\pi}(x)$ is equal to the integration provided in

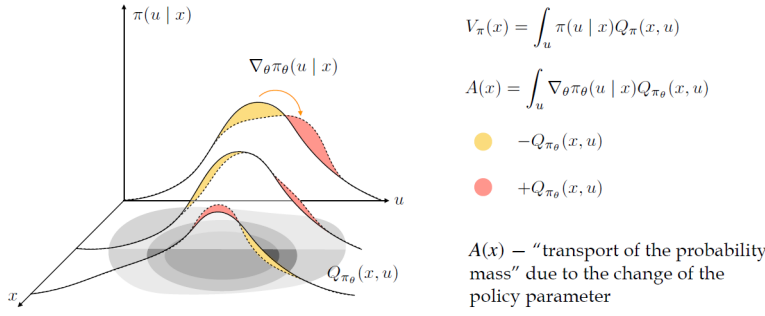


Figure 13.2: An intuitive illustration of how PGT works. In the graph, we show both the policy and the Q-function. The stochastic policy are the solid black line and the heat map which lies in the x - u plane represents the Q-function. We are moving probability mass from the area bounded by the solid black line to the hashed black line in order to minimize the π -weighted integration of Q in V_{π} .

the figure, which is a weighted sum of the values of the Q-function weighted by policy $\pi(\cdot)$. The Q-function is over the space spanned by x and u and is thus represented by a heat map in the x - u domain. The $A(x)$ is of the same ilk as $V_{\pi}(x)$, which can be seen in the form of the equation, but $A(x)$ is weighted with respect to the gradient of the policy $\pi(\cdot)$ and not π itself. Taking this gradient, we are changing the probability on the different state action pairs given that the integration of $\pi(\cdot)$ itself must stay at a value of one. As this gradient might have positive or negative values, we consider it as the transport of probability mass from the old distribution to the new distribution. Thus $A(x)$ may be considered to be the transport of the probability mass due to the change of the policy parameter.

However, $A(x)$ itself does not tell everything about $V_{\pi}(x)$. We must also consider $\eta(x)$ and how it affects the probability mass. As $\eta(x)$ describes the likeliness of visiting a specific state, the quantity $\eta(x)$ shapes how we update the probability mass distribution by collecting estimates of states that we are more likely to visit. Similarly, for states which we seldom or never visit case, we do not put as much weight around shifting of the probability mass as this is not where the more influential components act. That is when we calculate the gradient of the value function, we need both $\eta(x)$ and $A(x)$.

Dealing with Deterministic Policies We can also apply the same theorem for deterministic policy, it is just a slight change in interpretation if we regard the deterministic case as a special case of the stochastic policy. In a deterministic policy, the probability mass is concentrated to a certain point in u for any given x . This builds a surface for a specific policy, which is the surface u_1 in fig. 13.3. So the gradient of $\pi_{\theta}(\cdot)$ is shifting this surface around. How do we capture this change? First, we can describe $A(x)$ as

$$A(x) = Q_{\pi_{\theta}}(x, u_2) - Q_{\pi_{\theta}}(x, u_1). \quad (13.12)$$

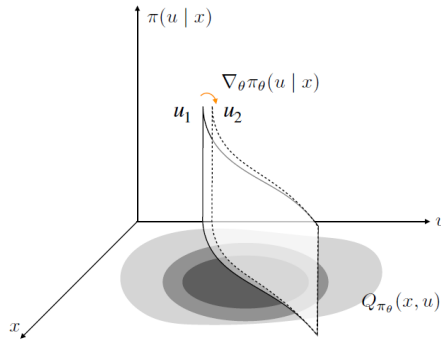


Figure 13.3: Policy gradient with a deterministic policy.

We then denote the deterministic mapping as $u = g_\theta(x)$, then rewrite $A(x)$ by taking the limit from u_2 to u_1 . With this infinitesimal difference, we can rewrite $A(x)$ as

$$A(x) = \nabla_u Q_{g_\theta}(x, g_\theta(x)) \nabla_\theta g_\theta(x). \quad (13.13)$$

The difference between the calculation of the stochastic and the deterministic gradients are large. The stochastic policy gradient (especially the corresponding $A(x)$) only involves the gradient of the policy, while the deterministic policy gradient ($A(x)$ in (13.13)) considers both the gradient of the policy and the gradient of Q . It may be difficult to evaluate the gradient of Q from samples, unless we have a parameterized function for Q , like in the deep deterministic policy gradient (DDPG) algorithm. If we fix the form of Q , then we can compute the form of the gradient offline.

13.3 REINFORCE

REINFORCE directly leverages the policy gradient theorem and uses samples to approximate the gradient. REINFORCE is a Monte Carlo method, where the key idea is to use samples to approximate $\nabla_\theta J(\theta_k)$ so that we can perform stochastic gradient descent

$$\theta_{k+1} = \theta_k - \alpha \widehat{\nabla_\theta J(\theta_k)}. \quad (13.14)$$

In the following discussion, we will introduce different approaches to obtain $\widehat{\nabla_\theta J(\theta_k)}$.

All-Actions Method In order to use samples, we need to turn the equation in Policy Gradient Theorem into an expectation. Let us do so by converting $\eta(x)$ to a probability measure $\mu(x)$ through normalization. Note that after this conversion, we no longer have an

equality, but rather a statement of proportionality:

$$\begin{aligned}\nabla_{\theta} V_{\pi_{\theta}}(x_0) &\propto \int_x \mu(x) \int_u \nabla_{\theta} \pi_{\theta}(u|x) Q_{\pi_{\theta}}(x, u) \\ &= \mathbb{E}_{\pi_{\theta}} \left[\int_u \nabla_{\theta} \pi_{\theta}(u|X_k) Q_{\pi_{\theta}}(X_k, u) \right],\end{aligned}$$

where we have converted the integral into an expectation which samples $X_k \sim \mu(\cdot)$. With this new expression, we can rewrite the gradient descent as

$$\theta_{k+1} = \theta_k - \alpha \int_u \nabla_{\theta} \pi_{\theta}(u|X_k) Q_{\pi_{\theta}}(X_k, u). \quad (13.15)$$

The problem with this approach is that we still have to integrate over all u 's, which is computationally expensive, especially if this action space is continuous. We fix this in the One-Action Method.

One-Action Method The general idea behind the one-action method is that we really want to take (or sample) a single action instead of calculating the value over the entire control space. So, how do we get rid of this integration over u ? Well, in the all-action method, we used expectation to get rid of an integral, we can kind of do the same thing. However, as we are missing probability distribution to draw control from, we need to modify the equation a little. We multiply through by $\pi_{\theta}(\cdot)$ over $\pi_{\theta}(\cdot)$, which is shown below

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\pi_{\theta}} \left[\int_u \pi_{\theta}(u|X_k) Q_{\pi_{\theta}}(X_k, u) \frac{\nabla_{\theta} \pi_{\theta}(u|X_k)}{\pi_{\theta}(u|X_k)} \right], \quad (13.16)$$

and then we consider the following relations

$$Q_{\pi_{\theta}}(X_k, U_k) = \mathbb{E}_{\pi_{\theta}}[G_k|X_k, U_k], \quad (13.17)$$

where $U_k \sim \pi_{\theta}(\cdot|X_k)$ and

$$\frac{\nabla_{\theta} \pi_{\theta}(u|X_k)}{\pi_{\theta}(u|X_k)} = \nabla_{\theta} \ln \pi_{\theta}(U_k|X_k) \quad (13.18)$$

which allows us to condense our earlier expression to

$$\nabla_{\theta} J(\theta) \propto \mathbb{E}_{\pi_{\theta}}[G_k \nabla_{\theta} \ln \pi_{\theta}(U_k, X_k)]$$

which we can then solve with the following form of stochastic gradient descent

$$\theta_{k+1} = \theta_k - \alpha G_k \nabla_{\theta} \ln \pi_{\theta}(U_k|X_k). \quad (13.19)$$

There are several advantages to this one-action method. Now, no more integration is needed, just sampling operations. However, this method does not get around the fact that we have to wait for an

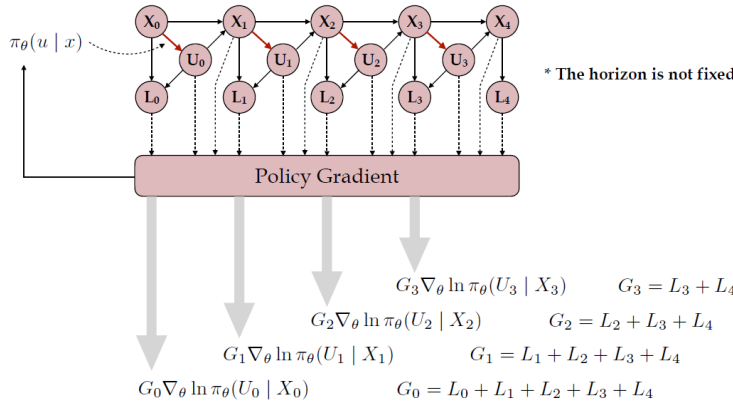


Figure 13.4: A depiction of the REINFORCE Algorithm.

entire episode to finish to get G_k before we can do an update. So the update is done in retrospect after we have seen all the run-time costs.

REINFORCE is an algorithm that leverages this one-action method approach and its procedure is illustration in fig. 13.4.

So, we start at X_0 and sample from policy π_θ to get U_0 , then from X_0 and U_0 we get a run-time cost L_0 and transition to X_1 . Do the same thing over and over until the event terminates. This means we do not need to fix the horizon! If we do not reach the terminal state, we can choose the horizon as long as possible. After we finish the episode, we collect all the L 's and U 's and use that information to do a policy gradient. This policy gradient updates the θ inside the π_θ . Then we repeat the process. The pseudo-code is provided below.

- Input: a differentiable policy parameterization $\pi_\theta(u|x)$
- Algorithm parameter: step size α
- Initialize policy parameter $\theta^{(0)}$
- For every episode i
 - Generate trajectory $X_0, U_0, L_0, X_1, U_1, L_1, \dots, X_{N-1}, U_{N-1}, L_{N-1}, X_N, L_N$ from the policy $\pi_{\theta^{i-1}}$
 - $\theta_0 = \theta^{i-1}$
 - For every time step of the episode $k = 0, 1, \dots, N-1$
 - * $G_k = \sum_{j=k}^N L_j$
 - * $\theta_{k+1} = \theta_k - \alpha G_k \nabla \ln \pi_\theta(U_k | X_k)$
 - $\theta^i = \theta_N$

Note that when we evaluate the gradient $\nabla \ln \pi_\theta(U_k | X_k)$, we need to use the parameters that generate the action. In the above case, we

always use θ_0 instead of θ_k . That is because θ_k is only used in the learning process, not in the action generation process.

Lifted REINFORCE vs Lifted ILC Since this gradient in REINFORCE is always with respect to the original θ , then we can lift the whole process by computing a summed gradient over all time steps:

$$\nabla_{\theta} J(\theta) \propto \sum_{k=0}^{N-1} G_k \nabla_{\theta} \ln \pi_{\theta}(U_k | X_k). \quad (13.20)$$

To simplify the notation, let us define $D_k = \nabla_{\theta} \ln \pi_{\theta}(U_k | X_k)$, which allows us to rewrite $\nabla_{\theta} J(\theta)$ as:

$$\nabla_{\theta} J(\theta) \propto [D_0, D_1, \dots, D_{N-1}] \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 0 & 1 & \dots & 1 & 1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & 1 & 1 \end{bmatrix} \begin{bmatrix} L_0 \\ L_1 \\ L_2 \\ \vdots \\ L_{N-1} \\ L_n \end{bmatrix}. \quad (13.21)$$

With the lifted form, we no longer need to update θ at every time k . Instead, we can do just one update for all time steps using (13.21).

You may have noticed that this lifted REINFORCE is similar to the lifted ILC we studied at earlier. Both update the control strategy in retrospect when an episode is completed. But what are the differences between these two? There are some differences, mainly that ILC is a feedforward method while REINFORCE is a feedback method. In general, ILC is better at rejecting repetitive disturbances that remain constant across different iterations/episodes. REINFORCE is better at rejecting state-dependent disturbances or low-frequency non-repetitive disturbances. However, the relation between the two still needs more study.

13.4 REINFORCE with Baseline

Recalling the policy gradient theorem

$$\nabla_{\theta} J(\theta) \propto \int_x \mu(x) \int_u \nabla_{\theta} \pi_{\theta}(u|x) Q_{\pi_{\theta}}(x, u), \quad (13.22)$$

we can generalize this expression to include a comparison of the Q value to some arbitrary baseline $b(x)$

$$\nabla_{\theta} J(\theta) \propto \int_x \mu(x) \int_u \nabla_{\theta} \pi_{\theta}(u|x) [Q_{\pi_{\theta}}(x, u) - b(x)]. \quad (13.23)$$

Considering our original interpretation of policy gradient theorem, we see that including this baseline does not affect the shifting of the probability mass as we considered above. This is because

$\int_u \nabla_\theta \pi_\theta(u|x) b(x) = 0$, since

$$\int_u \nabla_\theta \pi_\theta(u|x) b(x) = b(x) \int_u \nabla_\theta \pi_\theta(u|x) = b(x) \nabla_\theta \int_u \pi_\theta(u|x) = b(x) \nabla_\theta 1 = 0.$$

With the baseline, we get the following reformulation leveraging the expectation

$$\begin{aligned} \nabla_\theta J(\theta) &\propto \int_x \mu(x) \int_u \nabla_\theta \pi_\theta(u|x) [Q_{\pi_\theta}(x, u) - b(x)] \\ &= \mathbb{E}_{\pi_\theta} \left[\int_u \nabla_\theta \pi_\theta(u|X_k) [Q_{\pi_\theta}(X_k, u) - b(X_k)] \right] \\ &= \mathbb{E}_{\pi_\theta} \left[[Q_{\pi_\theta}(X_k, U_k) - b(X_k)] \frac{\nabla_\theta \pi_\theta(U_k|X_k)}{\pi_\theta(U_k|X_k)} \right] \\ &= \mathbb{E}_{\pi_\theta} [(G_k - b(X_k)) \nabla_\theta \ln \pi_\theta(U_k|X_k)] \end{aligned}$$

Given this change to the expectation, we modify our stochastic gradient descent update to:

$$\theta_{k+1} - \theta_k - \alpha (G_k - b(X_k)) \nabla_\theta \ln \pi_\theta(U_k|X_k). \quad (13.24)$$

Since the baseline is not a function of or sampled from U_k , the expected value of the update is unchanged. Meanwhile, an advantageously selected baseline may reduce the variance in our particular problem.

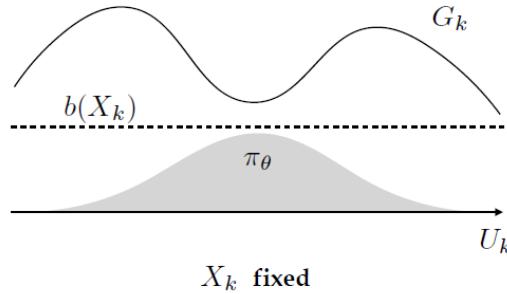


Figure 13.5: Selecting the correct baseline, $b(X_k)$, can reduce the variance we experience

If we multiplied the gradient $\nabla_\theta \ln \pi_\theta(U_k|X_k)$ with G_k , we may have a large variance. If we pick the baseline $b(X_k)$, then the area between G_k and $b(X_k)$ is smaller, thus reflecting a lower variance. A natural choice for the baseline is the value function $b(X_k) := V(X_k)$, since $\mathbb{E}_{\pi_\theta}[G_k] = V(X_k)$ if θ is the optimal parameter. In this case, the baseline will reduce the variance.

However, since the value function is unknown, we need to use the estimated value function $\hat{V}_w(X_k)$ parameterized by w to serve as the baseline. The update rule of the estimated value function is:

$$w_{k+1} = w_k + \alpha^w (G_k - \hat{V}_w(X_k)) \nabla_w \hat{V}_w(X_k). \quad (13.25)$$

The pseudo code for REINFORCE with Baseline is shown below:

- Input: a differentiable policy parameterization $\pi_\theta(u|x)$ and a differentiable estimate of the value function $\hat{V}_w(x)$
- Algorithm parameter: step size α^θ, α^w
- Initialize policy parameter $\theta^{(0)}, w^{(0)}$
- For every episode i
 - Generate trajectory $X_0, U_0, L_0, X_1, U_1, L_1, \dots, X_{N-1}, U_{N-1}, L_{N-1}, X_N, L_N$ from policy $\pi_{\theta^{i-1}}$
 - $\theta_0 = \theta^{i-1}, w_0 = w^{i-1}$
 - For every time step of the episode $k = 0, 1, \dots, N-1$
 - * $G_k = \sum_{j=k}^N L_j, \delta_k = G_k - \hat{V}_w(X_k)$
 - * $w_{k+1} = w_k + \alpha^w \delta_k \nabla_w \hat{V}_w(X_k)$
 - * $\theta_{k+1} = \theta_k - \alpha^\theta \delta_k \nabla_\theta \ln \pi_\theta(U_k|X_k)$
 - $\theta^{(i)} = \theta_N, w^{(i)} = w_N$

The lifted version has a slightly different formulation with the same initialization.

- For every episode i :
 - Generate trajectory $X_0, U_0, L_0, X_1, U_1, L_1, \dots, X_{N-1}, U_{N-1}, L_{N-1}, X_N, L_N$ from policy $\pi_{\theta^{i-1}}$
 - For every time step of the episode $k = 0, 1, \dots, N-1$
 - * Compute $D_k^\theta = \nabla_\theta \ln \pi_\theta(U_k|X_k)$
 - * Compute $D_k^w = \nabla_w \hat{V}_w(X_k)$

$$- w^{(i)} = w^{(i-1)} + \alpha^w [D_0^w, D_1^w, \dots, D_{N-1}^w] \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 0 & 1 & \dots & 1 & 1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & 1 & 1 \end{bmatrix} \begin{bmatrix} L_0 \\ L_1 \\ \vdots \\ L_{N-1} \\ L_N \end{bmatrix} - \begin{bmatrix} \hat{V}_w(X_0) \\ \hat{V}_w(X_1) \\ \vdots \\ \hat{V}_w(X_{N-1}) \end{bmatrix}$$

$$- \theta^{(i)} = \theta^{(i-1)} + \alpha^\theta [D_0^\theta, D_1^\theta, \dots, D_{N-1}^\theta] \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 0 & 1 & \dots & 1 & 1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & 1 & 1 \end{bmatrix} \begin{bmatrix} L_0 \\ L_1 \\ \vdots \\ L_{N-1} \\ L_N \end{bmatrix} - \begin{bmatrix} \hat{V}_w(X_0) \\ \hat{V}_w(X_1) \\ \vdots \\ \hat{V}_w(X_{N-1}) \end{bmatrix}$$

13.5 Actor Critic

There are still some problems with REINFORCE with Baseline. One large problem, touched on when we first introduced REINFORCE, was that we need to wait until the end of an episode to get G_k . There is no bootstrapping (updating the value estimate from the estimated values of subsequent states) in either REINFORCE or REINFORCE with baseline. But if we make the following transformation

$$G_k = L_k + \hat{V}_w(X_{k+1}) \quad (13.26)$$

which produces an -admittedly- biased estimate, we can turn REINFORCE into an online algorithm. Making this substitution, we see that our parameter update takes the form

$$\theta_{k+1} = \theta_k - \alpha(L_k + \hat{V}_w(X_{k+1}) - b(X_k)) \nabla_{\theta} \ln \pi_{\theta}(U_k | X_k). \quad (13.27)$$

We can also make our substitution with the estimated value function for the baseline and obtain the following equation

$$\theta_{k+1} = \theta_k - \alpha(L_k + \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)) \nabla_{\theta} \ln \pi_{\theta}(U_k | X_k). \quad (13.28)$$

It should be noted that $L_k + \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)$ is the one-step TD error. These changes enable us to execute the algorithm fully online. All the states, actions, and run-time costs are processed as they occur and are never revisited.

The pseudo-code for Actor-Critic is

- Input: a differentiable policy parameterization $\pi_{\theta}(u|x)$, a differentiable estimate of value function $\hat{V}_w(x)$.
- Algorithm parameter: step size, $\alpha^{\theta}, \alpha^w$
- Initialize policy parameter $\theta_0, w_0, k = 0$
- Loop forever (for each episode)
 - State initialization X_k
 - For every time step before termination
 - * $U_k \sim \pi_{\theta}(X_k)$
 - * Take action U_k and observe X_{k+1}
 - * $\delta_k = L_k + \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)$
 - * $w_{k+1} = w_k + \alpha^w \delta_k \nabla_w \hat{V}_w(X_k)$
 - * $\theta_{k+1} = \theta_k - \alpha^{\theta} \delta_k \nabla_{\theta} \ln \pi_{\theta}(U_k | X_k)$
 - * Increase k

Comparison We compare the structure of REINFORCE with Actor-Critic in fig. 13.6. In the upper-two layers, we show the the dynamics of the system, describing how we transition between the states and get the expected costs. In REINFORCE, after we get all the information from one episode, we can compute the update to the parameters. The bottom layers describe how we do this update. We see that the parameter update from step k to step $k + 1$ requires $D_k(\cdot)$, θ_k , and δ_k . The reason why we must wait for the end of the episode in REINFORCE is that the advantage function δ_k requires information (all of the costs) from the entire episode. Furthermore, look at the parameter vector θ , that is used to calculate the update. For REINFORCE, every time step uses θ_0 . Contrast this with the Actor-Critic approach, where the advantage function is only a function of the next state and current cost and the parameter is updated at each step. Furthermore, we use this new parameter estimate to find our new control.

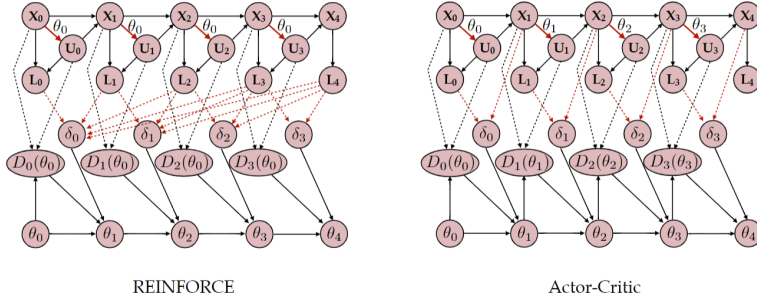


Figure 13.6: A diagram comparing REINFORCE with Actor-Critic

Furthermore, we can compare the functions used in different approaches in the following table. The equations are very similar, but there are some important differences. In the advantage functions, REINFORCE uses G_k , while Actor-Critic uses bootstrapping. For the parameter update, the structure is the same, but where/when we take the gradient changes. In REINFORCE, the parameter update occurs with respect to the starting parameters and weights. In Actor-Critic, we take the update with respect to the specific parameter/weight corresponding to the time step in which is taken. Thus, we can view REINFORCE as an offline learning approach and Actor-Critic is an online learning approach.

A3C If we use the same advantage function in Actor-Critic, but keep using the initial parameter to choose the actions, then we arrive at the algorithm called Asynchronous Advantage Actor-Critic (A3C) as shown in fig. 13.8. This approach is different from both Actor-Critic and REINFORCE with baseline.

REINFORCE with Baseline	Actor-Critic
$\delta_k = G_k - \hat{V}_w(X_t) \quad G_k = \sum_{j=k}^N L_j$	$\delta_k = L_k + \hat{V}_w(X_{k+1}) - \hat{V}_w(X_k)$
$w_{k+1} = w_k + \alpha^w \delta_k D_k^w(w_0)$	$w_{k+1} = w_k + \alpha^w \delta_k D_k^w(w_k)$
$\theta_{k+1} = \theta_k - \alpha^\theta \delta_k D_k^\theta(\theta_0)$	$\theta_{k+1} = \theta_k - \alpha^\theta \delta_k D_k^\theta(\theta_k)$
$U_k \sim \pi_{\theta_0}(X_k)$	$U_k \sim \pi_{\theta_k}(X_k)$
Offline Learning	Online Learning

Figure 13.7: Table comparing REINFORCE and Actor-Critic

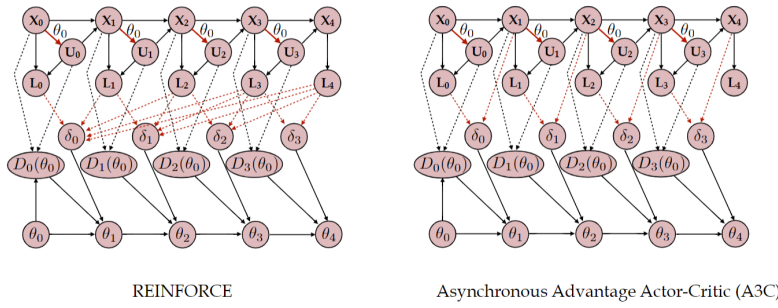


Figure 13.8: A figure comparing REINFORCE to A3C

A3C was designed for parallel training. The spirit of the algorithm is that multiple actors are trained in parallel and get synced with the global parameters from time to time. This may also be viewed as a parallelized reformation of minibatch-based stochastic gradient update, as the values of w or θ get corrected by a little bit in the direction of each training thread independently.

Pseudo-code for A3C is as follows. The initialization is the same as the Actor-Critic case

- Loop for each episode
 - Synchronize policy parameters with the main thread
 - For every time step before next synchronization
 - * Sample the policy and take action
 - * Compute $D_k^\theta = \nabla_\theta \ln \pi_\theta(U_k|X_k)$ and $D_k^w = \nabla_w \hat{V}_w(X_k)$

$$- \Delta w \propto [D_0^w, D_1^w, \dots, D_{N-1}^w] \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 0 & 1 & \dots & 1 & 1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & 1 & 1 \end{bmatrix} \begin{bmatrix} L_0 \\ L_1 \\ \vdots \\ L_{N-1} \\ L_N \end{bmatrix} - \begin{bmatrix} \hat{V}_w(X_0) \\ \hat{V}_w(X_1) \\ \vdots \\ \hat{V}_w(X_{N-1}) \end{bmatrix}$$

$$- \Delta\theta \propto [D_0^\theta, D_1^\theta, \dots, D_{N-1}^\theta] \begin{bmatrix} 1 & 1 & \dots & 1 & 1 \\ 0 & 1 & \dots & 1 & 1 \\ \vdots & \ddots & \ddots & \vdots & \vdots \\ 0 & \dots & \dots & 1 & 1 \end{bmatrix} \begin{bmatrix} L_0 \\ L_1 \\ \vdots \\ L_{N-1} \\ L_N \end{bmatrix} - \begin{bmatrix} \hat{V}_w(X_0) \\ \hat{V}_w(X_1) \\ \vdots \\ \hat{V}_w(X_{N-1}) \end{bmatrix}$$

13.6 Off-Policy Gradient

The policy gradient methods we discussed so far are all on-policy methods, which means that the behavior policy is equal to the target policy. Off-policy methods will result in several additional advantages including the reuse of past episodes (experience replay), which helps with efficient use of samples, and better exploration by choosing a behavior policy different from the target policy. We assume that the behavior policy for collecting samples is known and is given as $\beta(u|x)$. The performance measure is defined as

$$J(\theta) := \int_x \mu^\beta(x) V_{\pi_\theta}(x) = \int_x \mu^\beta(x) \int_u \pi_\theta(u|x) Q_\pi(x, u), \quad (13.29)$$

where $\mu^\beta(x)$ is the stationary distribution under the behavior policy $\beta(u|x)$. We want to learn the parameters, θ , in the target policy. Let us take the gradient of the performance measure:

$$\begin{aligned} \nabla_\theta J(\theta) &= \int_x \mu^\beta(x) \nabla_\theta \left[\int_u \pi_\theta(u|x) Q_\pi(x, u) \right] \\ &= \int_x \mu^\beta(x) \int_{x'} \eta^{\pi_\theta}(x') \int_u \nabla_\theta \pi_\theta(u|x) Q_{\pi_\theta}(x', u) \\ &\approx \mathbb{E}_\beta \left[\int_u \nabla_\theta \pi_\theta(u|X) Q_{\pi_\theta}(X, u) \right] \end{aligned}$$

where $X \sim \mu^\beta(\cdot)$. We introduced η^{π_θ} , which is the chance that we visit a certain state under the policy π_θ , but then we ignored η^{π_θ} as it does not have a large impact on the expectation term. However, we still want to get rid of the integration over u . To do this, we need to find the probability distribution using this β .

$$\begin{aligned} \nabla_\theta J(\theta) &\approx \mathbb{E}_\beta \left[\int_u \nabla_\theta \pi_\theta(u|X) Q_{\pi_\theta}(X, u) \right] \\ &= \mathbb{E}_\beta \left[\beta(u|X) \frac{\pi_\theta(u|X)}{\beta(u|X)} Q_{\pi_\theta}(X, u) \nabla_\theta \ln \pi_\theta(u|X) \right] \\ &= \mathbb{E}_\beta \left[\frac{\pi_\theta(u|X)}{\beta(u|X)} Q_{\pi_\theta}(X, u) \nabla_\theta \ln \pi_\theta(u|X) \right] \end{aligned}$$

where $U \sim \beta(\cdot|X)$. The first term in the last expression is known as the importance weighting, which basically tells how well aligned the

two policies are. We again use a diagram to describe what is going on in this Off-Policy gradient. In the Off-Policy Gradient, we are noticeably sampling from the behavior policy β and not θ .

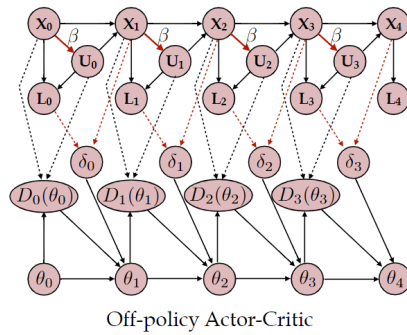


Figure 13.9: Actor-Critic Off-Policy Gradient

So the big questions relating these different Policy Gradient methods are:

- What θ for action generation?
- What θ for gradient computation?
- What advantage function δ to use?

which are all summarized in the following table. Yet, there are many other Policy Gradient Methods, and this is just a quick overview of all the basic methods.

θ for action	θ for gradient	Advantage δ	Method
θ_0	θ_0	Cost summation	REINFORCE
θ_0	θ_0	Cost summation - baseline	REINFORCE with Baseline
θ_k	θ_k	TD error	Actor-Critic
θ_0	θ_0	TD error	A3C
Some θ^*	θ_k	TD error	Off-policy Actor-Critic

Figure 13.10: A comparison of the three major components in different Policy Gradient algorithms.

Part VI

Stability and Safety

Lyapunov Stability

We have studied the definition of stability in the beginning of this course and also discussed ways to show stability using eigen-values. However, that eigen-value approach does not apply to general non-linear systems. To develop a general theory for stability, we need to reconsider what “stability” for a system actually implies. Any physical motion in the real world consumes energy, and the amount of energy that can be consumed is bounded. As a result, those physical motions will eventually converge to a steady state with no further energy consumption. For example, for a pendulum with friction, its total energy is a summation of its kinetic energy and its potential energy. These two energies can be converted into each other, but the total energy will always dissipate due to friction. Eventually, the pendulum will stop at the position with smallest potential energy. According to our definition of stability, this pendulum system is asymptotically stable. This example implies that if we can find a general way to specify such an “energy” function for an arbitrary system, then we can test its stability without checking their eigen-values. Indeed, Lyapunov’s 2nd method provides us such a handy tool to check stability, with a generalized “energy” function called the Lyapunov function. In the following discussion, we will first formally introduce Lyapunov stability analysis, then talk about the synthesis of the Lyapunov function, and finally talk about control synthesis using Lyapunov functions.

14.1 Lyapunov Stability

Note we have discussed Lyapunov 1st method earlier, which checks stability by checking the eigen-values of linearized nonlinear systems. This method cannot generalize to arbitrary nonlinear systems. Here we focus on the Lyapunov 2nd method which can be applied to arbitrary autonomous systems.

14.1.1 Lyapunov 2nd Method For Continuous Time Systems

Theorem 14.1. For the nonlinear system $\dot{x} = f(x)$ that has an equilibrium at the origin, if we can find a scalar function $V(x)$ such that

1. $V(0) = 0$;
2. Exist a K-class function¹ α such that $V \geq \alpha(\|x\|)$ (V is positive definite);
3. Exist a K-class function γ such that $\dot{V} \leq -\gamma(\|x\|)$ ($\dot{V}$ is negative definite);
4. $\alpha(\|x\|) \rightarrow \infty$ as $\|x\| \rightarrow \infty$ (V is radially unbounded)

¹ A K-class function is a mapping from positive real numbers to positive real numbers which is continuous, non-decreasing, and approaching zero at the origin.

Then the equilibrium point $x = 0$ is globally asymptotically stable (GAS). And we call the scalar function as the Lyapunov function.

None of these four conditions can be relaxed. The first two conditions guarantee that the Lyapunov function is a lower-bounded energy function. The third condition guarantees that the Lyapunov function is strictly decreasing outside the equilibrium point, *i.e.*, energy dissipation. The last condition guarantees that the direction for energy dissipation is pointing towards the equilibrium point.

14.1.2 Lyapunov Analysis on Continuous Time Linear Systems

Consider a continuous time linear system $\dot{x} = Ax$. Let us define a Lyapunov function $V(x) = x^T Px$. Then its derivative becomes $\dot{V}(x) = x^T (A^T P + PA)x$. According to Lyapunov 2nd method, as long as we can find a positive definite matrix P such that $A^T P + PA$ is negative definite, the linear system is globally asymptotically stable. Let $A^T P + PA = -Q$. Then the continuous time linear system is globally asymptotically stable if we can find positive definite matrices P and Q that solves the following Riccati equation:

$$A^T P + PA + Q = 0. \quad (14.1)$$

Before we derive the condition to ensure A is stable, let us first present several results regarding P and Q if A is stable.

Lemma 14.1 (Lyapunov Integral). For positive definite matrices P and Q that satisfy (14.1), we have

$$P = \int_0^\infty e^{tA^T} Q e^{tA} dt. \quad (14.2)$$

Proof. If we can find positive definite matrices that solve (14.1), then Lyapunov 2nd method states that the system is GAS, hence $x_\infty \rightarrow 0$.

We have

$$\begin{aligned}
 x_0^T P x_0 &= x_0^T P x_0 - x_\infty^T P x_\infty \\
 &= - \int_0^\infty \frac{d}{dt} [x_t^T P x_t] dt \\
 &= - \int_0^\infty x_t^T [A^T P + P A] x_t dt \\
 &= \int_0^\infty x_0^T e^{tA^T} Q e^{tA} x_0 dt.
 \end{aligned}$$

Since the above equality holds for all x_0 , then P and Q satisfy (14.2). $\square$

One direct implication of this lemma is that we can regard the Lyapunov function as a value function, since

$$V(x_0) = x_0^T P x_0 = \int_0^\infty x_t^T Q x_t dt, \quad (14.3)$$

where $x_t^T Q x_t$ can be viewed as the run-time cost. Note this is an autonomous system. So there is no control term in the run-time cost.

Let us compare this formulation with LQR. In LQR, the Riccati equation $PA + A^T P - PBR^{-1}B^T P + Q = 0$ has an additional term $PBR^{-1}B^T P$ compared to (14.1). Later we will see that this additional term is associated with control. Moreover, in LQR, we specify run-time cost first, then obtain the value function. Similarly, we can show that for any positive definite Q , there is a positive definite P to form the Lyapunov equation for the autonomous system, as long as the system is asymptotically stable.

Lemma 14.2 (Construction of Lyapunov Function). *If A is asymptotically stable, then for any positive definite Q , there is an explicit formula for the Lyapunov function $V(x) = x^T P x$ where P satisfies (14.2).*

Proof. By definition in (14.2), P is positive definite. To show that $V(x) = x^T P x$ is indeed a Lyapunov function, we only need to show that $A^T P + P A$ is negative definite.

$$\begin{aligned}
 A^T P + P A &= \int_0^\infty (A^T e^{tA^T} Q e^{tA} + e^{tA^T} Q e^{tA} A) dt \\
 &= \int_0^\infty \frac{d}{dt} [e^{tA^T} Q e^{tA}] dt \\
 &= e^{tA^T} Q e^{tA} \Big|_0^\infty \\
 &= -Q
 \end{aligned}$$

$\square$

Let us now present the condition for us to test whether A is stable.

Theorem 14.2 (Necessary and Sufficient Condition). *The system $\dot{x} = Ax$ is asymptotically stable, if and only if for any positive definite Q , there is a unique positive definite solution P for the Riccati Equation (14.1).*

Proof. The “if” part is justified by the Lyapunov 2nd method. The “only if” part is justified by lemma 14.2. $\square$

The necessary and sufficient condition essentially implies that for linear systems, if we cannot find a quadratic Lyapunov function, then the system is unstable. This implication is not true for nonlinear systems. Moreover, to test the condition on linear systems, it suffices to set $Q = I$. That is because for any $Q = C^T C$ where C is invertible, we have

$$(AC^{-1})^T PC^{-1} + (PC^{-1})^T AC^{-1} + I = 0. \quad (14.4)$$

By similarity transform, we have

$$\underbrace{(CAC^{-1})^T}_{\tilde{A}} \underbrace{C^{-T}PC^{-1}}_{\tilde{P}} + \underbrace{(C^{-T}PC^{-1})^T}_{\tilde{P}} \underbrace{CAC^{-1}}_{\tilde{A}} + I = 0, \quad (14.5)$$

which is equivalent to the case where $Q = I$.

Theorem 14.3 (Stability of Infinite Horizon Continuous Time LQR). *Consider open-loop dynamics $\dot{x} = Ax + Bu$ where A and B are controllable. Cost function $J = \int_{t=0}^{\infty} [x^T Q x + u^T R u] dt$ where Q and R are positive definite. The closed-loop system with the LQR control law $u = -R^{-1} B^T P x$ is asymptotically stable; and the value function $V(x) = x^T P x$ can serve as a Lyapunov function, where P is the solution of the Riccati equation*

$$PA + A^T P - PBR^{-1}B^T P + Q = 0. \quad (14.6)$$

Proof. By definition, $V(x)$ satisfies the first, second, and fourth conditions of the Lyapunov 2nd method. Now we show that $\dot{V}$ is negative definite. The closed-loop dynamics is $\dot{x} = [A - BR^{-1}B^T P]x$. Hence

$$\begin{aligned} \dot{V}(x) &= x^T \{ [A - BR^{-1}B^T P]^T P + P[A - BR^{-1}B^T P] \} x \\ &= x^T \{ A^T P + PA - 2PBR^{-1}B^T P \} x \\ &= x^T \{ -Q - PBR^{-1}B^T P \} x. \end{aligned}$$

The last equality uses the Riccati equation for LQR. Using the control gain $K = R^{-1} B^T P$, we have $PBR^{-1}B^T P = K^T K$, hence

$$\dot{V}(x) = -x^T \{ Q + K^T K \} x, \quad (14.7)$$

which verifies the negative definiteness. Hence the infinite continuous time LQR is asymptotically stable where the value function serves as the Lyapunov function. $\square$

14.1.3 Lyapunov Analysis on Discrete Time Linear Systems

The previous discussions are for continuous time systems. For discrete time systems, we just need to change all differential conditions in theorem 14.1 to difference conditions. Consider a discrete time linear system $x_{k+1} = Ax_k$. Define $V(x_k) = x_k^T Px_k$, then the difference between two consecutive time steps is

$$V(x_{k+1}) - V(x_k) = x_k^T (A^T PA - P)x_k. \quad (14.8)$$

If we can find a positive definite matrix P such that $A^T PA - P$ is negative definite, the linear system is globally asymptotically stable. The Riccati equation in the discrete time Lyapunov analysis becomes

$$A^T PA - P + Q = 0. \quad (14.9)$$

In the following discussion, we list the counterparts of lemma 14.1, lemma 14.2, theorem 14.2, and theorem 14.5 in discrete time.

Lemma 14.3 (Lyapunov Summation). *For positive definite matrices P and Q that satisfy (14.9), we have*

$$P = \sum_{k=0}^{\infty} (A^T)^k Q A^k. \quad (14.10)$$

Lemma 14.4. *If A is asymptotically stable, then for any positive definite Q , there is an explicit formula for the Lyapunov function $V(x) = x^T Px$ where P satisfies (14.10).*

Theorem 14.4. *The system $x_{k+1} = Ax_k$ is asymptotically stable, if and only if for any positive definite Q , there is a unique positive definite solution P for the Riccati Equation (14.9).*

Theorem 14.5 (Stability of Infinite Horizon Discrete Time LQR).

Consider open-loop dynamics $x_{k+1} = Ax_k + Bu_k$ where A and B are controllable. Cost function $J = \sum_{k=0}^{\infty} [x_k^T Q x_k + u_k^T R u_k]$ where Q and R are positive definite. The closed-loop system with the LQR control law $u = -[B^T PB + R]^{-1} B^T P A x$ is asymptotically stable; and the value function $V(x) = x^T Px$ can serve as a Lyapunov function, where P is the solution of the Riccati equation

$$P = Q + A^T PA - A^T PB [B^T PB + R]^{-1} B^T PA. \quad (14.11)$$

Proof. By definition, $V(x)$ satisfies the first, second, and fourth conditions in Lyapunov 2nd method. Now we show that $V(x_{k+1}) - V(x_k)$ is negative definite. The closed-loop dynamics is $x_{k+1} = [A - BK]x_k$ where $K = [B^T PB + R]^{-1} B^T PA$. Moreover, we can rewrite the Riccati equation as

$$P = Q + A^T PA - A^T PBK. \quad (14.12)$$

Hence

$$\begin{aligned}
V(x_{k+1}) - V(x_k) &= x_k^T \{ (A - BK)^T P (A - BK) - P \} x_k \\
&= x_k^T \{ A^T P A - P - 2A^T P B K + K^T B^T P B K \} x_k \\
&= x_k^T \{ -Q - A^T P B K + K^T B^T P B K \} x_k \\
&= x_k^T \{ -Q - K^T [B^T P B + R] K + K^T B^T P B K \} x_k \\
&= -x_k^T \{ Q + K^T R K \} x_k.
\end{aligned}$$

The third equality uses the Riccati equation for LQR. The fourth equality uses the fact that $A^T P B = K^T [B^T P B + R]$.

It is now verified that $V(x_{k+1}) - V(x_k)$ is negative definite. Hence the infinite continuous time LQR is asymptotically stable where the value function serves as the Lyapunov function. $\square$

14.2 Synthesizing Lyapunov Function

If we can find a Lyapunov function, then we can prove the system is (asymptotically) stable. However, for nonlinear systems, if we cannot find a Lyapunov function, we usually cannot conclude that the system is unstable. Then the problem becomes: how can we synthesize the Lyapunov function?

According to theorem 14.1, to prove stability of $\dot{x} = f(x)$, we need to find a scalar function $V(x)$ that satisfies the four conditions. To efficiently do that, we need a parameterization of $V(x)$. Since $V(x)$ should be positive definite and radially unbounded, it is natural to parameterize $V(x)$ as a quadratic form

$$V(x) = x^T P(x) x, \quad P(x) = \sum_{i=1}^N P_i \rho_i(x), \quad (14.13)$$

where $\rho_i(x)$ are predefined basis functions and P_i are tunable parameters. Then we need to translate the conditions on V and $\dot{V}$ into conditions on P_i . The condition for positive definiteness and unboundedness can be written as

$$V(x) \geq c_1 \|x\|^2 \Rightarrow \sum_{i=1}^N x^T P_i x \rho_i(x) \geq c_1 \|x\|^2. \quad (14.14)$$

The condition for negative definiteness can be written as

$$\dot{V} \leq -c_2 V \Rightarrow \sum_{i=1}^N \left[x^T P_i x \dot{\rho}_i + f^T(x) (P_i^T + P_i) x \rho_i + c_2 x^T P_i x \rho_i \right] \leq 0. \quad (14.15)$$

Note that both constraints are linear in the parameters P_i , hence should be straight forward to optimize over. However, the problem is

that these conditions should hold for all x which has infinitely many possible values. To deal with this infinite cardinality, we can either discretize the state space or resort to sum of square programming.

The standard form of a sum of square programming problem is

$$\min_{u_i} \sum_{i=1}^n \alpha_i u_i \quad (14.16)$$

$$\text{s.t. } \beta_0(x) + \beta_1(x)u_1 + \cdots + \beta_n(x)u_n \in \text{SOS}, \quad (14.17)$$

where SOS is the class of polynomials that are sums of squares, e.g., there exist an integer d and a positive semidefinite matrix Q such that the function can be represented as

$$\begin{bmatrix} 1 \\ x \\ \vdots \\ x^d \end{bmatrix}^T Q \begin{bmatrix} 1 \\ x \\ \vdots \\ x^d \end{bmatrix}. \quad (14.18)$$

We can turn the constraints (14.14) and (14.15) into SOS form in (14.16), if we regard the parameters P_i as the decision variables u_i . Then these SOS programs can be translated into semidefinite programs (SDP) and solved by SDP solvers.²

² Available tools: SOSTOOLS.
<https://www.cds.caltech.edu/sostools/>
 Systems Polynomial Optimization Toolbox. <https://github.com/mposa/spot>.

14.3 Control with Lyapunov Function

Now let us consider a non-autonomous nonlinear control-affine system

$$\dot{x} = f(x) + g(x)u \quad (14.19)$$

where $f(0) = 0$ and $g(x) \neq 0$. How can we design the controller to make the system stabilize at the origin? Choose a scalar positive definite function $V(x)$. If we want to make this $V(x)$ a Lyapunov function for the closed loop system, we need to choose a control law that makes $\dot{V}$ negative definite, i.e.,

$$\dot{V} = \frac{\partial V}{\partial x} \dot{x} = \underbrace{\frac{\partial V}{\partial x} f(x)}_{L_f V} + \underbrace{\frac{\partial V}{\partial x} g(x) u}_{L_g V} \leq -\gamma(\|x\|). \quad (14.20)$$

The variables $L_f V$ and $L_g V$ are called Lie derivatives.

It is not guaranteed that we can make any positive definite V a Lyapunov function. For example, it is possible that $L_g V = 0$, i.e., no matter what the control input is, it will not change the value of $\dot{V}$. Even if $L_g V \neq 0$, it is possible that there are additional constraints on the control such that the inequality (14.20) cannot be met. We may also synthesize the Lyapunov together with the control policy using the SOS programming.

Example Consider the control of a unicycle. The state is $x = [p_x, p_y, v, \theta]^T$; the control is $u = [\dot{v}, \dot{\theta}]^T$. The dynamics of the unicycle can be written as

$$\dot{x} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix} u. \quad (14.21)$$

Let us define a positive definite function

$$V(x) = \frac{p_x^2 + p_y^2 + v^2}{2}. \quad (14.22)$$

The time-derivative of $V(x)$ is

$$\dot{V}(x) = p_x \dot{p}_x + p_y \dot{p}_y + v \dot{v} \quad (14.23)$$

$$= p_x v \cos \theta + p_y v \sin \theta + v \dot{v} \quad (14.24)$$

$$= (p_x \cos \theta + p_y \sin \theta + \dot{v})v. \quad (14.25)$$

To make $\dot{V}(x)$ negative definite, we can design the control law to be

$$\dot{v} = -p_x \cos \theta - p_y \sin \theta - k_v v \quad (14.26)$$

$$\dot{\theta} = k_\theta [\text{atan}(-p_y, -p_x) - \theta]. \quad (14.27)$$

Then $\dot{V}(x) = -k_v v^2 < 0$.

Note the Lyapunov function we defined is only positive semi-definite. The target state is $p_x = p_y = v = 0$ with any θ . $\dot{V}(x)$ is negative semi-definite. If $v = 0$, then $\dot{V}(x) = 0$, no matter what (p_x, p_y) is. However, the system will never be trapped in $\dot{V}(x) = 0$ before (p_x, p_y) reaches the origin under the control law. When $v = 0$, if $\dot{v}$ is none zero, the system will not be trapped in $\dot{V}(x) = 0$. When both $v = 0$ and $\dot{v} = 0$, i.e., $p_x \cos \theta + p_y \sin \theta = 0$, we show that $\dot{\theta} \neq 0$. Note that

$$\begin{aligned} & \cos(\text{atan}(-p_y, -p_x) - \theta) \\ &= \cos(\text{atan}(-p_y, -p_x)) \cos \theta + \sin(\text{atan}(-p_y, -p_x)) \sin \theta \\ &= -p_x \cos \theta - p_y \sin \theta \\ &= 0. \end{aligned}$$

Hence $\dot{\theta} \neq 0$. And once θ changes, the term $p_x \cos \theta + p_y \sin \theta$ may no longer be zero, which results in a non-zero $\dot{v}$ and the system will no longer be trapped in $v = 0$.

Passivity and Hyperstability

Hyperstability theory is another type of energy-based stability analysis method. It is developed by Vasile M. Popov, who was born in 1928, Romania and retired from University of Florida in 1993. Hyperstability theory is developed independent of Lyapunov theory. The specific energy notion used in hyperstability is the notion of *passivity*. In the following discussion, we will first introduce passivity, then discuss the main theorem in the hyperstability theory. Finally, we will use hyperstability theory to analyze the stability of parameter adaptation.

15.1 Passivity

Definition 15.1 (Continuous time passivity). *A continuous-time system with input v and output w is called passive if*

$$\int_0^{t_1} w^T(t)v(t)dt \geq -\gamma^2, \forall t_1 \geq 0, \quad (15.1)$$

where γ depends on the initial conditions.

The intuition behind the definition is that since $\int_0^{t_1} w^T(t)v(t)dt$ is the work/supply provided to the system, it should be bounded below. Passive systems should not always generate energy. Similarly, we can make a similar definition for discrete time systems.

Definition 15.2 (Discrete time passivity). *A discrete-time system with input v and output w is called passive if*

$$\sum_{k=0}^{k_1} w^T(k)v(k) \geq -\gamma^2, \forall k_1 \geq 0, \quad (15.2)$$

where γ depends on the initial conditions.

The two inequalities (15.1) and (15.2) are called *Popov Inequalities*. We can introduce the notion of strict passivity by making the inequalities more restrictive.

Definition 15.3 (Continuous time strict passivity). *A continuous-time system with input v and output w is called passive if*

$$\int_0^{t_1} w^T(t)v(t)dt \geq -\gamma^2 + \delta \int_0^{t_1} v^T(t)v(t)dt + \epsilon \int_0^{t_1} w^T(t)w(t)dt, \forall t_1 \geq 0 \quad (15.3)$$

where γ depends on the initial conditions; and $\delta, \epsilon \geq 0$ but not both zero.

Definition 15.4 (Discrete time strict passivity). *A discrete-time system with input v and output w is called passive if*

$$\sum_{k=0}^{k_1} w^T(k)v(k) \geq -\gamma^2 + \delta \sum_{k=0}^{k_1} v^T(k)v(k) + \epsilon \sum_{k=0}^{k_1} w^T(k)w(k), \forall k_1 \geq 0 \quad (15.4)$$

where γ depends on the initial conditions; and $\delta, \epsilon \geq 0$ but not both zero.

Passivity definition treats the system as a “black box”. We do not need to know the internal structure of the system. As long as the system does not continually generate energy, the system is passive. Passivity plays an important role in hyperstability, which will be illustrated below. There are some nice properties of passive systems. For example, if two systems are both passive, then the following parallel and feedback combination of these two systems are also passive. From an energy perspective, if the feedback system consists of only passive systems, the overall system will eventually be stable.

15.2 Hyperstability: Main Results

Hyperstability analyzes closed-loop systems with linear time invariant open loop dynamics and nonlinear feedback blocks. Such LTI-nonlinear feedback systems can be used to describe many dynamics. For example, in semi-gradient Q-learning, we can treat the LTI block as the parameter update from the TD error multiplying the gradient

$$w \leftarrow w + \alpha dw, dw = [L + \delta \min_u \hat{Q}_w(X', u) - \hat{Q}_w(X, U)] \nabla_w \hat{Q}_w(X, U),$$

while the nonlinear block as how w affects dw in the subsequent rounds. Note that w affects the action selection, hence nonlinearly affecting dw .

Next we introduce the formal definition of hyperstability and asymptotical hyperstability.

Definition 15.5 (Hyperstability). *The LTI-nonlinear feedback system is hyperstable if and only if there exist positive constants $\delta > 0$ and $\gamma > 0$ such that*

$$\|x(t)\| < \delta[\|x(0)\| + \gamma], \forall t > 0 \text{ or } \|x(k)\| < \delta[\|x(0)\| + \gamma], \forall k > 0 \quad (15.5)$$

For all nonlinear feedback blocks that satisfy the Popov inequality (15.1) or (15.2).

Definition 15.6 (Asymptotical hyperstability). *The LTI-nonlinear feedback system is asymptotically hyperstable if and only if 1) it is hyperstable; 2) for all bounded w satisfying the Popov inequality, we have $\lim_{k \rightarrow \infty} x(k) = 0$.*

Now the problem becomes how to check whether a system is hyperstable or asymptotically hyperstable. In the following discussion, we present the theorem first, then discuss the key concepts in the theorem.

Theorem 15.1 (Hyperstability). *The LTI-nonlinear feedback system is hyperstable if and only if 1) the nonlinear block satisfies Popov inequality (i.e., it is passive); and 2) the LTI transfer function is positive real.*

Theorem 15.2 (Asymptotical hyperstability). *The LTI-nonlinear feedback system is asymptotically hyperstable if and only if 1) the nonlinear block satisfies Popov inequality (i.e., it is passive); and 2) the LTI transfer function is strictly positive real.*

We are not going to show how these theorems are proved, but will provide intuitions on understanding the theorems and illustrate how to use these theorems. But before we illustrate the usage, you may notice that there are two more concepts: positive real and strictly positive real. These are two very important concepts for LTI systems.

Definition 15.7 (Continuous time positive real systems). *A single-input-single-output (SISO) transfer function $G(s)$ is called positive real (PR) if 1) $G(s)$ is real for real values of s ; and 2) $\operatorname{Re}(G(s)) > 0, \forall \operatorname{Re}(s) > 0$.*

Equivalently, to check whether $G(s)$ is PR, we just need to check the following conditions:

- $G(s)$ does not possess any pole in $\operatorname{Re}(s) > 0$ (no unstable poles);
- Any pole on the imaginary axis jw_0 does not repeat and the associated residue $\lim_{s \rightarrow jw_0} (s - jw_0)G(s)$ is non-negative;
- $\forall w \in \mathbb{R}$ where $s = jw$ is not a pole of $G(s)$, $G(jw) + G(-jw) = 2\operatorname{Re}(G(jw)) \geq 0$.

Definition 15.8 (Continuous time strictly positive real systems). *A single-input-single-output (SISO) transfer function $G(s)$ is strictly positive real (SPR) if the following conditions are satisfied*

- $G(s)$ does not possess any pole in $\operatorname{Re}(s) \geq 0$;

- $\forall w \in \mathbb{R}, G(jw) + G(-jw) = 2\text{Re}(G(jw)) > 0$.

PR and SPR contain stronger notions than merely stability (i.e., the poles being on the negative half plane). For example, consider the following two systems:

- System 1: $\dot{y} = -y + u$ where $G_1(s) = (s + 1)^{-1}$;
- System 2: $\dot{y} = -y - u$ where $G_2(s) = -(s + 1)^{-1}$.

Both systems are stable since their eigen values are both -1 . The only difference is the sign before the control u . Due to that difference, system 1 is strictly positive real since $G_1(jw) + G_1(-jw) = 1$; while system 2 is not positive real since $G_2(jw) + G_2(-jw) = -1$.

Similarly, we can define the notion of positive real and strictly positive real for discrete time systems.

Definition 15.9 (Discrete time positive real systems). *A single-input-single-output (SISO) transfer function $G(z)$ is called positive real (PR) if the following conditions are satisfied*

- It does not possess any pole in outside the unit circle;
- Any pole on the unit circle does not repeat and the associated residue is non-negative;
- $\forall |w| \leq \pi$ where $z = e^{jw}$ is not a pole of $G(z)$, $G(e^{jw}) + G(e^{-jw}) = 2\text{Re}(G(e^{jw})) \geq 0$.

Definition 15.10 (Discrete time strictly positive real systems). *$G(z)$ is strictly positive real (SPR) if the following conditions are satisfied*

- $G(z)$ does not possess any pole outside of on the unit circle;
- $\forall |w| \leq \pi, G(e^{jw}) + G(e^{-jw}) = 2\text{Re}(G(e^{jw})) > 0$.

Why is the concept positive real important? That is because it implies passivity.

Lemma 15.1 (Passivity of LTI systems). *Consider an LTI system $\dot{x} = Ax + bu, y = Cx + Du$. It is passive if $G(s) = C(sI - A)^{-1}B + D$ is positive real. It is strictly passive if $G(s)$ is strictly positive real.*

Analogous results hold for discrete-time systems. And the lemma explains why we need the LTI system to be PR. Essentially, to ensure hyperstability, we want both the LTI block and the nonlinear feedback block in the closed-loop system to be passive. Popov inequality ensures passivity of the nonlinear feedback block, while PR ensures passivity of the LTI block. Let us further consider the following example to better understand the hyperstability theorem.

Example Consider the following mass-spring-damper system as shown in fig. 15.1:

$$m\ddot{p} + b\dot{p} + kp = u \quad (15.6)$$

$$v = \dot{p}. \quad (15.7)$$

Hence the transfer function is

$$G_{u \rightarrow v}(s) = \frac{s}{ms^2 + bs + k}. \quad (15.8)$$

According to definition 15.8, $G_{u \rightarrow v}(s)$ is strictly positive real.

Suppose the control u depends nonlinearly on the measurement v . The block diagram for this systems is shown in fig. 15.2.

Note that $\int_0^{t_1} u(t)v(t)dt$ is the total energy supplied to the system. Suppose the nonlinear feedback block satisfies the Popov inequality

$$\int_0^{t_1} w^T(t)v(t)dt \geq -\gamma^2, \forall t_1 \geq 0. \quad (15.9)$$

Then from the fact that $u(t) = -w(t)$, the energy supplied to the system is bounded by $\int_0^{t_1} u^T(t)v(t)dt \leq \gamma^2, \forall t_1 \geq 0$. According to conservation of energy,

$$mv_t^2 + kp_t^2 - mv_0^2 - kp_0^2 = 2 \int_0^{t_1} u(t)v(t)dt \leq \gamma^2, \quad (15.10)$$

where $\frac{1}{2}(mv_t^2 + kp_t^2)$ is the system energy at time t . Let the system state be $x = [p; v]$. The conservation of energy implies that the state is bounded.

The intuition behind the example is that:

- The nonlinear block satisfying Popov inequality assures bounded supply to the LTI system.
- Based on the conservation of energy, the energy of the LTI system is bounded.
- When the energy function is positive definite with respect to the states, then the states will be bounded.
- Moreover, if the LTI system is strictly passive, it consumes energy. The bounded supply will eventually be all consumed. Then the states will converge to the equilibrium.

15.3 Hyperstability of Parameter Adaptation

In the following discussion, we are going to show how to use the hyperstability theorem to analyze adaptation systems. The general procedure in the analysis is summarized below.

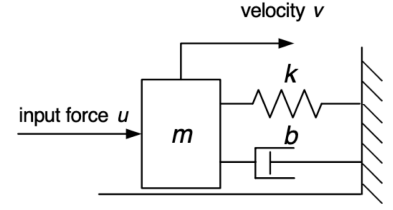


Figure 15.1: Illustration of the mass-spring-damper system.

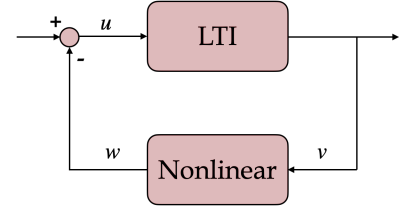


Figure 15.2: Illustration of the mass-spring-damper system.

- Step 1: translate the adaptation algorithm to a feedback combination of a LTI block and a nonlinear block.
- Step 2: verify that the feedback block satisfies the Popov inequality.
- Step 3: check that the LTI block is strictly positive real.
- Step 4: show that the output of the feedback block is bounded.
Then from the definition of asymptotic hyperstability, we conclude that the state (the parameter that is being adapted) converge.

Let us now try out this procedure with Semi-gradient TD(o), an algorithm to learn the parameterized Q values for a fixed policy using bootstrapping. The parameter update rule is

$$W_{k+1} = W_k + \alpha [L_k + \delta \hat{Q}_W(X_{k+1}, U_{k+1}) - \hat{Q}_W(X_k, U_k)] \nabla_w \hat{Q}_W(X_k, U_k). \quad (15.11)$$

We have the following assumptions to simplify the analysis. First, assume the underlying policy is deterministic, i.e., $U_k = \pi(X_k)$. Second, the ground truth Q value is $Q_\pi(X_k, U_k) = \phi_k^T W^*$ where ϕ_k includes features regarding X_k and U_k ; and the parameterized Q function is $\hat{Q}_w = \phi_k^T W$. For example, in LQR, we can write the SoS parameterization of the Q function as $\hat{Q}_W(x, u) = [x^2; u^2; 2xu]W$. Now we are going to use hyperstability theorem to show that the learning is stable and converging under a specific learning gain (to be derived later). Note Semi-gradient TD(o) only uses samples to perform policy evaluation, hence is a prediction problem, not yet a control problem. And we are investigating the stability of the adaptation loop, not the stability of the adaptation plus the control loop.

Step 1: Transforming the system Since we are going to analyze the stability of the adaptation loop, we use focus the error terms: parameter estimation error $\tilde{W}_k := W^* - W_k$ and the TD error $E_k = L_k + \delta \hat{Q}_W(X_{k+1}, U_{k+1}) - \hat{Q}_W(X_k, U_k)$. Let us first rewrite the TD error as a function of the parameter error

$$E_k = L_k + \delta \hat{Q}_W(X_{k+1}, U_{k+1}) - \hat{Q}_W(X_k, U_k) \quad (15.12a)$$

$$= Q_\pi(X_k, U_k) - \delta Q_\pi(X_{k+1}, U_{k+1}) + \delta \hat{Q}_{W_k}(X_{k+1}, U_{k+1}) - \hat{Q}_{W_k}(X_k, U_k) \quad (15.12b)$$

$$= \phi_k^T \tilde{W}_k - \delta \phi_{k+1}^T \tilde{W}_k. \quad (15.12c)$$

Then by subtracting W^* on both sides of (15.11), we have

$$\tilde{W}_{k+1} = \tilde{W}_k - \alpha \phi_k E_k \quad (15.13)$$

With these two equations, we can construct the following LTI-nonlinear feedback system as shown in fig. 15.3. The LTI block is just an identify (in order for it to be positive real). The nonlinear feedback block contains (15.12) and (15.13). However, since these two update rules are run sequentially, we must include a one-step delay in the block diagram if we term them into a closed-loop feedback system. The one-step delay cannot be included in the LTI block since z^{-1} is not positive real. Then the one-step delay has to be included in the nonlinear block. However, there is no free lunch, with the one-step delay, it will be difficult to prove Popov inequality for the nonlinear block since E_k does not directly affect Y_k in the current representation, which implies that $(-E_k)Y_k$ can be arbitrarily small, violating the Popov inequality.

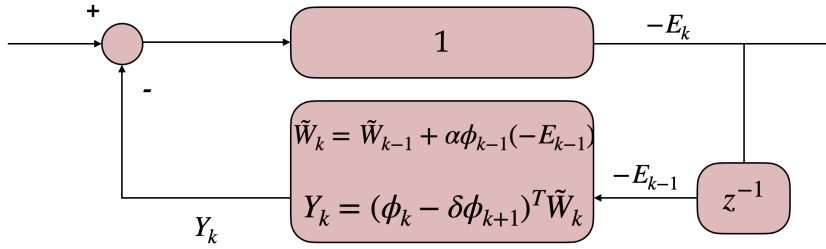


Figure 15.3: The LTI-feedback system for Semi-Gradient TD(o). The feedback block contains a delay.

To absorb the one step delay, we need to turn the a priori TD error into the a posteriori error, a trick that we have used multiple times to show stability of parameter adaptation. Since the a priori error is

$$E_k = (\phi_k - \delta\phi_{k+1})^T \tilde{W}_k, \quad (15.14)$$

we define the a posteriori error as

$$E_k^* = (\phi_k - \delta\phi_{k+1})^T \tilde{W}_{k+1} \quad (15.15a)$$

$$= (\phi_k - \delta\phi_{k+1})^T \tilde{W}_k - \alpha(\phi_k - \delta\phi_{k+1})^T \phi_k E_k \quad (15.15b)$$

$$= (1 - \alpha(\phi_k - \delta\phi_{k+1})^T \phi_k) E_k \quad (15.15c)$$

where the second equality used (15.13) and the last equality used the expression of the a priori error again.

With the a posteriori error E_k^* , we should also rewrite the parameter error dynamics to be a function of E_k^* instead of E_k . Starting from (15.13), we have

$$\tilde{W}_{k+1} = \tilde{W}_k - \alpha\phi_k E_k \quad (15.16a)$$

$$= \tilde{W}_k - \frac{\alpha\phi_k}{1 - \alpha(\phi_k - \delta\phi_{k+1})^T \phi_k} E_k^*, \quad (15.16b)$$

where the last equality uses (15.15). However, now the expressive before E_k^* is quite messy. Let us choose a learning gain α to simplify the expression. The goal is to select α such that

$$\frac{\alpha \phi_k}{1 - \alpha(\phi_k - \delta\phi_{k+1})^T \phi_k} = F(\phi_k - \delta\phi_{k+1}) \quad (15.17)$$

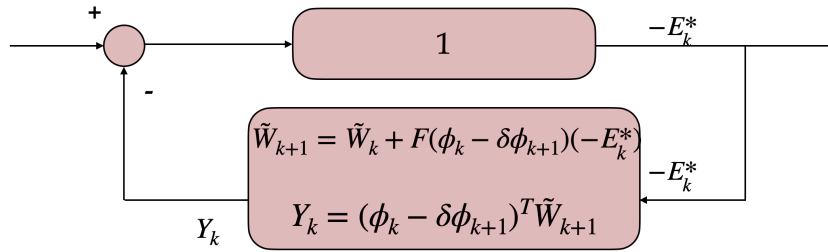
where F is the new scalar constant learning gain with respect to the a posteriori error. The advantage of this arrangement will show up when we prove Popov inequality. From (15.17), we can compute that¹

$$\alpha = \frac{F\phi_k^T(\phi_k - \delta\phi_{k+1})}{\phi_k^T \phi_k + F[(\phi_k - \delta\phi_{k+1})^T \phi_k]^2}. \quad (15.18)$$

Now the parameter dynamics become

$$\tilde{W}_{k+1} = \tilde{W}_k - F(\phi_k - \delta\phi_{k+1})E_k^*. \quad (15.19)$$

The transformed dynamics are shown in fig. 15.4. Due to the use of the a posteriori error, there is no delay in the feedback block any more.



¹ From (15.17), we first have $\alpha \phi_k = F(\phi_k - \delta\phi_{k+1}) - \alpha F(\phi_k - \delta\phi_{k+1})(\phi_k - \delta\phi_{k+1})^T \phi_k$. By multiplying ϕ_k^T on both sides, we have $\alpha \phi_k^T \phi_k = F\phi_k^T(\phi_k - \delta\phi_{k+1}) - \alpha F[(\phi_k - \delta\phi_{k+1})^T \phi_k]^2$. After rearranging the expression, we get to .

Figure 15.4: The alternative LTI-feedback system for Semi-Gradient TD(o). Since we are using the a posteriori error, there is no delay in the feedback block.

Remark 15.1. We used two tricks to transform the system. The first is to consider a posteriori error dynamics to remove the one-step delay. Usually the one-step delay will make the system not passive. The second is to select a unique learning gain for α . Semi-gradient TD(o) may not always be stable; but we will show that the selected learning gain will ensure stability and convergence.

Step 2: Verifying Popov Inequality The goal of this step is to show that $\sum_{k=0}^{k_1} Y_k^T(-E_k^*) \geq -\gamma^2, \forall k_1 \geq 0$. Since $Y_k = (\phi_k - \delta\phi_k)^T \tilde{W}_{k+1}$, we have

$$\sum_{k=0}^{k_1} Y_k^T(-E_k^*) = \sum_{k=0}^{k_1} \tilde{W}_{k+1}^T(\phi_k - \delta\phi_{k+1})(-E_k^*) \quad (15.20a)$$

$$= \sum_{k=0}^{k_1} \tilde{W}_{k+1}^T F^{-1}(\tilde{W}_{k+1} - \tilde{W}_k) \quad (15.20b)$$

where the second equality uses (15.19). Since we picked the unique α , now $\sum_{k=0}^{k_1} Y_k^T(-E_k^*)$ only depends on the parameter error $\tilde{W}_k$ not the dynamic features ϕ_k . And we will show that $\sum_{k=0}^{k_1} \tilde{W}_{k+1}^T F^{-1}(\tilde{W}_{k+1} - \tilde{W}_k)$ is uniformly bounded below for any k_1 .

$$\sum_{k=0}^{k_1} \tilde{W}_{k+1}^T F^{-1}(\tilde{W}_{k+1} - \tilde{W}_k) \quad (15.21a)$$

$$= \sum_{k=0}^{k_1} [\tilde{W}_{k+1}^T F^{-1} \tilde{W}_{k+1} - \frac{1}{2} \tilde{W}_k^T F^{-1} \tilde{W}_k + \frac{1}{2} \tilde{W}_k^T F^{-1} \tilde{W}_k - \tilde{W}_{k+1}^T F^{-1} \tilde{W}_k] \quad (15.21b)$$

$$= \sum_{k=0}^{k_1} \frac{1}{2} [\tilde{W}_{k+1}^T F^{-1} \tilde{W}_{k+1} - \tilde{W}_k^T F^{-1} \tilde{W}_k] + \sum_{k=0}^{k_1} \frac{1}{2} [\tilde{W}_{k+1}^T F^{-1} \tilde{W}_{k+1} - 2\tilde{W}_{k+1}^T F^{-1} \tilde{W}_k + \tilde{W}_k^T F^{-1} \tilde{W}_k] \quad (15.21c)$$

$$\geq \sum_{k=0}^{k_1} \frac{1}{2} [\tilde{W}_{k+1}^T F^{-1} \tilde{W}_{k+1} - \tilde{W}_k^T F^{-1} \tilde{W}_k] \quad (15.21d)$$

$$= \frac{1}{2} [\tilde{W}_{k_1+1}^T F^{-1} \tilde{W}_{k_1+1} - \tilde{W}_0^T F^{-1} \tilde{W}_0] \quad (15.21e)$$

$$\geq -\frac{1}{2} \tilde{W}_0^T F^{-1} \tilde{W}_0 \quad (15.21f)$$

The inequality is due to the fact that $\tilde{W}_{k+1}^T F^{-1} \tilde{W}_{k+1} - 2\tilde{W}_{k+1}^T F^{-1} \tilde{W}_k + \tilde{W}_k^T F^{-1} \tilde{W}_k = [\tilde{W}_{k+1} - \tilde{W}_k]^T F^{-1} [\tilde{W}_{k+1} - \tilde{W}_k] \geq 0$.

Hence we have shown that the nonlinear feedback block satisfies the Popov inequality.

$$\sum_{k=0}^{k_1} Y_k^T(-E_k^*) \geq -\frac{1}{2} \tilde{W}_0^T F^{-1} \tilde{W}_0, \forall k_1 \geq 0 \quad (15.22)$$

Step 3: Checking SPR of the LTI Block Since the LTI block is defined to be identity, it is then strictly positive real.

Now from the first three steps, we can conclude from theorem 15.2 that the adaptation system is asymptotically hyperstable, which means that the output of the system $-E_k^*$ is bounded. However, will E_k^* converge to zero? This will answered in step 4.

Step 4: Boundness and convergence Since $-E_k^*$ is bounded, then $Y_k = E_k^*$ is also bounded. By definition of asymptotical hyperstability in definition 15.6, we have $\lim_{k \rightarrow \infty} E_k^* = 0$. Hence now we have shown that the learning in semi-gradient TD(o) is stable; and the a posteriori error converges to zero.

Another way to show convergence without invoking the asymptotical hyperstability definition and theorem is to directly consider the LTI block in the Popov inequality. The LTI block ensures that

$Y_k = E_k^*$. Hence according to (15.22), we have

$$\sum_{k=0}^{k_1} E_k^{*2} = - \sum_{k=0}^{k_1} Y_k^T (-E_k^*) \leq \frac{1}{2} \tilde{W}_0^T F^{-1} \tilde{W}_0, \forall k_1 \geq 0 \quad (15.23)$$

In order for this inequality to hold, we must have $\lim_{k \rightarrow \infty} E_k^* = 0$.

Conclusion If we use semi-gradient TD(o) to learn $Q_\pi(X_k, U_k) = \phi_k^T W^*$ for a deterministic policy (with the special form of α), the a posteriori error $E_k^* = (\phi_k - \delta \phi_{k+1})^T \tilde{W}_{k+1}$ converges to zero. It neither implies that the parameter error converges to zero, nor implies that the TD error converges to zero.

The tricks that we used in the proof can be used to prove stability of other adaptation algorithms in prediction problems. For example, can you use a similar procedure to show that RLS is stable? To extend these methods to show stability and convergence of adaptation algorithms in control problems will be much more trickier. That is because now the dynamic features ϕ_k will then depend on the parameter error $\tilde{W}_k$. The derivation will be more complex.

Safe Learning and Adaptation

To ensure safe learning, we first need to understand what “safety” implies. In certain cases, we can explicitly define safety as satisfying a set of constraints. That mathematical definition is called as a safety specification. For example, if we want to ensure that the agent does not collide with any obstacle in the environment, the safety specification can be: the distance between the agent to any obstacle in the environment is greater than zero. However, in some other cases, we may not have the safety specification in the design phase. The agent needs to interact with the environment in order to figure out what “safety” implies. In the previous example, this situation corresponds to the case that the agent does not know colliding with obstacles is bad beforehand, but receives a huge run-time penalty once it collides with an obstacle. Then the agent should be able to infer from the experience that such behavior is unsafe and be sure to behave safely in the following episodes.

In the following discussion, we will first introduce safety specifications and reachability analysis, then discuss methods for safe learning with known specifications, followed by safe learning with unknown specifications.

16.1 Safety Specification

Safe Set

In most cases, safety is specified as a requirement that the system state should be constrained in a set $\mathcal{X}_S$ called the safe set. For example, in the collision avoidance case, the state of the agent should be constrained in a set of collision-free states. With the notion of the safe set, there are two important problems to solve.

- Problem 1 (Forward Invariance):

For $x(t_0) \in \mathcal{X}_S$, how to choose $u(t)$ such that $x(t) \in \mathcal{X}_S$ for all $t > t_0$?

- Problem 2 (Finite Time Convergence):

For $x(t_0) \notin \mathcal{X}_S$, how to choose $u(t)$ such that $x(t) \in \mathcal{X}_S$ for some finite $t > t_0$?

The first problem is to ensure that trajectories start from the safe set should remain in the safe set. The second problem is to ensure that trajectories start outside the safe set should go back to the safe set in finite time.

These two problems may seem trivial, in fact so trivial that there exists a straight-forward strategy to enforce safety:

- Do nothing when the current state is inside the safe set;
- Choose a control such that $\dot{x}$ is pointing to the interior of the safe set if the current state is on the boundary of the safe set;
- Choose a control such that $\dot{x}$ is pointing to the closest boundary of the safe set if the current state is outside the safe set.

However, it is actually not that trivial to implement this strategy on a dynamic system. There may be measurement noise such that the measurement of the current state is inaccurate. There may be constraints on control that the desired control is not attainable. There may be dynamic noises or model uncertainties that the executed $\dot{x}$ may deviates from the desired $\dot{x}$. The actual challenge is how to make the simple strategy work on real world problems.

Safety Index

To better distinguish between the states inside and outside the safe set, we can define a scalar function $\phi : \mathcal{X} \rightarrow \mathbb{R}$ such that the states in $\mathcal{X}_S$ are mapped to non-positive values; and states not in $\mathcal{X}_S$ are mapped to positive values. The scalar function can be called as a safety index, a potential function, or a barrier function. It works in a way very similar to the Lyapunov function. The analogy will be shown in the following discussion. We call this scalar function as a safety index. With the safety index, we can turn the forward-invariance constraint from $x \in \mathcal{X}_S$ to $\phi(x) \leq 0$. Nonetheless, depending on the type of system dynamics, we may enforce the constraint in different ways:

- Worst case situation, where we require $\max_t \phi(x(t)) \leq 0$. This constraint only works if the disturbance is bounded. Otherwise, the inequality will be ill-posed.
- Adversarial situation, where we model the disturbance w as an adversary in the dynamics $\dot{x} = f(x, u, w)$ and require that

$\max_w \min_u \phi(x) \leq 0$. The adversarial constraint can be viewed as an lower bound of worst case constraint, *i.e.*, $\max_w \min_u \phi(x) \leq \max_t \phi(x(t))$.

- Stochastic situation, where we require that the probability of violating the safety constraint is very small $\mathbb{P}[\phi(x) > 0] < \epsilon$. This constraint works for the case that the disturbance is unbounded.

16.2 Safety Index and Reachability Analysis

Given a safe set, there are multiple ways to construct the safety index. We may define the safety index as a signed distance function to the boundary of the safe set. For example, in $\mathbb{R}^2$, if the safe set is defined as $\mathcal{X}_S := \{x \mid \|x\| \geq 1\}$, then the safety index can be define as $\phi(x) = 1 - \|x\|$. Although this definition is easy to quantify the geometric distance to the safe set, it ignores the system dynamics. For example, if $\dot{x} = [-1; 0]$ at the following two points $x_1 = [2; 0]$ and $x_2 = [-2; 0]$, then x_1 is more dangerous than x_2 , since the system is getting “into” the unsafe set at x_1 while the system is getting “out of” the unsafe set at x_2 . The values of the signed distance function at these two points are both -1 , which is not able to create proper distinctions. In order to resolve this problem, we can synthesize the safety index using reachability analysis.

16.2.1 Forward Reachability vs Backward Reachability

Consider a dynamic system $\dot{x} = f(x)$. The forward reachable set $\mathcal{R}(f, \mathcal{X}_0)$ from a set of initial states $\mathcal{X}_0$ is defined as a set of states that can be reached along the system trajectories from the initial states, *i.e.*,

$$\mathcal{R}(f, \mathcal{X}_0) := \{x \mid \exists x_0 \in \mathcal{X}_0, t \geq 0, \text{ s.t. } x = x(t), x(0) = x_0, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [0, t]\}. \quad (16.1)$$

The system is safe if the forward reachable set is separated from the unsafe set.

The backward reachable set $\mathcal{B}(f, \mathcal{T})$ from a target set $\mathcal{T}$ (usually chosen as the unsafe set, *i.e.*, $\mathcal{T} := \mathcal{X} \setminus \mathcal{X}_S (= \mathcal{X}_S^c)$) is defined as a set of states such that the system will enter the target set if the system start on those states and move along the system trajectories, *i.e.*,

$$\mathcal{B}(f, \mathcal{T}) := \{x \mid \exists x^* \in \mathcal{T}, t \geq 0, \text{ s.t. } x = x(-t), x(0) = x^*, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [-t, 0]\}. \quad (16.2)$$

The system is safe if and only if the initial states are out of the backward reachable set of the unsafe set.

The previous definitions do not depend on time. We can also define time-dependent reachable sets.

- Forward reachable set at time t is the set of states that can be reached at time t .

$$\mathcal{R}_t(f, \mathcal{X}_0) := \{x \mid \exists x_0 \in \mathcal{X}_0, \text{ s.t. } x = x(t), x(0) = x_0, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [0, t]\}. \quad (16.3)$$

- Backward reachable set at time t is the set of states that will end up in the target set at t time ahead.

$$\mathcal{B}_t(f, \mathcal{T}) := \{x \mid \exists x^* \in \mathcal{T}, \text{ s.t. } x = x(-t), x(0) = x^*, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [-t, 0]\}. \quad (16.4)$$

- Forward reachable set within time t is the set of states that can be reached within time t .

$$\mathcal{R}_{\leq t}(f, \mathcal{X}_0) := \{x \mid \exists x_0 \in \mathcal{X}_0, l \in [0, t], \text{ s.t. } x = x(l), x(0) = x_0, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [0, l]\}. \quad (16.5)$$

- Backward reachable set within time t is the set of states that will end up in the target set within t time ahead.

$$\mathcal{B}_{\leq t}(f, \mathcal{T}) := \{x \mid \exists x^* \in \mathcal{T}, l \in [0, t], \text{ s.t. } x = x(-l), x(0) = x^*, \dot{x}(\tau) = f(x(\tau)) \forall \tau \in [-l, 0]\}. \quad (16.6)$$

Example In $\mathbb{R}^2$, consider a set $\mathcal{S} = \{x \mid \|x\| \leq 1\}$ and the dynamics $f(x) = [-1; 0]$. Note we take L2 norm in $\|x\|$. The forward reachable sets can be computed as

$$\mathcal{R}_t(f, \mathcal{S}) = \{x \mid \|x + t\| \leq 1\} \quad (16.7)$$

$$\mathcal{R}_{\leq t}(f, \mathcal{S}) = \bigcup_{\tau \in [0, t]} \{x \mid \|x + \tau\| \leq 1\} \quad (16.8)$$

$$\mathcal{R}(f, \mathcal{S}) = \bigcup_{\tau \geq 0} \{x \mid \|x + \tau\| \leq 1\}. \quad (16.9)$$

Similarly, the backward reachable sets can be computed as

$$\mathcal{B}_t(f, \mathcal{S}) = \{x \mid \|x - t\| \leq 1\} \quad (16.10)$$

$$\mathcal{B}_{\leq t}(f, \mathcal{S}) = \bigcup_{\tau \in [0, t]} \{x \mid \|x - \tau\| \leq 1\} \quad (16.11)$$

$$\mathcal{B}(f, \mathcal{S}) = \bigcup_{\tau \geq 0} \{x \mid \|x - \tau\| \leq 1\}. \quad (16.12)$$

Note that $x_2 = [-2; 0]$ does not lie in any backward reachable set, while $x_1 = [2; 0]$ does lie in the backward reachable set $\mathcal{B}_{\leq t}(f, \mathcal{S})$ for $t \geq 1$. To incorporate reachability, we may define the safety

index ϕ^* as the negative time it takes for a state to enter the unsafe set. Hence, $\phi^*(x_1) = -1$ and $\phi^*(x_2) = -\infty$, which provides a better measurement of safety than the signed distance function since the distances from x_1 and x_2 to the unsafe set are the same. In the following discussion, we introduce formal ways to construct such safety index.

16.2.2 Hamilton Jacobi Formulation of Reachability

In the following discussion, we mainly use the backward reachability. Forward reachability can fit into the framework by taking the inverse of time.

Suppose the target set in the backward reachability analysis satisfies $\psi(x) \leq 0$. The scalar function ψ is not a safety index and it may be chosen as the signed distance function to the boundary of the actual target set. For a non-autonomous system $\dot{x} = f(x, w)$, $w \in \mathcal{W}$ tries to drive the system to the target set. If the target set is the unsafe set, w can be understood as a disturbance. If the target set is the safe set, w can be understood as the control input we can choose to maintain safety. In either case, we want to quantify how likely it is for different states to eventually enter the target set under w . We can do that by defining the following value function, which evaluates the minimum achievable value of ψ at time 0 if we start from x at time $t \leq 0$:

$$V(x, t) = \min_{w \in \mathcal{W}} \psi(x(0)). \quad (16.13)$$

The reason we call it as a value function is because we can view the optimization as an optimal control problem with zero run-time cost and a non-zero terminal cost. By Bellman principle, we have

$$V(x(t), t) = \min_{w \in \mathcal{W}} V(x(t+dt), t+dt) \quad (16.14)$$

$$= \min_{w \in \mathcal{W}} V(x(t), t) + \dot{V}(x(t), t)dt + \nabla_x V(x(t), t)dx, \quad (16.15)$$

which implies the following HJB equation

$$\dot{V}(x(t), t) + \min_{w \in \mathcal{W}} \nabla_x V(x(t), t)f(x, w) = 0. \quad (16.16)$$

The boundary condition is $V(x, 0) = \psi(x)$. The HJB equation may not have a smooth solution. But it will always have a viscosity solution.

How can we use the value function in practice? For a given time t , the sub-level set $V(\cdot, t) \leq 0$ is the backward reachable set within time t . If the target set is the unsafe set, then we can choose $-V(\cdot, t)$ as our safety index where different choices of t corresponds to different safety margins. The safety index tells us the range of dangerous

regions that once entered, there exists some disturbance to drive the system into the unsafe set. If the target set is the safe set, then we can evaluate whether there exist a control law that ensures forward invariance, *i.e.*, $V(x, t) \leq 0$ for $\psi(x) \leq 0$ and all t , and finite time convergence, *i.e.*, there exists a finite t such that $V(x, t) \leq 0$ for any $\psi(x) > 0$.

16.2.3 Hamilton Jacobi Isaacs (HJI) Formulation of Reachability

We can extend the analysis to systems with multiple inputs. Consider $\dot{x} = f(x, u, w)$ where u wants to drive the system away from the target set and w wants to drive the system into the target set. This situation may arise in robust control where one of the input is the control signal and the other is the disturbance. The situation may also be understood as a dynamic zero-sum game as the objective of the two inputs are opposite. In this case, we can define a value function similar to the previous case and obtain a partial differential equation:

$$\dot{V}(x(t), t) + \max_{u \in \mathcal{U}} \min_{w \in \mathcal{W}} \nabla_x V(x(t), t) f(x, u, w) = 0. \quad (16.17)$$

The HJI equation may not have a smooth solution. But it will have a viscosity solution.

How can we use the value function in practice? For a given time t , the sub-level set $V(\cdot, t) \leq 0$ is the backward reachable set within time t . If the target set is the unsafe set, the value function tells us how best the control can do in terms of safety when we encounter worst case disturbances. We can choose $-V(\cdot, t)$ as our safety index where different choices of t corresponds to different safety margins.

16.3 Safe Learning with Known Specifications

16.3.1 Notion of safety during learning

During learning, there are two different notions of safety. The first is that the learned control law should be safe regardless of what happens during the learning process. The second is that the system should stay safe during the learning process. Mathematically, if we denote the parameterized control policy as π_θ , the learning process generates a sequence of θ_k which converges to the final θ^* . The first safety notion requires π_{θ^*} to be safe, while the second safety notion requires π_{θ_k} to be safe for all k .

The first safety notion is often used in offline learning, while the second safety notion is often used in online learning. If we know the safety specification, we can enforce the first safety notion through

cost shaping (or reward shaping), *i.e.*, penalizing the action that leads to unsafe states and gradually increasing the penalty until the policy converges to a safe policy. This method is similar to the interior point method for optimization. For online learning, if we know the safety specification, we may embed a safety monitor in the policy ϕ_{θ_k} to ensure that all control actions are safe. In the following discussion, we will discuss how to design and implement such a safety monitor. In literature, this safety monitor has been called as safety-shield, safety layer, etc.

16.3.2 Safety index and safe control

The safety specifications we defined earlier are all on the state space. However, to test whether a control policy is safe or not, we need a specification on the control space. Nonetheless, we can turn the state-space specification into a control-space specification without redefining the safety notion.

In the state space, we have the safety index ϕ that indicates safety level of different states. Note that ϕ is not a function of time, which can be identified as $V(x, t)$ when $t \rightarrow \infty$ in the reachability analysis. The time derivative of the safety index $\dot{\phi}$ is

$$\dot{\phi} = \frac{\partial \phi}{\partial x} \dot{x} = \frac{\partial \phi}{\partial x} f(x, u), \quad (16.18)$$

which depends on the control strategy. Hence ϕ can be used to judge whether a control input is safe or not.

Essentially, a control law solves problem 1 (forward invariance) if it satisfies

$$\dot{\phi}(x) \leq 0, \forall x \in \{x : \phi(x) = 0\}. \quad (16.19)$$

A control law solves problem 2 (finite convergence) if it satisfies

$$\dot{\phi}(x) < -\eta, \forall x \in \{x : \phi(x) > 0\}, \quad (16.20)$$

for some constant $\eta > 0$. The time it takes for an unsafe state to go back to the safe set is bounded by $\phi(x)/\eta$. In the following discussion, we call (16.19) the *invariance condition* and (16.20) the *convergence condition*. Note these conditions are only sufficient conditions for forward invariance and finite convergence, not necessary conditions, meaning that they may be overly conservative than it needs to be.

16.3.3 Safety index and Lyapunov function

The way we use the safety index to obtain the constraints on control is similar to how we use the Lyapunov function. Lyapunov functions are scalar functions that are used to prove the stability of an equilibrium of an ODE, which needs to satisfy the following conditions:

- $V(x) \geq 0$ and $V(x) = 0$ if and only if $x = 0$ (where we assume that 0 is the equilibrium point);
- $\dot{V}(x) < 0$ for $x \neq 0$;
- $\dot{V}(0) = 0$.

A safety index is used to prove forward invariance and finite convergence of a set. There exists a one-to-one correspondence between the conditions that the Lyapunov function needs to satisfy and the conditions that the safety index needs to satisfy. The first condition (positive definiteness) for the Lyapunov function corresponds to the requirement that $\{x : \phi(x) \leq 0\} \subseteq \mathcal{X}_S$, *i.e.*, the safety index should be positive for unsafe states and only be negative for safe states. Here the safe set can be viewed as generalized equilibrium. The second condition (negative definiteness on non equilibrium states) corresponds to the finite convergence condition (16.20), *i.e.*, the derivative at any state that is outside the safe set or the equilibrium should be pointed towards the safe set or the equilibrium. The third condition (zero derivative at the equilibrium) corresponds to the forward invariance condition (16.19), *i.e.*, the system should never escape the safe set of the equilibrium once enters. Due to the similarity as discussed above, the safety index can be viewed as a generalized Lyapunov function.

16.3.4 Safe control as a constraint

Now let us derive the constraint on control with respect to (16.19) and (16.20). Assume the system dynamics are affine, *i.e.*, $\dot{x} = f(x) + g(x)u$. Then the time derivative of the safety index equals to

$$\dot{\phi} = \underbrace{\frac{\partial \phi}{\partial x} f}_{L_f \phi} + \underbrace{\frac{\partial \phi}{\partial x} g}_{L_g \phi} u, \quad (16.21)$$

where $L_f \phi$ and $L_g \phi$ are Lie derivatives. Then the constraints on $\dot{\phi}$ can be transformed to a linear constraint on u ,

$$\dot{\phi} \leq -\eta(\phi) \Leftrightarrow L_f \phi + L_g \phi u \leq -\eta(\phi), \quad (16.22)$$

where $\eta(\phi)$ generalize the margin in the safety constraints. For example, we can choose $\eta(\phi)$ to be a positive constant for $\phi \leq 0$ and $\eta(\phi)$ to be negative infinity for $\phi < 0$. In general, $\eta(\phi)$ should be a non-decreasing function such that $\eta(0) \geq 0$. We call the resulting constraint on u as the *set of safe control*, which is a half space in the control space for every state:

$$U_S(x) := \{u \mid L_g \phi u \leq -\eta(\phi) - L_f \phi\}. \quad (16.23)$$

The normal direction of the half space is along the Lie derivative $L_g\phi$, which means $-L_g\phi$ is the safest direction. The control component that is perpendicular to $L_g\phi$ does not affect safety.

Up to this point, we have connected the safety specifications on different spaces, which can be mainly divided into two categories: the static safety specification which can either be specified as a set in the state space or a constraint with respect to a scalar function on the state space; and the dynamic safety specification which concerns more on the “motion” of the system according to the vector fields defined by the system dynamics. For the dynamic safety specification, we first pose the requirements on $\dot{\phi}$ (as a scalar constraint), then the requirements can be translated to $\dot{x}$ (a constraint on a vector field), and finally translated to u (a constraint on another vector field). As we mentioned earlier, the safe control is not “one particular” control input, but a set of control inputs that satisfy safety requirements. During the translation from $\dot{\phi}$ to u , singularity may happen when $L_g\phi = 0$, in which case the set of safe control is either the whole space or empty. Hence, by design, we should require $\frac{\partial \phi}{\partial u} = L_g\phi \neq 0$. We may enforce this requirement when we synthesize the safety index using the Hamilton Jacobi approach, or directly enforce this requirement by including higher order derivatives of ϕ in to the design of the safety index.

16.3.5 Safety monitor for online learning

Now that we have derived the safety constraint on the control. Then the natural way to apply the safety monitor during the learning process is to project the control signal u^r generated by the nominal learning controller to the set of safe control (16.23) and apply the resulting safe control u to the system. There are multiple ways to perform the projection. We can either do a direct sum by adding a large vector Δu along $-L_g\phi$:

$$u = u^r + \Delta u. \quad (16.24)$$

This method is used in the potential field algorithms and the sliding mode algorithms. We may also compute the optimal Δu to be added through the following quadratic programming:

$$u = \arg \min_u \|u - u^r\|_Q^2 \quad (16.25)$$

$$\text{s.t. } L_f\phi + L_g\phi u \leq -\eta(\phi). \quad (16.26)$$

This method is used in control barrier function and the safe set algorithm.

Nonetheless, correct computation of the set of safe control depends on the following factors:

- The safety specification in the safety index ϕ ;
- Understanding of the system dynamics $\dot{x} = f(x) + g(x)u$;
- Measurement of the current state x .

During the learning process, it may be difficult to obtain the system dynamics beforehand, although we can assume knowledge of ϕ and noise-free measurement of the current state x . Then the set of safe control should also account for potential model mismatch and uncertainties in system dynamics. Let us now consider different modes of dynamic uncertainties:

- Case 1: Known g , unknown f .

Example: human-robot system; human dynamics are encoded in f , which is unknown; the robot knows its own dynamics.

- Case 2: Known f , unknown g .

Example: known system with additional actuations (unknown consequence of the additional actuation)

- Case 3: Unknown f , unknown g .

Example: unknown robotic system.

In the following discussion, let us consider case 1 and assume that we can perform online learning of f . Suppose the model uncertainty is capture by the uncertain set $\mathcal{F}$, i.e., $f \in \mathcal{F}$. Then to ensure safety, we need introduce robust safe control which takes the intersection of all safety constraints under different $f \in \mathcal{F}$:

$$\bar{U}_S(x) := \cap_{f \in \mathcal{F}} U_S(x) = \{u \mid L_g \phi u \leq -\eta(\phi) - \max_{f \in \mathcal{F}} L_f \phi\}. \quad (16.27)$$

We are going to illustrate how to compute and apply this robust safe control constraint in the following example on human-robot interaction.

Example Now let us consider a human-robot system where the robot state is denoted x_R and the human state is denoted x_H . The robot dynamics is $\dot{x}_R = f_R(x_R) + g_R(x_R)u$ where u is the robot control. Note an implicit assumption here is that the open-loop dynamics of the robot is independent from the human. The human dynamics is $\dot{x}_H = f_H(x_H, x_R)$, which is a closed-loop model of the human behavior. The closed-loop human behavior depends on the robot state since the human reacts to the robot motion. Let the system state be $x = [x_R; x_H]$. The closed-loop dynamics of the human-robot

system then become:

$$\dot{x} = \underbrace{\begin{bmatrix} f_R(x_R) \\ f_H(x) \end{bmatrix}}_{f(x)} + \underbrace{\begin{bmatrix} g_R(x_R) \\ 0 \end{bmatrix}}_{g(x)} u. \quad (16.28)$$

Suppose the safety specification requires that the robot and the human be at least d_{min} distance away from each other, i.e., $\phi(x) = d_{min}^2 - (x_R - x_H)^T Q (x_R - x_H)$ where Q is a weight matrix. Then the set of safe control satisfies:

$$L_g \phi u \leq -\eta(\phi) - L_f \phi. \quad (16.29)$$

Since $L_g \phi = \frac{\partial \phi}{\partial x_R} g_R$ and $L_f \phi = \frac{\partial \phi}{\partial x_R} f_R - \frac{\partial \phi}{\partial x_H} f_H$, we can rewrite the control constraint as

$$\frac{\partial \phi}{\partial x_R} g_R u \leq -\eta(\phi) - \frac{\partial \phi}{\partial x_R} f_R - \frac{\partial \phi}{\partial x_H} f_H. \quad (16.30)$$

The function f_H is unknown to the robot and it needs to be inferred online by observing the human's reaction to the robot. According to the robust safe control formulation, we need to choose a control u that is safe under all possible f_H . If we have a bounded range of possible f_H , then we can directly apply (16.27).

However, if we learn the parameters in f_H using the Kalman Filter type of online model learning methods, the uncertainty associated with the learned model follows a Gaussian distribution, which is unbounded. In the human-robot example, that means $\dot{x}_H = f_H(x) \sim \mathcal{N}(m(x), \Sigma(x))$. Then the constraint (16.30) is ill-posed since $f_H(x)$ is unbounded. To deal with this problem, we need to introduce chance constraint, which requires the constraint (16.30) to be satisfied with high probability instead of for every point. Then chance constraint can be written as

$$\mathbb{P}_{\dot{x}_H \sim \mathcal{N}(m, \Sigma)} \left[\underbrace{\frac{\partial \phi}{\partial x_R} g_R}_L u \leq \underbrace{-\eta(\phi) - \frac{\partial \phi}{\partial x_R} f_R - \frac{\partial \phi}{\partial x_H} \dot{x}_H}_{S(\dot{x}_H)} \right] > 1 - \epsilon. \quad (16.31)$$

Recall that 3σ value can almost bound all uncertainties (with probability 0.997). We can set ϵ to be 0.003 and compute the robust bound as

$$\bar{U}_S(x) = \{u \mid Lu \leq \bar{S}\}, \text{ where } \bar{S} := \min_{(\dot{x}_H - m)^T \Sigma^{-1} (\dot{x}_H - m) \leq 9} S(\dot{x}_H) \quad (16.32)$$

The expression to $\bar{S}$ is a quadratically constrained linear program, which has an analytical solution. Essentially, the worst case human

motion can be computed as

$$\dot{x}_H^* = m + 3 \frac{\Sigma(\frac{\partial \phi}{\partial x_H})^T}{\sqrt{(\frac{\partial \phi}{\partial x_H})\Sigma(\frac{\partial \phi}{\partial x_H})^T}}. \quad (16.33)$$

The resulting robust bound $\bar{S}$ is

$$\bar{S} = \underbrace{-\eta - \frac{\partial \phi}{\partial x_R} f_R - \frac{\partial \phi}{\partial x_H} m}_{S(m)} - 3 \sqrt{(\frac{\partial \phi}{\partial x_H})\Sigma(\frac{\partial \phi}{\partial x_H})^T}. \quad (16.34)$$

In this example, the algorithm for safe learning is summarized below.

- Step 1: compute the set of safe control in the general form, e.g., $U_S(x) = \{u \mid L u \leq S(\dot{x}_H)\}$;
- Step 2: estimate the robust bound given the uncertainty, e.g., $\dot{x}_H^*$;
- Step 3: map the reference control from the learning controller to the set of robust safe control $\bar{U}_S(x) = \{u \mid L u \leq S(\dot{x}_H^*)\}$.

Step 2 solves an optimization with linear objective and quadratic constraint, while step 3 solves an optimization with quadratic objective and linear constraint. Both have analytical solutions. Note this method works under two assumptions. First, the uncertainty only exists in the system inertia term. Second, the uncertainty follows a Gaussian distribution.

16.3.6 Open problems

One big problem associated with the safety monitor is that there is no exploration in the safety monitor. The learning controller should consider strategies to enhance exploration. While the learning controller wants to explore more states and the safety controller wants to maintain inside the safe set, there might be deadlocks between the learning control and the safe control. Stability issues may also arise if the system keeps switching between the learning control and the safe control. Moreover, although there are different safe control methods that can be combined with learning modules, there still lacks systematic methods for performance evaluation of those control methods in interactive environment.

16.4 Safe Learning with Unknown Specifications

If the safety specification unknown, the agent needs to “learn” the safety specification during the interaction with the environment. The underlying assumptions include:

- the agent should be able to evaluate the current safety level online;
- the safety specification is regular so that the local evaluation can be generalized into a global function;
- the agent needs to start from an initially safe configuration.

This situation is similar to a robot navigating in an unknown terrain. The robot may not know which place is safe, but can evaluate whether the current location is safe or not. And the robot has local sensors to sense the safety of a small neighbourhood around its current location. Then the robot can move along the boundary of the known safe region and continuously sense its neighbourhood to get better estimation of the safety region. Meanwhile, the robot may also need to conduct learning to minimize other cost objectives. Then the two learning needs to be conducted in a balanced way. In the following discussion, we introduce the *safe Bayesian learning* which provides an elegant mathematical framework for agents to learn to safely minimize cost objectives while both the objectives and the safety specifications are unknown.

Suppose we need to solve the following constrained optimization problem

$$\min_{u \in \mathcal{U}} J(u), \text{ s.t. } \phi_i(u) \leq 0, \forall i = 1, \dots, k. \quad (16.35)$$

The functions J and ϕ_i are unknown a priori, which are modeled as Gaussian processes. All we know is that an initial safe set $\mathcal{U}_0 \subseteq \mathcal{U}$ for which the constraints are fulfilled; the functions ϕ_i is Lipchitz where

$$|\phi_i(u) - \phi_i(u')| \leq L_i \|u - u'\|. \quad (16.36)$$

Recall that we can use past samples to infer the a posteriori distribution of J and ϕ_i . Suppose at step n , we have applied control $u_1, \dots, u_n$. Denote the measured J and ϕ_i values at these control points as

$$\hat{\Phi}_i^n = [\hat{\phi}_i(u_1) \ \cdots \ \hat{\phi}_i(u_n)]^T, \hat{\mathbf{J}}^n = [\hat{J}(u_1) \ \cdots \ \hat{J}(u_n)]^T, \quad (16.37)$$

Then the lower and upper confidence bounds of ϕ_i and J can be computed as

$$\bar{\phi}_i^n(u) = \mathbb{E} [\phi_i(u) \mid \hat{\Phi}_i^n] + \beta_n^{1/2} \text{VAR} [\phi_i(u) \mid \hat{\Phi}_i^n] \quad (16.38)$$

$$\underline{\phi}_i^n(u) = \mathbb{E} [\phi_i(u) \mid \hat{\Phi}_i^n] - \beta_n^{1/2} \text{VAR} [\phi_i(u) \mid \hat{\Phi}_i^n] \quad (16.39)$$

$$\bar{J}^n(u) = \mathbb{E} [J(u) \mid \hat{\mathbf{J}}^n] + \beta_n^{1/2} \text{VAR} [J(u) \mid \hat{\mathbf{J}}^n] \quad (16.40)$$

$$\underline{J}^n(u) = \mathbb{E} [J(u) \mid \hat{\mathbf{J}}^n] - \beta_n^{1/2} \text{VAR} [J(u) \mid \hat{\mathbf{J}}^n] \quad (16.41)$$

where β_n reflects the confidence interval of the GP.

According to the upper bound $\bar{\phi}_i^n(u)$ can using the Lipschitz condition, we have

$$\bar{\phi}_i^n(u') \leq \bar{\phi}_i^n(u) + L_i \|u - u'\|. \quad (16.42)$$

Hence, the control u' should be safe if for some $u \in \mathcal{U}_n$, it satisfies that

$$\bar{\phi}_i^n(u) + L_i \|u - u'\| \leq 0. \quad (16.43)$$

Hence the safe set at the next iteration is defined as

$$\mathcal{U}_{n+1} = \cap_i \cup_{u \in \mathcal{U}_n} \{u' \in \mathcal{U} \mid \bar{\phi}_i^n(u) + L_i \|u - u'\| \leq 0\}. \quad (16.44)$$

Then we can choose the next control u_{n+1} by solving the following constrained optimization,

$$u_{n+1} = \arg \min_{u \in \mathcal{U}_{n+1}} J^n(u). \quad (16.45)$$

Now the problem comes as how should be balance the exploration of the safe set and the optimization of the objective function?

Let us define two critical sets:

- The set of control that can improve the minimizer, *i.e.*, the set of control such that the lower bound of the objective function is smaller than the minimum value of the upper bound of the objective function within the safe set $\mathcal{U}_{n+1}$:

$$\mathcal{M}_{n+1} = \{u \in \mathcal{U}_{n+1} \mid J_n(u) \leq \min_{u' \in \mathcal{U}_{n+1}} \bar{J}_n(u')\}. \quad (16.46)$$

- The set of control that can enlarge the safe set

$$\mathcal{G}_{n+1} = \{u \in \mathcal{U}_{n+1} \mid m(u) > 0\}, \quad (16.47)$$

where $m(u)$ is the measure of the safe set that can be potentially enlarged

$$m(u) = |\{u' \in \mathcal{U} \setminus \mathcal{U}_{n+1} \mid \exists i, \bar{\phi}_i^n(u) + L_i \|u - u'\| \leq 0\}|. \quad (16.48)$$

If we pick u as our next action and we are lucky that the true value of $\phi_i(u)$ lies on the lower bound of our estimate, then in the next round, any u' that satisfies $\phi_i^n(u) + L_i \|u - u'\| \leq 0$ will be considered safe. And in particular, we want to measure how many new u' will be considered safe. Hence we only count u' that is outside $\mathcal{U}_{n+1}$. As long as there are u' outside the current safe set that can be made safe, the action u will result in a positive measure $m(u)$. And the collection of those actions is the set of control that is good for us to explore the safe set.

The controls in $\mathcal{M}_{n+1}$ will help us improve the objective function, while the controls in $\mathcal{G}_{n+1}$ will help us explore the safe set. We only need to perform the optimization in the union of these two sets. The trade off between $\mathcal{M}_{n+1}$ and $\mathcal{G}_{n+1}$ can be handled by selecting the most uncertain element across all performance and safety functions

$$u_{n+1} = \arg \max_{u \in \mathcal{M}_{n+1} \cup \mathcal{G}_{n+1}} \max \{ \bar{J}_n(u) - \underline{J}_n(u), \max_i \bar{\phi}_i^n(u) - \underline{\phi}_i^n(u) \}. \quad (16.49)$$

The algorithm for safe Bayesian optimization can be summarized below:

- Given n past observations, compute the lower and upper bounds of all functions;
- Compute the safe set in (16.44);
- Compute the set to improve the minimizer (16.46);
- Compute the set to enlarge the safe set (16.47);
- Obtain the new evaluation point (16.49);
- Get new measurement and repeat the process.

Bibliography

Index

asymptotically stability, [26](#)

controllability, [19](#)

license, [2](#)

stability, [26](#)